从零开始的网安学习之路——爆破皮卡丘靶场

此份笔记记录了本人从零开始学习爆破皮卡丘靶场的心得体会，所使用的工具主要有BurpSuite社区版和Python脚本工具。坦白地说，这是一个探索过程的真实记录，并不是专业网络攻防者的攻略说明，因此必然存在诸多疏漏之处，还请各位谅解。

倘若此份笔记能够给大家一点启发，那将是我莫大的荣幸。

一、配置说明

在爆破开始之前，首先需要进行靶场和相关工具的配置。由于网上相关教程已经十分详备，此处仅作简要概述，同时贴出原文链接，可供参考。

1、安装皮卡丘靶场

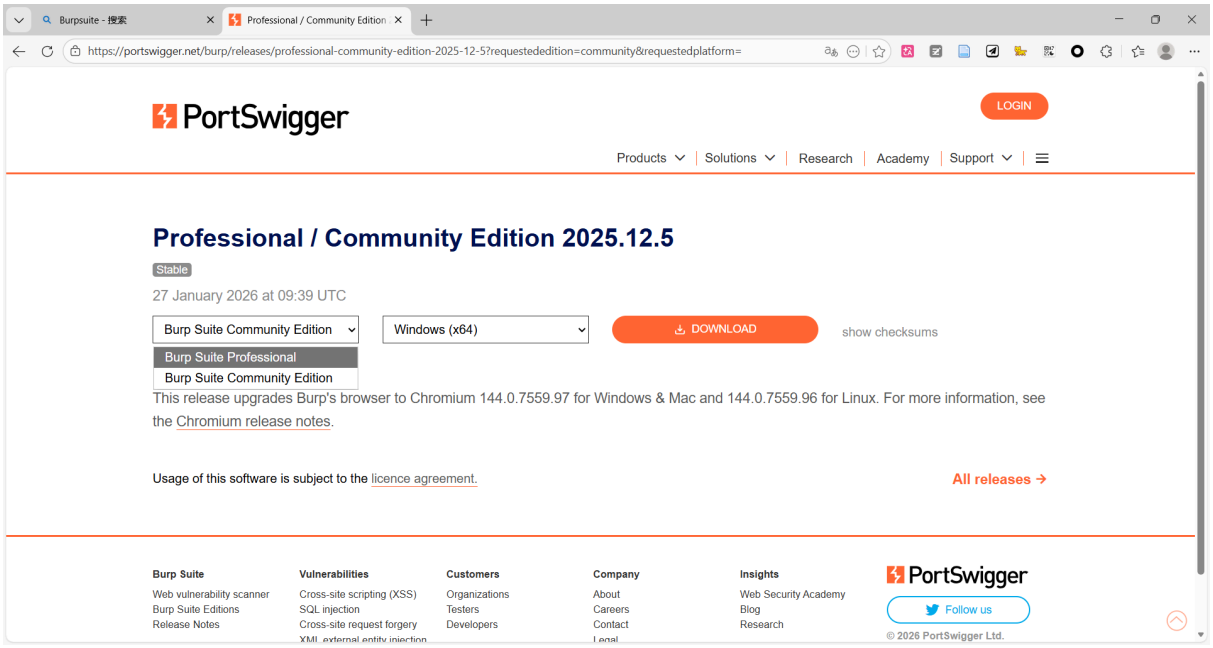
在皮卡丘靶场安装前，我们需要将靶场运行的环境配置好，我选用的是**phpstudy集成开发环境**。在环境配置完成之后，就可以前往GitHub下载皮卡丘靶场的源代码，解压到专门目录即可。

完整的配置说明可以参考下方链接：

pikachu 靶场环境搭建 完整版（手把手保姆级教学）_pikachu靶场搭建-CSDN博客

2、安装BurpSuite

BurpSuite 是一款集成化 Web 应用安全测试工具，你需要前往官网Professional / Community Edition 2025.12.5 | Releases下载安装包，可以选择需要的版本（专业版Professional/社区版Community）。



不过，由于BurpSuite采用Java语言编写，运行时需要配置Java环境。具体配置方法可以参考下方链接：

java 环境配置(详细教程)_java环境配置-CSDN博客

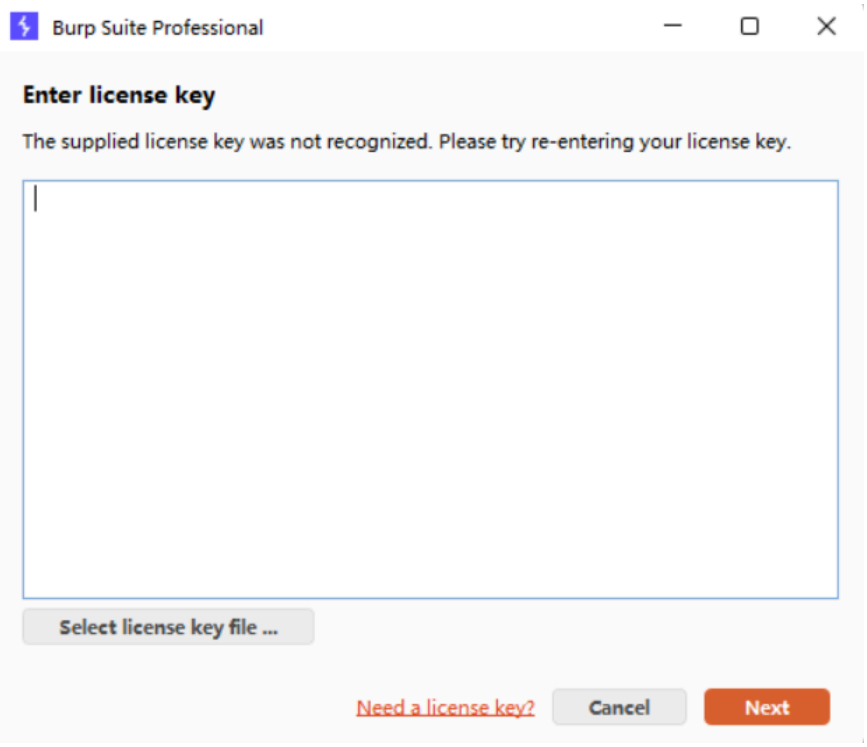
此链接的博客时间虽然比较久了，但是编程环境的配置都是同一套步骤，而且环境这种东西并不是越新越好，还要考虑到其兼容性和稳定性。你也可以直接百度搜索其他的配置说明，不再赘述。当然，如果你的目的只是为了配置BurpSuite，而不考虑成为java工程师的话，那么倒不用考虑太多。

配置完成后，你可以在终端输入java -version来检验。如果成功显示java版本，那么恭喜：你的java版本已经配置完成了。

```
(base) PS D:\> java -version
java version "25.0.1" 2025-10-21 LTS
Java(TM) SE Runtime Environment (build 25.0.1+8-LTS-27)
Java HotSpot(TM) 64-Bit Server VM (build 25.0.1+8-LTS-27, mixed mode, sharing)
```

如果你选择的是社区版，和我一样，那么就双击下载完成后的安装包，按照安装说明一步一步操作即可（记得安装路径要改到D盘），安装完成后双击打开BurpSuite，开箱即可用。

当然，社区版的BurpSuite功能比较基础，很多进阶的功能没有配置。如果你希望手里有一个工具更齐全、功能更强大的网安工具箱，可以下载专业版。不过专业版需要进行激活，如下图。如果你是从官网下载的安装包的话，就必须通过官网Burp Suite Professional - PortSwigger获取激活码了，一般来说都是需要付费的。



如果你想要白嫖专业版的话，不要从官网下载安装包，可以直接通过网盘等途径下载他人的安装包，具体操作方法可以参考下面的链接：

BurpSuite下载和安装保姆级教程（附官网安装包，非常详细）_burpsuite官网下载-CSDN博客

下面这个链接包含了上述专业版BurpSuite配置的全流程：

Burpsuite安装及环境配置（超详细解说）_burpsuite配置java环境-CSDN博客

此外，值得注意的是，2020年之后的BurpSuite版本没有内置汉化包。如果你不想被语言所累，可以参考下面的汉化教程：

burp suite改为中文(小白必看！3分钟搞定Burp Suite改中文全流程,手把手教学,零基础也能轻松上手) - Burp Suite中文网站，Portswigger中文网站

3、配置Python环境

为什么要配置Python环境？主要是我安装的是社区版，有很多功能需要Python脚本来打个辅助。当然，我相信你既然已经开始搞网安了，Python环境肯定早就配置好了吧(●'◡'●)，这里就不多赘述了。

4、配置网络代理

如果你需要使用BurpSuite进行抓包，就需要配置好代理，这里的建议是不要更改系统的网络代理，可以通过一个插件**ZeroOmega**来实现，具体配置方法可以参考下面的链接：

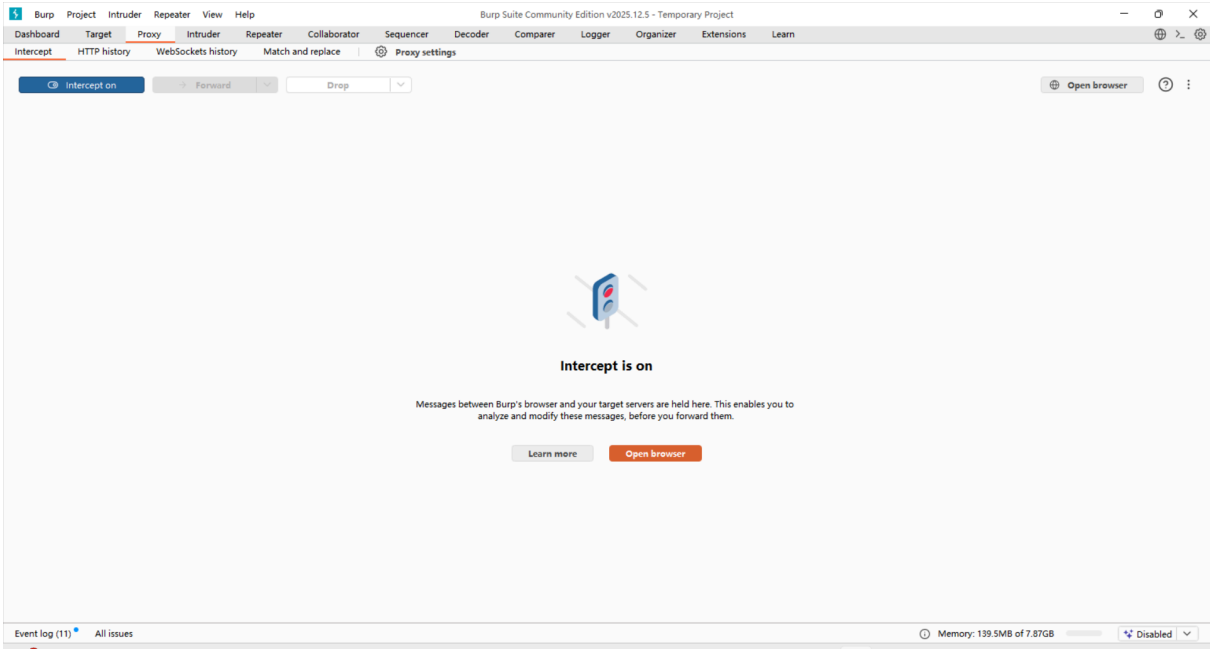
谷歌+burpsuite+ZeroOmega-CSDN博客

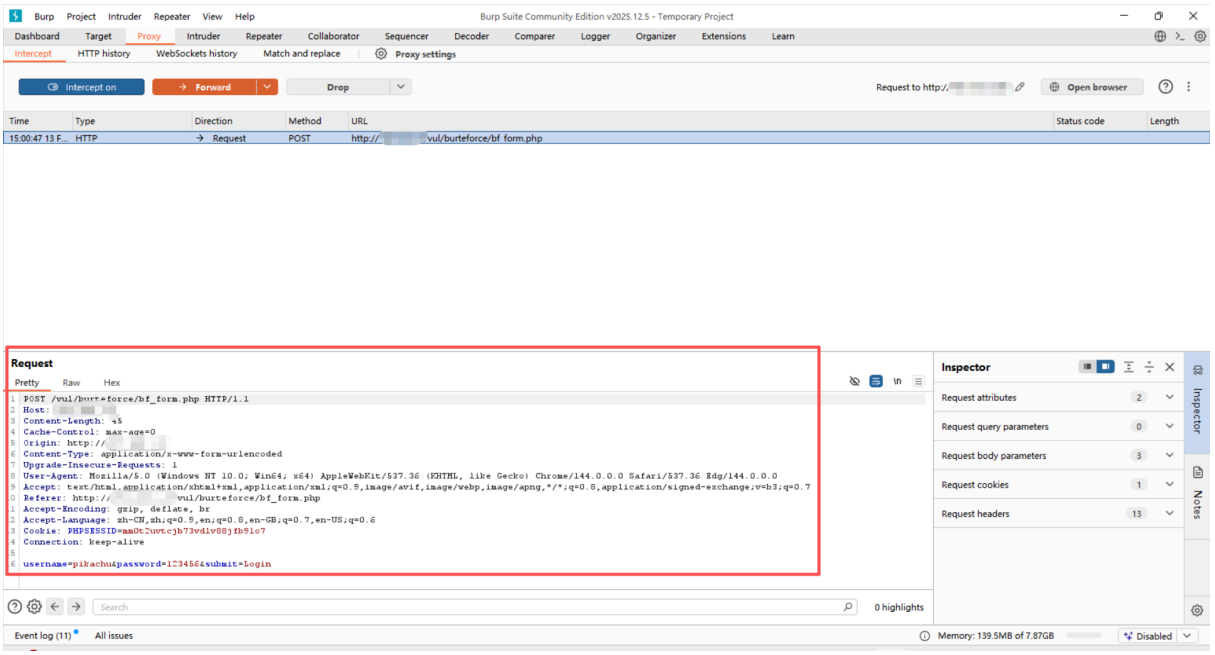
配置完成后，插件的“系统代理”选项就是采用系统的IP，你可以正常浏览网页；而如果你需要进行抓包，就可以打开proxy选项。

此外，由于BurpSuite抓包默认不会拦截本地127.0.0.1传输的信息，而皮卡丘靶场是本地部署的，默认通过127.0.0.1访问，因此你需要通过自己本机IP地址来访问皮卡丘靶场，具体配置方法如下：

BurpSuite抓不到本地（127.0.0.1）靶场的包解决方法（超简单傻瓜教程）_burpsuite抓不到127.0.0.1-CSDN博客

配置完成后，你可以做一个测试，访问皮卡丘靶场后，首先打开BurpSuite的Proxy选项，开启intercept，然后开启**ZeroOmega**插件的proxy选项，随意在靶场的一个登录入口输入一段用户名和密码，点击login，立马就会跳转到BurpSuite页面，你就可以看到自己抓包的信息啦！





OK！事到如今，配置部分已经大功告成，你可以正式开启自己的靶场爆破之路啦！

二、暴力破解

引用皮卡丘靶场关于暴力破解的概述如下：

> ***Burte Force（暴力破解）概述***
>
> *"暴力破解"是一攻击具手段，在web攻击中，一般会使用这种手段对应用系统的认证信息进行获取。 其过程就是使用大量的认证信息在认证接口进行尝试登录，直到得到正确的结果。 为了提高效率，暴力破解一般会使用带有字典的工具来进行自动化操作。*
>
> *理论上来说，大多数系统都是可以被暴力破解的，只要攻击者有足够强大的计算能力和时间，所以断定一个系统是否存在暴力破解漏洞，其条件也不是绝对的。 我们说一个web应用系统存在暴力破解漏洞，一般是指该web应用系统没有采用或者采用了比较弱的认证安全策略，导致其被暴力破解的"可能性"变的比较高。 这里的认证安全策略, 包括：*
>
> *1.是否要求用户设置复杂的密码；2.是否每次认证都使用安全的验证码（想想你买火车票时输的验证码～）或者手机otp；3.是否对尝试登录的行为进行判断和限制（如：连续5次错误登录，进行账号锁定或IP地址锁定等）；4.是否采用了双因素认*

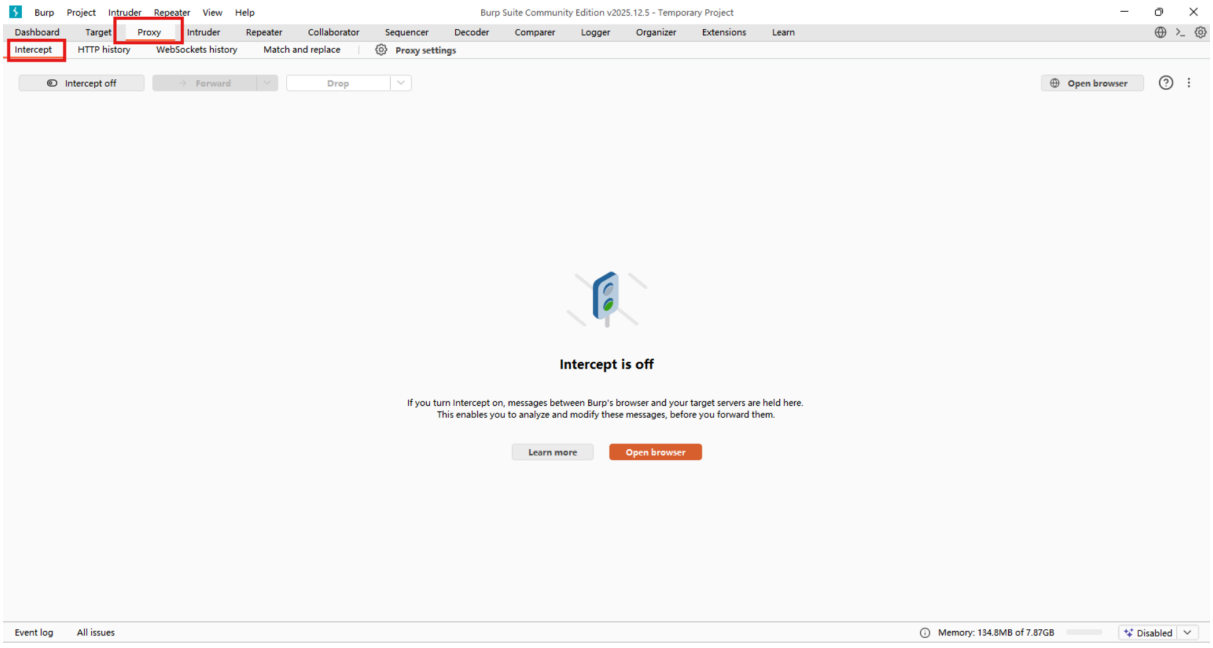
> *证；...等等。千万不要小看暴力破解漏洞,往往这种简单粗暴的攻击方式带来的效果是超出预期的!你可以通过"BurteForce"对应的测试栏目，来进一步的了解该漏洞。*
>
> *从来没有哪个时代的黑客像今天一样热衷于猜解密码 ---奥斯特洛夫斯基*

现在你可以开始使用BurpSuite暴力破解密码啦!

不过在开始之前，我们需要介绍BurpSuite中非常重要的一个工具，Proxy。这个工具是用来抓包的，后续暴力破解密码必备。它实际起到的作用就是拦截——用大白话解释一下，你在和网页互动时，你在网页上进行需要与服务器交互的操作（比如登录、提交表单、搜索）时，浏览器会把你的输入打包成一个请求，发往服务器，服务器收到信息之后再给出反馈，这就是一次你与网页的互动过程；而抓包就是把这个请求拦截在服务器接收之前，转而先发送给BurpSuite。这样的话，你就可以对这个请求数据做些手脚了（doge），之后你可以放行这个请求，服务器接收到的就是这个做过手脚的数据了。

而暴力破解密码的过程，就存在于这个抓包之后的做手脚的环节。它实质上就是替换掉你输入的用户名、密码等信息，代之以许多数据的排列组合，全都发送给服务器，从而根据反馈信息猜出正确的账号密码。

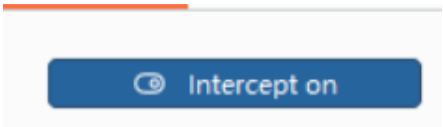
现在，请你启动**phpstudy**，访问皮卡丘靶场，开启**ZeroOmega**的proxy选项，打开BurpSuite的Proxy页面，开启暴力破解之旅吧!



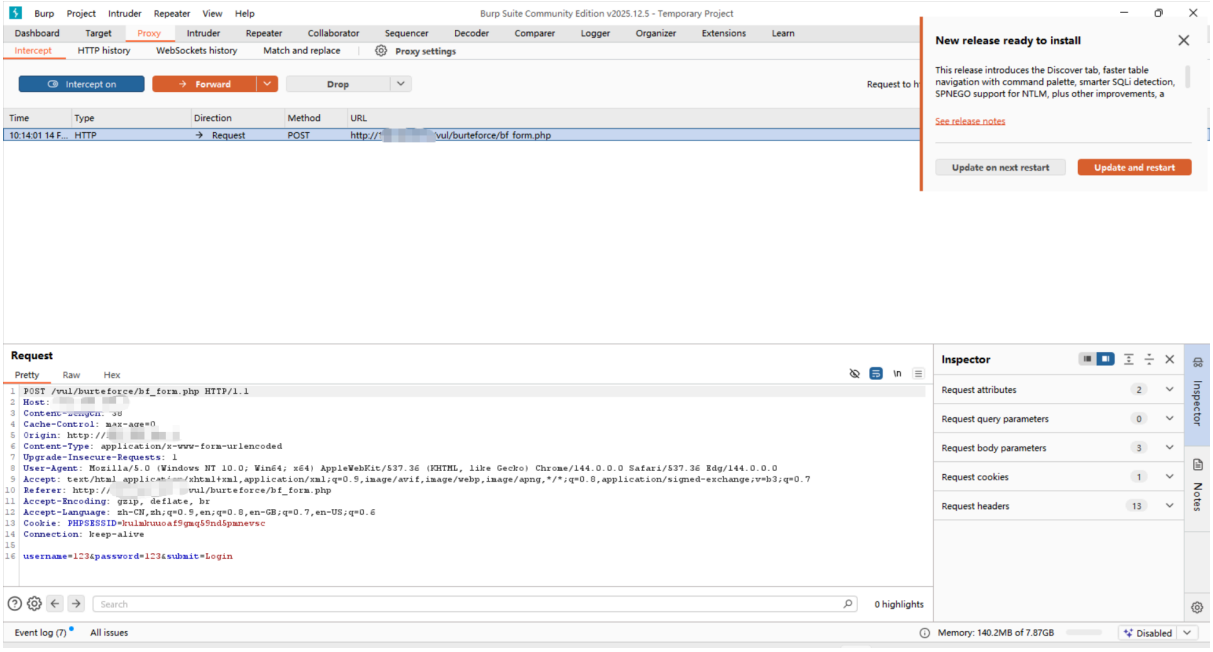
1、基于表单的暴力破解

我们可以先简单尝试一下，随便输入一对用户名和密码，看到前端反馈“用户名或密码不存在”。

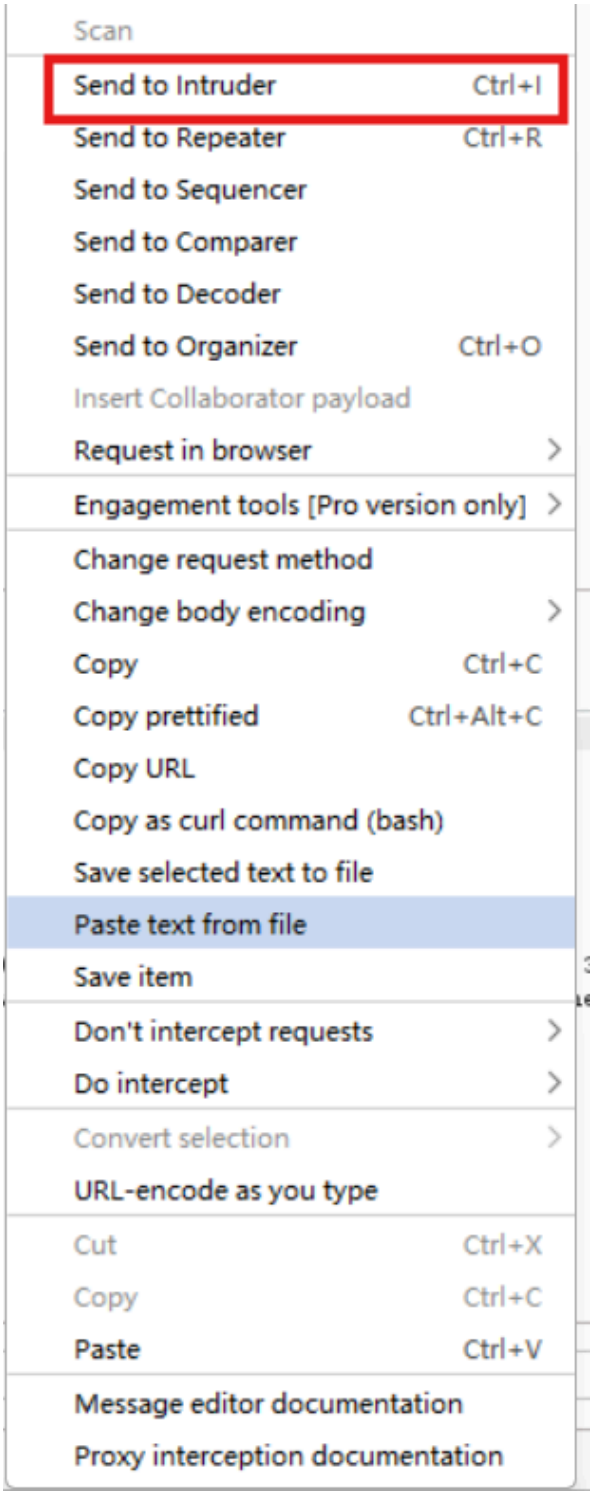
现在我们进一步验证，开启BurpSuite的Proxy功能后，再次点击login。



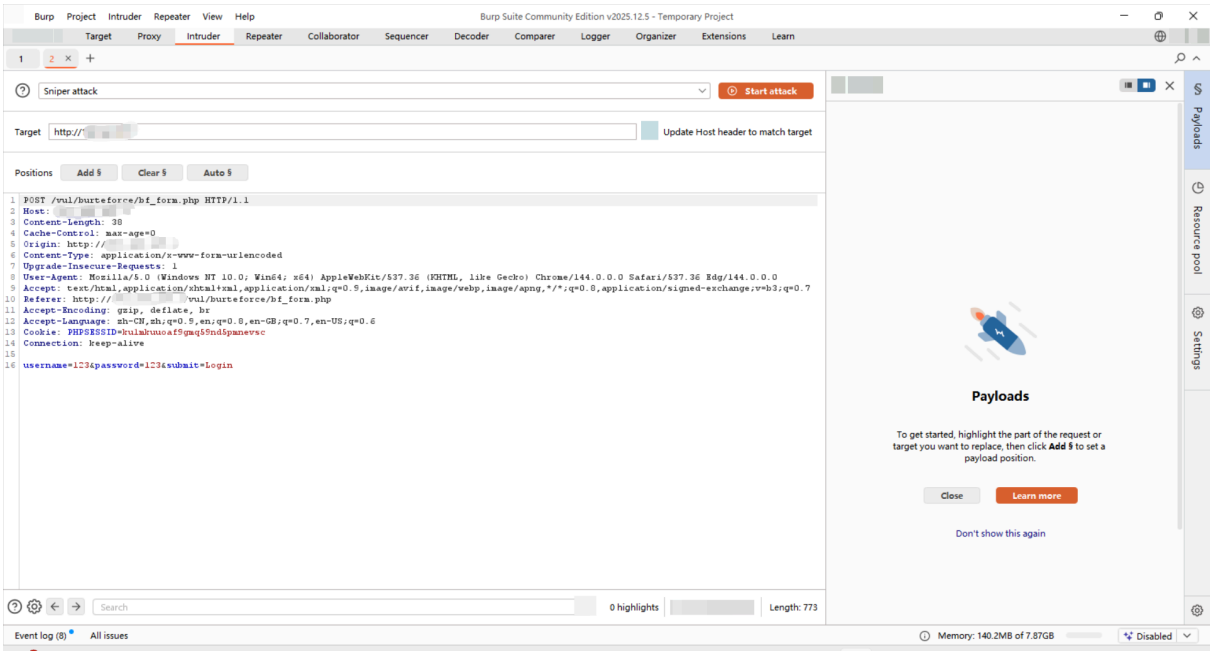
会发现自动跳转到BurpSuite页面，显示收到了Request的数据，重点是最后一行的信息，可见是你输入的用户名、密码，没有token（关于token功能，后面会提及）。因此，这就是一个最简单的输入验证模块，可以直接采用字典爆破的方法尝试出正确的用户名-密码对。



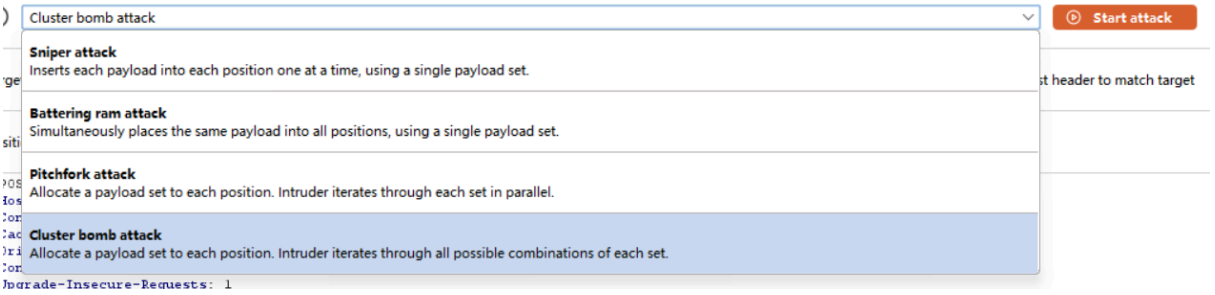
在Request功能区右键，点击“Send to Intruder”（之后你也可以直接用快捷键）。如果说Proxy主要负责抓包获取数据，那么Intruder（攻击器）就是专门负责自动化测试的工具，主要用于暴力破解、模糊测试或枚举。把抓包的数据发送给Intruder之后，你就可以根据数据的格式进行针对性的破解了。



下面是发送数据后Intruder的显示：

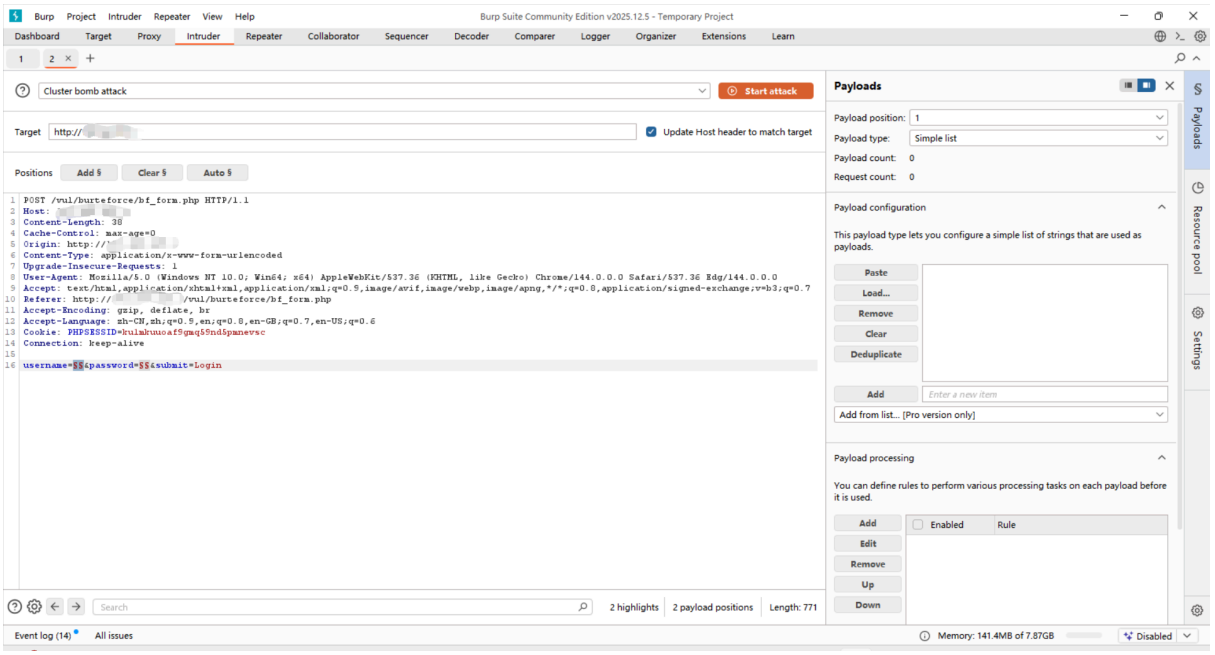


暴力破解选用此处的“Cluster bomb attack”（集束炸弹）功能，这是最灵活的的攻击模式，支持多组字典组合的攻击模式。



代入这个破解场景中，就是需要分别对用户名和密码导入字典。因此需要删掉自己在前端输入的用户名和密码，替换成SS，即“AddSS”功能。

替换后如下图所示：



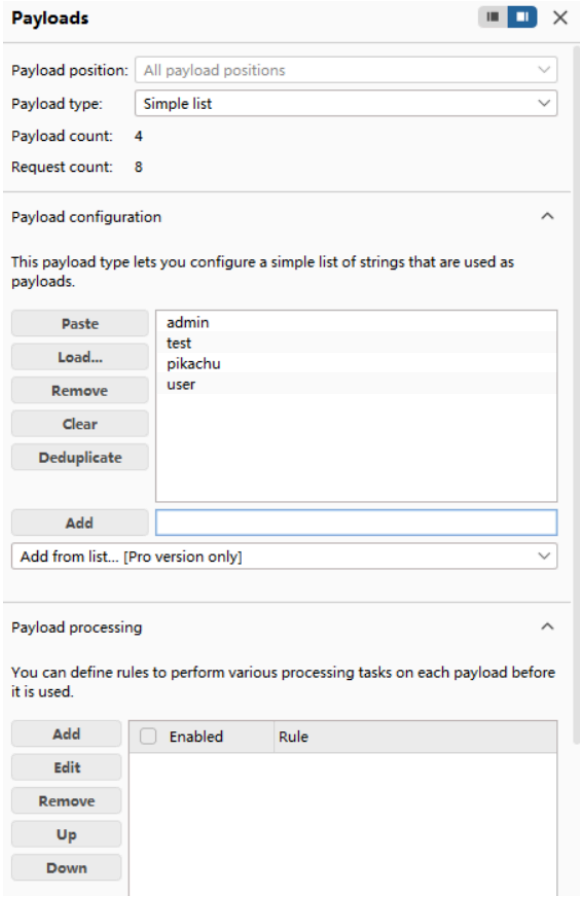
可以看到，当你把光标插入“SS”之间后，右侧出现Payloads面板，这就是插入“用户名/密码”字典的地方。你可以下载常用字典，直接导入到此功能区，参考链接如下：

【免费下载】 简易常用爆破字典-CSDN博客

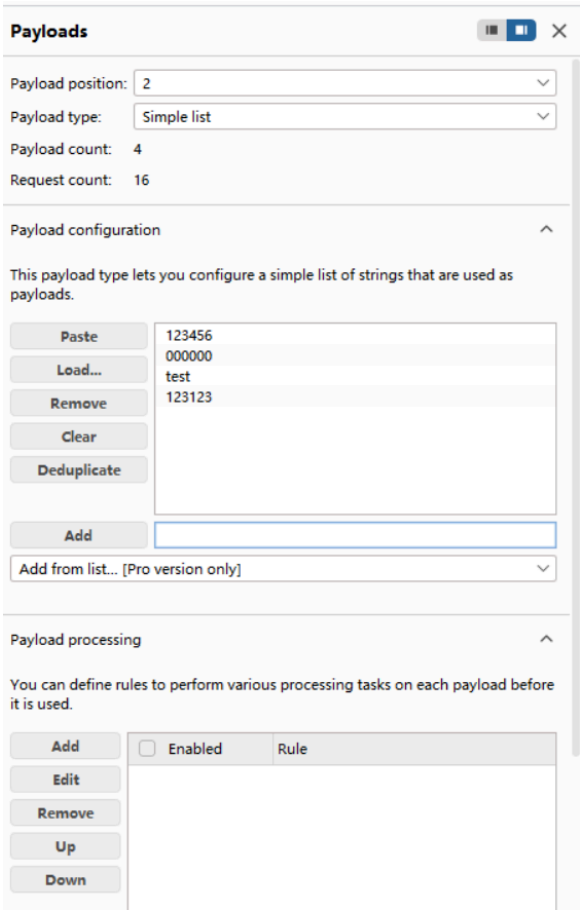
不过由于皮卡丘靶场的用户名/密码是有提示的，总共就三对组合：

所以我们这里就输入一些简单的用户名/密码，仅作暴力破解的演示。而在真实场景中，当然是需要导入字典可能对弱密码的暴力破解起到作用。

把光标放在用户名的"SS"之间，在右侧面板输入几组用户名：

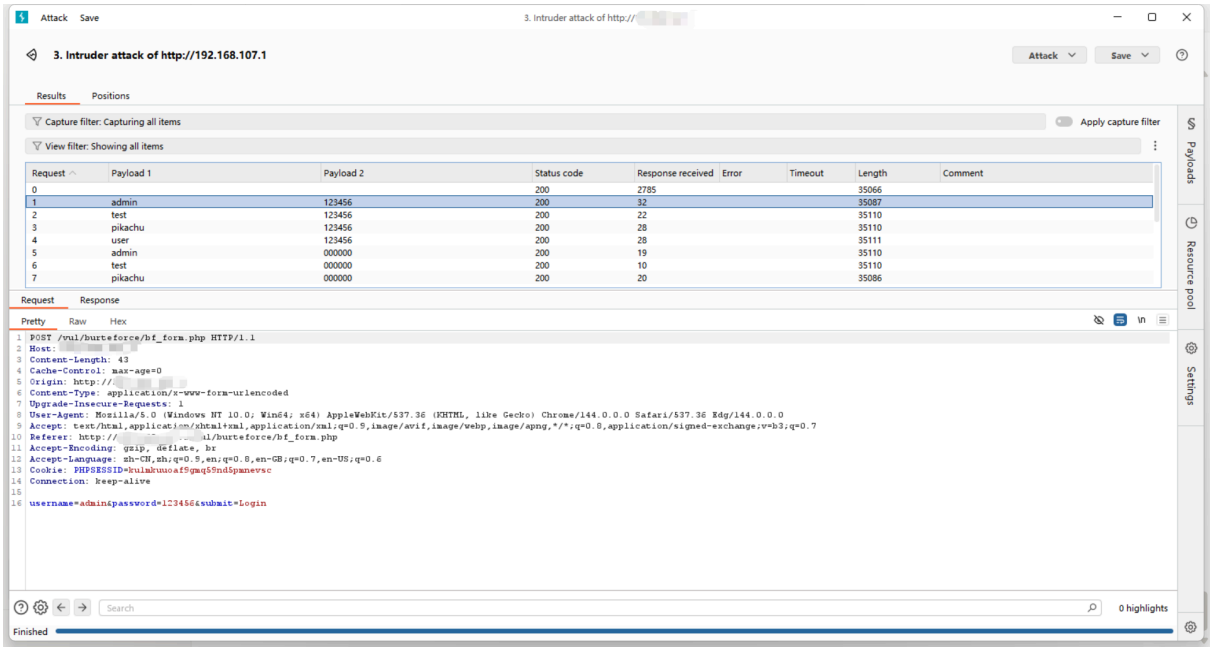

Payloads
Payload position: All payload positions
Payload type: Simple list
Payload count: 4
Request count: 8
Payload configuration
This payload type lets you configure a simple list of strings that are used as payloads.
Paste
Load...
Remove
Clear
Deduplicate
admin
test
pikachu
user
Add
Add from list... [Pro version only]
Payload processing
You can define rules to perform various processing tasks on each payload before it is used.
Add
Edit
Remove
Up
Down
Enabled
Rule

然后是密码：

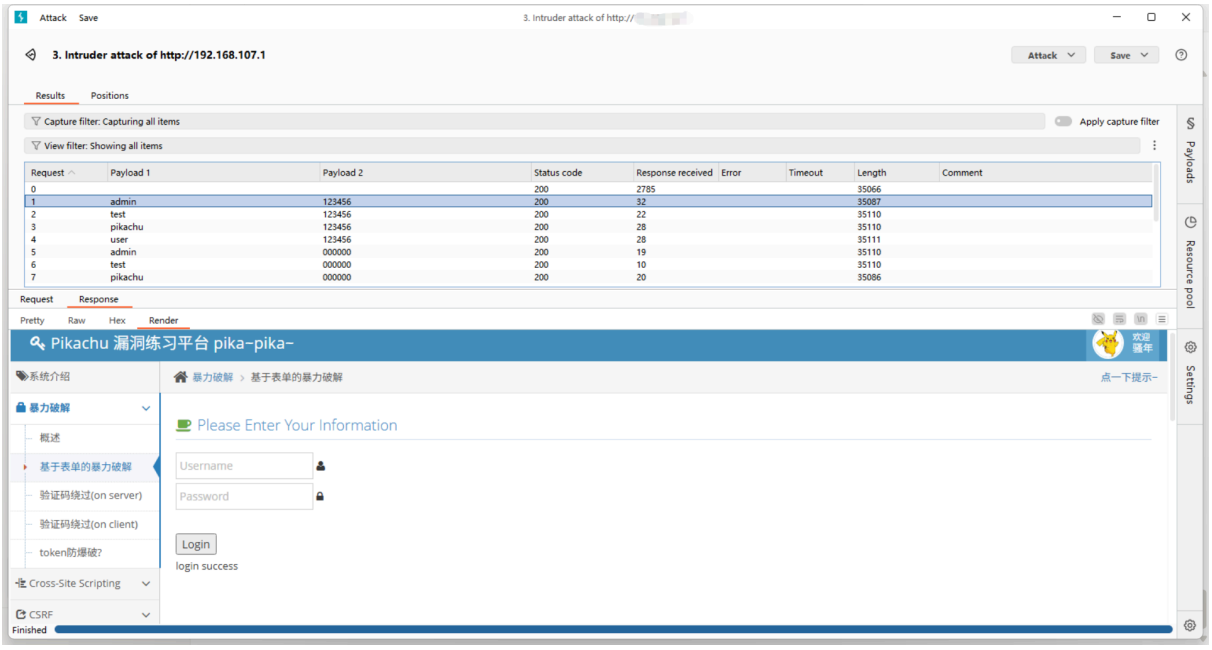


现在，就可以点击“Start attack”展开攻击啦!

这是攻击完成后的页面。可以看到，BurpSuite向服务器发送了所有组合的请求，并获取了反馈信息。其中，Length长度与其他有明显区别的大概率就是正确的用户名/密码对。



选中一个可能正确的组合，将下侧面板切换到Response-Render，你可以预览到前端页面，更加直观地看到结果。前端显示"login success"，说明这就是正确的用户名/密码对。



恭喜！至此你已经你成功通过了皮卡丘靶场暴力破解的第一关！

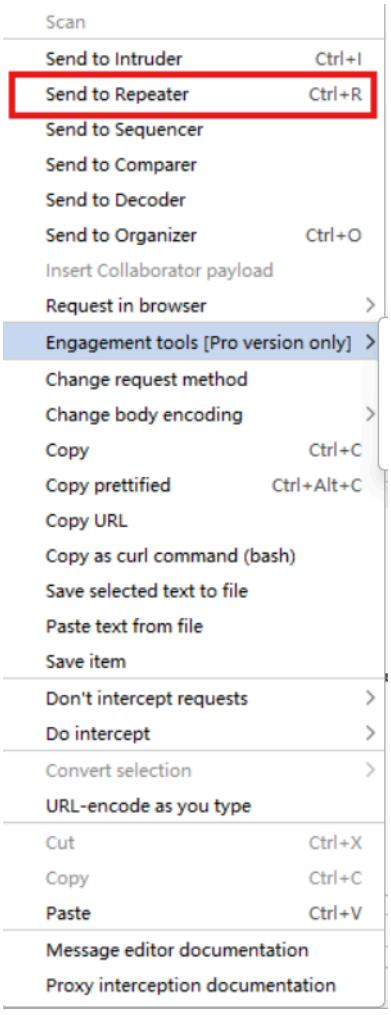
2、验证码绕过(on server)

进入第二关，发现这里出现了验证码验证。我们先做一些简单的测试，发现验证码不能为空，也不能输入错误，而且每次登陆失败，前端显示的验证码也发生了"变化"。

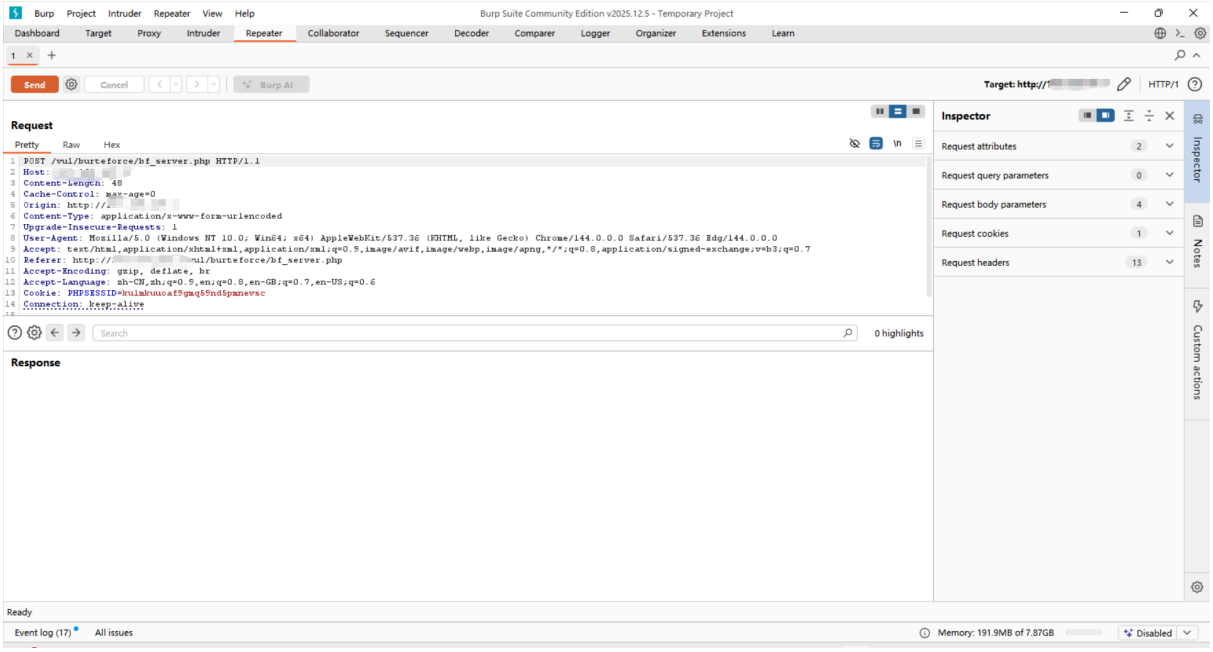
怎么办？如果每次都需要进行验证码输入的话，那么显然是无法通过简单的暴力破解来成功登录了，因为验证码的识别对于破解工具而言较为困难。

遇事不决，我们还是先抓包一次看看，同样的操作，抓到的数据如下图所示：

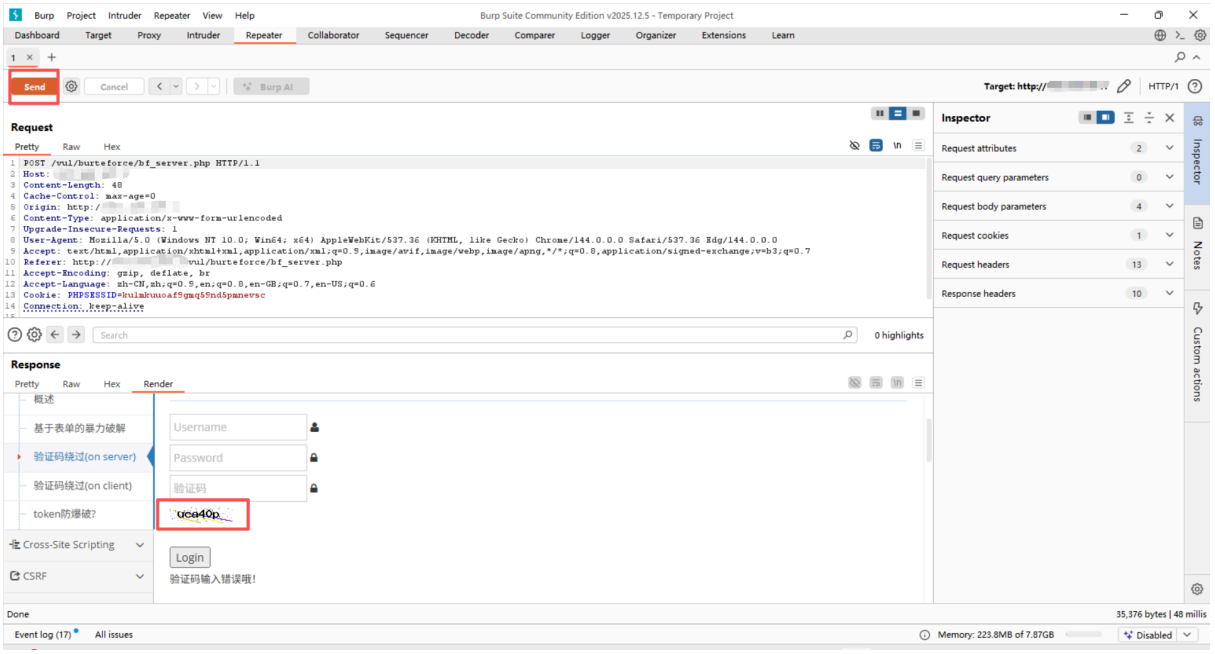
看最后一行，多出了一个vcode的数据，这就对应着验证码。看起来似乎无计可施了，不过我们还是多验证几次看看，把数据发送给Repeater工具。



Repeater（中继器）工具的核心作用是让你反复多次修改请求数据，并发送到服务器，在这个过程中你可以观察细节，查看反馈，尝试找到一些规律。如下图所示：



点击Send，查看Response-Render的前端反馈：



还是提示验证码输入错误，正常；但是有一个细节很奇怪，这个验证码好像在哪见过？

这不是偶然，多Send几次，发现每一次这个验证码都是一样的，没有发生丝毫变化——也就是说，反复向服务器发送请求之后，验证码实际上是不变的！

这是为什么呢？

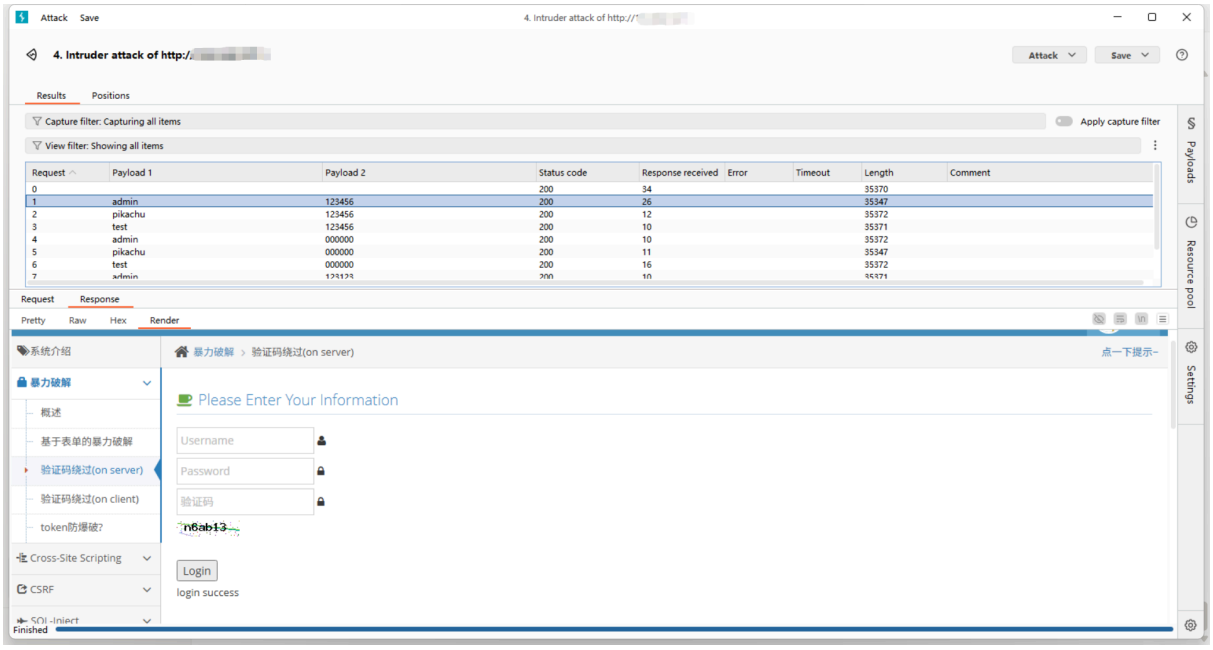
正常情况下，浏览器每次请求验证码图片，服务器就会生成新的验证码并存入数据库，用户输入验证码提交，服务器收到请求后比对数据库中的验证码，如果正确则放行。

但是，在这个情景下，显然是开发者贪图省事（bushi），服务器只在第一次生成了验证码，却没有在每次验证后更新数据库中的验证码！至于前端验证码图片发生的变化，不过是浏览器自行发生的行为，是“假”的，在没有刷新的情况下，服务器保存的仍然是初始生成的验证码！这就是典型的验证码绕过（服务器端验证码未更新）漏洞。

也就是说，只要第一次输入正确的验证码，后续就可以反复尝试暴力破解，而始终不变的验证码就被成功绕过了！

那么，就按照我们第一关暴力破解的逻辑进行操作即可，首先随意输入用户名和密码，正确输入验证码，抓包之后如下图所示：

然后，就是将数据发送给Intruder，替换掉用户名和密码（不要动验证码），采用"集束炸弹"的破解方法，就可以破解成功啦！



恭喜！第二关你也成功通过啦！

回顾一下，本关的标题"验证码绕过（on server）"实际上就指向了服务器端的漏洞。

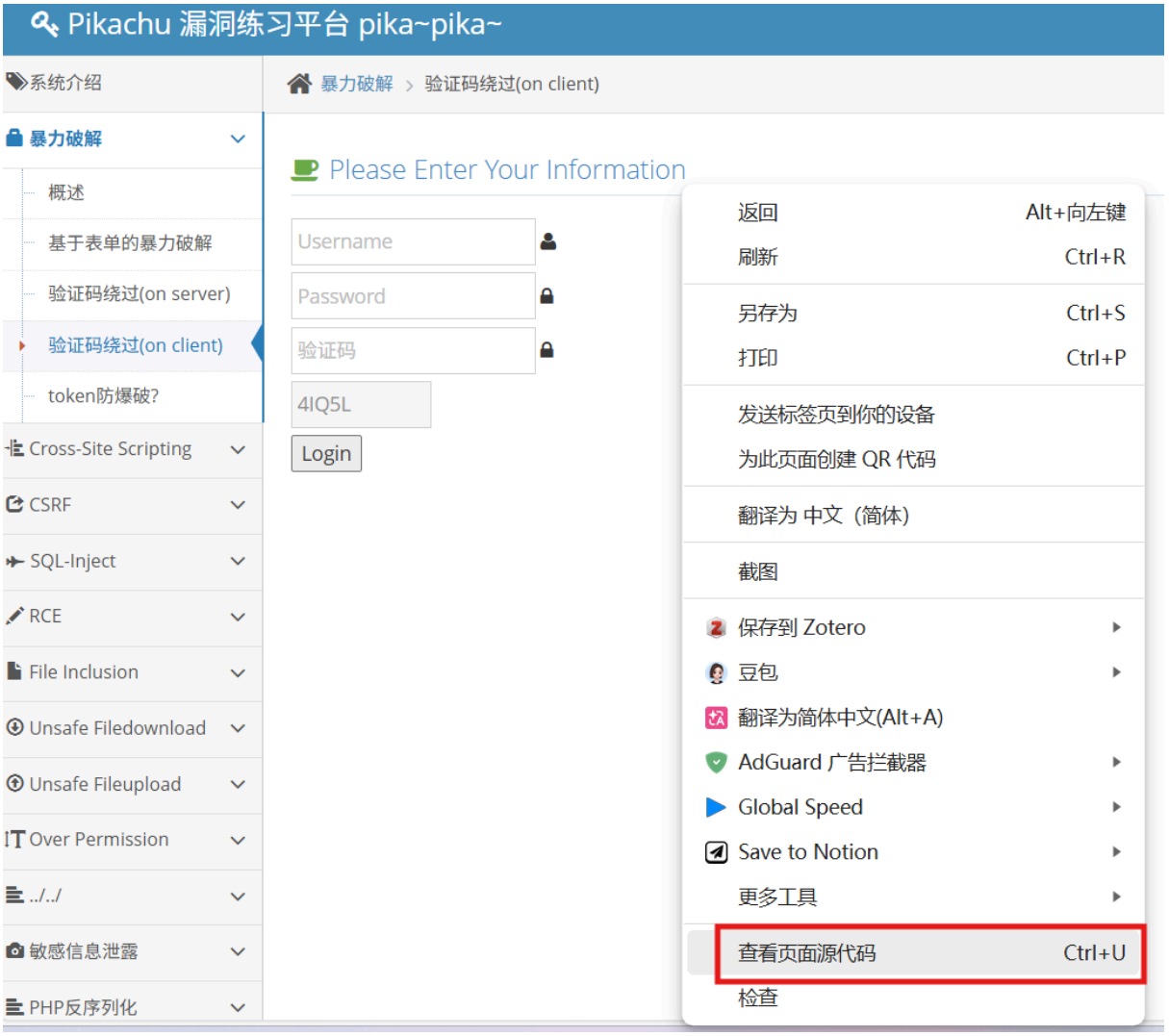
3、验证码绕过(on client)

来到第三题，首先我们可以关注到，这道题目的验证码样式和上一题有明显区别：

这个到底是个啥玩意儿？点击一下验证码就立马变换了，我们不妨还是先抓包一下看看。

奇怪，我们发现并没有抓取到任何数据，反而是前端跳出了一个弹窗提示——也就是说，验证码的校对环节并不存在与服务器的通信，而是完全在浏览器页面实现的；至于弹窗提示，则是JavaScript的典型特征。这说明是JavaScript代码负责验证码的校对！

我们可以查看前端源代码来验证这一判断，右键点击网页，点击查看页面源代码选项（我用的是Edge浏览器，其他浏览器都有类似的查看源代码方法）。



现在我们跳转到了一个新的页面，全是恼人的代码，我们采用最朴素的中文定位法，容易看到这样一段代码：

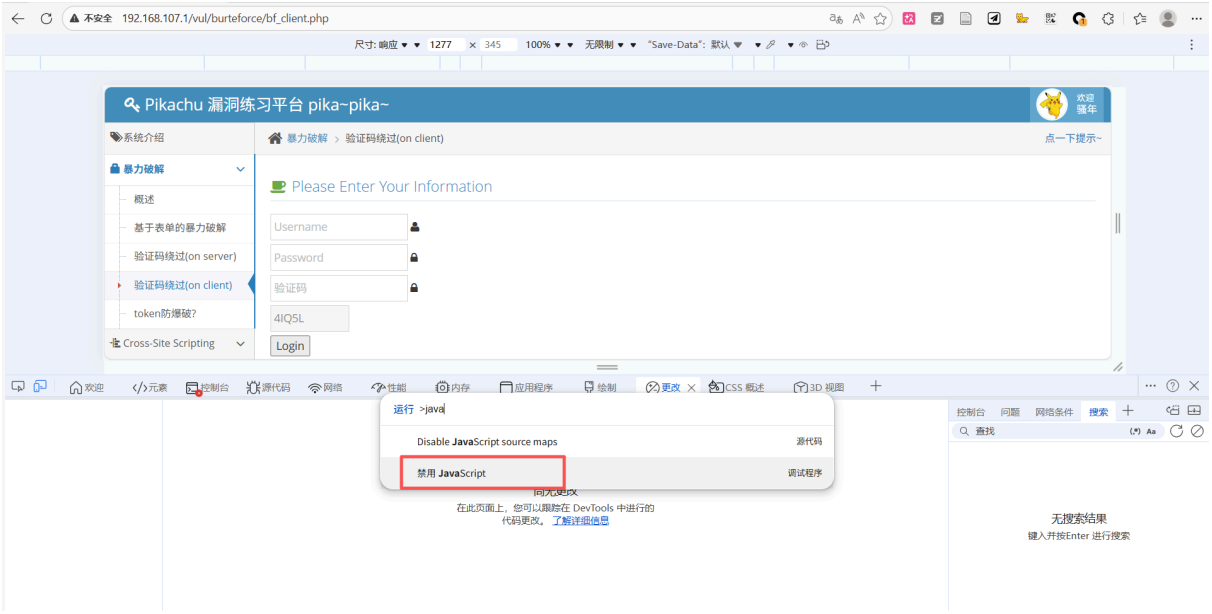
```
<script language="javascript" type="text/javascript">
    var code; //在全局 定义验证码
    function createCode() {
        code = "";
        var codeLength = 5;//验证码的长度
        var checkCode = document.getElementById("checkCod
e");
        var selectChar = new Array(0, 1, 2, 3, 4, 5, 6, 7,
8,
9,'A','B','C','D','E','F','G','H','I','J','K','L','M','N','O'
所有候选组成验证码的字符，当然也可以用中文的
```

```
        for (var i = 0; i < codeLength; i++) {
            var charIndex = Math.floor(Math.random() * 36);
            code += selectChar[charIndex];
        }
        //alert(code);
        if (checkCode) {
            checkCode.className = "code";
            checkCode.value = code;
        }
    }

    function validate() {
        var inputCode = document.querySelector('#bf_client
.vcode').value;
        if (inputCode.length <= 0) {
            alert("请输入验证码! ");
            return false;
        } else if (inputCode != code) {
            alert("验证码输入错误! ");
            createCode();//刷新验证码
            return false;
        }
        else {
            return true;
        }
    }


    createCode();
</script>
```

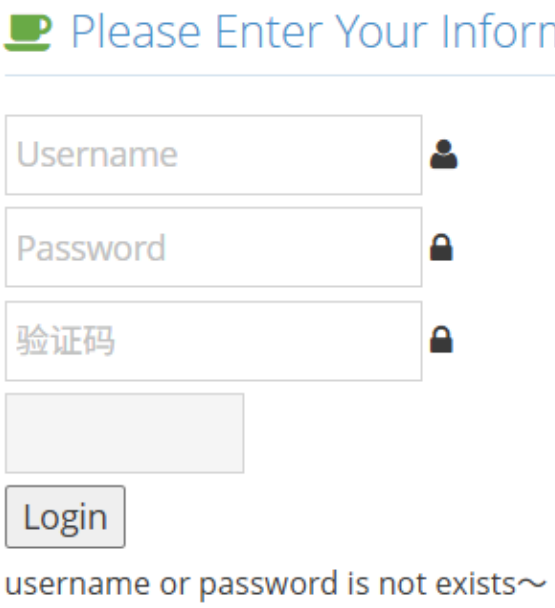
看不懂？那也没关系，毕竟代码的中文很丰富（doge）——我们不难发现这段代码本质上就是起到一个生成验证码和校对验证码的功能，并通过跳出弹窗的方式进行提示。

也就是说，我们的判断没有错，所有的验证码功能都通过网页前端的JavaScript代码实现，而我们知道浏览器本身就有一个关闭JavaScript的选项，那么关闭后会发生什么呢?

在Edge浏览器中，你可以通过“Ctrl+Shift+I”快捷键打开开发者工具，“Ctrl+Shift+P”快捷键打开命令菜单，输入JavaScript即可查找到禁用JavaScript的命令（此方法仅在当前页面起作用）：

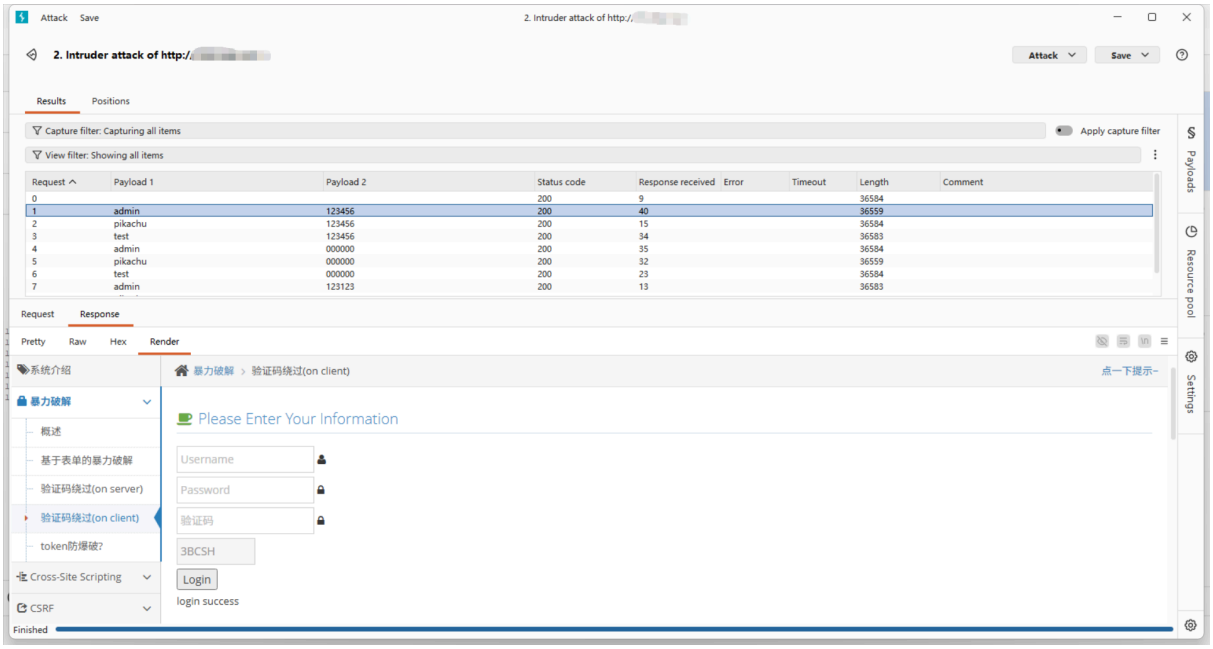


保持禁用JavaScript，我们这次不输入验证码试试看：



这次的报错是用户名或密码错误，没有再出现弹窗提示“验证码为空”，说明禁用JavaScript成功绕过了验证码，本题也就转换为了暴力破解第一题。

禁用JavaScript之后，按照第一题的破解思路，很容易得到正确的用户名/密码对：



解完题目后，我们可以对照一下上一题，对上一题我们也查看页面源代码，不难找到下面这段代码：

```
                <label>
                        <span>
                            <input type="text" name="vcode"
placeholder="验证码" />
                            <i class="ace-icon fa fa-lock">
</i>
                        </span>
                </label>
                </br>
                <label>
                    <img src="../../inc/showvcode.php" oncl
ick="this.src='../../inc/showvcode.php?'+new Date().getTime
();" />
                </label>
```

我们不需要特别懂这段代码，但是其中一串代码有一个鲜明的img（图片）标识——而上一题的验证码正是图片形式——这个img指向了一个php文件，但这个php文件不存在于前端，那么就必然是后端服务器的代码。因此，这个验证码便是通过服务器生成后提供给前端网页的。

而除了这串代码之外，我们就再也找不到其他关于验证码的逻辑，也就是说，上一题的验证码生成与浏览器无关，这就是这两道题目的区别！

4、token防爆破?

来到第四题，倒是颇有一种返璞归真的感觉——这不就和第一题一样吗?

但是，看着这道题目的标题，理智告诉我们，绝对没有这么简单！

不管那么多了，先抓包一下看看：

这次抓包返回的请求数据好像不太一样，最下面一行除了用户名和密码，还多了一个"token"的值?

如果重新抓包的话，我们会发现token的值和第一次相比发生了变化，也就是说，每次登录都会有一个新的token？那么这个token是干什么用的呢？

百度一下，我们了解到，token也是一种验证机制，相当于你在数据世界里的一张临时身份证，每次登录之后，服务器就会给你发放一个token值，如果后续再次登录时，token验证无误，你就无需再次输入用户名和密码进行验证。

但是在这里，token却成了我们的阻碍，因为我们根本就登不进去——如果我们还是像第一题一样不管token进行暴力破解，那么每次登录就会由于已经生成了新的token而旧的token过期无效导致验证失败，重复就变得毫无意义了。

怎么办？只有获取到新的token，而且是在每次登录时都获取到这个新的token，把它作为新的token输入，才能规避token过期无效的问题——在此基础上的暴力破解才有意义。

那么，如何做到每次登录时就更新一遍token值呢？遗憾的是，社区版的BurpSuite似乎不具备这个功能，而网上的教程基本上都是围绕专业版BupSuite进行解题的。

这里，就要用到Python来打辅助了。

在编写Python脚本之前，我们首先需要了解网页源代码，看看token到底是如何生成的，以便在脚本中准确抓取到每次登陆的token值。

还是按照第三题的方法，查看网页源代码：

```
<div class="bf_form">
    <div class="bf_form_main">
        <h4 class="header blue lighter bigger">
            <i class="ace-icon fa fa-coffee green"></i>
            Please Enter Your Information
        </h4>

        <form id="bf_client" method="post" action="bf_toke
n.php" ">
<!--            <fieldset>-->
                <label>
                    <span>
                        <input type="text" name="username"
placeholder="Username" />
                        <i class="ace-icon fa fa-user"></i>
                    </span>
                </label>
                </br>
```

```
                <label>
                    <span>
                        <input type="password" name="passwo
rd" placeholder="Password" />
                        <i class="ace-icon fa fa-lock"></i>
                    </span>
                </label>
                </br>

                <input type="hidden" name="token" value="87
44369906c0000f3b695591858" />

                <label><input class="submit"  name="submit"
type="submit" value="Login" /></label>

        </form>


    </div><!-- /.widget-main -->

</div><!-- /.widget-body -->
```

我们看到，这里和username、password并列的就是token值，这里type="hidden"说明这个值默认隐藏，不会显示在网页上，也就是你看不到，但是代码里面是会动态生成的。

根据token处代码格式特征，我们可以通过python的正则表达式提取到token：

```
token_match = re.search(r'<input\s+type="hidden"\s+name="to
ken"\s+value="([^"]*)"\s*/?>', response.text, re.IGNORECAS
E)
if token_match:
    return token_match.group(1)
```

这段正则表达式属于Python的相关知识，此处就不必过多介绍了。我们只需要知道，根据页面源代码，我们可以观察到token的分布特点，然后针对性地设计代码捕获token，并将其更新为下一次登录的token值。

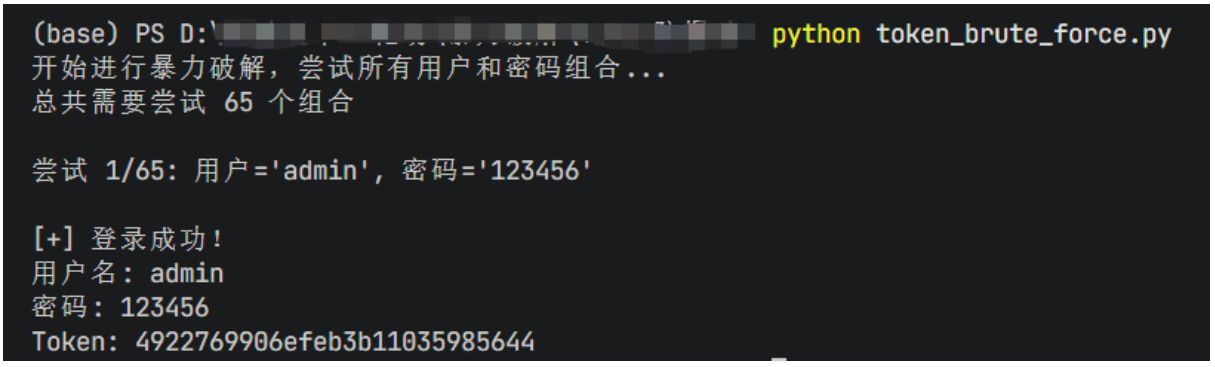
既然都做到这一步了，我们就可以完整地通过Python脚本来实现暴力破解了，完整代码如下：

```
import requests
import re

# 目标URL
target_url = "http://192.XXX.X.X/vul/burteforce/bf_token.php"#替换为实际URL

# 用户名和密码字典
usernames = ["admin", "test", "user", "root", "pikachu"]
passwords = ["123456", "admin", "password", "root", "12345678", "123456789", "1234", "12345", "123", "1", "test", "1234567", "1234567890"]

# 会话对象，保持cookie
session = requests.Session()

def get_token():
    """获取登录页面的token"""
    response = session.get(target_url)
    if response.status_code == 200:
        # 使用更宽松的正则表达式提取token，忽略空格和大小写
        token_match = re.search(r'<input\s+type="hidden"\s+name="token"\s+value="([^"]*)"\s*/?>', response.text, re.IGNORECASE)
        if token_match:
            return token_match.group(1)
    return None

def brute_force():
    """执行暴力破解"""
    print("开始进行暴力破解，尝试所有用户和密码组合...")
    total_attempts = len(usernames) * len(passwords)
    print(f"总共需要尝试 {total_attempts} 个组合")

    attempt_count = 0
```

```
    for username in usernames:
        for password in passwords:
            attempt_count += 1
            print(f"\n尝试 {attempt_count}/{total_attempt
s}: 用户='{username}', 密码='{password}'")

            # 获取新的token
            token = get_token()
            if not token:
                print("[-] 获取token失败，跳过此次尝试")
                continue

            # 构造登录数据
            data = {
                "username": username,
                "password": password,
                "token": token,
                "submit": "Login"
            }

            # 发送登录请求
            response = session.post(target_url, data=data)

            # 检查登录是否成功
            if len(response.text) > 34000:  # 登录成功后页面
会变得很长
                print(f"\n[+] 登录成功! ")
                print(f"用户名: {username}")
                print(f"密码: {password}")
                print(f"Token: {token}")
                return True
            elif "密码错误" in response.text:
                print("[-] 密码错误")
            elif "用户不存在" in response.text:
                print("[-] 用户不存在")
            else:
                print("[-] 尝试失败")
```

```
        print("\n[-] 暴力破解完成，未找到正确的用户名和密码组合")
        return False

if __name__ == "__main__":
    brute_force()
```

运行这个脚本，终端返回结果：



好！第一次尝试就成功了（doge）！

至此，暴力破解部分的四道大题你就全部通关！我们即将迎来一片崭新的大陆！

三、Cross-Site Scripting

老规矩，先摘录一下皮卡丘靶场关于此篇章的概述：

> **XSS（跨站脚本）概述**
>
> Cross-Site Scripting 简称为“CSS”，为避免与前端叠成样式表的缩写"CSS"冲突，故又称XSS。一般XSS可以分为如下几种常见类型：
>
> 1.反射性XSS;
>
> 2.存储型XSS;
>
> 3.DOM型XSS;
>
> XSS漏洞一直被评估为web漏洞中危害较大的漏洞，在OWASP TOP10的排名中一直属于前三的江湖地位。XSS是一种发生在前端浏览器端的漏洞，所以其危害的对象也是前端用户。形成XSS漏洞的主要原因是程序对输入和输出没有做合适的处理，导致“精心构造”的字符输出在前端时被浏览器当作有效代码解析执行从而产生

> 危害。因此在XSS漏洞的防范上，一般会采用“对输入进行过滤”和“输出进行转义”的方式进行处理：输入过滤：对输入进行过滤，不允许可能导致XSS攻击的字符输入；输出转义：根据输出点的位置对输出到前端的内容进行适当转义;你可以通过“Cross-Site Scripting”对应的测试栏目，来进一步的了解该漏洞。

在正式爆破之前，我们首先需要了解一下XSS攻击的实质——上面的概述提到“‘精心构造’的字符输出在前端时被浏览器当作有效代码解析执行”，对于懂行的人来说当然很容易理解，但是似乎还是有点空，我们不妨把靶场中XSS的第一题作为一个例子。

1、反射型xss(get)

我们随便输入一串数字，抓包一下，看看发送到服务器的请求是什么：

BurpSuite抓取到的请求如下：

可以看到，点击submit之后，浏览器向服务器发送了请求，指向的是`xss_reflected_get.php`这一个文件。由于皮卡丘靶场是本地部署的，所以我们可以找到这个文件，看看源代码，其中我们定位到这样一段代码：

```
if(isset($_GET['submit'])){
    if(empty($_GET['message'])){
        $html.="<p class='notice'>输入'kobe'试试-_-</p>";
    }else{
        if($_GET['message']=='kobe'){
            // 这里存在XSS漏洞！！！
            $html.="<p class='notice'>愿你和{$_GET['messag
e']}一样，永远年轻，永远热血沸腾！</p><img src='{$PIKA_ROOT_DIR}
assets/images/nbaplayer/kobe.png' />";
        }else{
            // 这里也存在XSS漏洞！！！
            $html.="<p class='notice'>who is {$_GET['messag
e']},i don't care!</p>";
        }
    }
}
```

当输入是"kobe"时，输出为：

```
<p class='notice'>愿你和kobe一样，永远年轻，永远热血沸腾！</p>
```

当输入不是"kobe"时，比如输入"詹姆斯"，输出为：

```
$html.="<p class='notice'>who is 詹姆斯},i don't care!</p>"
```

可以看到，`{$_GET['message']}`实现的功能就是一个简单的替换，它不管你输入了什么，都会完整地把输入的内容替换到这个语句中去。但是，如果我们输入一段特殊的构造代码，如`<script>alert('XSS')</script>`，那么会发生什么呢？

```
// $_GET['message'] = "<script>alert('XSS')</script>"
// 因为不是"kobe"，所以进入else分支
$html .= "<p class='notice'>who is {$_GET['message']},i do
n't care!</p>"
// 实际拼接后：
```

```
$html .= "<p class='notice'>who is <script>alert('XSS')</sc
ript>,i don't care!</p>"
```

也就是说，最终输出为：

```
<p class='notice'>who is <script>alert('XSS')</script>,i do
n't care!</p>
```

但是，在浏览器解析的过程中，它不会把`<p class='notice'>who is <script>alert('XSS')</script>,i don't care!</p>`当成普通文本一样输出，而是看到：

```
1. <p>标签开始
2. 文本："who is "
3. <script>标签开始
4. JavaScript代码："alert('XSS')"
5. </script>标签结束
6. 文本：",i don't care!"
7. </p>标签结束
```

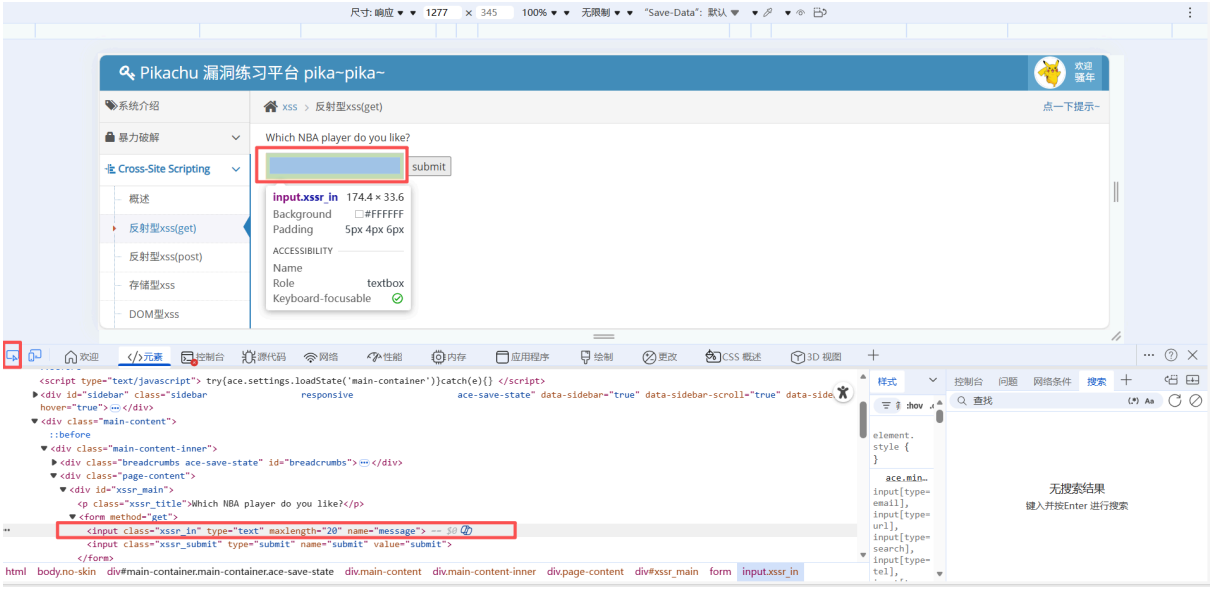
也就是说，原本的输入文本由于具有<script>标签，而被浏览器解析为了一个命令！而这个命令就是两个<script>之间的代码"alert('XSS')"，即跳出一个弹窗，内容为"XSS"！

一个弹窗当然不值得大惊小怪，但是关键在于<script>之间的内容是可以自定义的——我们可以开个玩笑一般，只是让它跳出一个弹窗，当然也可以做出诸如向服务器窃取用户cookie，篡改页面内容，攻击内网等等具有极大破坏性的操作！事实上，2005年MySpace Samy蠕虫、2018年英国航空公司数据泄露等重大事件全是黑客利用XSS漏洞导致的重大网络风险事件！

大致了解了XSS的原理，我们知道，在这里我们需要做的和前一章的暴力破解完全不同——暴力破解本质上是猜解密码，存在于前后端通信的过程中，所以抓包是首要的一步，登录即成功；而XSS本质上是构造，而且主要是在浏览器的格式解析中发生的，因此核心就是理解浏览器的代码结构，构造出特定的输入，引起异常反应即成功——当然，皮卡丘靶场是本地部署在我们自己的电脑上的，咱们就老老实实让浏览器跳出弹窗就算成功。

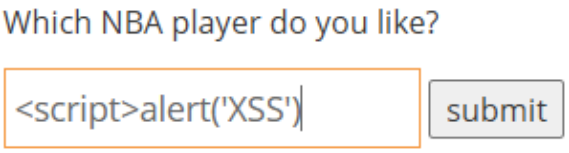
现在，我们可以正式看看第一题了。

我们已经知道XSS的核心是了解代码结构，那么和BurpSuite就没多大关系了，主要利用的是浏览器自带的开发者工具。我们通过快捷键"Ctrl+Shift+I"打开Edge的开发者工具面板，使用"选择元素"工具定位到输入框，可以看到如下图所示的一行代码：



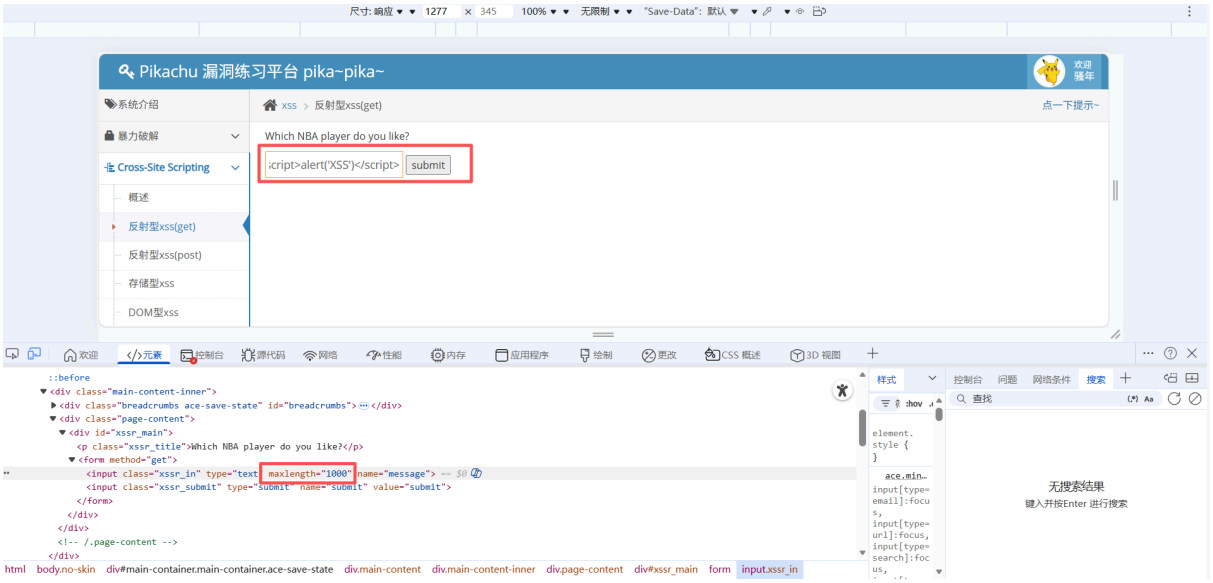

```
<input class="xssr_in" type="text" maxlength="20" name="mes
sage" />
```

这行代码是典型的html语言，它告诉我们。这个输入框名字叫做"message"，输入长度最大为20，最终将会将数据发送给服务器后端。我们就按一般的方法构造即可，在输入框输入"<script>alert('XSS')</script>"：

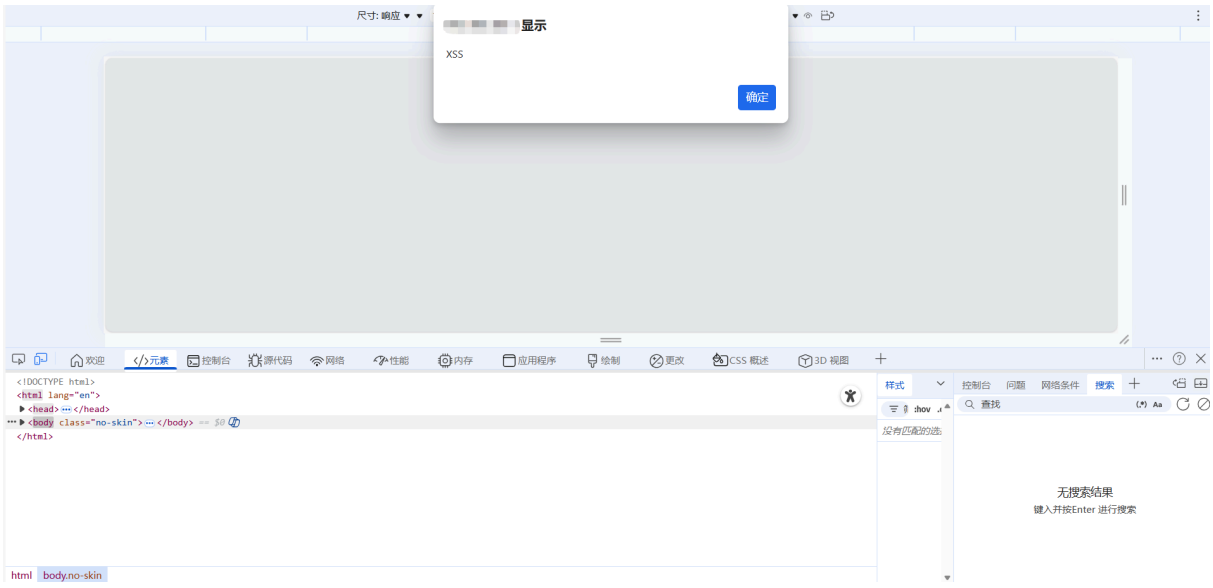


咦？输入到这里怎么输入不了了？哦，对了，输入有最大长度限制！刚才我们看到的代码maxlength="20"，就是用来约束长度的！

不过，只要是浏览器前端代码，我们实际上都是可以直接更改的（想当初直接把校园网通报表扬页面里提到的人名全都改成了同班同学，拿到班级群里耍，笑死了）。通过开发者工具，直接把maxlength的数值改成1000，这样就可以输入完整的命令了：



点击submit，就可以看到弹窗跳了出来，说明我们构造成功了：



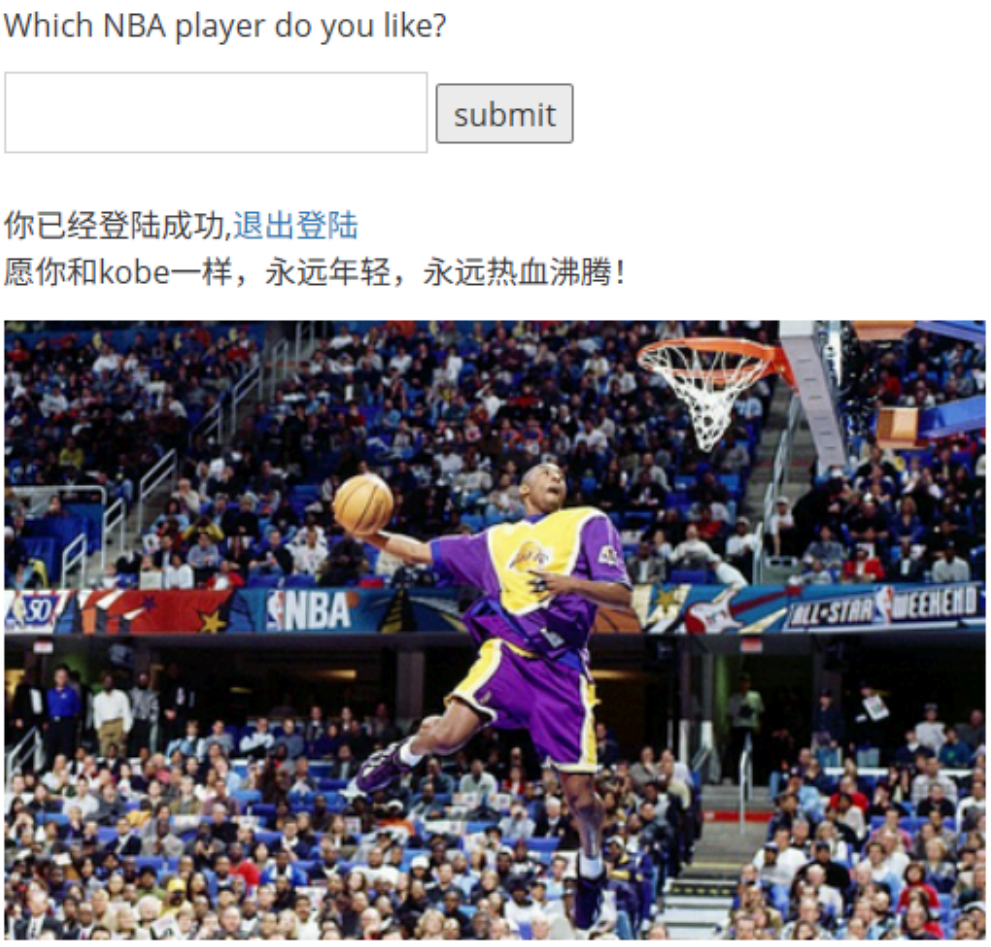
恭喜！我们成功通关了第一个XSS漏洞！不过回过神来，为什么这道题目叫做”反射型xss(get)“呢？下一题又叫做”反射型xss(post)“，那还是一起看看再说吧！

2、反射性xss(post)

一上来，我们就看到要先进行登录验证，这就和上一篇章的暴力破解一样啊！但是我们很懒，不想再暴力破解一次了，索性点击提示看看：

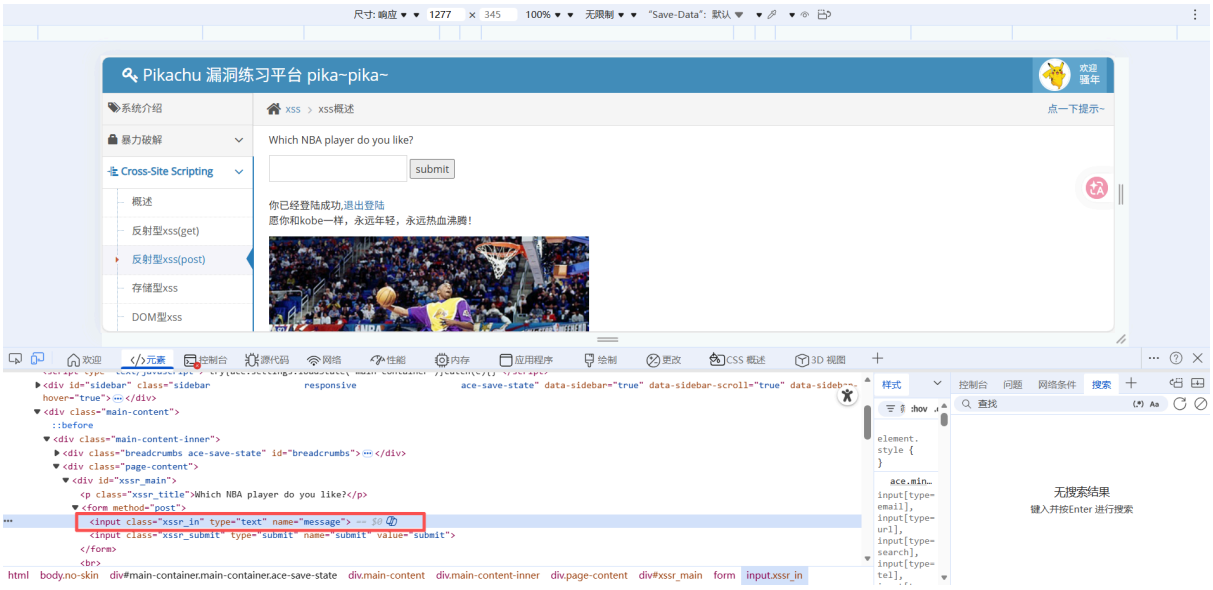
这个开发者人真好啊！那咱们直接登录。

还是一样的提交页面，那么还是输入"kobe"看一下：

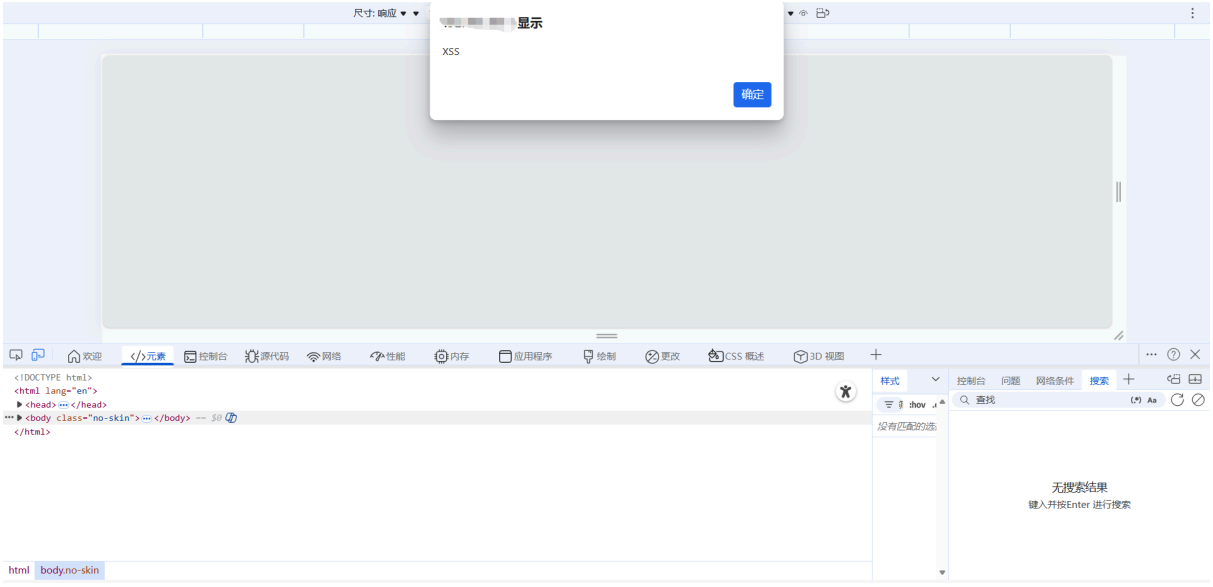


嗯，不禁热血沸腾起来了！谢谢牢大！

按照上一题的思路，我们还是先用开发者工具看一下：



这次没有长度限制，不妨还是先试试看直接构造，输入：`<script>alert('XSS')</script>`，看看效果：



还是一样，弹窗正常跳出了，看起来和上一题差不多?

不过我们还是能观察到一点细节，看看submit之后的网址呢?

```
http://XXX.XXX.XXX.X/vul/xss/xsspost/xss_reflected_post.php#
```

好像和上一题不太一样，上一题网址是什么来着？我们回顾一下，重新输入提交一下：

```
http://XXX.XXX.XXX.X/vul/xss/xss_reflected_get.php?message=123&submit=submit
```

确实不一样——上一题是get类型，网址里面就已经包含了输入的信息，看来是把整个网址作为输入传输给后端服务器的；但是这一题是post类型，输入之后网址没有发生任何变化，应该是通过一个接口把数据传递给服务器？

很有可能，我们抓包一下看看：

没有错，最后一行有我们输入的信息，这个叫做请求体（我也是刚刚才知道），看来数据是这样发送给服务器的。

那么上一题呢？我们还是抓包：

显然，这里就没有请求体的位置了，反而是最上面Get的那一行包含了我们输入的信息。

咦？这个Get不就是题目标题里提到的吗，以及上一题抓包数据里面最上面一行的Post，也是对应着上一题的标题！

一切都连起来了！

我们可以百度一下二者的区别，得到下面这个表格：

维度	GET型XSS	POST型XSS
数据传输	URL参数中	请求体中

维度	GET型XSS	POST型XSS
可见性	地址栏可见	地址栏不可见
攻击方式	直接发链接	需要构造表单/配合CSRF
攻击难度	低	中高
真实场景	搜索框、URL参数	留言板、登录表单、评论
长度限制	有（URL限制）	无

所以说，理论上我们可以通过大量发送精心构造的链接来实现GET型XSS攻击（2005年的Samy蠕虫就是一个典型案例）；而POST型和GET型相比，多出了一个登录页面，攻击起来要麻烦许多，不过一旦成功登录，XSS的构造都是类似的——无论数据怎么传过来，只要后端把数据输出到页面，浏览器都会执行其中的代码。

3、存储型xss

先看看题目：

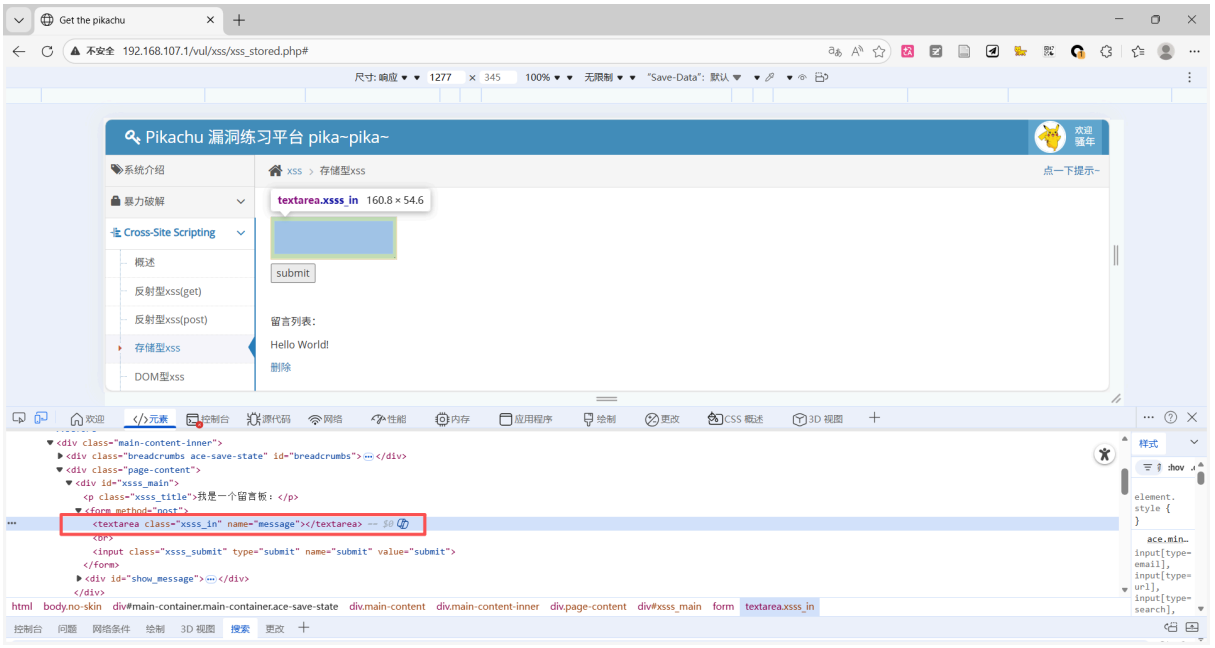
好像和前两题有一点区别，下方多了一个留言列表，这个是干什么用的呢？联系一下本题的标题，“存储型xss”，这个大概率就是用来存储输入的信息的。咱们先试试看！

点击submit之后，下方留言列表更新了我输入的信息，看来猜测没有错，本题将留言存储在了数据库中（具体怎么实现在这里倒不用太在意），并呈现在网页上。

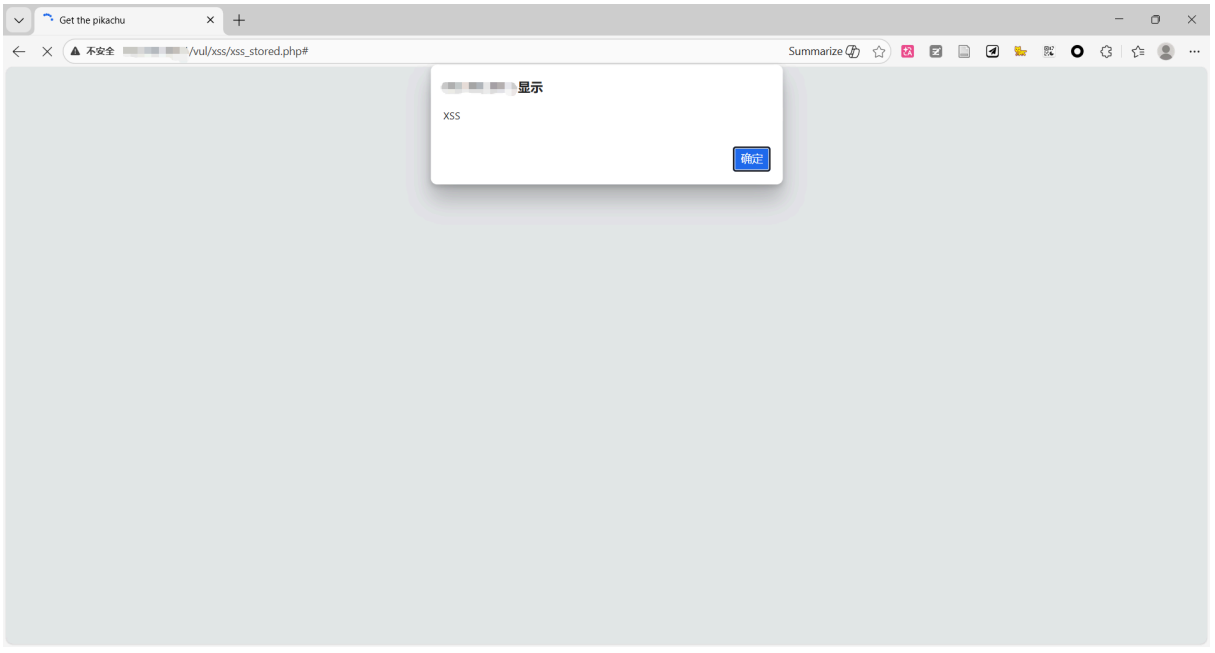
回顾一下前两题，都是反射型xss，有很明显的区分。通过百度，我们知道所谓“反射型”就是将输入的信息处理后直接“反弹”回来，而不会存储在数据库中，浏览器对你的输入是没有记忆的，是“阅后即焚”的。

那么，在这种情况下，我们构造的输入会产生什么样的效果呢？

还是先通过开发者工具看一下输入框的代码，看看有没有什么约束条件：

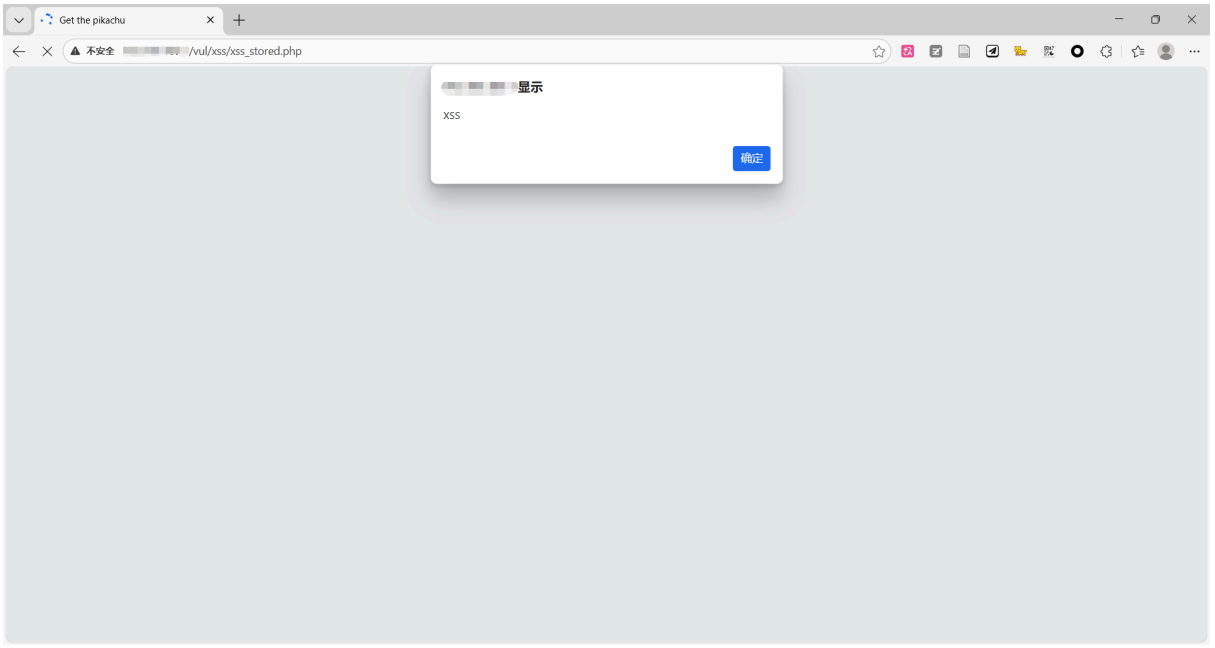


前端代码没有任何关于输入的约束，那我们还是输入`<script>alert('XSS')</script>`看看：



浏览器的响应很符合我们的预期，跳出了一个弹窗。但是别忘了这一次我们的输入已经被存储到了数据库中，那么每一次刷新浏览器的时候，数据库中的数据都会被重新加载一次，那么`alert('XSS')`也会被浏览器重新执行。所以我们预计，刷新之后，什么也不用输入，页面还是会跳出这样的一个弹窗！

我们测试一下：



页面刷新之后，果然再次出现了这个弹窗，我们点击“确定”：

关注一下留言列表，多了一个“删除”按钮，但是却没有与之对应的内容。为什么呢？别忘了我们输入的是 `<script>alert('XSS')</script>`，<script>是标签，alert('XSS')是命令，会直接被浏览器执行，而不会被作为内容存储，而是作为命令存储，所以在网

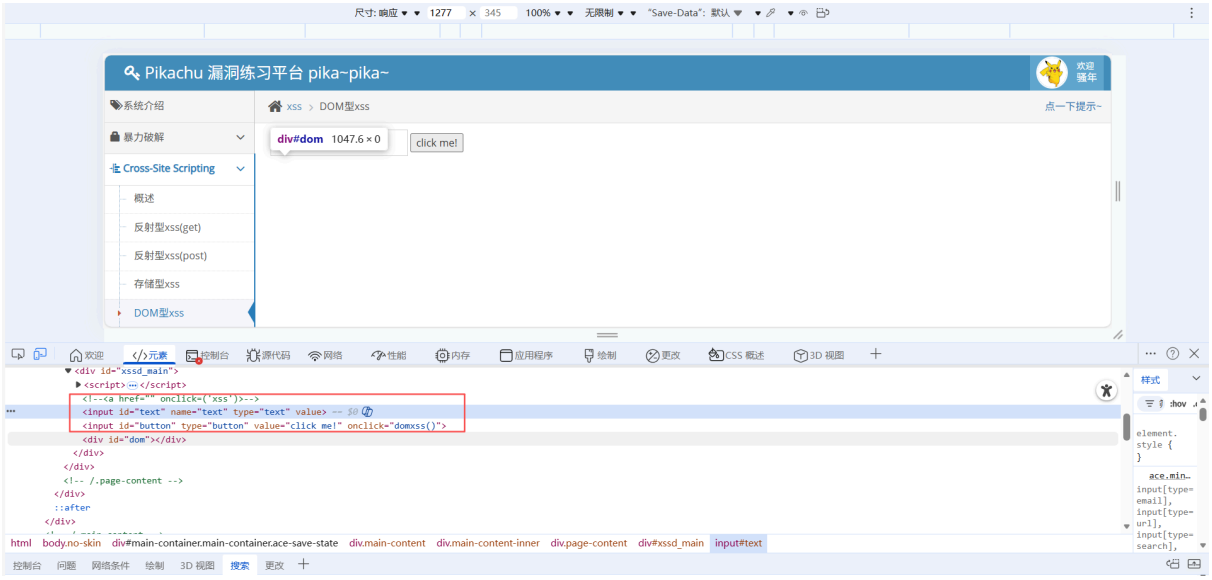
页上列表这一项不会显示任何东西，自然是空的。不过由于我们submit了一次，所以列表还是会更新一条内容，自然就只剩下一个删除选项了。

当然，这个删除是有用的，点击删除之后，我们先前输入的命令被删除；因而重新刷新页面，就不会再出现弹窗了。

我们不妨发散一下，如果攻击者采用存储型XSS进行攻击（比如<script>之间是获取用户cookie的命令），而这个删除按钮并不对用户开放，那么每次用户访问这个网页，浏览器都会执行一次构造出来的命令，收集访客的cookie——一次注入，永久生效，无差别攻击所有访客！

4、DOM型xss

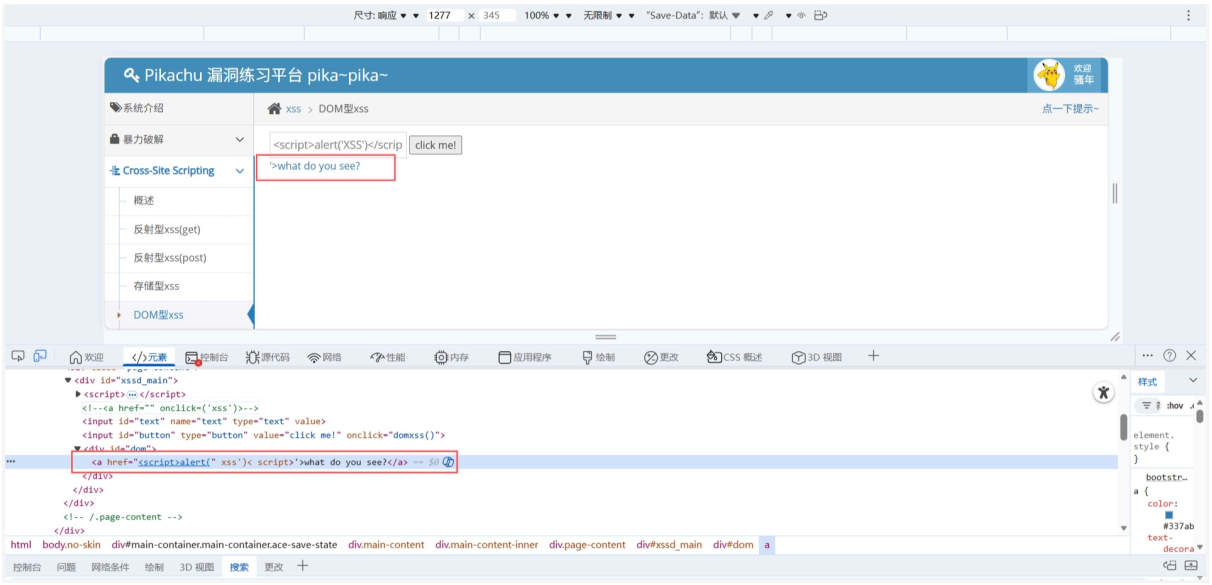
看下一题，光看页面似乎还是老样子，我们还是先打开开发者工具看一下输入框对应的代码：



看起来输入框代码和先前的类似，但是我们发现上方有一行注释代码我们之前没见过：

```
<!--<a href="" onclick=('xss')>-->
```

现在我们先不用考虑那么多，不妨还是先试试看输入`<script>alert('XSS')</script>`：

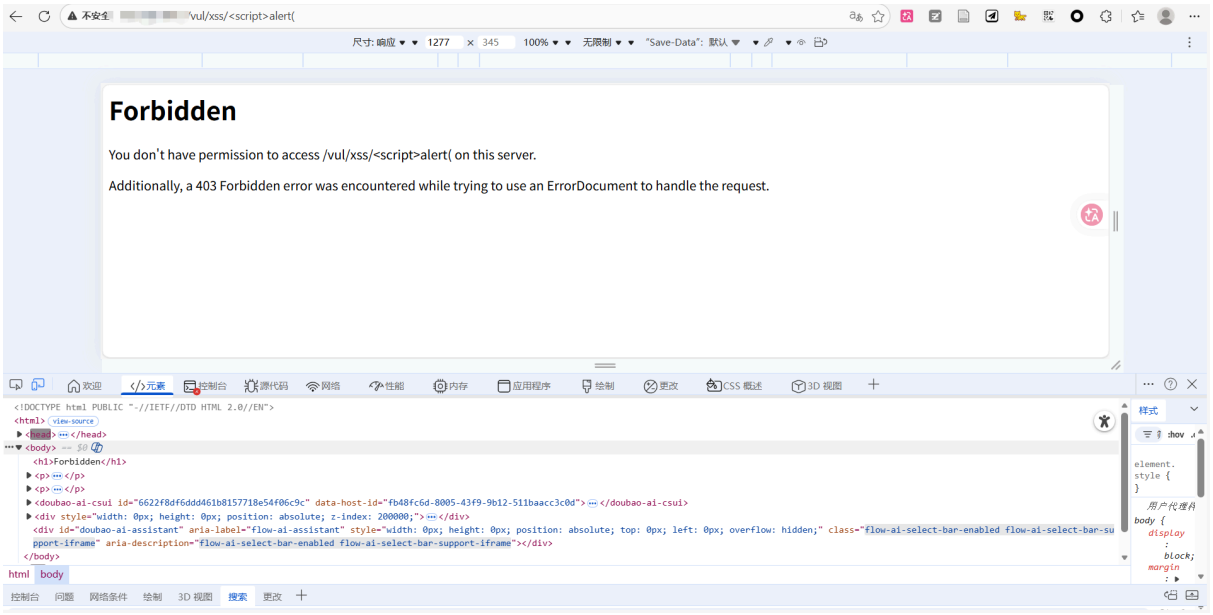


这一次没有直接触发弹窗，反而是出现了一行文字“what do you see”。我们使用开发者工具选中这个元素，发现其对应这样一行代码：

```
<a href="&lt;script&gt;alert(" xss')<="" script="">'&gt;wha
t do you see?</a>
```

又出现了这个”a”？这个问题不大，百度一下我们知道，这个就是<a>标签——<a>标签就是网页里的“传送门”——用户一点它，就能跳到另一个地方（另一个页面、另一个位置，或者执行一段代码）。不过，这一次我们输入的文本似乎被处理过，中间多出了一些东西？

尽管心怀疑惑，但是我们发现下面”what do you see”似乎是可以点击的，那当然要点一下！



这是什么情况？跳转到了一个新的页面，和我们的预期完全不符。但别慌，我们仔细看看这个页面的内容。

首先是网址中包含的 `/vul/xss/<script>alert(`，vul/xss是文件位置（因为皮卡丘靶场是本地部署的），这个无需在意，但是末尾的“<script>alert”却是一个命令，之前我们就知道，这是一个弹窗命令——这个命令之前明明是在<script>里面，怎么现在跑到网址里面去了？

再看网页的提示，大概是说我们没有权限访问alert服务，什么叫没有权限呢？

这里，我们需要从头理一下了——这无非说明我们的构造失败了，但是为什么失败？

最初，我们输入的是 `<script>alert('XSS')</script>`，但是对照一下开发者工具里显示的代码，最终在开发工具里这个命令却被处理成了 `<script>alert(" xss')<="" script`，中间由于被加入了特殊字符（多了‘，<，=，以及两个“），实际上这个命令已经被截断了，已经被无害化处理了。从最早多出来的那个单引号开始截断，这个命令实际上只剩下了：

```
<script>alert('
```

此后，“`<a href="<script>alert(" xss')<="" script="">'>what do you see?</a>`”告诉我们这是一个<a>标签，其中包含着被截断的输入内容——也就是说浏览器将我们的输入转换成了<a>标签what do you see，点击它之后，就会访问问 `/vul/xss/<script>alert(` 链接！而这个链接当然会访问失败，因为根本就不存在这样一个目录！

所以，本质上来说是我们构造失败了，这里的代码设计显然考虑到了XSS攻击中<script>的构造方法！因此我们就必须要转换思路了。

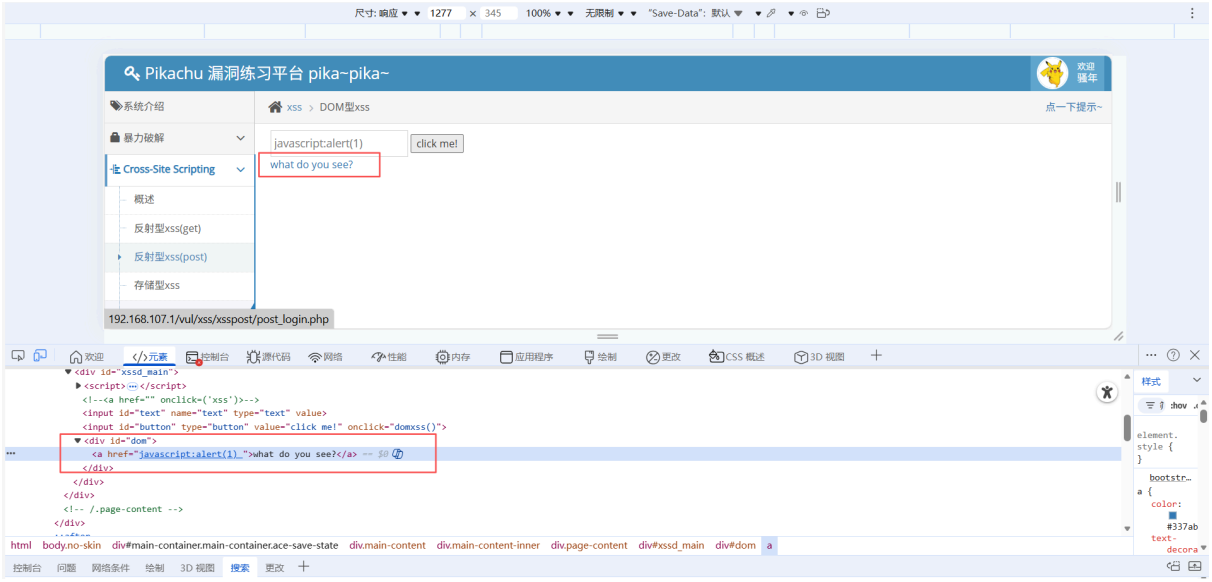
那么除了<script>的构造方法，还有什么办法呢?

这里，我们需要重新理解一下<script>构造方法的本质了——<script>本质上一个标签，只要在执行命令中出现这个标签，浏览器就会自动执行<script>之间的命令。因此，一旦这个标签被送达浏览器，之后的过程就与用户无关了。

实际上，浏览器本身就有三套执行JavaScript的机制，第一种是标签型，特点是只要出现就自动执行，以 `<script>` 标签为代表，这也是我们先前的构造方法；第二种是事件型，特点是不自动执行，只有触发后才会执行，这种机制以 `on click` 、 `on error` 等命令为代表；第三种是伪协议型，特点是点击链接时才被执行，以 `javascript:` 为代表。

在这一题中，我们已经通过测试了解到了命令的触发方式——首先是将命令存储在<a>标签中，然后用户点击<a>标签，跳转到对应的网页从而执行命令。那么，很明显这与第三种执行机制相符，所以，我们优先尝试一下 `javascript:` 试试看。

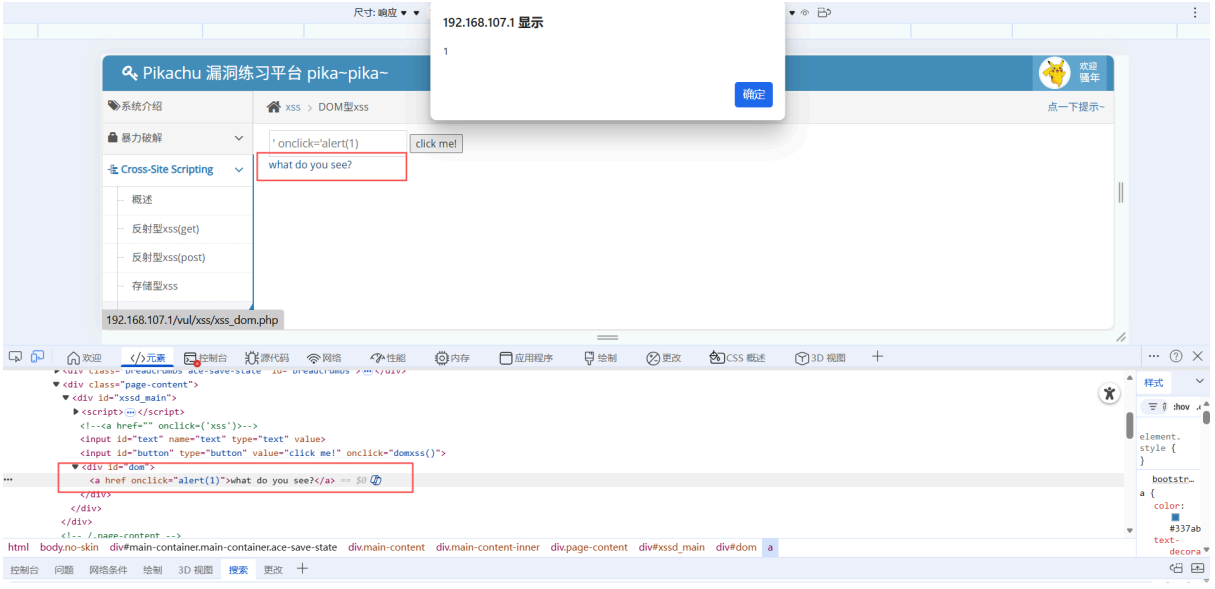
和 `<script>` 构造方法类似， `javascript:` 后面接的就是我们需要执行的命令，不妨构造为： `javascript:alert(1)` ：



可以通过开发者工具发现，现在的<a>标签内含的内容就是 `javascript:alert(1)` ，与我们的输入一致，说明这里并没有设计应对此类XSS攻击的方法，咱们十有八九已经成功了。

现在点击 `what do you see` ，发现正常跳出了弹窗，与我们的预期一致，攻击成功!

不过我们自然也能想到，这里命令的执行既是通过访问链接实现的，也同样是通过点击实现的。我们不妨再尝试一下事件型的XSS攻击，输入点击触发的 `'onclick='alert(1)` ，点击生成的<a>标签，页面仍然跳出了弹窗，也成功了。



这里做一个补充（是我写到后面时发现的）：注意 `'onclick='alert(1)` 的引号，实际上是为了构造——当内容被拼接到href中时，会自动在两端添加引号来“引用”内容，会导致很多构造无效化。因此此处输入也要有引号，从而与新增的引号拼接形成“空”，抵消引号的影响。

但是为什么 `javascript:alert(1)` 不需要引号？因为这种构造本身就是伪协议型，引号本身就是其构造的一部分——换言之，它就是专门为了应对这种“自动添加引号”的机制而存在的！

最后，我们还要再试着用BurpSuite抓包一次，发现没有抓到任何请求，所以整个过程中完全是在浏览器中实现的，并不存在前后端通信的过程。

最后，我们回顾一下本题的标题，就大概能理解什么叫做DOM型xss了——浏览器把网页转换成的一棵“树”——每个标签（如 、<a>，此处是<a>标签）都是树上的一个节点，JavaScript可以随意修改这棵树（增、删、改节点），而这些修改完全在本地发生，不需要告诉服务器。因此，攻击者利用这种机制，让JavaScript把恶意代码作为新节点插入DOM树（比如本题就是写进<a>标签的href里），用户点击时代码就在本地执行——整个过程不经过服务器，所以我们抓不到任何请求数据。

那么，下一题的DOM型xss-x又有什么变化呢？

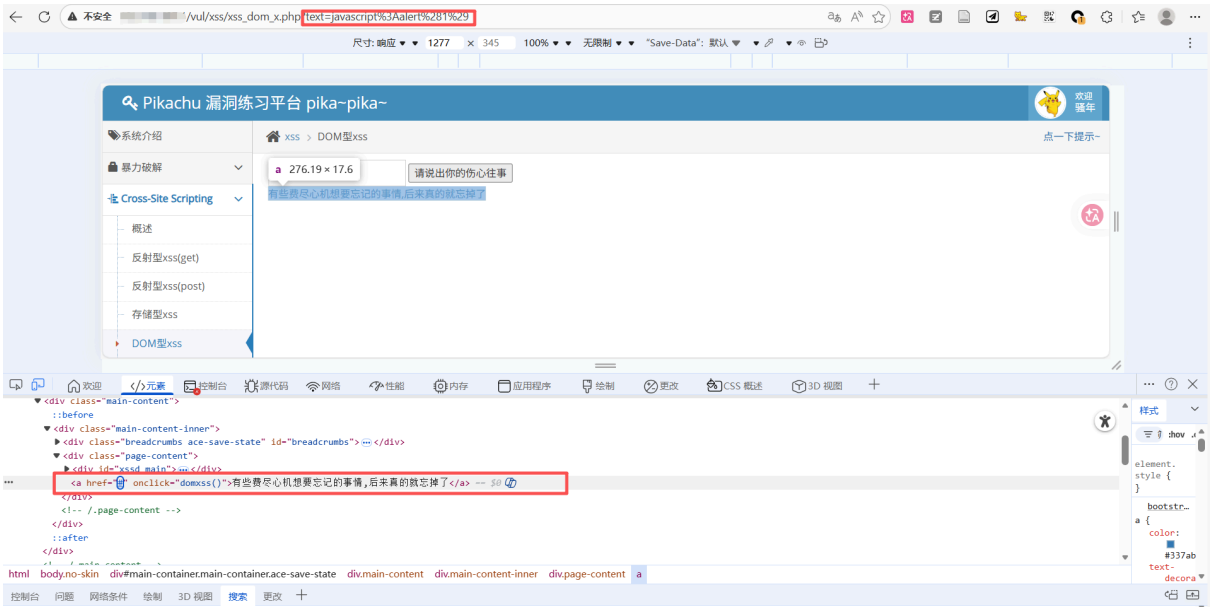
5、DOM型xss-x

进入下一题，其实我们要弄明白的点只有一个：同是DOM型xss-x，末尾多出来的这个-x是什么含义？

我们不妨按照上一题的方法先行测试——既然类型一致，那么解题的流程走下来应该也相似。我们输入 `javascript:alert(1)` ，发现出现了一个<a>标签，而点击这个<a>标签后，又出现了一个新的<a>标签。当然我们也没有别的办法，再次点击这个新的<a>标签，网页成功跳出了弹窗——说明我们的破解思路正确，仍然是利用伪协议型的JavaScript触发机制，看起来只是中间多出了一步<a>标签的转换罢了。

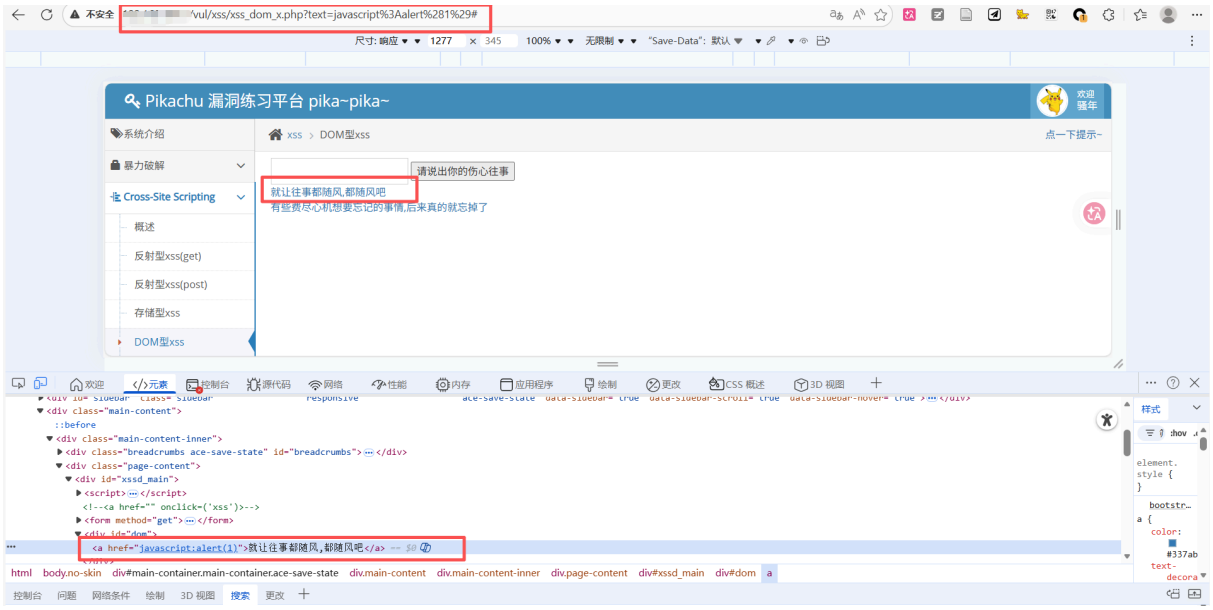
然而，真的就只是这么简单吗？我们还是打开开发者工具，追踪每一步进行时的页面代码变化。

首先，输入 `javascript:alert(1)` 后点击“请说出你的伤心事”按钮：



这里，我们看到有两个不同寻常的地方——第一，网址栏后面显示了我们的输入，说明这里是将我们的输入作为网址来处理的；第二，<a>标签对应的代码的href内容仅仅显示一个"#"，而不是向上一题一样包含我们输入的内容。

咱们姑且把这两个疑问放在一边，点击生成的<a>标签"有些费尽心机想要忘记的事情，后来真的就忘掉了"：



这里，我们选中新生成的<a>标签"就让往事都随风,都随风吧"(当时我不禁哼起了歌，"就让往事随风~都随风~都随风~心随你动~")，发现这里的<a>标签href是有内容的，显示了我们输入的 `javascript:alert(1)` ，单看此处基本上就还原为了上一题。

但是，我们注意到此时网址的后缀仍然是我们输入的内容，这是一个很值得关注的点。

此时点击<a>标签，自然就跳出了弹窗，此处当然没什么好说的。

让我们重新梳理一下刚刚发现的不同之处：

第一，输入 `javascript:alert(1)` 之后，网址发生了变化，我们的输入作为后缀被包含在网址中；

第二，生成的第一个<a>标签对应的代码中href内容为"#"，并不是我们输入的内容；

第三，生成的第二个<a>标签对应的代码中href是我们输入的 `javascript:alert(1)` 。

考虑到网址后缀的变化，我们猜测输入没有被直接塞到<a>标签之中，而是作为端点被写入了网址之中，我们不妨仔细分析一下网址的结构：

```
~/vul/xss/xss_dom_x.php?text=javascript%3Aalert%281%29#
```

通过询问AI我们知道（doge），`~/vul/xss/xss_dom_x.php` 这部分是代码的目录文件位置，无需在意；而 `?` 表示查询参数的开始；`text=` 是参数名；`javascript%3Aalert%281%29` 是经过编码之后的输入内容。

因此，综合来看，这个网址的含义就是：把 `javascript:alert(1)` 作为 `text` 参数的值，传给服务器端的 `xss_dom_x.php` 文件处理。

而第一个<a>标签href中的"#"，则是一个特殊的空链接：点击不会跳转发任何页面，但是会触发点击事件。而在这一题中，这个 `href="#"` 其实是一个"触发器"，作用仅仅是用来触发点击后绑定的JavaScript事件（可以联想到上一题中我们构造的 `onclick` ）。

然后，在我点击这个 `href="#"` 的链接后，页面里的JavaScript代码就被触发了，这段代码这才去读取URL参数里的 `javascript:alert(1)` ，把它写进第二个<a>标签的href里。因此，我们就能在第二个<a>标签中看到我们的输入，进而点击后就会触发弹窗事件！

所以我们看到，这个所谓的DOM型xss-x实际上最终仍然要通过<a>标签的href来触发我们输入的命令，只是中间多了一步——先把输入的内容写到网址的后缀中，再读取后缀生成新的<a>标签，便回到了一般的DOM型xss！
那么，中间多出来的这一步——写入网址的过程，有什么实际意义吗？对于我们的xss攻击而言，一个直接包含命令参数的网址无疑具有极其强大的传播性和可操作性。

试想一下，如果按照上一题的方法，比如你想登录你兄弟的游戏账号，你必须要把原始网址发给他，让他点击进去，然后你说："兄弟，你在这个输入框里输

入 `javascript:alert(document.cookie)`，再点一下‘what do you see’，然后把弹窗的内容告诉我”。

那么问题来了，对面是傻子吗？

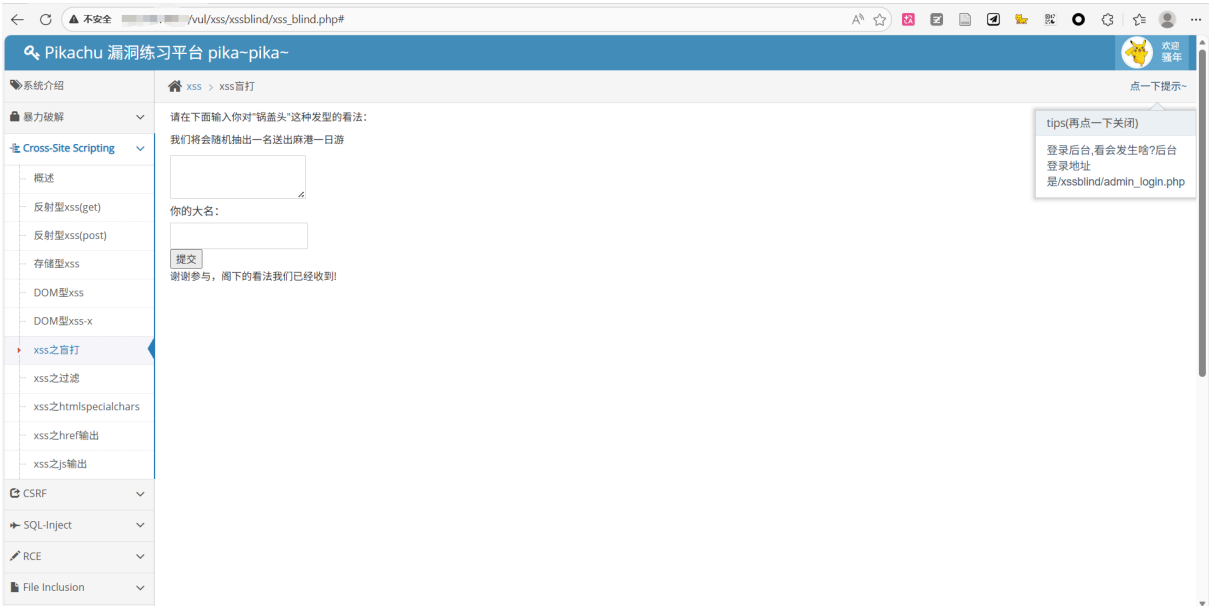
哈哈，开个玩笑。通过这个例子，我们知道了如果没有后面的这个”-x”，即便你掌握了攻击原理，也终究不过是屠龙术罢了，没有任何实际的意义。而一旦有了这个”-x”，即把命令直接写到网址中的过程，你就可以直接发给对方一个链接，这个链接后缀里包含了一个自动获取token且发送到你的邮箱的功能，对方只要稍不留神一点击，然后再在网页里随便点了某一个链接，他的token立马就被你盗取了。之后你就可以大摇大摆地用你兄弟的token畅享他的游戏账号了！

当然，以上纯粹是举个例子，为的是方便理解，真要这么干的话也太缺德了。

一句话，”-x”把DOM型xss攻击从理想拉到现实。

6、xss盲打

这又是一个新的名词，我们态度还是一样的——不管怎么样，先试试看，遇到问题再去解决问题。那么还是输入最典型的注入命令 `<script>alert('xss')</script>` ：



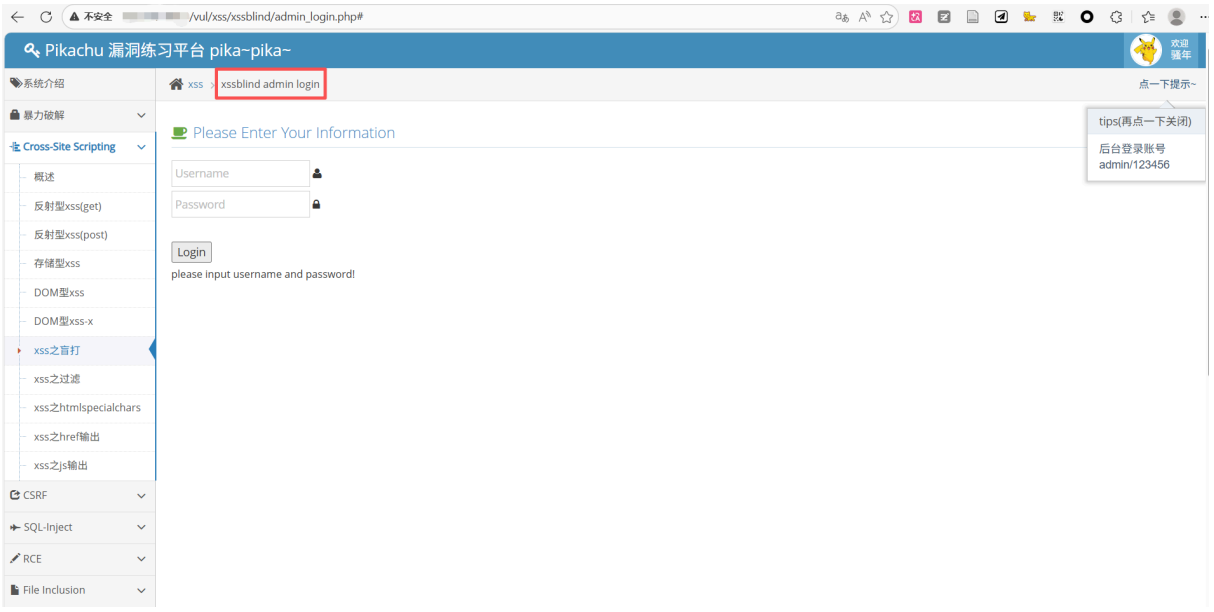
我们看到，网页给我们唯一的反馈就是开发者的礼貌——“谢谢参与，阁下的看法我们已经收到!”。我们可以再打开开发者工具找找线索，但似乎还是一无所获。

但是，这是不是提示我们什么？既然“已经收到”，那么管理员是如何查看自己收到的信息的呢？如果他能查看的话，那么这个命令会不会在他的页面执行呢？

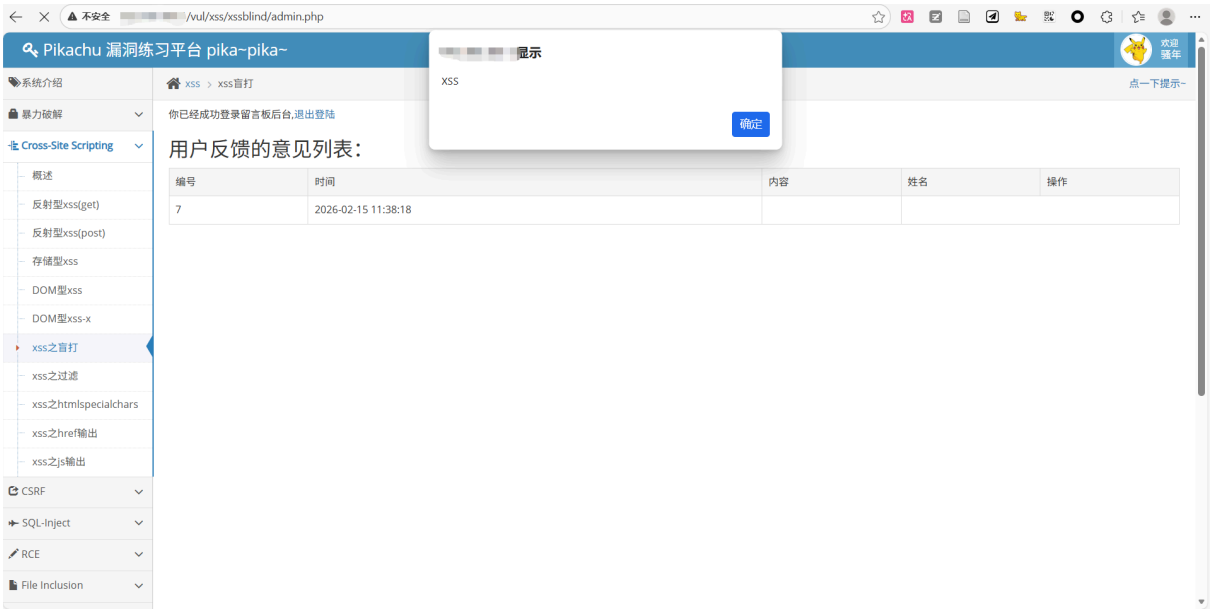
这里我们查看一下提示，也是提醒我们去查看管理员页面：

那就照做呗！

让我们用本地IP访问这个目录：`http://XXX.XXX.XXX.X/vul/xss/xssblind/admin_login.php`



根据提示，输入用户名和密码，点击登录：



咦？页面上立刻跳出了弹窗！与我们先前输入的命令完全匹配！

也就是说，在这一题中，本该在用户界面里出现的弹窗，我们自己看不到，反而在管理员的后台管理界面上出现了！回顾一下本题的标题，盲打——是不是立马有那味了！

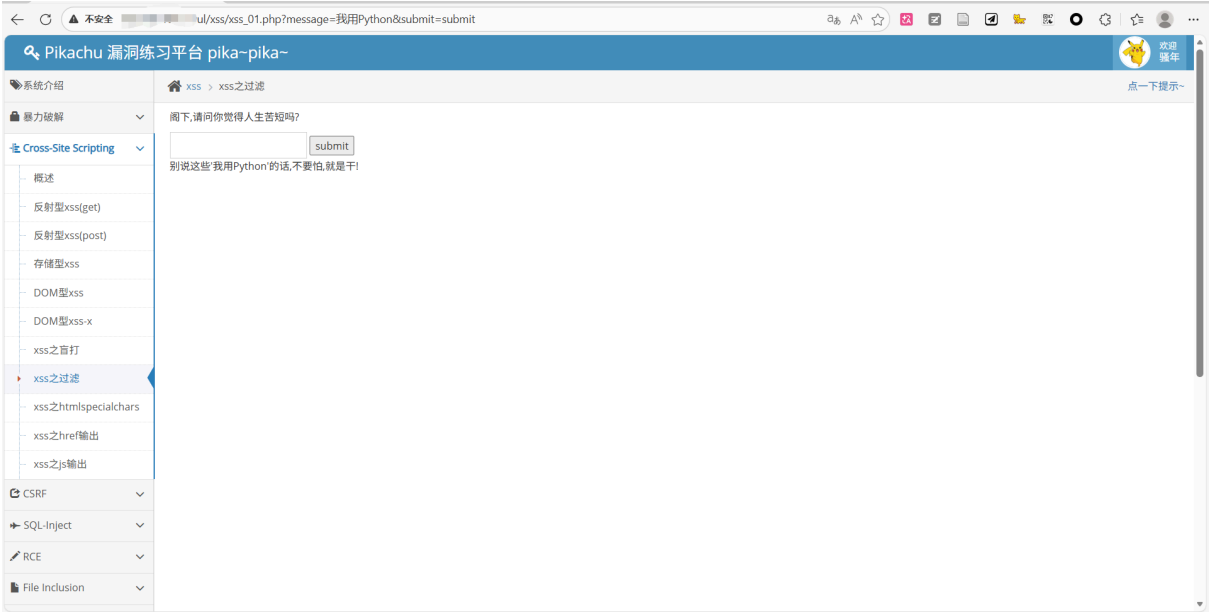
实际上，xss盲打就是这样一种技术：攻击者在一个看不见反馈的地方（比如留言板、反馈表单、评论框）注入XSS代码，这些代码不会在攻击者自己的页面上执行，而是会在后台管理员或其他用户的页面上悄悄执行。因为你看不见执行结果，就像“盲人”一样，所以叫盲打。

那么，“盲打”有什么实际应用呢？如果说一般的XSS主要是面向用户的攻击，那么盲打型XSS就是面向管理员的攻击——普通用户被攻击，最多丢个cookie、账号被盗；但管理员一旦中招，整个网站就易主了，而所有存储的用户信息也就全都被盗走了。

这就是“盲打”——你看不见子弹飞，但子弹可能正飞向整个网站最值钱的目标。

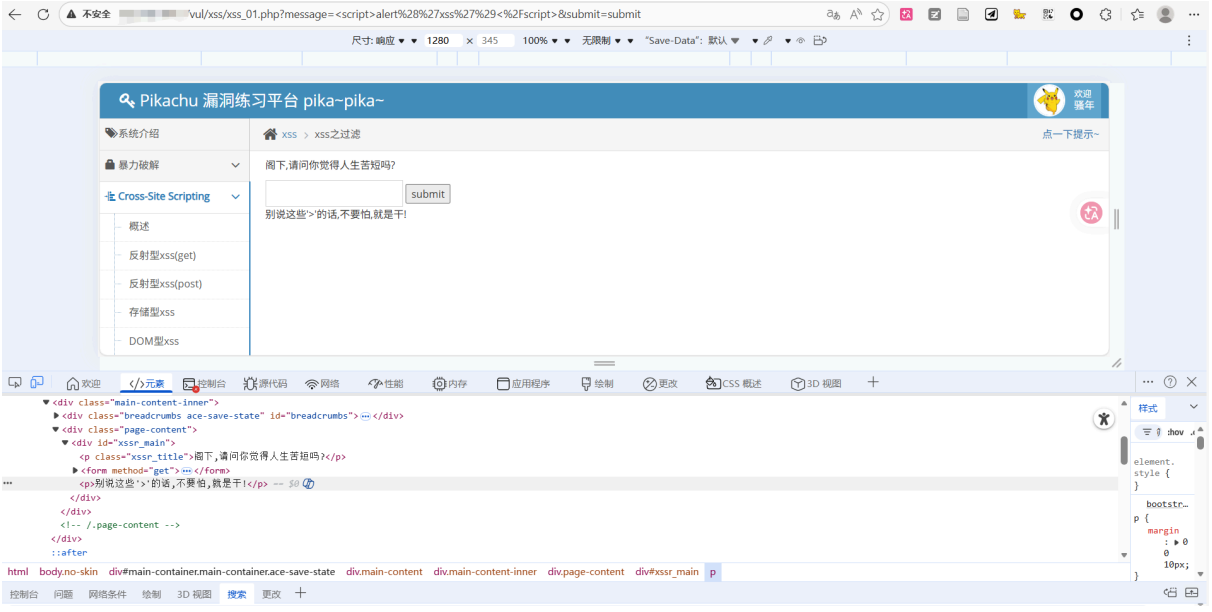
7、xss之过滤

现在，我们进入下一题，还是简洁的一个输入框，看起来和第一题很类似——但是也别急着攻击，毕竟这个提问很有意思（doge），不妨先输入“我用Python”：



点击submit之后，下面出现了一段话“别说这些'我用Python'的话,不要怕,就是干!”好吧，看起来开发者是在鼓励我们去尝试世界上最好的语言PHP呢！

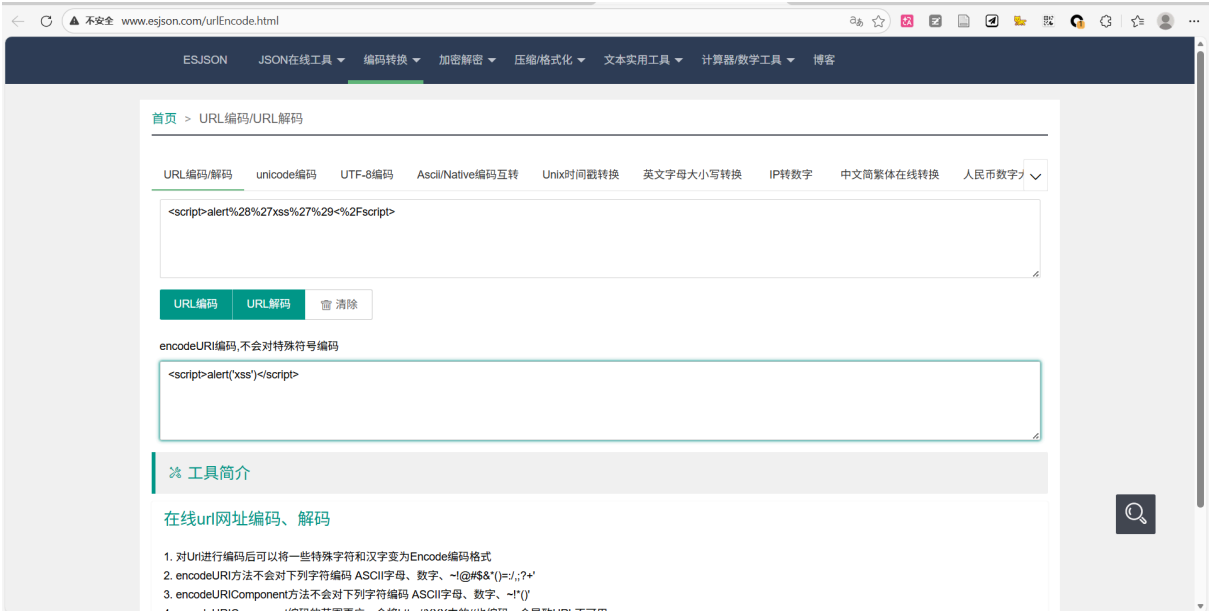
不开玩笑了，其实我们多试几次可以发现，这个其实就是一个简单的内容代换，将输入的内容填充到引号之间。从这里其实我们看不出更多的东西，我们不妨还是按照XSS攻击的一般思路，先输入 `<script>alert('xss')</script>` 试试看：



没有跳出弹窗，看起来攻击失败了——但是也别慌，失败很正常，但我们于此同时能够观察到很多有趣的细节：

第一，引号之间的内容并不是我们输入的内容 `<script>alert('xss')</script>` ，而仅仅只是一个<，这说明我们的命令被改变了，被截断而仅剩一个无效的<作为输入；

第二，观察网址，不难注意到这样一串后缀 `?message=<script>alert%28%27xss%27%29<%2Fscript>&submit=submit` ，这种后缀我们之前见过，它的含义就是把我们的输入 `<script>alert%28%27xss%27%29<%2Fscript>` 传递给服务器对应目录——我们当然有理由怀疑这个输入也许有点不太对劲，毕竟我们构造失败了，或许在浏览器前端就已经给我们的输入处理过了呢！让我们把它解码还原一下，看看被发往服务器的内容究竟是什么：



呃，这下没话说了，看来浏览器还真是老老实实把我们的输入原样发送给了服务器——不过，这其实也就说明问题一定出在服务器层面上，一定是后台哪段代码给我们的命令做了手脚！

既然这里可能存在前后端通信的过程，那么不妨抓包看看这段内容被发到哪里去了：

很明显，这段内容被发到了 `/vul/xss/xss_01.php` 目录。

等下，这个目录不就是我们在网址栏看到的吗？那这个不是前端吗？这里就涉及到一个很有趣的问题：皮卡丘靶场是用“世界上最好的语言”PHP编写的，而PHP则是典型的嵌入式语言，其基本特征就是可以在HTML中穿插PHP代码！所以，我们才看到了这种数据发送给同一个目录的情况。不过，请读者相信，我们这样抓包一次还是很有

必要的——当代的网页开发基本都是前后端分离了，很少会出现这种“自己发给自己”的情况，因此搞清楚来龙去脉还是很有必要的。

那么，让我们打开xss_01.php文件，查看源代码，看看服务器究竟是怎么处理我们输入的内容的吧！

在源码中，我们可以定位到这样一段代码：

```
$html = '';
if(isset($_GET['submit']) && $_GET['message'] != null){
    //这里会使用正则对<script进行替换为空,也就是过滤掉
    $message=preg_replace('/<(.*)s(.*)c(.*)r(.*)i(.*)p(.*)
t/', '', $_GET['message']);
//    $message=str_ireplace('<script>',$_GET['message']);

    if($message == 'yes'){
        $html.="<p>那就去人民广场一个人坐一会儿吧!</p>";
    }else{
        $html.="<p>别说这些'{$message}'的话,不要怕,就是干!</p
>";
    }
```

这里最重要的是正则表达式 `$message=preg_replace('/<(.*)s(.*)c(.*)r(.*)i(.*)p(.*)t/', '', $_GET['message']);`，它的目标很明确，只要你的输入里以 `<` 开头，后面按顺序有 `s`、`c`、`r`、`i`、`p`、`t` 这几个字母（中间可以夹带任何东西），整个直接就删掉！回顾一下我们的输入，这个典型的构造 `<script>alert('xss')</script>`。

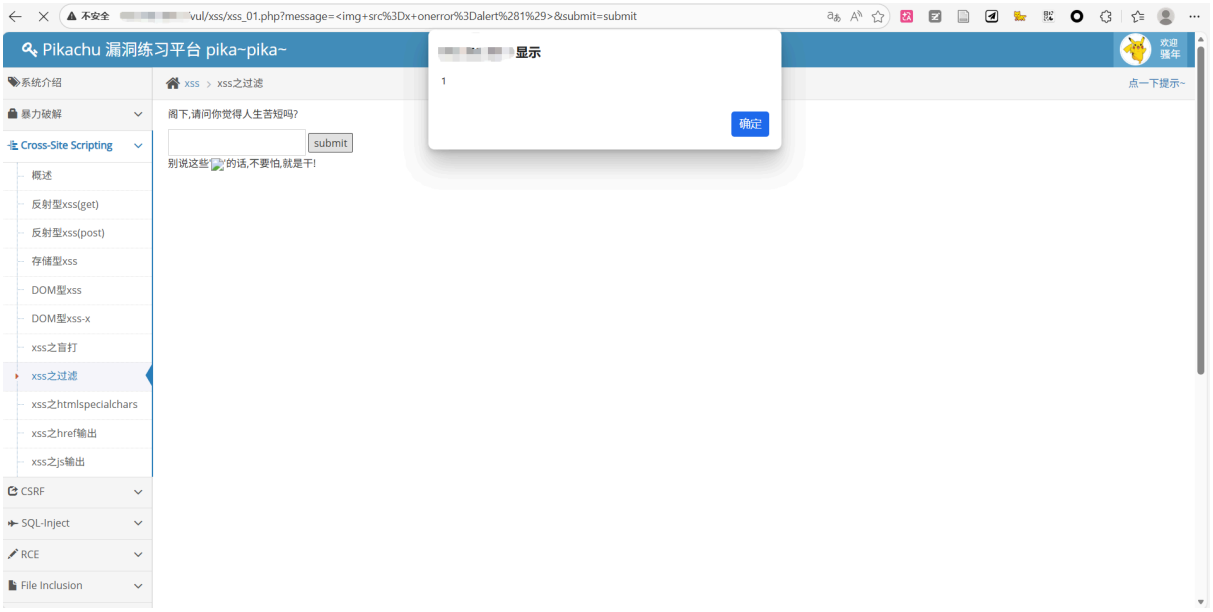
恰好就符合这个正则表达式过滤的目标，因此我们的输入就被全部删除了——我们构造失败了。

所以，我们就需要考虑如何绕过服务器的过滤机制，其实看到源代码之后我们就很清楚了——因为后端代码的过滤机制十分“单纯”，只要不是“以 `<` 开头，后面按顺序有 `s`、`c`、`r`、`i`、`p`、`t` 这几个字母”，那就可以绕过这个过滤。当然有一点值得注意，这里的过滤规则由于存在 `*`，因此是不会区分大小写的，因此将小写改成大写并没有实际的意义。

回看一下，这里采用的是GET类型的传参方法，即直接通过网址传递数据，所以我们不能采用伪协议型的XSS（`javascript:alert(1)`），可以考虑标签型XSS或者事件型XSS。

对于标签型XSS，除了 `<script>alert('xss')</script>` 这种典型构造，我们当然还有其他构造方法，比如 `<img src=x onerror=alert(1)>`。我们输入之后点击submit，网页果然

跳出了弹窗：



不过严格来说，`<img src=x onerror=alert(1)>` 其实是一个标签型和事件型的混合构造，其本质上是依靠 `onerror` 这一事件（标签加载失败）来触发的，但因为具有img标签，所以到也没必要太严格地区分。在这里，无非是说明一个道理——针对后端服务器过滤机制，需要通过尝试不同的构造方法来绕过机制，达成攻击目的！

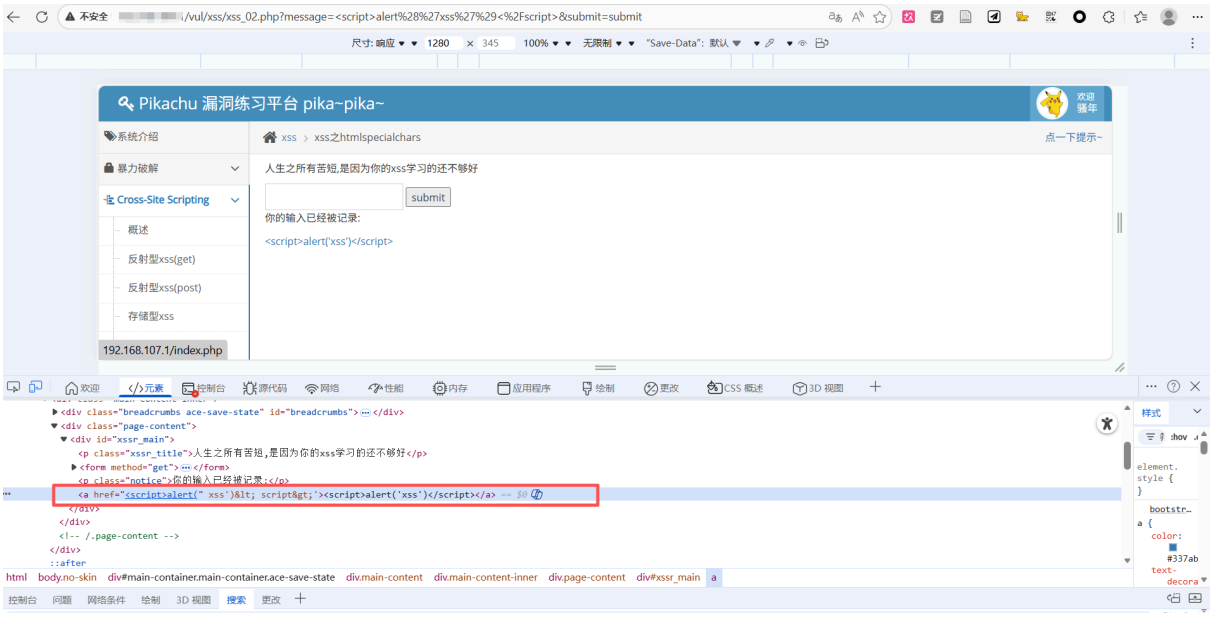
这里，我们首先是找到了源代码，才进行新的构造的。但是真实情况下我们当然不可能直接找到过滤的源代码（要是真找到了，那你不如直接控制对方服务器算了~），所以需要做的就是不断尝试，直到找到成功触发的那一段XSS命令！

但我们倒也不必太过烦恼，毕竟过滤机制永远是存在局限的，永远只能过滤掉有限可能的情况——而攻击者的办法却是无限的！

8、xss之htmlspecialchars

又遇到一个新的名词！

好吧，还是老样子先试试看。这里还是先用最典型的标签型xss试试水——输入 `<script>alert('xss')</script>` ，点击submit之后，我们看到了似曾相识的东西，下面又蹦出来一个 `<a>` 标签，用开发者工具选中这个 `<a>` 标签：



很明显了，又是熟悉的href，那就需要看看包含的内容是啥了：

```
<a href="&lt;script&gt;alert(" xss')&lt;="" script&gt;'="">
&lt;script&gt;alert('xss')&lt;/script&gt;</a>
```

可以看到，这确实是我们输入的内容——只不过被从中间切开了，加入了一些意义不明的字符，那么按照先前的经验，我们知道构造肯定是失败了，开发者在这里埋入了预防机制，故意把内容打乱了。但也别急着去尝试其他XSS构造，这里也有一些很有趣的东西，是我们之前没有仔细讨论的——这个“预防机制”究竟是如何起作用的呢？

前端当然不会有线索（因为我们之前已经知道前端代码其实是可以被访客随意更改的，此处的预防机制没有实际意义），所以我们还是找到源代码文件xss_02.php，可以提取到这样一段代码：

```
$message=htmlspecialchars($_GET['message']);
$html2.="<a href='{$message}'>{$message}</a>";
```

这里，我们再次看到了那个新词，htmlspecialchars。这下我们不得不严肃面对了。

通过询问Deepseek（无广告），我们知道 `htmlspecialchars()` 是一种PHP内置的工具，它的作用是把特殊字符转换成HTML实体，请看下面的表格。

字符	原本含义	转换后
<	标签开始	<
>	标签结束	>
"	双引号	"

字符	原本含义	转换后
&	实体开始	&

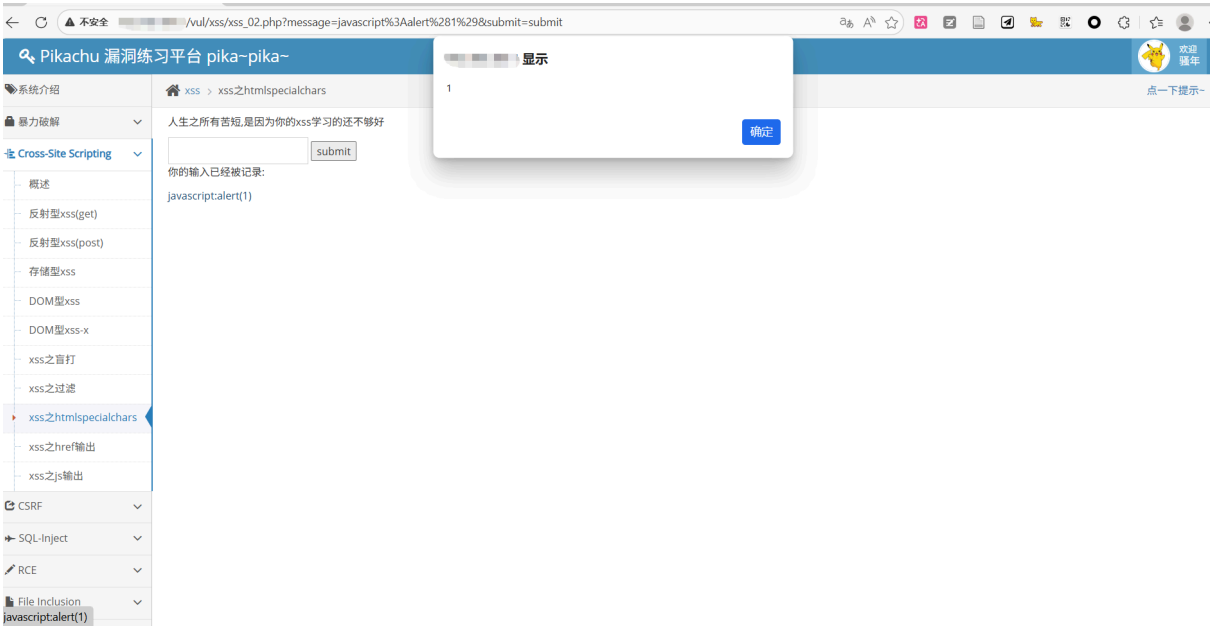
但是，这种方法默认不会转换单引号 `'`！因此，我们看到原先的代码 `<script>alert('xss')</script>` 经过 `htmlspecialchars()` 之后，就变成了 `<script>alert('xss')<=script>` 。至于多出来的一个引号，则是在 `<a>` 标签生成时被拼进字符串时发生的（写入href时会自动添加引用，会防御很多构造攻击，但是新增的引号会与已有的引号发生反应，一一配对，会发生很多奇妙反应），因此这里 `'` 最终也被转换过了。

所以它就变成了 `<a href="<script>alert(" xss')<="" script>'=""><script>alert('xss')</script></a>` 的 `<a>` 标签形式。

所以我们看到，在第一步转换后，即 `htmlspecialchars()` 的转换之后，我们的XSS攻击实际上已经被成功防御了，这段命令被切分成了错乱的字符，已经毫无意义。

但是，读者不难发现，这一题和上一题过滤机制有相似之处——上一题是“发现即消灭”的过滤，这一题则是“发现即同化”的转化，其前置条件都是“发现”！

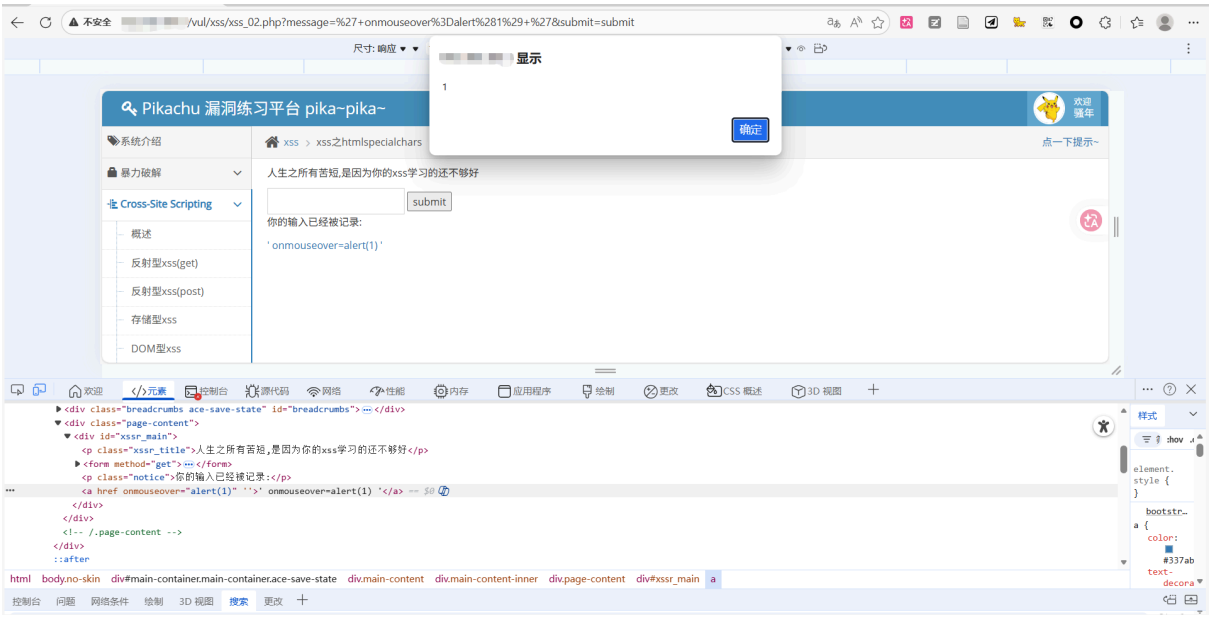
所以我们的思路也就确定下来了，只要在构造时不用涉及到 `htmlspecialchars()` 方法的四个字符，同时避免使用 `'` 即可。这里，由于存在<a>标签，所以我们自然想到了前面已经攻克的DOM型XSS，因此不妨先试试 `javascript:alert(1)` ：



果然！一发入魂！（doge）

当然，这里并不是完全不能用 `'`，实际上我们还可以输入 `' onmouseover=alert(1) '`（引号一定要有，且引号与字符之间一定要有空格！换成 `' onclick=alert(1) '` 也是一

个道理，可以回顾一下DOM型xss那一题）。这是一个事件型的XSS，当鼠标移到生成的 <a> 标签上时，就能触发弹窗：



观察这里<a>标签的的href，发现引号也发生了变化——由于存在空格的关系，引号不会直接和我们构造的命令发生作用，而是首先和href拼接时自带的引号发生作用——第一个引号与拼接字符串时生成的引号组合配对，两个引号之间没有内容，相当于前面有一个"空"；第二个引号同理，在末尾形成了另一个"空"，因此反而没有影响整个命令的完整性，当然也能够实现XSS攻击——不管怎么说，核心逻辑是一样的，通过各种方法绕过防御机制，不要让代码影响到自己构造的命令。

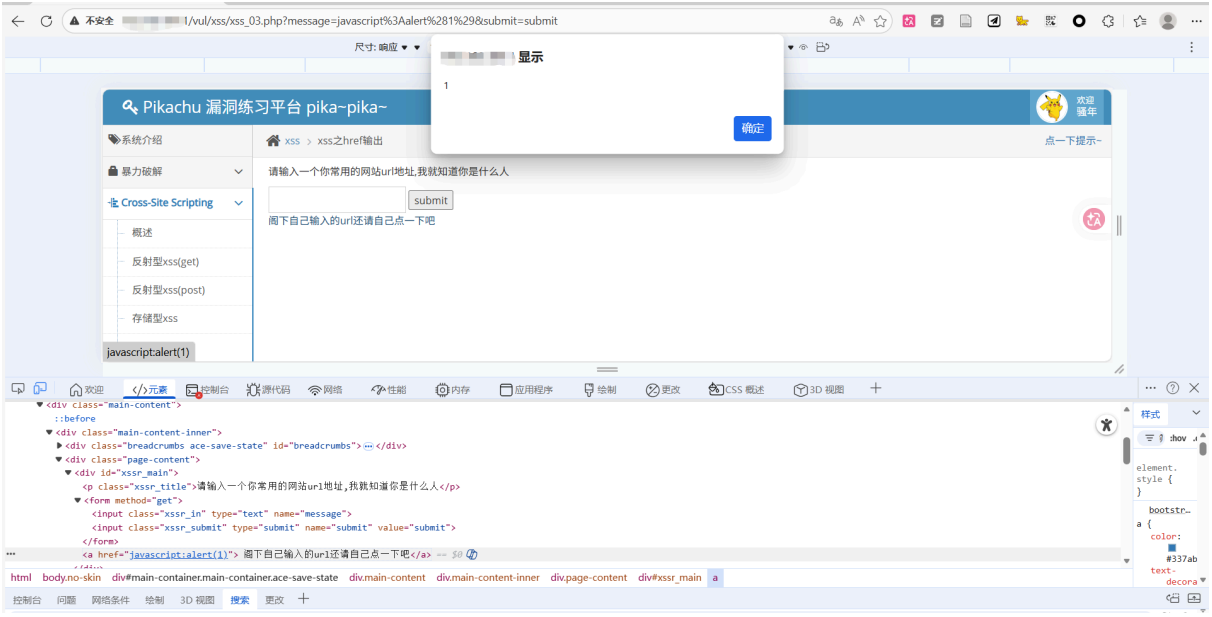
9、xss之href输出

光看这个标题，其实我们心里大致就有数了——又是这个href！那大概率还是和<a>标签有关，所以待会不管怎么样，咱们都先拿"对策卡" `javascript:alert(1)` 试试看！

还是用开发者工具选中新生成的<a>标签，好吧，和我们的预期完全一致，捕获到这样一串代码：

```
<a href="javascript:alert(1)"> 阁下自己输入的url还请自己点一下吧
</a>
```

无需多言了，这就是我们最熟悉的情况，现在点击这个<a>标签，就正常跳出弹窗了：



现在回想一下，其实这种情况我们已经遇见很多次了，我们在DOM型xxs那两题遇到过，在xss之htmlspecialchars那一题也同样遇见了它。这里我们终于了解到了关于这

类攻击特征的官方名称“href输出”——换言之，就是把用户输入的内容，直接输出到了HTML标签的 href 属性里。

因此，“href输出”不是一个新漏洞，而是一类漏洞的“官方命名”。

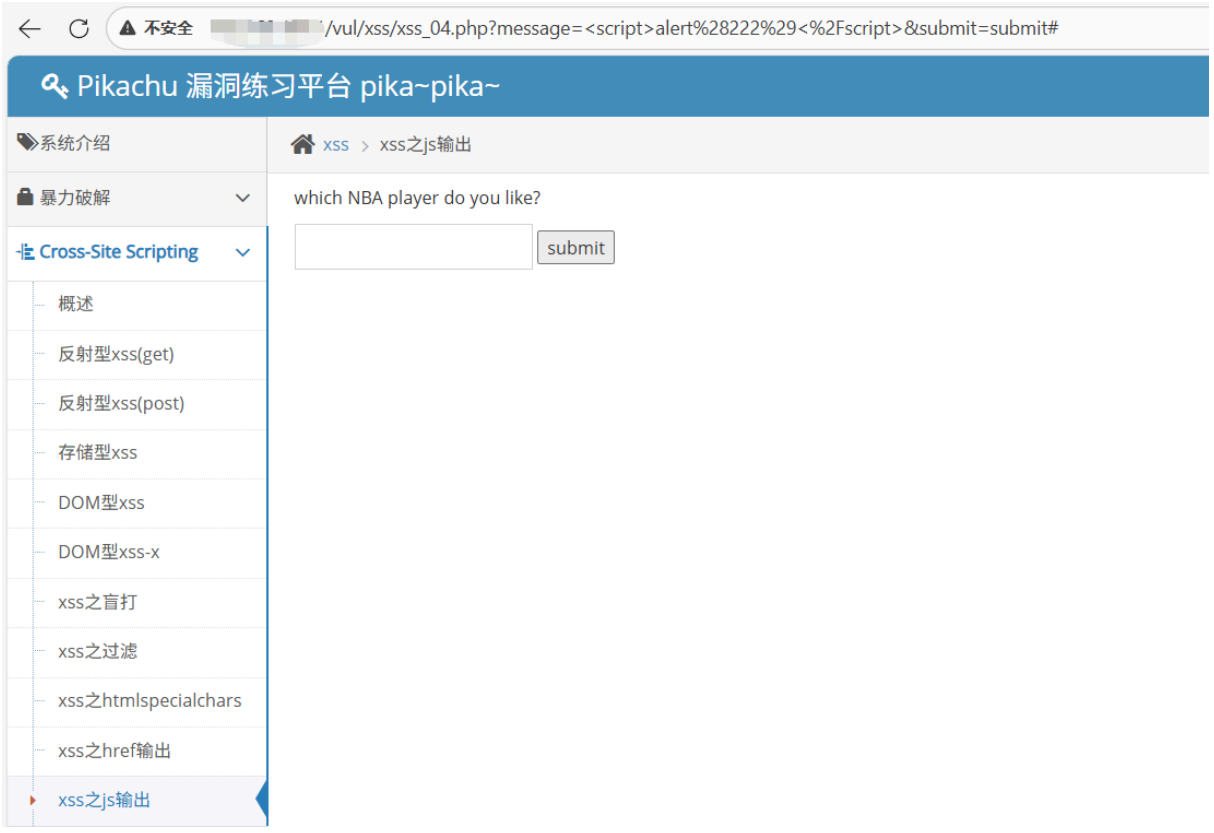
它提醒我们：只要输入被拼进了 href，不管前面经过了多少层过滤、转义，只要没拦住 `javascript:`，攻击就能成。

所以，面对 href 输出，第一反应就应该是：试试 `javascript:alert(1)`，大概率有惊喜哦！

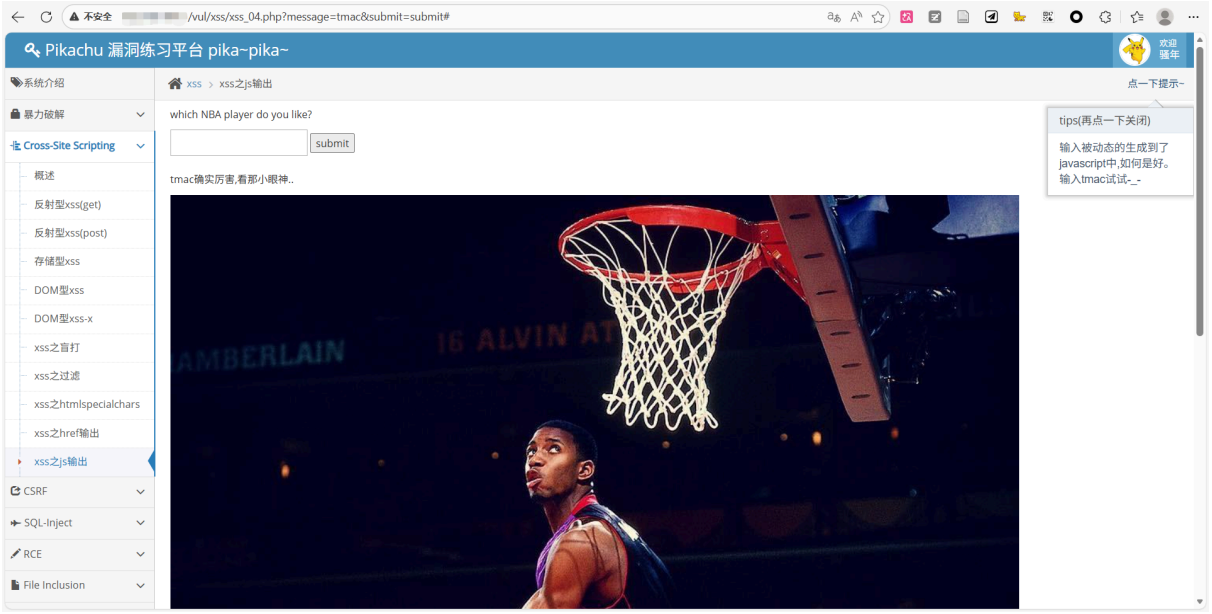
10、xss之js输出

下一题，很明显与上一题形成了对照：所谓“js输出”，就是javascript输出，那到底是怎么一回事呢？

遇事不决，还是先把 `<script>alert('xss')</script>` 甩上去看看！



好吧，似乎无事发生，这次老实一点，输入一个正常的内容看看，不如就试试提示里的tmac。



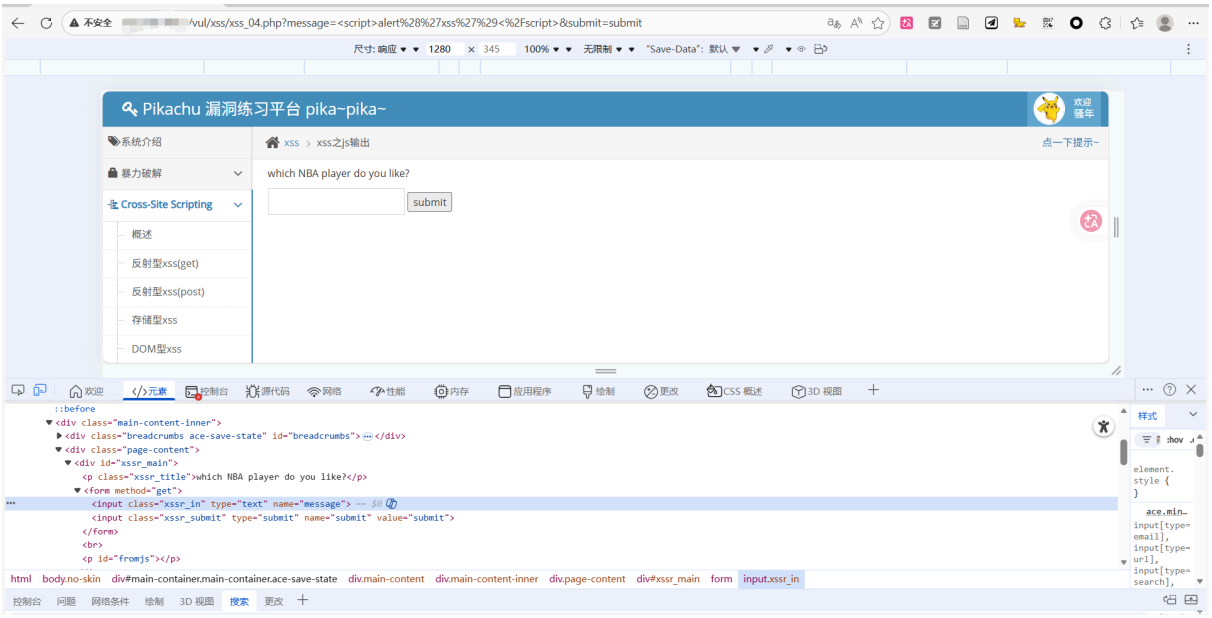
看来是正常有输出的，如果试试kobe呢？（我并非篮球爱好者，只是好奇U•ェ•*U）

无论如何不要放弃心中所爱——好吧，xss的第一题你不是还爱着kobe嘛！喜新厌旧的开发者！

由来只有新人笑，有谁听见旧人哭~

好吧好吧，至少从这几次尝试中我们大概明白，正常的输入下应该是有输出的！而我们第一次关于`<script>alert('xss')</script>`的尝试，很显然是不正常的情况！那么问题究竟出在哪里呢？

这次仔细观察一下：

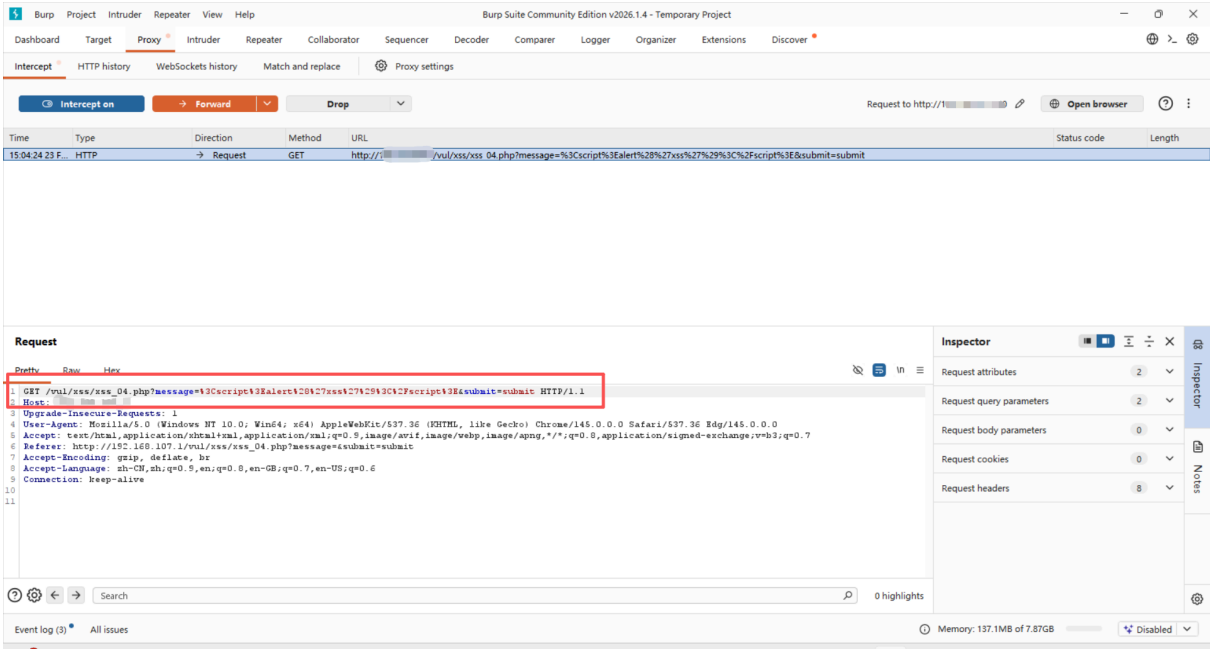


在下面的开发者工具里面似乎看不出什么线索，但是网址发生了变化——我们的输入被当作后缀拼接到网址中去了！

```
~/vul/xss/xss_04.php?message=<script>alert%28%27xss%27%29<%2Fscript>&submit=submit
```

那么，很明显，回想一下之前我们所遇到的题目，很明显这里同样是把数据通过网址发送给了服务器进行处理——这里必然存在前后端通信的过程！

那么·，先抓包一下看看：



抓打包了，这很符合我们的预期，看来，数据被发送到了“xss_04.php”目标文件中进行处理，那么让我们找到这个文件，尝试定位到有关的源代码.

在 "xss_04.php" 文件里，处理输入的核心代码是这一段：

```
if(isset($_GET['submit']) && $_GET['message'] != null){
    $jsvar=$_GET['message'];  // ⚠ 直接取用户输入，没有任何过滤!
    //    $jsvar=htmlspecialchars($_GET['message'],ENT_QUOTES);  // 这行被注释掉了
    if($jsvar == 'tmac'){
        $html.="<img src='{$PIKA_ROOT_DIR}assets/images/nba player/tmac.jpeg' />";
    }
}
```

然后下面这里把用户输入的内容直接拼进了 JavaScript：

```
<script>
    $ms='<?php echo $jsvar;?>';  // ⚠ 直接输出，没有转义!
    // ...
</script>
```

也就是说，我们的输入被原封不动地塞进了JavaScript代码的字符串里！

在这里，我们不妨再举个例子，假如说我们输入的是tmac，那么生成的代码就是：`$ms='tmac';` ，就会对应读取那张tmac的“小眼神”的图片，并返回到前端网页。假如说我们输入的还是那个典型的`<script>alert('xss')</script>` 构造，那么就会因为这两端的引号而导致内容为空！

也就是说，我们这里构造的方法就是：先把`$ms='` 的引号兑掉（也就是需要一个引号，与这个引号配对形成一个“空”，以防这个引号对我们的构造产生影响），再加一个分号，把这一条代码直接截断掉，最后在末尾加上注释标记`//`，把这段代码最后的那个`'`注释掉！也就是说，我们构造的格式为：`';……//`，那么最终会形成这样一段代码：

```
$ms='';……//';
```

也就是说，前面这段代码就被构造为一个空命令，不会产生任何影响，后面的引号被`//`注释掉，同样没有影响——而省略号处就可以自由构造我们自己的xss攻击命

令！

那么，现在我们就可以让浏览器跳出弹窗了！输入 `'; alert(1)\\` ，点击submit！

成功乃是意料之中！

和上一题联系一下，其实这就是xss攻击的两种经典命题，请看下面的表格：

输出位置	典型输入	核心逻辑
HTML标签属性（如 `href` ）	`javascript:alert(1)`	伪协议在属性内直接执行
JavaScript字符串	`'; alert(1)//`	闭合字符串，注入新语句

看似完全不同的两种Payload，背后是同一个逻辑：让浏览器把我的输入当成代码执行，而不是普通数据。最终，我们都要通过网页特征和不断的尝试找出一个真正能够绕过浏览器防御手段的构造！

到此，xss章节就全部结束。让我们期待下一章“CSRF”吧！

四、CSRF概述

当然，还是先摘录一下靶场开发者的概述：

> **CSRF(跨站请求伪造)概述**
>
> Cross-site request forgery 简称为“CSRF”，在CSRF的攻击场景中攻击者会伪造一个请求（这个请求一般是一个链接），然后欺骗目标用户进行点击，用户一旦点击了这个请求，整个攻击就完成了。所以CSRF攻击也成为"one click"攻击。 很多人搞不清楚CSRF的概念，甚至有时候会将其和XSS混淆,更有甚者会将其和越权问题混为一谈,这都是对原理没搞清楚导致的。这里列举一个场景解释一下，希望能够帮助你理解。
>
> **场景需求：**小黑想要修改大白在购物网站tianxiewww.xx.com上填写的会员地址。
>
> **先看下大白是如何修改自己的密码的：**登录---修改会员信息，提交请求---修改成功。所以小黑想要修改大白的信息，他需要拥有：1，登录权限 2，修改个人信息的

> 请求。
>
> 但是大白又不会把自己xxx网站的账号密码告诉小黑，那小黑怎么办？于是他自己跑到www.xx.com上注册了一个自己的账号，然后修改了一下自己的个人信息（比如：E-mail地址），他发现修改的请求是：【http://www.xxx.com/edit.php?email=xiaohei@88.com&Change=Change】
>
> 于是，他实施了这样一个操作：把这个链接伪装一下，在小白登录xxx网站后，欺骗他进行点击，小白点击这个链接后，个人信息就被修改了,小黑就完成了攻击目的。
>
> **为啥小黑的操作能够实现呢。有如下几个关键点：**
>
> 1.www.xxx.com这个网站在用户修改个人的信息时没有过多的校验，导致这个请求容易被伪造;---因此，我们判断一个网站是否存在CSRF漏洞，其实就是判断其对关键信息（比如密码等敏感信息）的操作(增删改)是否容易被伪造。
>
> 2.小白点击了小黑发给的链接，并且这个时候小白刚好登录在购物网上;---如果小白安全意识高，不点击不明链接，则攻击不会成功，又或者即使小白点击了链接，但小白此时并没有登录购物网站，也不会成功。
>
> ---因此，要成功实施一次CSRF攻击，需要“天时，地利，人和”的条件。当然，如果小黑事先在xxx网的首页如果发现了一个XSS漏洞，则小黑可能会这样做： 欺骗小白访问埋伏了XSS脚本（盗取cookie的脚本）的页面，小白中招，小黑拿到小白的cookie，然后小黑顺利登录到小白的后台，小黑自己修改小白的相关信息。
>
> ---所以跟上面比一下，就可以看出CSRF与XSS的区别：CSRF是借用户的权限完成攻击，攻击者并没有拿到用户的权限，而XSS是直接盗取到了用户的权限，然后实施破坏。因此，网站如果要防止CSRF攻击，则需要对敏感信息的操作实施对应的安全措施，防止这些操作出现被伪造的情况，从而导致CSRF。
>
> 比如：--对敏感信息的操作增加安全的token；--对敏感信息的操作增加安全的验证码；--对敏感信息的操作实施安全的逻辑流程，比如修改密码时，需要先校验旧密码等。
> 如果你没有读太明白,不要犹豫,请再读一遍啦你可以通过“Cross-site request forgery”对应的测试栏目，来进一步的了解该漏洞。

开发者举的例子特别好，通过这段概述，我们能够明白所谓CSRF攻击，就是在对方已经登陆的条件下，通过诱导对方点击自己构造的链接，达到修改对方个人信息的目的。那么为什么这个链接能在别人的账号上生效？因为这个链接里没有标识“这是谁的账号”，服务器只认当前登录的cookie——所以谁登录着点它，改的就是谁的信息。因此，构造这个链接，就是我们在本节需要做的事情。

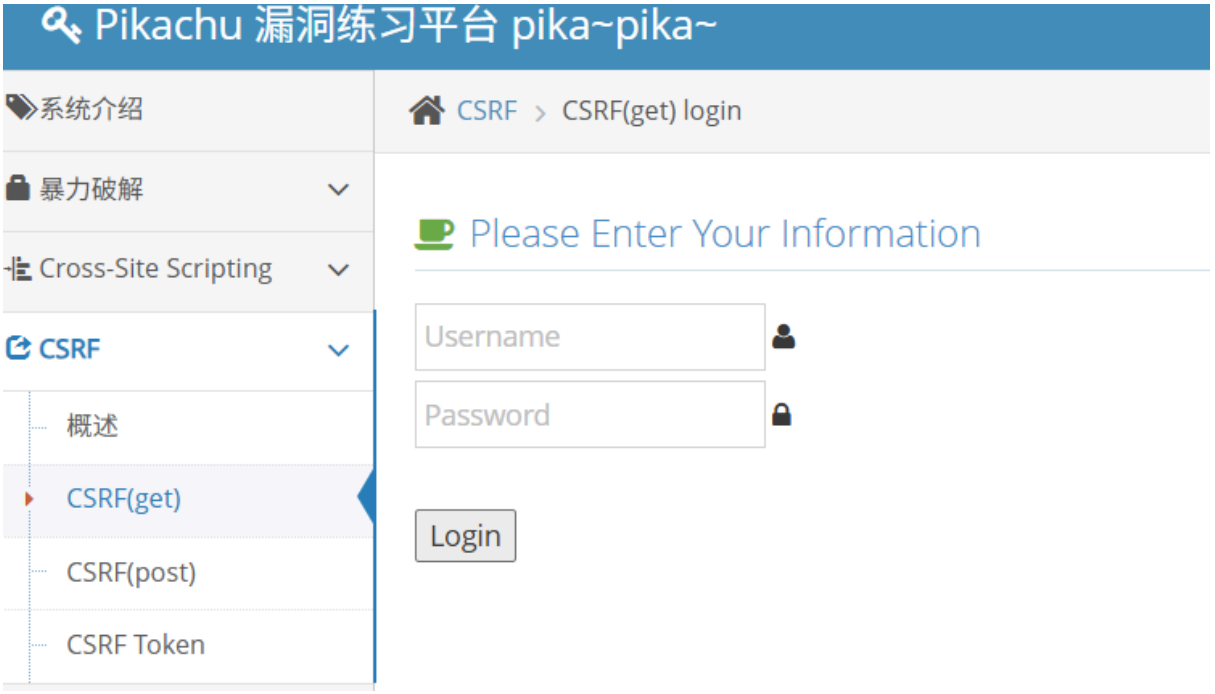
那么，通过CSRF攻击，修改对方个人信息之后有什么意义吗？当然，我们确实没有获取到对方的登录权限，也无法直接登录对方账户——但是，现代的网站往往具备多种登录方式，还存在“忘记密码”这种找回密码机制，而这种机制往往是直接和用户个人信息产生关联的。例如，找回密码往往需要通过个人邮箱或者短信进行验证，而你当然就可能把这些信息修改成自己的，这样验证信息就全部发往你处，而你就可以通过找回密码的方法修改对方的密码了进而实现登录对方账户了！

因此，我们在本章节主要需要用到的就是BurpSuite的抓包功能：首先自己创建一个账号，进入修改个人信息页面，修改后点击提交，而我们就是要抓包这个发往服务器的请求——在这个请求中，我们就能提取到这个链接。最后，自己登录另一个账号后，点击这个链接进行验证，如果发现个人信息被修改，那么我们的攻击就成功了。

好，那么我们就开始这趟“自己攻击自己”的旅程吧！

1、CSRF(get)

先看第一题，首先是一个登录页面：



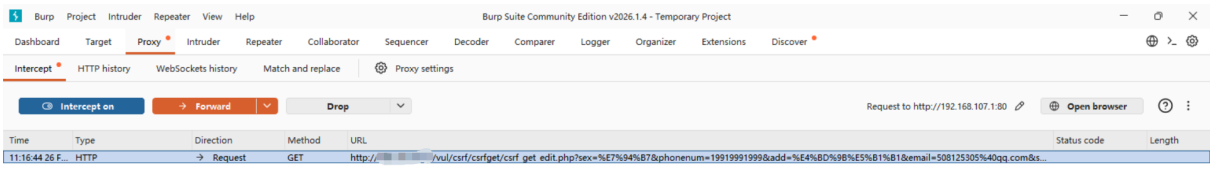
熟悉的页面。不过我们当然没必要手痒再去试试暴力破解，在本章我们要做的是另一回事。点开提示，直接获取账号密码进行登录。

一眼看过去，看来账号的主人都是些篮球巨星啊！那很有攻击的价值了（doge）！

那么先用vince账号进行登录，可以看到这样一个简陋的个人页面，只有一个修改个人信息的功能——那就是我们主要的战场（邮箱是本人真实邮箱doge）：

那么，我们现在开始修改个人信息：

做好抓包的准备，点击submit之后，我们成功抓取到这样的请求：



我们复制粘贴出来：

```
GET /vul/csrf/csrfget/csrf_get_edit.php?sex=%E7%94%B7&phone
num=19919991999&add=%E4%BD%9B%E5%B1%B1&email=508125305%40q
q.com&submit=submit HTTP/1.1
Host: XXX.XXX.XXX.X
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) Apple
```

```
WebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36 Edg/145.0.0.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://XXX.XXX.XXX.X/vul/csrf/csrfget/csrf_get_edit.php
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
Cookie: PHPSESSID=9b5tnubudjii5rcm2il68iivln
Connection: keep-alive
```

这里，我们需要把Host和Get拼接在一起，最终得到这样一行链接：

```
http://XXX.XXX.XXX.X/vul/csrf/csrfget/csrf_get_edit.php?sex=%E7%94%B7&phonenum=19919991999&add=%E4%BD%9B%E5%B1%B1&email=508125305%40qq.com&submit=submit
```

而这个链接，就是CSRF攻击的核心。

我们不妨仔细分析一下这个链接：

`http://XXX.XXX.XXX.X/vul/csrf/csrfget/csrf_get_edit.php` 是地址（只是因为皮卡丘靶场是本地部署的，所以并非一个域名，而是指向文件目录）。而 `sex=%E7%94%B7&phonenum=19919991999&add=%E4%BD%9B%E5%B1%B1&email=508125305%40qq.com` 则记录了我们的输入，包括性别、手机号、地址、邮箱四个类，对应即为我们输入的个人信息（所有中文都被转码过了）；`submit=submit` 则是请求的类型。这三个部分通过特定字符拼接在一起，成为了一个新的网址。

接下来我们要做的，就是模拟被攻击的对象，登录账号，然后点击这个链接！

那么就登录kobe的账号吧！

现在在网址栏输入刚才我们得到的链接 http://XXX.XXX.XXX.X/vul/csrf/csrfget/csrf_get_edit.php?sex=男&phonenum=19919991999&add=佛山&email=508125305%40qq.com&submit=submit （别忘了把XXX.XXX.XXX.X改成真实的ip地址），直接访问：

页面刷新了一下，然后就看到了我们预期的结果——kobe的个人信息已经全部被修改！内容和我们之前的输入如出一辙！

我们退出登录，再次登一下kobe的帐号，发现个人信息还是我们修改后的状态，看来信息确实是入库了——攻击成功！

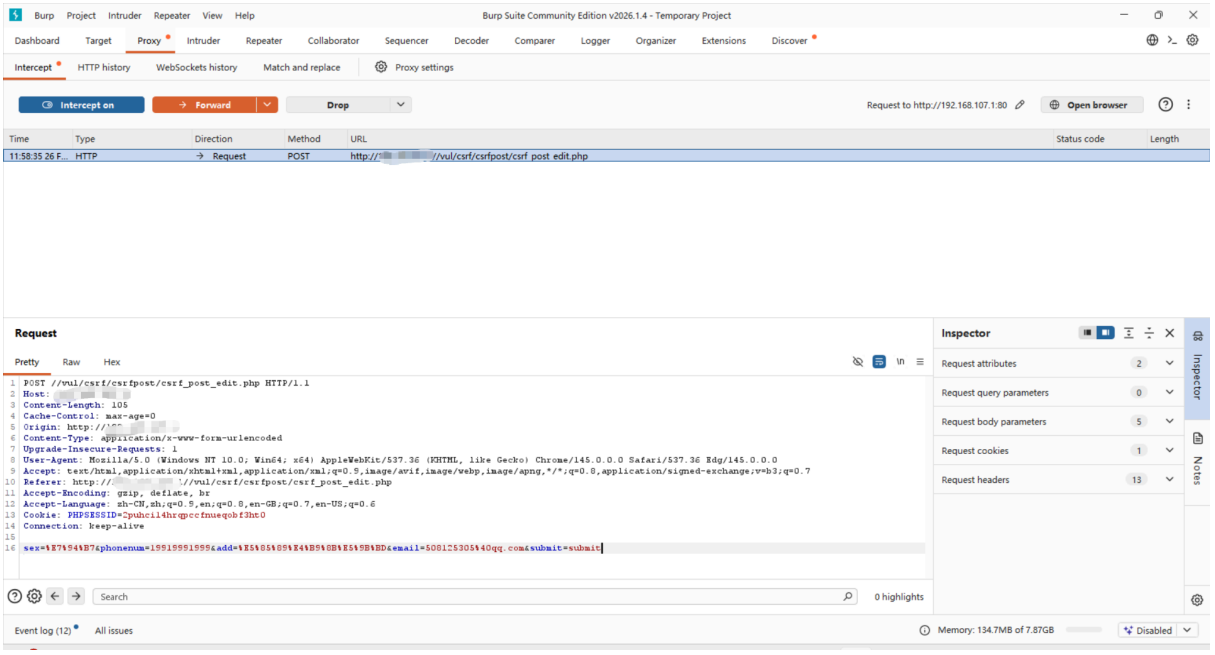
回顾一下刚才我们抓包到的信息，会发现这是一个Get类型的请求，没有验证账号信息，而是直接执行信息修改——因此无论谁处于登录状态下点击这个链接，他的信息就会被直接修改。而这就是CSRF攻击的内涵！

2、CSRF(post)

看到这个标题，再和上一题对比一下，我们不难联想到上一章的反射型xss同样是分成了这两类。显然它们的逻辑应该是差不多的，我们合理推测一下，本题和上一题的差别就在于，上一题是通过拼接网址后缀的方法传递参数给服务器，而本题应该是通过构造请求体来传递参数的，我们不妨实验一下。

还是按照上一题的逻辑，先修改个人信息：

然后点击submit，抓包这个修改个人信息的请求：



接下来还是把抓包到的数据摘录下来：

```
POST //vul/csrf/csrfpost/csrf_post_edit.php HTTP/1.1
Host: XXX.XXX.XXX.X
Content-Length: 105
Cache-Control: max-age=0
Origin: http://XXX.XXX.XXX.X
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) Apple
WebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/5
37.36 Edg/145.0.0.0
Accept: text/html,application/xhtml+xml,application/xml;q=
0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/
signed-exchange;v=b3;q=0.7
Referer: http://XXX.XXX.XXX.X//vul/csrf/csrfpost/csrf_post_
edit.php
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;
q=0.6
Cookie: PHPSESSID=2puhci14hrqpccfnueqobf3ht0
Connection: keep-alive

sex=%E7%94%B7&phonenum=19919991999&add=%E5%85%89%E4%B9%8B%E
5%9B%BD&email=508125305%40qq.com&submit=submit
```

显然，这次的数据和上一次有很明显的区别，首先就是第一行那个醒目的POST，提示我们这一次的传参方式；其次是最后一行，可以看到这就是我们输入的信息，被包含在了一个请求体之中。

但是或许你会想，这和我们要做的有什么区别呢？我们接下来不还是要根据抓包数据进行链接的构造，而所有的数据不是已经齐全了吗？不难构造为：

```
http://xxx.xxx.xxx.x/vul/csrf/csrfpost/csrf_post_edit.php?s
ex=%E7%94%B7&phonenum=19919991999&add=%E5%85%89%E4%B9%8B%E
5%9B%BD&email=508125305%40qq.com&submit=submit
```

接下来，登录受害者的账号，访问这个链接，会发生什么呢？

跳转到了下一题的登录页面？虽然很奇怪（我也不知道咋回事，或许是某种浏览器的重定向机制吧，登录异常就跳转到登录页面），不过可以肯定的是，我们攻击失败了，为什么呢？

原因很简单，因为传参方式不一样，在Post类型中，我们构造的链接根本就不适配！因为这里传参方式根本就不是通过网址进行的，而是通过请求体进行的，因此我们一厢情愿地按照Get类型构造出来的这个链接根本就不是一个正确的命令！

那么，该怎么办呢？其实还是需要构造链接，因为这是CSRF攻击的本质，但是这个链接必须要符合Post类型的格式！换句话说，我们可以通过构造表单来实现！

我们可以新建一个txt文件，把抓包的数据整理成下面这种形式：（为了更直观，这里写成了点击手动提交的方式）

```
<html>
<body>
  <h2>测试表单</h2>
  <form action="http://192.168.107.1/vul/csrf/csrfpost/csrf
_post_edit.php" method="POST">
    <input type="text" name="sex" value="男" />
    <input type="text" name="phonenum" value="19919991999"
/>
    <input type="text" name="add" value="暗之国" />
    <input type="text" name="email" value="508125305@qq.co
m" />
    <input type="text" name="submit" value="submit" />
    <input type="submit" value="点我手动提交" />
  </form>
```

```
</body>
</html>
```

现在，将其保存为html格式的文件，比如命名为“CSRF(post).html”，然后保持另一个账号（kobe）的登录状态（也就是保留那个登陆状态的标签页），双击打开这个文件，点击提交：

我们会发现，这个页面直接跳转到了登录页面，而kobe的个人信息也成功被我们修改了！

（顺带一提，专业版BurpSuite有专门的CSRF攻击工具，但是社区版没有，只能靠这种笨笨的构造方法来攻击了。不过这并不影响我们去理解这种攻击的实质(*^_^*)）

对比一下本题和上一题，可以总结一个经验：GET型是“明码标价”，参数全写在链接里，点一下就中招；POST型是“暗箱操作”，参数藏在表单里，得骗你打开页面才生效。

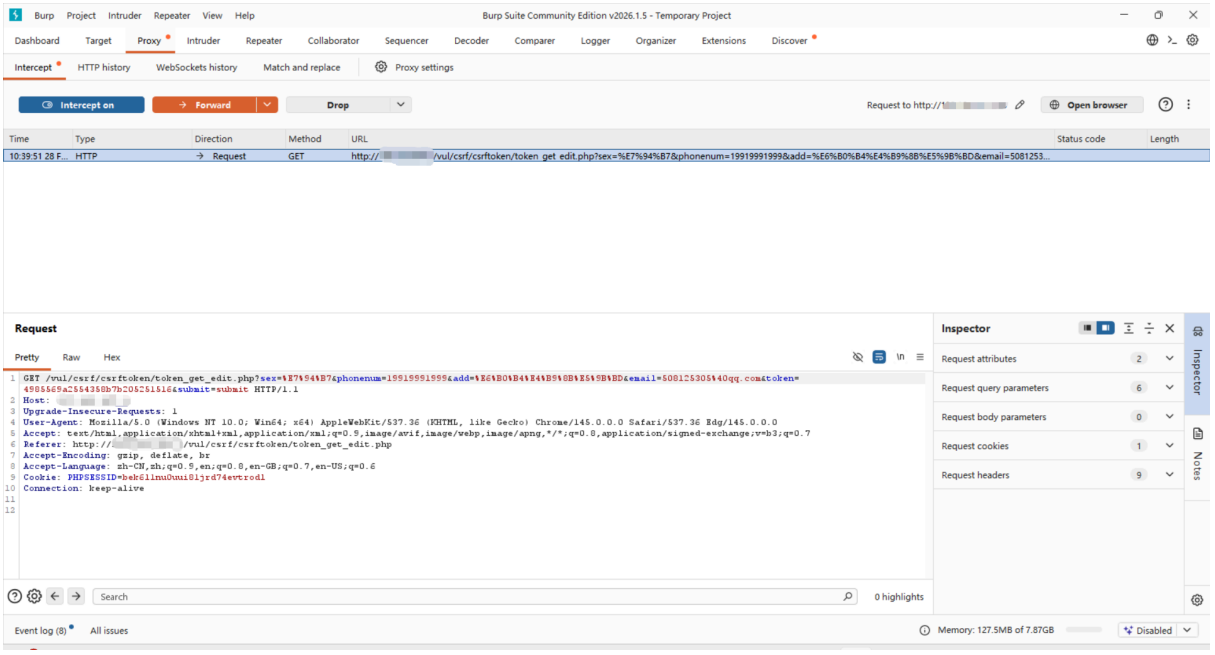
那么，网页链接和html文件，你更喜欢点击哪一个呢？（doge）

3、CSRF Token

来到这一题，看起来登录页面是一样的，但是标题的”Token”提示我们这一题一定有什么不同之处。其实我们之前就见识过token这种机制——在暴力破解那一章节的最后一题，有一个”token防爆破”的设计。在那里，我们是通过Python脚本在每次尝试时

都捕获新的token来绕过这层防御机制的。那么，在这里，我们能否利用类似的方法成功突破token的防御呢？

空想无益，我们还是按照先前的思路，先做一个抓包看看：



我们把有用的部分提取出来（实际上就是第一行）：

```
GET /vul/csrf/csrftoken/token_get_edit.php?sex=%E7%94%B7&phonenum=19919991999&add=%E6%B0%B4%E4%B9%8B%E5%9B%BD&email=508125305%40qq.com&token=4985569a2554358b7b205251516&submit=submit HTTP/1.1
```

对比一下这行代码和CSRF(Get)那一题我们抓包的代码，不难发现这里除了输入的数据之外，还多了一个字段：`token=4985569a2554358b7b205251516` 。看来，这里就是变数所在——而这个token值必然是用来进行校验的，但是，具体是如何做到的呢？

不妨先搁置一边，我们还是按照之前的思路把链接构造出来，先完整地走一遍看看：

```
http://XXX.XXX.XXX.X/vul/csrf/csrftoken/token_get_edit.php?sex=%E7%94%B7&phonenum=19919991999&add=%E6%B0%B4%E4%B9%8B%E5%9B%BD&email=508125305%40qq.com&token=4985569a2554358b7b205251516&submit=submit
```

然后登录另一个账号（还是始终如一“迫害”kobe），访问这个链接：

咦，页面跳转回了最初的登录页面。看来是因为验证失败自动定向到了初始页面——那么，绝对就是那个token惹的祸！

我们不妨多尝试几次，发现在进入修改个人信息页面时，这个token值每次都会变化！当然，这符合我们的预期，毕竟token也是老相识了。

但是问题在于，如果每次进入修改个人信息页面，浏览器都会随机生成一个token值的话，那么就意味着我们通过抓包数据构造的链接中的token值永远都是失效的！

因为，每次我们登录自己的账号修改信息，并通过抓包数据构造出链接时，那个token是当时生成的，那么必然只适用于当时！而如果我们还是依葫芦画瓢地把链接发送给攻击对象，而对方访问这个链接时，就相当于重新进入修改个人信息的页面，同时重新获取了一次token，那么服务器一经比对这两个token值，就必然验证出错，而我们的攻击也必定失败！

那么，或许会想到，之前我们通过脚本的方式实时获取新的token值得绕过方法，能否适用于这一题呢？很遗憾的是，基本不可能。

先前之所以我们能够通过脚本实时更新token值，那是因为我们全程都是在自己的电脑上运行，访问用户就是我们自己，我们要做的只是更新自己的token而已！而这里却不一样，我们需要把自己电脑上获取的token值更新为对方电脑上获取的token值！那么，就必然只能寄希望于于在对方电脑上执行脚本——但是谁又会蠢到在自己电脑上去运行来路不明的脚本呢？

退一步说，就算对方真的运行了这个脚本，那么为什么攻击者不直接做些更大的事情呢？实际上脚本能做的事情太多了，譬如一个来路不明的exe程序能够神不知鬼不觉地把对方所有数据全部偷走，监控对方操作，甚至直接破坏电脑硬件！如果攻击者只是为了更新token以便修改对方个人信息，那么岂不是性价比太低了吗？

虽然完全不必去编译这个脚本，不过我们可以简单地描述一下这个脚本的大致思路——让用户处于登录状态下时，运行这个脚本，就直接从这个登录页面中获取用户的token，然后直接按我们预先设计的格式进行构造，并自动访问这个链接——这样折腾一圈，终于修改了对方账号的个人信息了。

但是还是要重新说回来，既然此情景下用户愿意运行这个脚本，那么就完全没必要拘泥于“修改个人信息”了。要是兜了这么一大圈只是为了做这么一点小事，那也太让人笑掉大牙了（doge）。

看来，在CSRF中引入token验证确实是一个很好的防御手段——它的意义在于，不仅用简单的验证机制很好地防御了大多数CSRF攻击，而且使得能够绕过此机制的许多攻击手段由于成本过高而没有实操意义。

从这里也能看到网安的一个核心思维——不仅是技术的博弈，还是社会工程学的较量。一个成本高昂、设计复杂的的防御机制，往往不如一个虽然简单却使得对方由于攻击成本过高而自主放弃的防御机制来得好。

那么，对于普通人来说，有什么启发呢？现代网站防御机制已经十分健全，没有哪个黑客脑子有病专门针对你的账号展开“酷炫”的攻击，而你要做的只是——

永远不要随便运行来路不明的脚本！

五、Sql Inject

这一章可谓是重头戏了，我想读者就算没有了解过，也应该多少听说过这个可怕的名词——SQL注入。

先看看开发者要发表什么感慨吧：

> **Sql Inject(SQL注入)概述**
> 哦,SQL注入漏洞，可怕的漏洞。
>
> 在owasp发布的top10排行榜里，注入漏洞一直是危害排名第一的漏洞，其中注入漏洞里面首当其冲的就是数据库注入漏洞。**一个严重的SQL注入漏洞，可能会直接导致一家公司破产！**
>
> SQL注入漏洞主要形成的原因是在数据交互中，前端的数据传入到后台处理时，没有做严格的判断，导致其传入的“数据”拼接到SQL语句中后，被当作SQL语句的一部分执行。 从而导致数据库受损（被脱裤、被删除、甚至整个服务器权限沦陷）。
>
> 在构建代码时，一般会从如下几个方面的策略来防止SQL注入漏洞：
>
> 1.对传进SQL语句里面的变量进行过滤，不允许危险字符传入；

> 2.使用参数化（Parameterized Query 或 Parameterized Statement）；
>
> 3.还有就是,目前有很多ORM框架会自动使用参数化解决注入问题,但其也提供了"拼接"的方式,所以使用时需要慎重!
>
> SQL注入在网络上非常热门，也有很多技术专家写过非常详细的关于SQL注入漏洞的文章，这里就不在多写了。你可以通过“Sql Inject”对应的测试栏目，来进一步的了解该漏洞。

概述内容已经十分清晰。简单来说，所谓SQL注入漏洞，就是传入后台处理的数据被当作数据库语法而被执行，而这个语法能够直接操作后台数据库的数据！（关于具体构造后的影响，我们将在后面题目的解说中看到）那么，在这一章中，我们当然没必要去把数据库搞崩（毕竟这是我们自己的数据库），不妨就将获取到数据全部数据作为我们成功的标志。

其实，单从这种攻击方式来看，其实它和XSS漏洞很相似，都是在输入栏精心构造一串代码——只不过后者是构造前端代码，操作的是浏览器，进而可以通过传递参数的方式影响后端；而SQL注入构造的是数据库语法，操作的直接就是后端数据库。

归根结底，它们存在的本质问题在于：开发者永远在相信用户输入的数据，而攻击者永远在利用这种相信。所谓信任的成本，大抵如此！

接下来，让我们开始实操吧！

1、数字型注入(post)

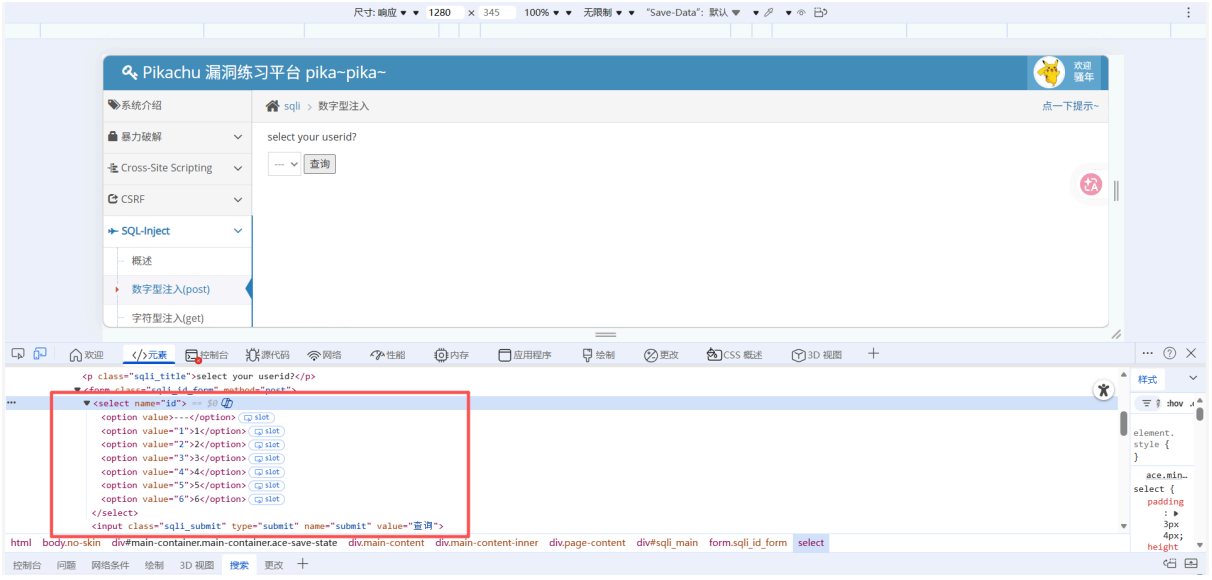
来到第一题，看起来这是一个简单的查询选项。不妨先试试看呗！

随便在列表中选一个数字，点击查询：

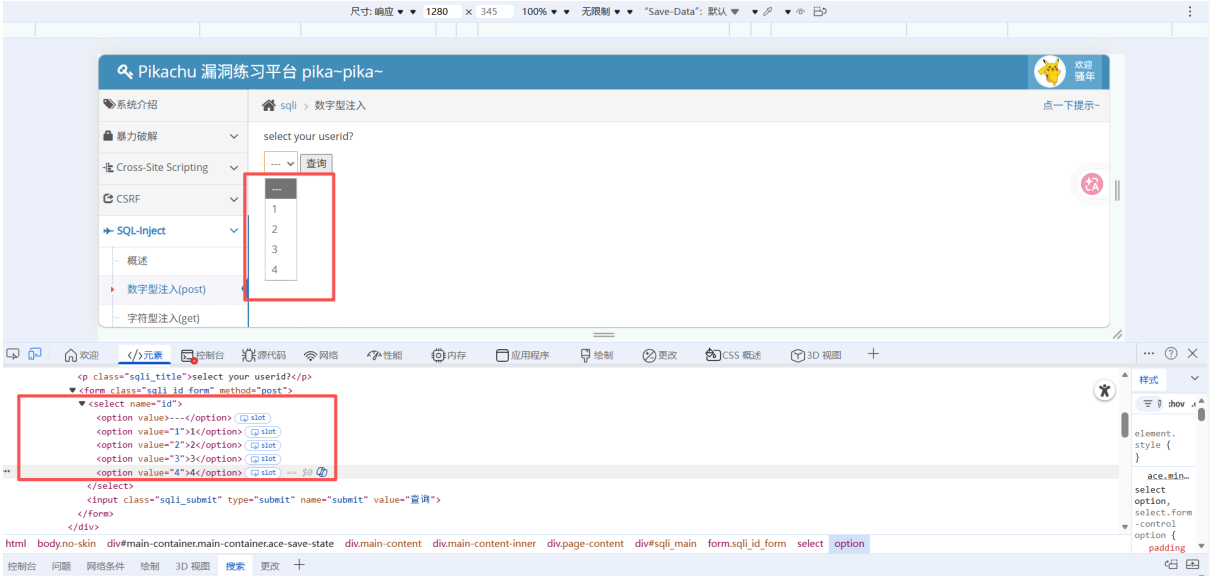
整个过程是很直观的，实际上就是一个输入查询的过程，根据用户输入的内容，匹配到数据库中对应的数据，并传递到前端显示。只是这个输入被开发者通过加入列表的方式约束了，因此表面上看只能选择列表中固定的几个选项。

但是直觉告诉我们，事情绝不是这么简单！具体而言，这个列表是怎么约束的呢？但从这里来看，应该是通过浏览器前端代码的方式实现的——而我们很早就知道，网页前端的代码是可以直接改的！那么如果是这样，这个约束自然也可以被我们直接绕过！

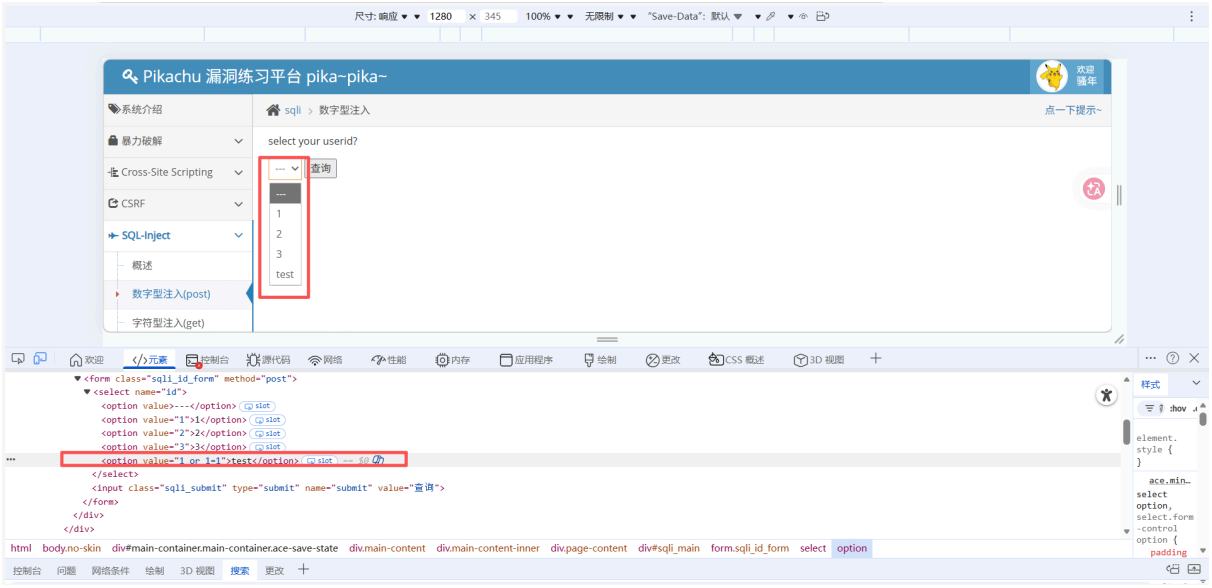
我们打开开发者工具验证一下我们的猜想：



没有错，这的确是通过select语法实现的一个列表选择，使得用户只能选择固定的几个选项。这里我们可以删掉几个元素试试看，比如我把选项5和6对应的元素删除后，点击展开列表后，会发现只剩下了1、2、3、4这四个选项。

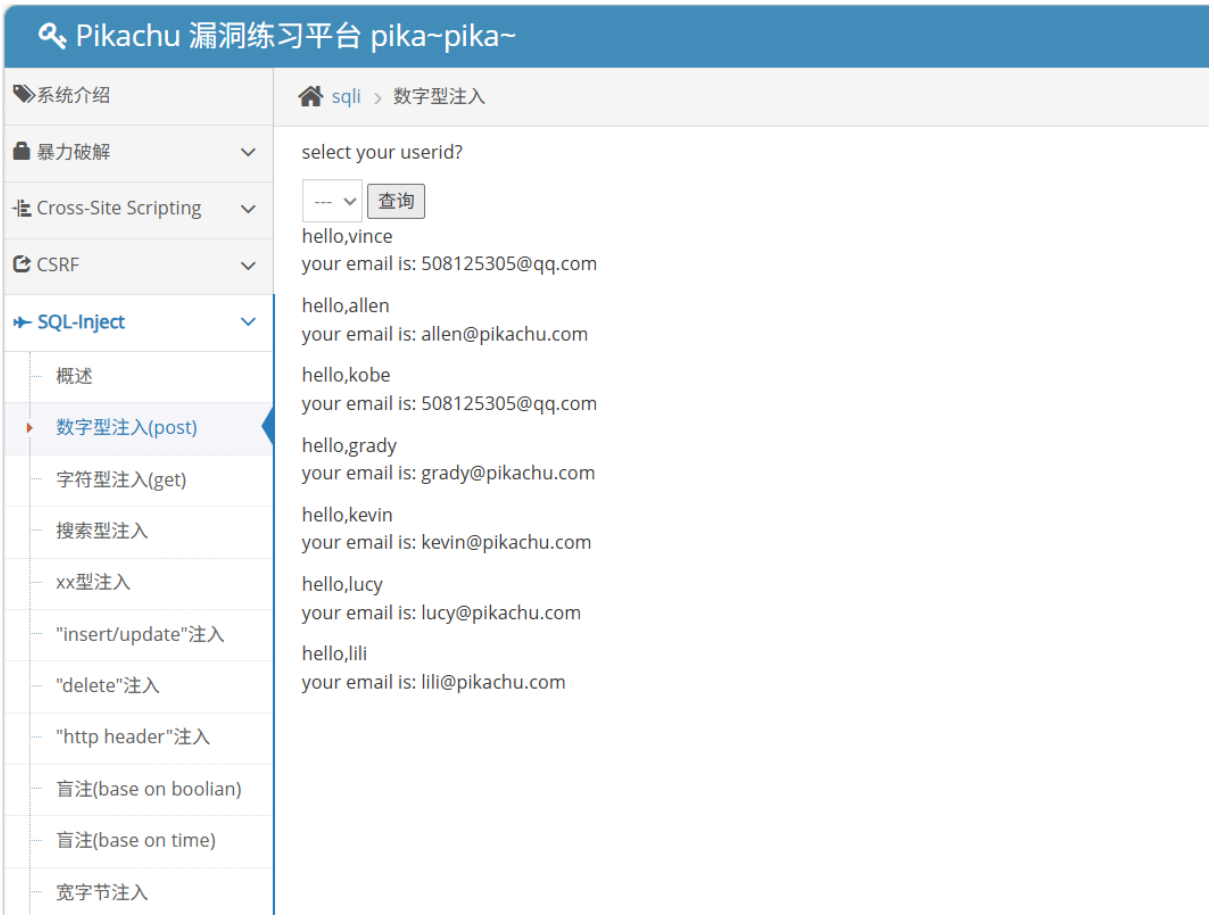


所以，为了成功实施SQL注入攻击，我们不妨直接把其中一个选项改成SQL攻击的语法。这里，关于SQL注入，最典型的语法当然是 `or 1=1` 。先不去想这行代码会引发什么后果，咱们先试试看！



请看，如此一来，第四个选项被我们改名为test，其值被改为 `1 or 1=1` ，也就是之后将传递给服务器的数据。那么就查询看下效果！

看，页面直接返回了数据库中所有数据！而这就意味着成功！



我们显然不应该就此止步，更有趣的问题是，为什么这行输入会引发这样的错误，其究竟在数据库中引发了什么样的连锁反应——这才是更需要关注的问题。

那么，又到了分析源代码的环节咯！我们首先定位到 `sqli_id.php` 文件中的这段代码：

```
if(isset($_POST['submit']) && $_POST['id']!=null){
    $id=$_POST['id'];                    // 直接取用户输入，没有任何过滤！
    $query="select username,email from member where id=$id";  // 直接拼接到SQL语句
    $result=execute($link, $query);
```

正如注释所说的，这里直接读取了用户的输入，没有做任何过滤处理，因此我们的输入就被原样拼接到了SQL语句之中。让我们把这个SQL语句提取出来看看：

```
select username,email from member where id=$id
```

好吧，这不仅唤起了我上学期学数据库的痛苦回忆~好吧，不过这行代码比较简单，用大白话来翻译就是：从member表中选出id=x对应的username和email字段，而这个x就是我们输入的内容。显然，这里期望的就是前端代码所约束的列表中显示的那些数字，所以这里叫做数字型注入。

那么，如果输入我们构造的语句 `1 or 1=1` ，会发生什么呢？

输入的内容被原原本本地传入后端，拼接到SQL语法之中，那么就变成了：

```
select username,email from member where id=1 or 1=1
```

其中， `id=1 or 1=1` 是一个条件判断语句，如果匹配到对应的数据使得这个条件为真，则返回对应的数据。在正常情况下，只有id相同的数据才能满足条件，其他数据自然就被过滤掉了；但是一旦引入了 `or 1=1` ，由于 `1=1` 恒为真，而在 `or` 的语法中，二者一真即真，所以整个条件判断语句恒为真！因此，所有的数据都能满足这个条件，当然会返回所有数据！

而这，就是SQL注入的典型案例。实际上，在特定情形下（默认的MySQL配置下，这种依次执行多条语句的行为操作是被禁止的），我们可以做的更过火一些，比如在后面再加上一段命令，构造为： `1 or 1=1; drop table member` ，如此一来，后端输出所有数据之后会把整张表直接删除！

当然，我们这里只是一个简单的操作，把 `or` 语法传输到后端让数据库执行。咱们对自己的电脑就别折腾了哦！不过通过这个例子，我想你已经能够理解这种SQL注入的实质了，举一反三应该不是难事吧——那就让我们继续闯下去！

最后，要做一个补充，除了直接利用开发者工具修改前端代码从而绕过约束，也可以利用BurpSuite的抓包工具，直接修改传入后端的请求，同样可以绕过前端的制约。一句话，前端约束简单，但并不可靠，我们能够通过很多方法绕过这种约束，因此网安攻防的重点往往就聚焦在与后端代码的斗智斗勇中——毕竟，前端是给人看的，后端是给数据跑的，你说呢？

2、字符型注入（get）

来到下一题，什么是“字符型注入”呢？这和上一题数字型注入有什么区别呢？先不着急看源代码，不妨还是先输入我们最经典的SQL注入的构造命令 `1 or 1=1` 试试看：

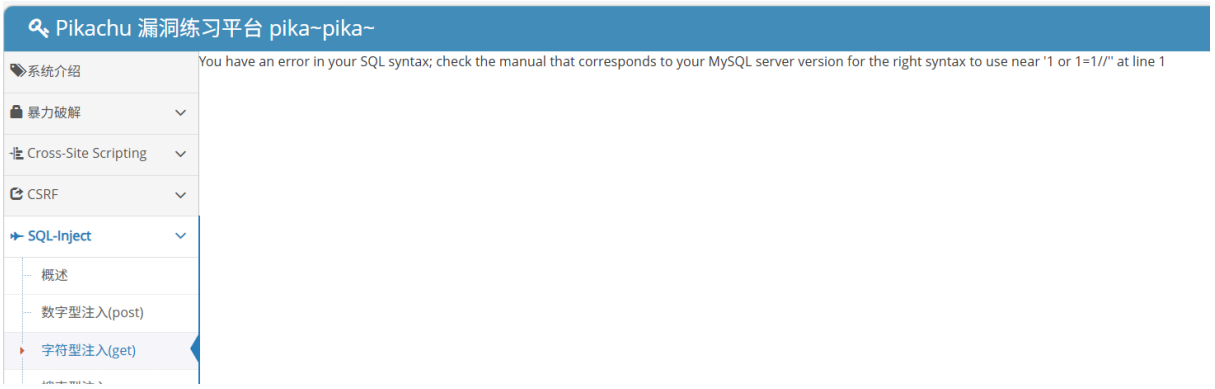
好吧，失败了——不过也是预料之中，其实我们很自然地就能做出猜测，后端是不是对我们输入的内容进行了格式筛选，所以我们原本的数字型的指令就被排除在外了？

那么，如何把一个数字转换成字符呢？在Python中，对于字符串类型一般都是加上引号，不妨联想一下后端是怎么处理我们的输入的呢？或许也是给我们的输入加上了引号？既然整体打上了引号，也就是我们的输入被转换成了一个大的字符串，没有实际的语法含义，因此就变成了查询数据库中有没有 `1 or 1=1` 这样一个id！那当然会显示内容不存在！

不必着急，我们不妨回忆一下“xss之js输出”那一题，同样是存在一个引号的包裹，那时我们是怎么构造输入的呢？先是在最前面加一个引号形成和第一个引号形成闭合，然后在末尾加上代表注释的 `//` ，把第二个引号给注释掉。相同的思路或许可以用在这里？

不妨试试看！（为什么现在不看源代码？现在不正是发散思维的好时机嘛！如果打云端靶场哪有源代码给你看呢doge）

输入 `'1 or 1=1//` ，提交！



好吧，一败涂地。但是让我们看看前端的反馈，这是说我们的输入格式错误，而不是查询不到！也就是说，我们的构造实际上是有意义的，它确实造成了后端解析代码的报错！

反思一下我们的构造，我们其实是按照`javascript`的拼接原理进行构造的，但是从上一题的经验中我们知道，这里应该是把输入的内容拼接到SQL语句中去了！所以，本质上其实是语法的不同，而不能说我们的思路错了！

让我们再联想一下，除了“兑掉”引号、“注释掉”引号，还有没有其他办法呢？其实我们还有一个方法，利用引号！不一定非要让引号之间被配对为空，其中当然也可以有内容，只是这个内容被引号转义为字符而已，但它未必会造成不利的影响！

我们不妨假定这里处理输入的方式就是简单的“两端加引号”，那么就需要利用这两个引号，让它们和我们主动添加的引号产生反应，将输入的内容转义成新的却符合语法的命令！

那么，不妨就把这里不一样的数字转换为字符吧！将`1 or 1=1`写成`1' or '1'='1`，如此一来，被拼接到SQL语句之后，两端再加上引号，整个命令就被转换为`'1' or '1'='1'`——整体是完整的，语法上也没有错！

那就试试！输入`1' or '1'='1`！

成功！那看来这道题对于输入的处理方法和我们的猜想类似！

到这里，如果是打云端靶场，那么已经可以宣告结束了，但是我们既然抱着学习的目的，就不能止于一个“成功”的符号——具体这道题的代码是如何工作的，和我们预期的有什么差距，我们仍然存疑。

那好！我们还是得追溯到后端处理的源代码上来（这就是本地部署给我们带来的便利），让我们找到sqli_str.php文件，容易定位到这样一段代码：

```
if(isset($_GET['submit']) && $_GET['name']!=null){
    $name=$_GET['name'];                    // 直接取用户输
入，没有任何过滤！
    // 这里的变量是字符型，需要考虑闭合
    $query="select id,email from member where username='$na
```

```
me'";
    $result=execute($link, $query);
```

一路代码看过来，其实也就慢慢熟悉这些个老朋友了，世界上最好的语言PHP！很明显，这里的处理方式和我们的猜想是完全一样的，没有对用户输入的内容进行任何过滤，就只是在两边加上了引号而已！

因此当我们输入 `1' or '1'='1` 之后，最终就形成了这样一段SQL语句：

```
select id,email from member where username='1' or '1'='1'
```

和上一题类似的解读方式——原本是要在member表中查询username='x'的数据，并返回其id和email，但是经过我们"别有用心"的构造，`'1'='1'`恒为真，这串命令的条件判断语句`username='1' or '1'='1'`变成了必定成立，因此所有数据都符合条件，而此命令一执行，所有数据也就被和盘托出了！

在这里，如果读者熟悉SQL语法的话，其实按照我们构造javascript的那一题的思路，在最后打一个空格，再加一个SQL中的注释标签`#`的话，效果也是一样的，可以简化命令（比如输入`1' or 1=1 #`）——无论怎样，本质上，就是要防止这个拼接时产生的引号对我们整体构造的影响！

允许我事后诸葛亮一句，在之后形形色色的构造中，这个思路始终如一——我们后面就会看到，无论如何构造，无论我们的输入多么花里胡哨，本质上有用的就只是那其中一部分，而剩下的全是为了骗服务器的拼接、过滤机制而加入的"迷彩服"——核心目的只有一个，就是为了让服务器把我们的输入当作SQL语法的一部分而加以执行！

那么，继续！

3、搜索型注入

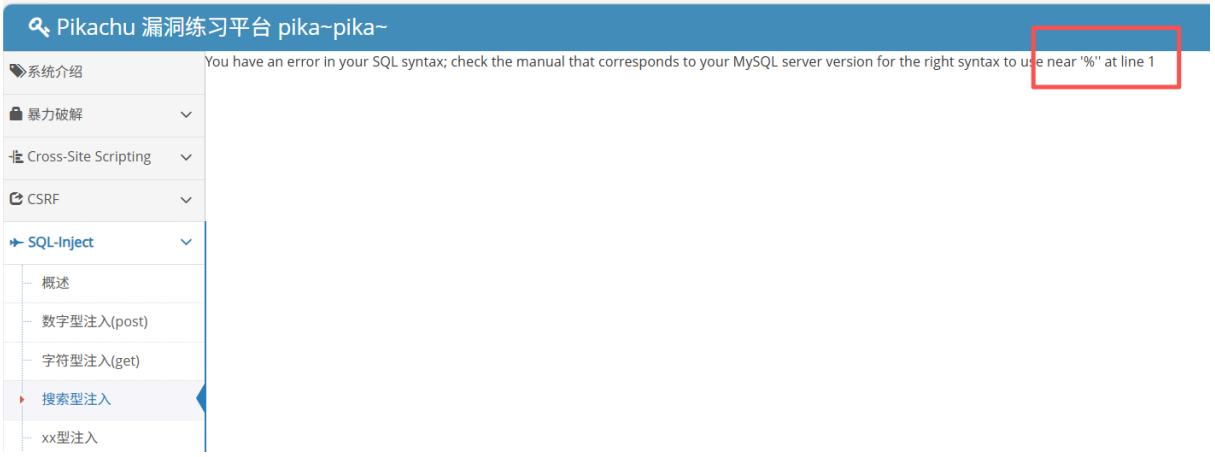
来到下一题，这是一个搜索型注入：

有了前面两道题目的经验，其实我们能感受到，不同类型的注入本质上是取决于输入的内容在源代码中的处理方式。因此，我们的核心任务就是通过不断尝试，根据前端反馈的信息去猜测我们的输入究竟是如何被后端处理的——第一道题是直接代入后端，第二道题是加入了引号，这一题又是如何处理的呢?

换句话说，如果我们能够理解所谓"搜索"是如何在后端通过代码实现的，这一题的构造方式也就呼之欲出了。

所以，我们现在要做的事情就是打破常规，不要输入正常内容，但也不要强求直接构造出一个完整的命令，不妨先输入一些特殊字符——如果后端没有响应的过滤机制，那么说明本题很有可能有SQL注入的漏洞!

不妨试试，输入上一题我们用于闭合的字符 ' ，点击搜索：



报错了！这正是我们期望的！而且这里报错信息未经任何处理而直接反馈到前端，我们得以窥探到一点“天机”：

```
You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '%'' at line 1
```

这不仅提示我们发生了MySQL语法错误，同时也提醒我们这里的错误原因是关于%的。如果读者熟悉SQL语法的话，这里我们几乎已经可以下定论了——搜索功能+%，因此这里用的绝对是SQL中的like语法！

到这里，我们基本上已经能够确定了后端的处理方式——输入的内容被后端拼接为`'%input%'`的格式——而当我们猜到这种格式的时候，构造也就不再是难事了。

到这里，不妨直接对照对应的源码，验证我们的猜想！

找到sqli_search.php文件，容易定位到下面的代码：

```
if(isset($_GET['submit']) && $_GET['name']!=null){
    //这里没有做任何处理，直接拼到select里面去了
    $name=$_GET['name'];

    //这里的变量是模糊匹配，需要考虑闭合
    $query="select username,id,email from member where username like '%$name%'";
    $result=execute($link, $query);
```

很明显，这里代码的处理逻辑就是将输入的内容直接拼接到SQL语句中去，而SQL语句中是带有引号和百分号的——正是like模糊匹配的语法！

也就是说，我们这里需要构造一个特别的式子，通过特定位置的字符把引号和百分号的影响取消，并且包含一个有效的恒等式以获取所有数据！

注意在这里，百分号是通配符，也就是本身可以代表任意长度的字符，因此我们不仅需要闭合引号（所谓闭合，就是我们之前一直在做的，拼凑出几个对子！），也需要把百分号处理掉，要使它作为一个正确的语法而存在。这里最简单的方法就是直接用一对引号将它包住，也就是说我们要在构造的开头加一个引号。

实际上都到了这一步了，就没有任何难度了，最简单的方法是在首部构造闭合，尾部直接注释掉，也就是SQL注入宇宙中最万能的构造 `' or 1=1 #` ：

看，前端返回了所有数据，再次成功！

回顾一下后端代码，当我们输入 `' or 1=1 #` 之后，SQL语句就变成了：

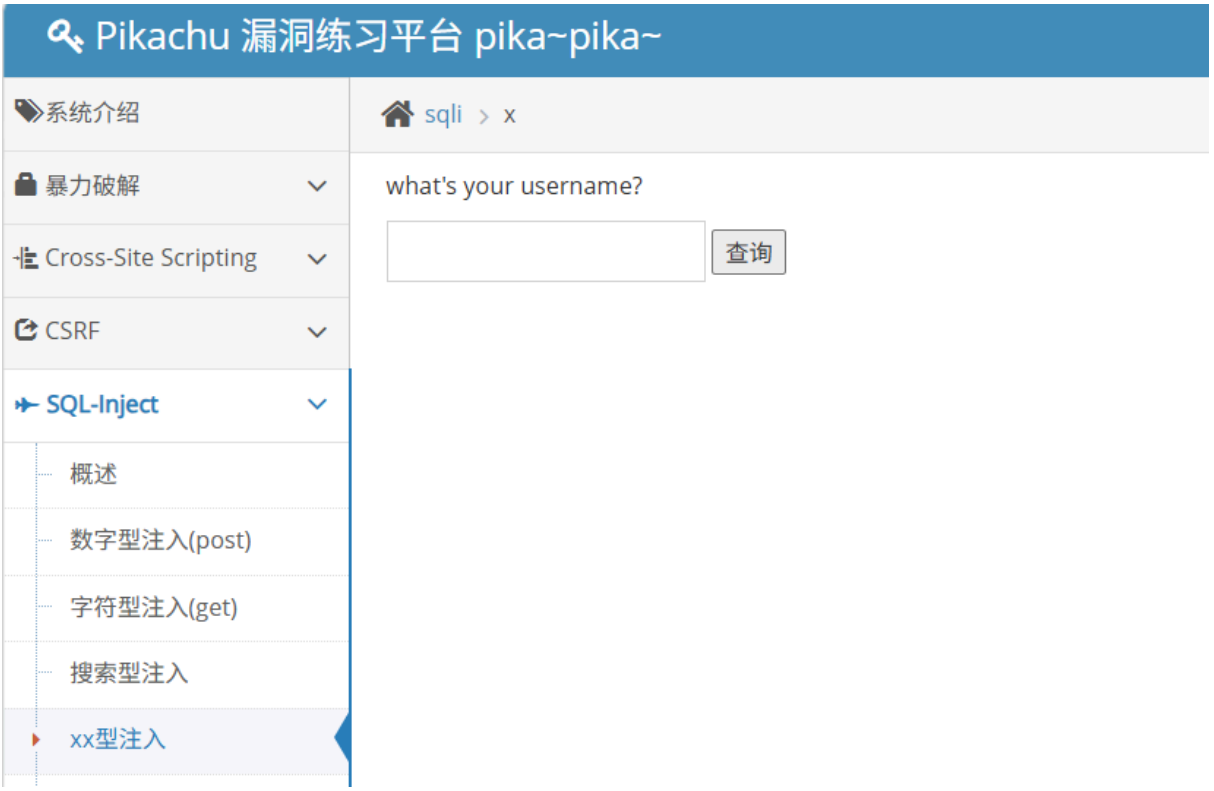
```
select username,id,email from member where username like
'%' or 1=1 #%'
```

这段代码的含义是：在member表中选取username字段满足“任意字符开头”和“恒真”的两个条件的数据，并返回其username,id和email。和前面我们构造的含义是类似的——都是通过恒真的条件把所有数据一网打尽！

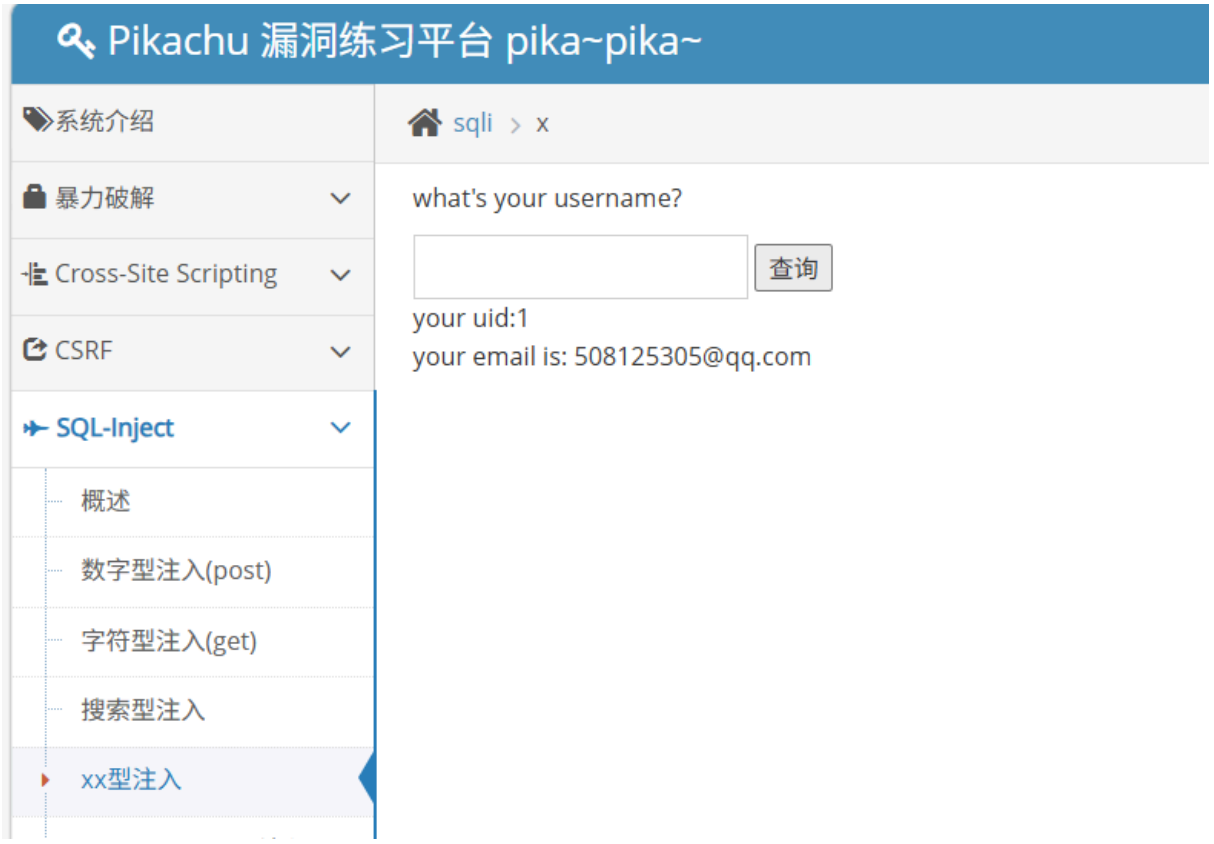
趁热打铁，下一题！

4、xx型注入

到了这一题，看上去似乎没什么特别的，也没什么提示，就只是问问我们的用户名而已，如果不看标题，还真不知道这一题和前面几题有什么区别：

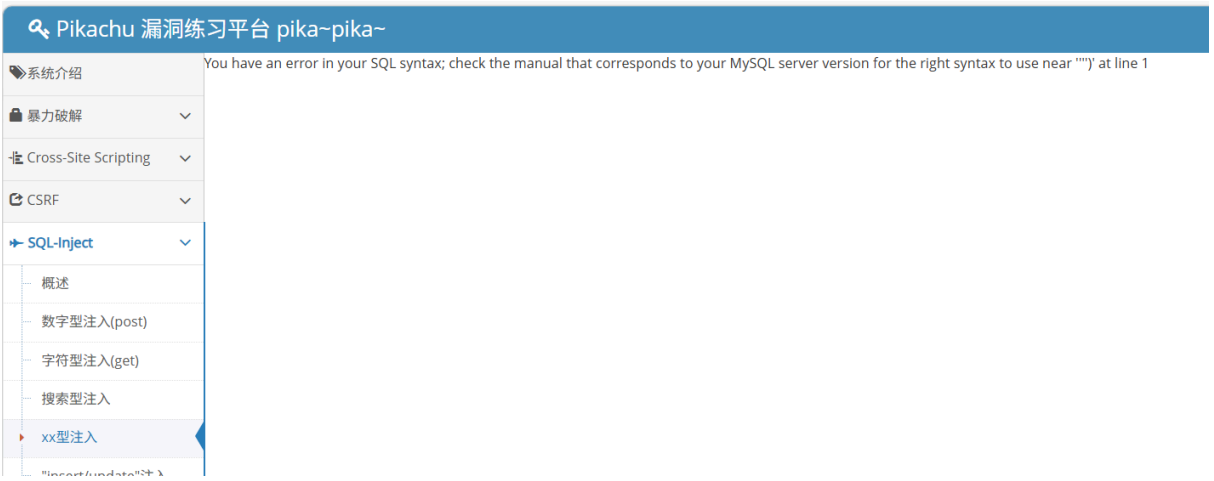


不如还是先输入一个真实的用户名试试水，还是用vince吧（这里几个数据库都是共用的）：



看起来很正常，应该就是一个简单的在数据库中的用户名匹配。好像没什么很明显的思路，那只能做好多次尝试的准备了（很正常的啦）。

还是先试试一个单独的引号，看看会不会报错：



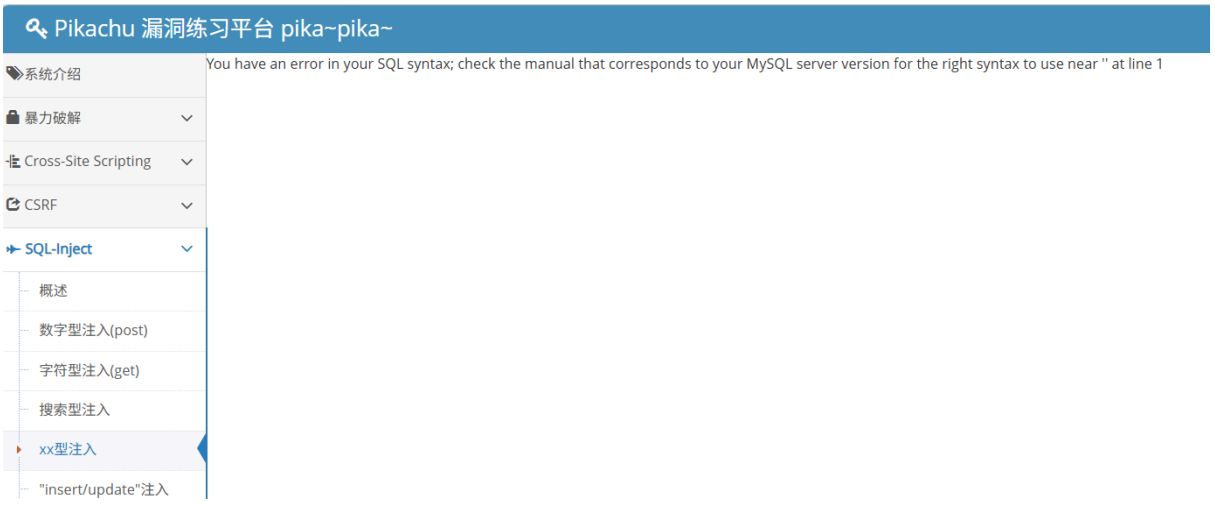
很好，报错了，这个报错信息好像有点东西：

```
You have an error in your SQL syntax; check the manual that
corresponds to your MySQL server version for the right synt
```

```
ax to use near ''''')' at line 1
```

可以看到，错误信息发生在 `''')` （把最外层引号脱掉！）附近，我们只是输入了一个引号，那么为什么会导致这种错误呢？而且这次不一般的地方在于——有好多引号！我们自己输入的引号在哪里？另外，这个括号又是什么意思？

好吧，似乎还是没什么头绪，那不妨先试试那个万能的构造 `' or 1=1 #` ——总之先碰碰运气嘛！



呃，还是报错了，所谓"万能"也有失灵的一天啊！不过看看这个报错信息，和刚才的又不太一样：

```
You have an error in your SQL syntax; check the manual that
corresponds to your MySQL server version for the right synt
ax to use near '' at line 1
```

这次如果把引号脱去，中间竟然什么也不剩了？太奇怪了，我们必须要冷静分析分析了。

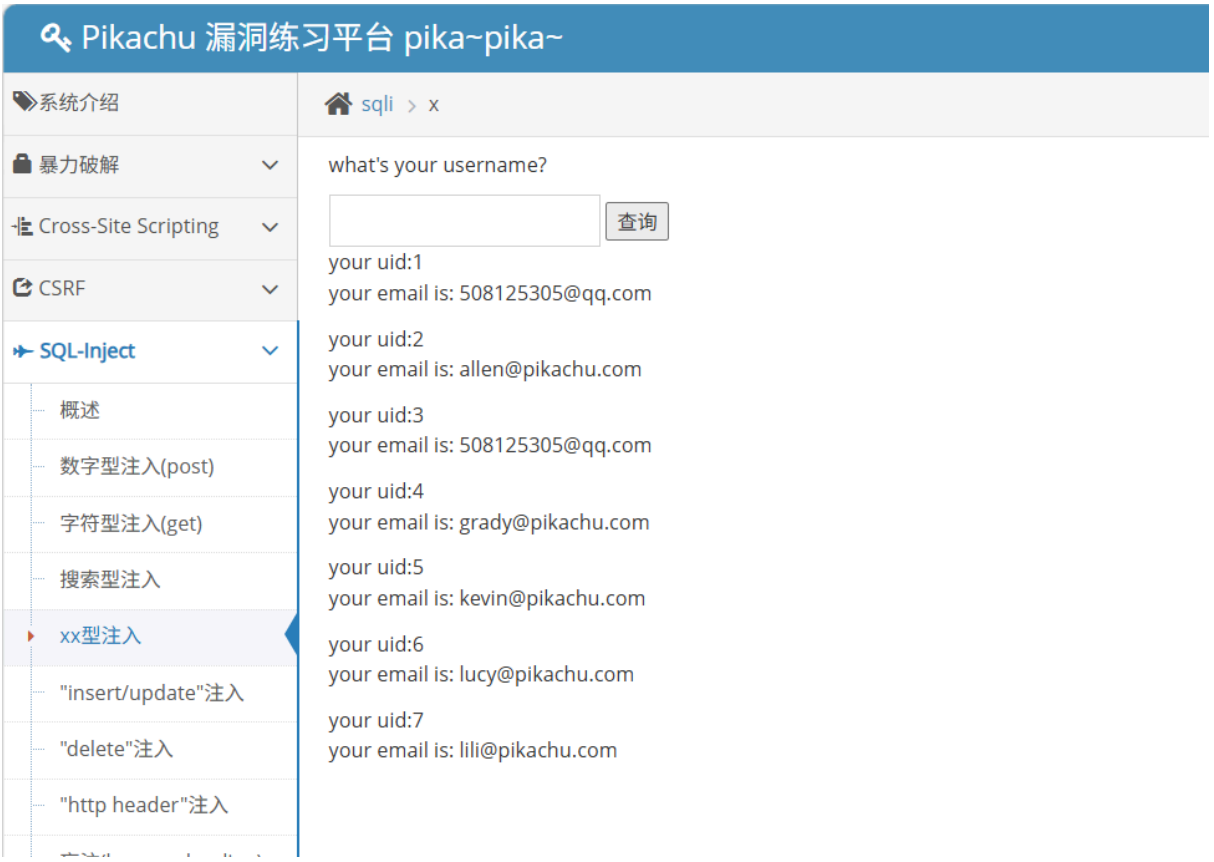
对比一下

这两次报错信息，前一个有括号，后一个没有，再看看我们构造的输入，第二次后面有一个注释标识符#，那么可以推断：这个括号应该是被注释掉了，而且位置一定是在输入的后面——但尽管如此，前面这个引号还是会引发报错！

那又是什么原因？如果后面有括号的话，是不是不太对称呢？如果读者熟悉SQL语言的话（或者其他语言，甚至数学），应当能够猜测到一定有另一个括号与之匹配——因为括号绝对不可能单独出现，必须要有另一个括号用来匹配才能构成完整语法，那么这个括号一定出现在最前面！

所以我们模模糊糊地能够猜测，这里后端的语法应该是存在一对括号的！我们只是注释掉后面的括号，却忽视了前面的括号的影响，只是不假思索地加上一个引号用来闭合——如果形成了 `'(` 形式，那么不存在括号的闭合问题，不会发生报错，所以必定是形成了 `('')` 形式，如此才存在括号未闭合的语法错误！

到这里，我们的思路就比较清晰了，我们需要在前面再加一个 `)` 专门用来闭合后端代码中的第一个括号！而且这个括号的位置一定是在第一个引号后面，试试 `') or 1=1 #` ：



成了！看来 `') or 1=1 #` 就是正确的构造，而从这个构造中，我们就能推理出后端对应的语法——应该是类似 `('input')` 这种结构！

那么——看看源码？

```
if(isset($_GET['submit']) && $_GET['name']!=null){
    //这里没有做任何处理，直接拼到select里面去了
    $name=$_GET['name'];
    //这里的变量是字符型，需要考虑闭合
    $query="select id,email from member where username=('$name')";
```

正如注释所提示的，还是把用户的输入直接拼接到SQL语句中，而把这个SQL语句提取出来：

```
select id,email from member where username=('$name')
```

和我们的预期完全一致！到了这个地步，读者应该完全能读懂这段SQL语句了：从member表中选出所有username匹配的数据，并返回其id、email数据。当我们输入 `') or 1=1 #` 之后，这段代码就变成了：

```
select id,email from member where username=('') or 1=1 #')
```

同样的，这里"1=1"没有被任何字符影响（全都被我们闭合掉了），再次发挥了其"恒真"的伟大力量，所有数据都符合条件，因此前端返回了所有数据的相应信息！

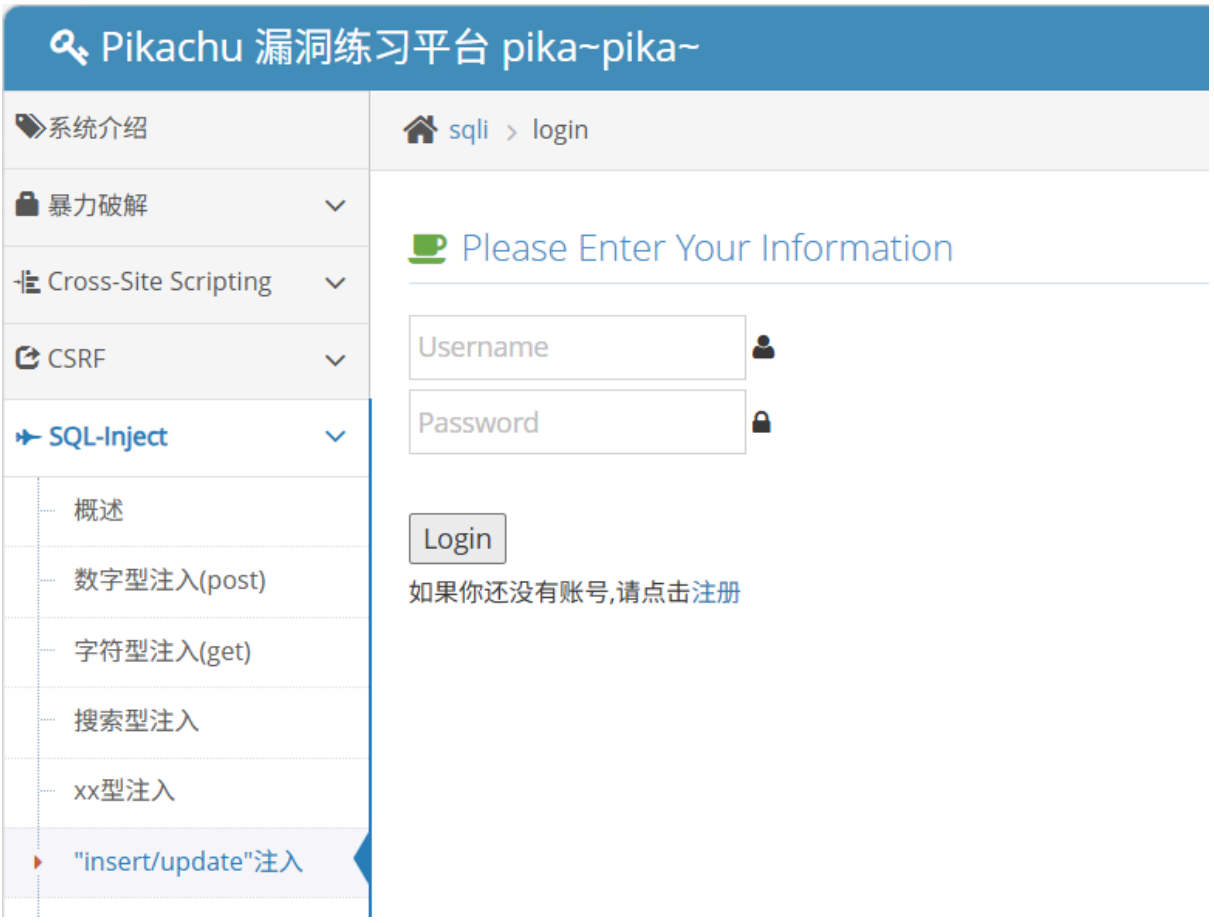
而只有到了这一步，再回头看这一题的标题：xx型注入，搜索一下，我们就知道这种注入的核心特征了——括号型注入，语法特点就是那多出来的一对括号。

诚然如本题开发者在提示中所言："管tmd的什么型,能够制造出闭合,就是本事。"

如果单独看这道题，谁知道这是什么型呢？只有一次又一次尝试，看看报错信息，才能一点一点推理出来呀！

5、"insert/update"型注入

来到这一题，看起来和前面的题目有很大区别，是一个登录注册页面：

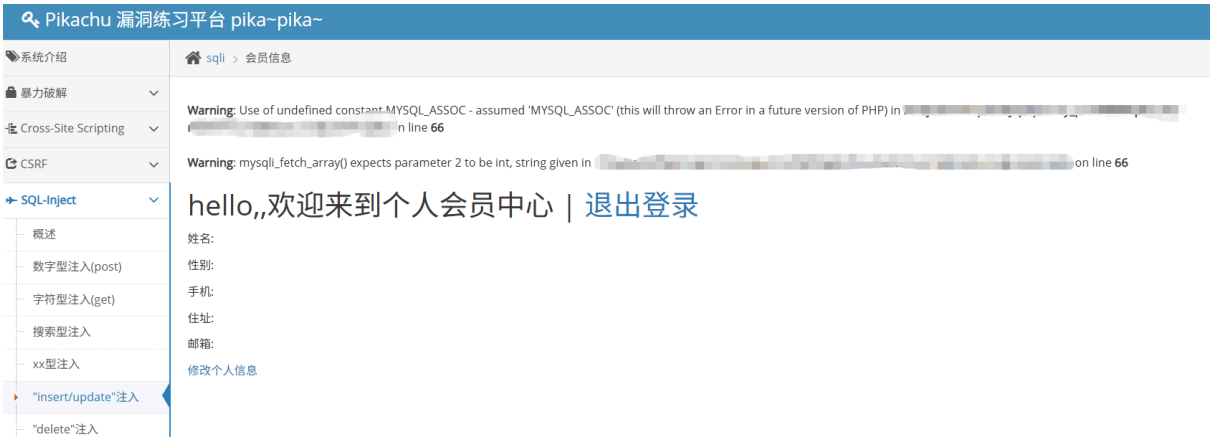


但是，有了之前SQL注入的经验，其实这一题也没有什么大不了的——正如之前我们一直在做的，只要有输入框，就存在SQL注入的可能！而且说到底，所谓SQL注入就是构造语法——从本质上来说，和我们之前一直在做的事情应该是异曲同工的。

不管怎么样，咱们先试试看！

先注册一个账号试试看咯，不然连登录都进不去：

输入用户名和密码，点击提交之后，返回登录页面，用我们刚才注册的信息登录一下看看：



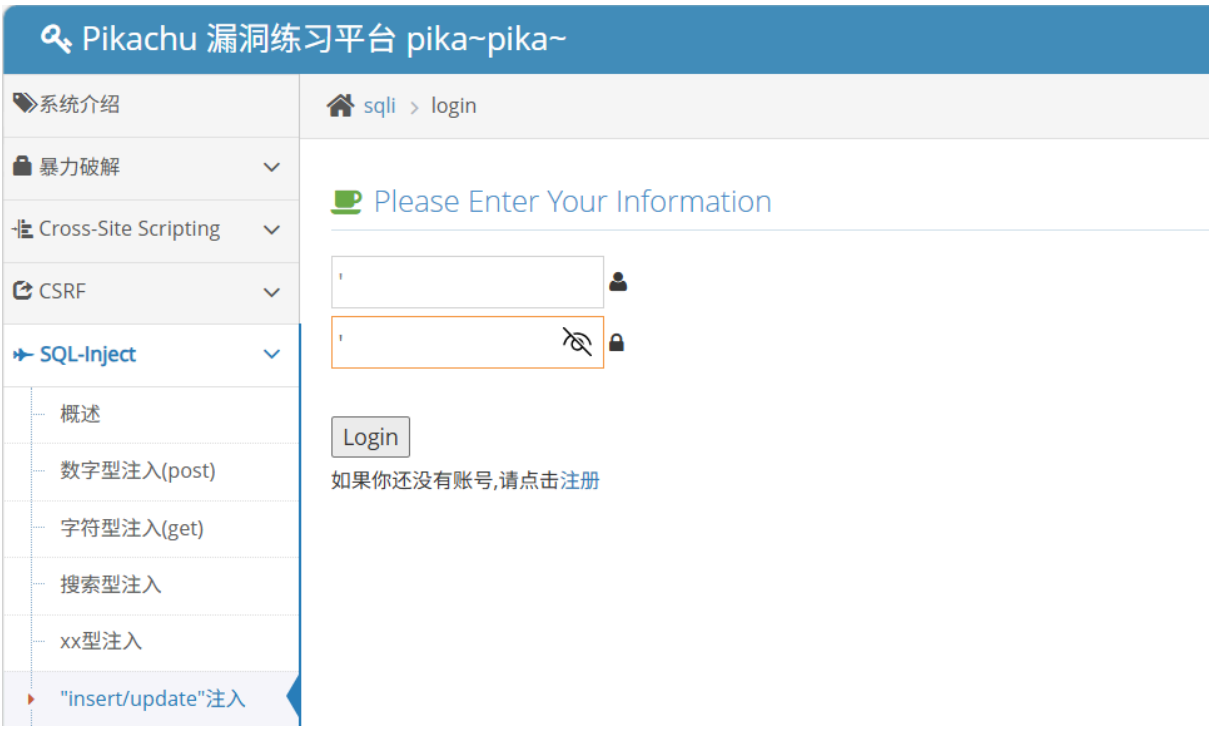
很好，成功进入了个人信息界面（还请读者无视上方的两行警告，那个大概率是我最初配置皮卡丘靶场的时候留下的隐患，但是不影响正常功能(っ °Д °;)っ）

那么，这里我们就做好了准备工作，接下来就是正式进行SQL攻击了！

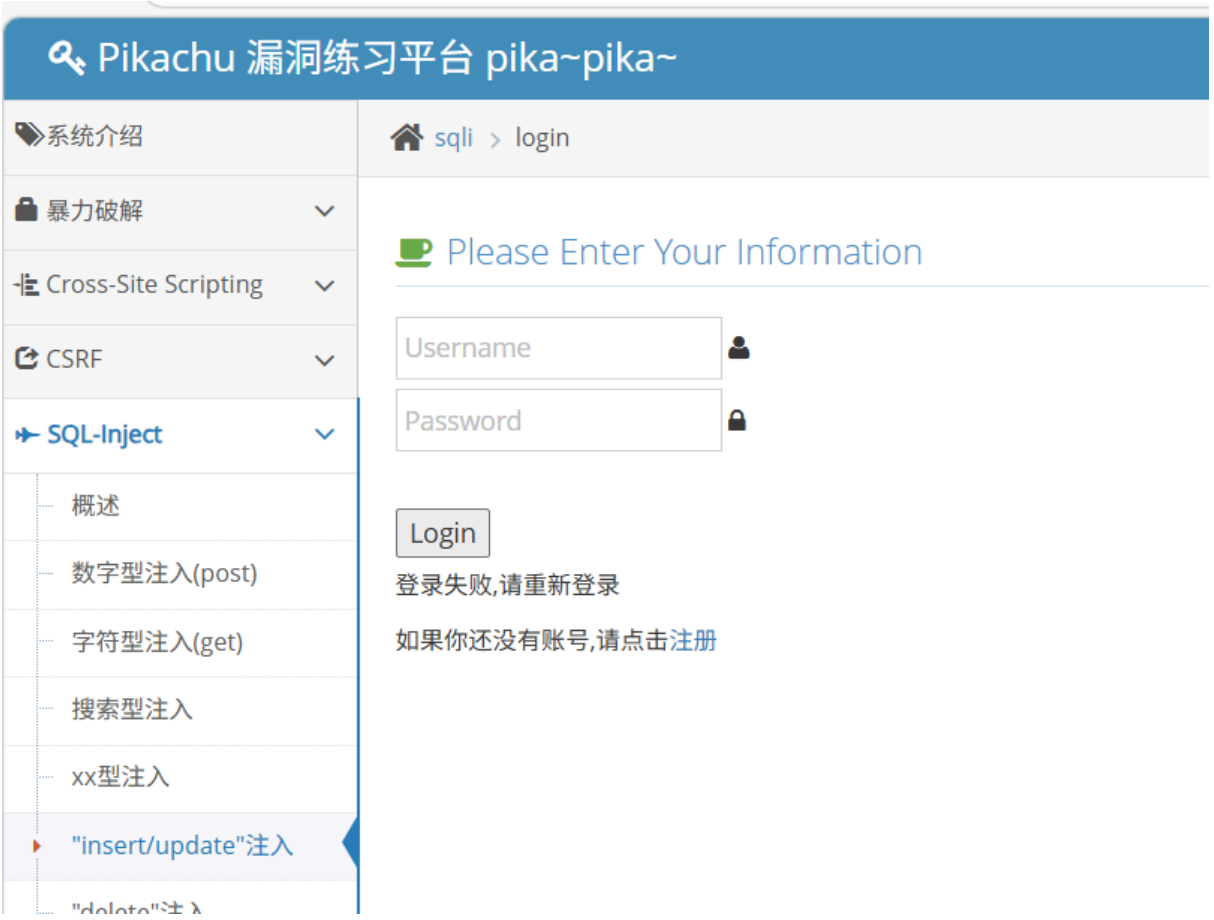
等下！这里我们要追问，为什么这个过程叫做“准备工作”？

所谓SQL注入，本质上是在数据库中的一种操作，那么首先得有这个数据库，而且这个数据库中得有数据，这样我们的注入才有实际意义——而注册操作就是往数据库中“填数据”的过程（这里本题的数据库默认为空）。而有了这个数据，我们才能验证自己的注入操作。

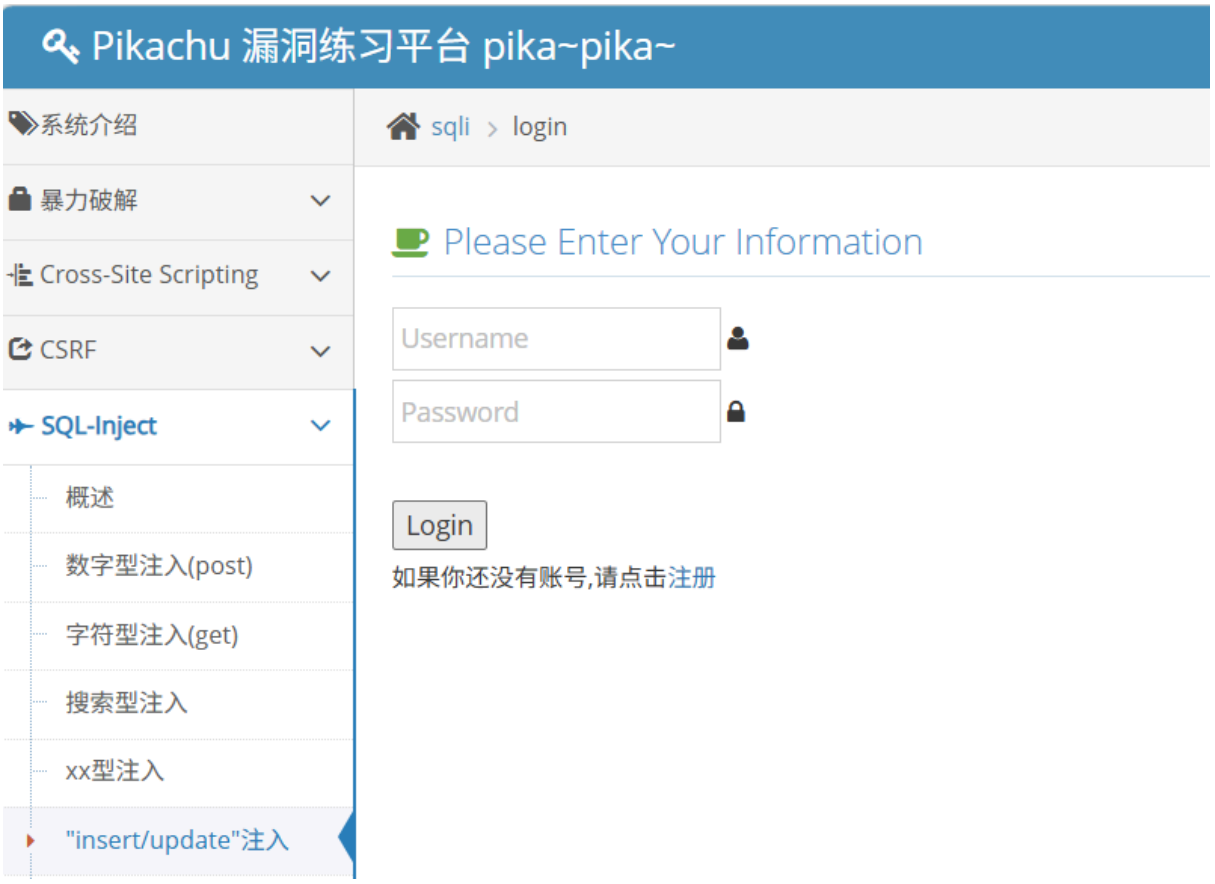
很好，那么我们还是老样子，先要在登录页面搞出一个报错，先试试为我们立下汗马功劳的引号：



先试试看，点击登录：



好吧，登录失败了。但别灰心，这次不输入密码，直接在用户名框输入引号，密码框留空，点击登录：

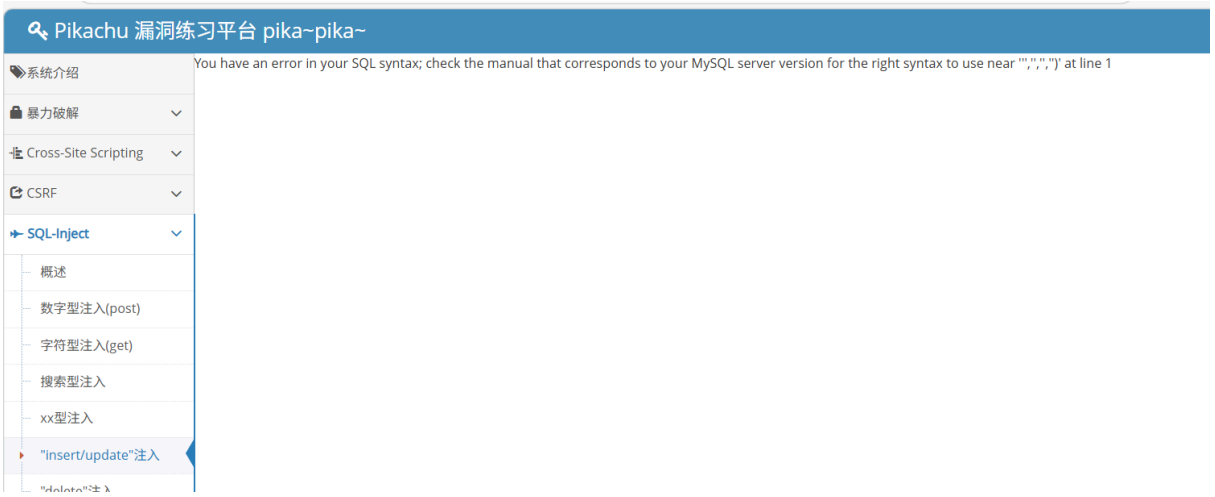


这次还是失败，不过看起来好像不太一样：前端没有显示“登录失败”，而是闪了一下跳回了最初的登录页面——这很可能是引发了网页的重定向机制，看来前端应该是有对于数量的检查，用户名和密码都要填才行。

先别急着下论断吧，毕竟现在信息还是不太够，先试试其他输入框吧！

这次咱们来到注册页面，还是输入引号（这里用户名和密码都是必填，否则就会提示“必填项不能为空”，这里就不展示了）：

点击提交之后，发现页面返回了报错，而且这次有具体的错误信息（来自未来的补充说明：只有用户名和密码都仅输入一个引号才能引发报错，否则会注册成功）：



错误信息如下：

```
You have an error in your SQL syntax; check the manual that
corresponds to your MySQL server version for the right synt
ax to use near ''',''',''')' at line 1
```

这次错误发生在 `''',''',''')'` 附近，这个好像有些端倪——用逗号分隔，中间是一对引号，而且刚好是三个逗号分隔出了4个空间！对比一下我们的注册页面，刚好也是四个输入框，那么合理推断，这就是输入框对应的后端代码格式！

而且更进一步地分析，我们可以看到，错误的原因就在于我们输入的引号导致了引号不配对（剩余了引号）！信息收集到这里，基本上有些眉目了。

这里，我们要结合一些SQL语法的知识，能够推断出后端的代码结构大致是这样：

```
INSERT INTO 某张表 (字段1, 字段2, 字段3, 字段4) VALUES ('$inpu
t1', '$input2', '$input3', '$input4');
```

虽然有一些细节还没有完全搞清楚，但是对于SQL攻击而言，很多攻击都是在模糊的情况达成的。那么，现在不妨先试试看构造一个注入命令。

这里，我们的思路就是通过构造，取消引号的影响。那么，不妨还是用我们"万能构造"的思路——闭合前面，注释后面：输入 `admin') #` 试试看！

这里由于用户名和密码必填，所以我们也象征性地给密码栏也填上——其实密码栏填什么都可以（这里只是一个前端约束而已），因为在理想情况下，我们的当前输入会被拼接形成这样一行代码：

```
INSERT INTO 表名 VALUES ('admin') #','$input2', '$input3',
'$input4')
```

可见，后面所有代码都被注释掉了，实际上就只剩下了第一个用户名数据是有效的。

虽然似乎还有些没搞懂的地方，但不妨先试试看呗！

好吧，报错了，报错信息是 `Column count doesn't match value count at row 1` ，也就是说我们输入的值的数量不匹配。看来后端应该是有一个检查数量的逻辑，如果不满足六个值（包含空值），那就会返回错误。不过这个不难，我们可以通过构造解决。

这一次，我们稍作修改，构造为 `admin', '123456','','','','') #` ：

点击提交，显示注册成功，那么我们的构造就成功了，这里我们注册了一个账号：表面上看上去用户名为 `admin', '123456','','','','') #` ，密码为1；但实际上用户名是admin，密码是123456，后面有四个占位的空值。因为这里第一个输入框把后面的数据给注释掉了，实际上登录框就已经包含了用户名和密码，当然也可以进一步包含下列所有数据！

这就是insert注入（勉强可以算作吧），也是本题的第一个部分，但是这并不是一个完整的SQL注入——因为它根本就没攻击！事实上，此处是一个新增数据的过程，我们当然能猜测到它不会去匹配数据库其他数据，所以我们在这个页面能做的大体上也就“到此为止”了吧！

不管怎么说，这个逻辑是无价的——所谓登录，不过是查询匹配罢了，那就回到了我们前面的故事了：如何一步一步测试，找出登录对应的后端代码的形式，就能够实施攻击！

那么，回到我们第一次尝试时的登录页面，我们又能如何进行SQL注入攻击呢？

回头看看这个登录页面，它的基本逻辑是：去数据库里找用户名和密码都匹配的那一行数据，如果找到就登录成功，如果找不到就登录失败。那么，什么叫做匹配呢？所谓匹配，在最初的那两题就已经为我们揭示了，或者是直接匹配数字，或者是加上引

号匹配字符，那么在这里考虑到用户名和密码的通用性，基本判断为字符型的匹配方式。

那么，我们就需要构造引号的闭合！

不妨还是先试试万能构造，先用引号闭合前面的引号，再注释掉后面的代码，比如构造为`admin' #`——如果能够成功的话，就会不管密码而只是查询有没有用户名为admin的数据，会是一个很好的线索。

那么，用户名输入`admin' #`，密码随便输入（反正都要被注释掉）：

点击登录——咦？登录失败？那很坏了。难道是我们构造有问题？

这里读者可以自己去试一试，会发现似乎不管怎么构造，都会返回登录失败——或许有一种可能，后端对我们构造的字符进行了过滤！

如果后端真的有过滤的话，那就不好办了。我们不得不去考虑新的方法——想一想这一道题其实是由两部分构成，一个是注册，一个是登录，二者的关联能否加以利用？如果说，我们一开始就在注册的时候埋雷呢？

不是不可能！回忆一下我们刚才在注册页面的构造，输入了那么多引号和括号，都是可以正确响应的，那么至少在注册的后端代码处是没有过滤机制的！

所以，我们现在的方向就很明确了，要尝试一下在注册的时候埋雷！

而我们埋的雷，就和我们刚才在登录页面的思路一样，都是要拼接，那么就在这个时候拼接！但是，这里发现了问题所在——

考虑到拼接的要求、括号的存在以及后端对于数量的校验，我们如何能够做到既满足注册页面的代码闭合，又能实现登录时的埋雷（一般是需要构造恒等式或者末尾留有注释）呢？我反正是没招了.....

不过，值得宽慰的是，看看标题，这里没有要求在登录时候进行注入攻击啊！insert注入我们已经实现了，剩下的那个是update啊！所谓update，就是更新，而用自己注册的账号登录之后，确实是有一个修改个人信息的功能！那么，应该是这里存在漏洞吧！

而且，看看这个修改个人信息的页面，很像啊！这和我们注册的页面差不多啊，那么我们当然理所当然地会想到，应该是一样的构造方法，这里就不再赘述了。

但是，直到这里，我们仍然只是在做一些“测试注入点”的工作，但还不能算一次完整的注入攻击——一个完整的SQL注入应当满足的效果是，通过注入获取了额外的数据或达成了非预期的效果！

那么，该怎么办呢？

坦白地和读者说：到这里，我已经有点崩溃了，这也是本人真实的尝试的极限。

现在，我要去查攻略了！

好！我查到答案了，这里第一栏输入 `' and extractvalue(1,concat(0x7e,(database()))) and '1'='1` ，剩下几栏随意输入，就可以看到这样的响应：

响应的信息是：`XPATH syntax error: '~pikachu'`，这是什么意思呢？查查看吧。

原来，`extractvalue()`是MySQL的一个XML函数，它本来的用途是解析XML文档并提取XPath路径（一种在 XML 文档中查找信息的语言，类似用路径找文件）对应的值。但是，如果我们故意给它一个格式错误的XPath路径，它就会报错，并且在报错信息里把错误的路径显示出来！

这里，`0x7e`是波浪号`~`的十六进制表示，`concat(0x7e,(database()))`就是把波浪号和当前数据库名拼在一起，形成`~pikachu`——这显然不是一个合法的XPath路径，所以报错了，而报错信息里就带出了数据库名`pikachu`！

这就是传说中的报错注入——不通过页面回显，而是通过报错信息把数据“吐”出来。

回头看看，这一题确实绕了好大一个弯子——一开始在注册页面折腾半天，以为insert注入只能“插入”不能“攻击”，结果没想到是用报错注入的方式，直接把数据库里的数据给“炸”出来了。

而且仔细想想，这个报错注入的点其实是在注册页面，而不是我之前一直纠结的登录页面。题目叫“insert/update注入”，意思就是注入点发生在insert语句里，而不是select语句里——只不过我们习惯了从select里拿数据，一时没转过弯来。

至此，这一题也算是拿下了。虽然中间崩溃过、查过攻略，但还是注入成功了，值了！

最后，让我们来看看后端代码，看看是怎么回事吧！以sqli_reg.php文件的源代码为例，我们定位到这样一段代码：

```
//没转义,导致注入漏洞,操作类型为insert
$getdata=$_POST;
$query="insert into member(username,pw,sex,phonenum,email,address)
        values('{$getdata['username']}',md5('{$getdata['password']}'),'{$getdata['sex']}','{$getdata['phonenum']}','{$getdata['email']}','{$getdata['add']}')";
```

这里的重点在于：第一，没转义——开发者特意注释了“没转义,导致注入漏洞”；第二，6个字段：username, pw, sex, phonenum, email, address；第三，密码用了md5()——你输入的任何密码，都会先被 MD5 哈希（一种单向性的加密算法）再存进去。

为什么我们输入 `' and extractvalue(1,concat(0x7e,(database()))) and '1'='1` 之后，能够显示数据库名？

把它填在 username 框里，拼进 SQL 后长什么样？

```
insert into member values(
    '' and extractvalue(1,concat(0x7e,(database()))) and '1'='1',
    md5('随便填的密码'),
    'sex值',
    'phone值',
    'email值',
    'add值'
)
```

这里最关键的一点是：我们构造出的引号把代码自带的第一个引号闭合，然后又利用第二个引号闭合了1!

那 `extractvalue()` 是怎么执行的？

这就不得不提到一个MySQL的特性了，在 MySQL 里，如果你写：

```
select '' and extractvalue(1, '~') and '1'='1';
```

猜猜会发生什么？——`extractvalue()` 会被执行！

因为 `and` 是逻辑运算符，它会强制把两边的表达式转换成布尔值。
而 `extractvalue()` 是一个函数，不管最终结果如何，它都会被调用，返回一个值（或者报错）。

所以在我们的 INSERT 语句里，第一个字段的值不是简单的字符串，而是一个表达式：

```
'' and extractvalue(...) and '1'='1'
```

这个表达式会被 MySQL 计算，因为存在函数，就必须执行 `extractvalue()`，一执行就报错，报错信息里带出了 `database()`！

这就是为什么我们能在 INSERT 语句里用 `extractvalue()` 报错注入！

坦白地说，这真的是我最汗流浃背的一题了。就算是教程，也写得很简略，我也完全看不懂，还是靠着AI才能勉强理解一点。SQL的宇宙，真是神奇啊！

6、“delete”注入

来到下一题，是一个留言板，下面有一个留言列表：

先输入一段留言试试看，搞清楚这道题的机制。这里输入“我是一个正经的人”，点击提交：

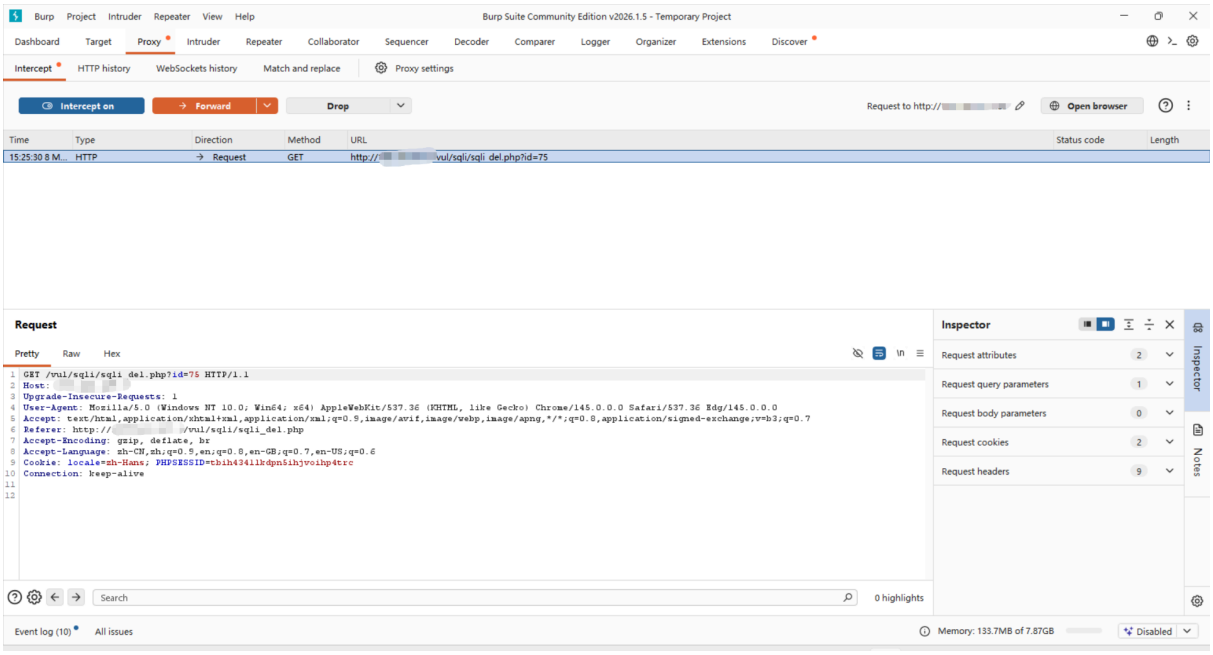
提交后，下方的留言列表更新了我们刚才的留言。看来，这里的工作机制应该是：前端将我们的输入传递到后端，存储在数据库中；而服务器读取数据库内容，并返回前端显示。问题在于，SQL注入究竟发生在哪个阶段呢？

所以，我们还是一样的思路，要先搞清楚后端对于我们数据的处理方式，还是先用引号试试水：

留言列表正常更新了，看来这里引号并没有引发报错！那么，再试试其他字符？这次直接输入#，看看能不能发生报错：

好吧，还是不行，看起来这里一定是存在一些防御机制，不过我们目前还猜不到。但是，如果在"存储到数据库"这个过程中我们的构造失败的话，或许可以试试"删除数据"这个操作。和前者单一的存储操作相比，后者多了一个查询和删除的逻辑，也许有更大的操作空间！

这里，如果只是点击删除，当然看不出什么端倪，但是这里既然是数据库操作，就必然存在前后端通信，那么具体是如何通信的呢——抓包一下看看！



把抓包信息提取出来：

```
GET /vul/sqli/sqli_del.php?id=75 HTTP/1.1
Host: XXX.XXX.XXX.X
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) Apple
WebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/5
37.36 Edg/145.0.0.0
Accept: text/html,application/xhtml+xml,application/xml;q=
0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/
signed-exchange;v=b3;q=0.7
Referer: http://XXX.XXX.XXX.X/vul/sqli/sqli_del.php
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;
q=0.6
Cookie: locale=zh-Hans; PHPSESSID=tbih4341lkdpn5ihjvoihp4tr
c
Connection: keep-alive
```

看着很复杂，但是重要的数据其实只有一行：

```
GET /vul/sqli/sqli_del.php?id=75 HTTP/1.1
```

这说明，删除操作的传参方式是Get（参数直接写进网址），参数名是id，参数值是75。这是一个数字型参数！而且有了前几题的经验，我们知道这个参数一定是被拼接到了SQL语句中，并作为SQL语法执行。

我们大致上可以猜到（猜不到就要去查查SQL基本语法啦！），后端删除操作的语法应该是类似这种：

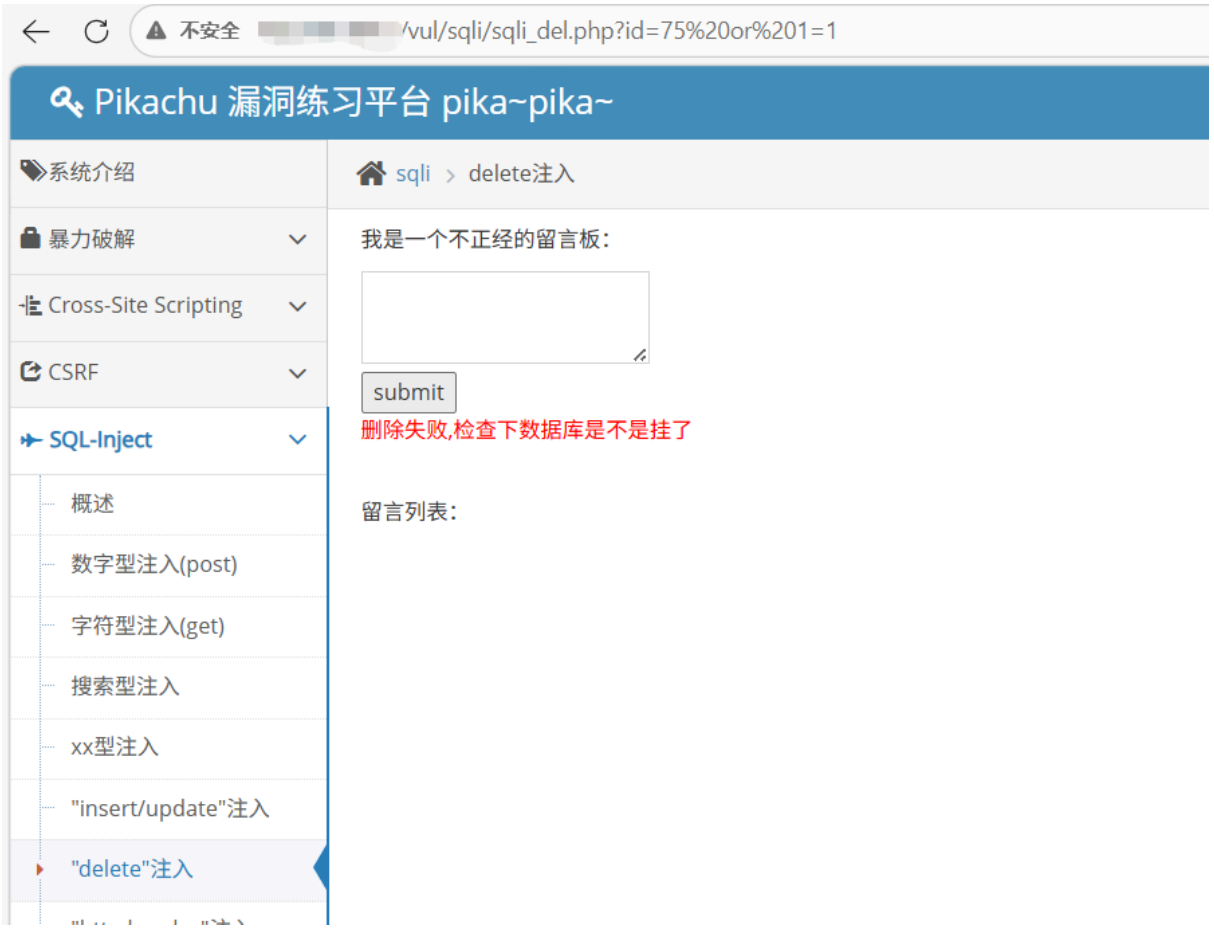
```
DELETE FROM 某张表 WHERE id = X
```

在最理想的情况下，如果这个数字型参数没有经过任何处理就被拼接到SQL语句中的话，我们就可以沿用第一题数字型注入的逻辑进行构造攻击！至少这应该是我们首先应该尝试的方向！

当然，由于这里传递的是id，所以在数据库中的查询条件也是id是否匹配，而与我们输入的内容无关，所以在留言板中的操作是毫无意义的。而由于这里是Get类型传参方式，我们可以直接在网址栏进行构造。不妨就模仿我们在第一题中的构造格式 `1 or 1=1` ，将网址构造为：

```
http://XXX.XXX.XXX.X/vul/sqli/sqli_del.php?id=75 or 1=1
```

理想情况下，访问后会发生和第一题类似的效果——所有数据都满足删除的条件，因此全都被删除！



效果拔群！所有留言列表都被删的一干二净，此外这里显示："删除失败，检查下数据库是不是挂了"。看来开发者也是早有预料啦！

在这里，SQL注入的形式和之前都大不相同，不再是在前端的输入框中进行构造，而是直接在网址栏进行构造！其实本质是一样的——读者不妨回忆一下我们最初在XSS那一章节中接触到的post和get两种传参类型，先前我们在输入框的一顿操作就属于post，而本题就属于get——无论如何，都是在构造输入，都是为了将输入构造为SQL语法从而被后端执行！

好啦！现在看看源代码吧！找到sqli_del.php文件，容易定位到这样一段代码：

```
// 注意看这里——原本有一行被注释掉的代码
// if(array_key_exists('id', $_GET) && is_numeric($_GET['id'])){

// 真正执行的是这一行——没有任何过滤！
if(array_key_exists('id', $_GET)){
    $query="delete from message where id={$_GET['id']}";
```

```
    $result=execute($link, $query);
    if(mysqli_affected_rows($link)==1){
        header("location:sqli_del.php");
    }else{
        $html.="<p style='color: red'>删除失败,检查下数据库是不
是挂了</p>";
    }
}
```

定位到那行SQL语句：

```
delete from message where id={$_GET['id']}
```

很明显这里传入的参数没有经过任何过滤，就被拼接到SQL语句中加以执行，而当我们在网址栏构造命令之后，这行SQL就变成了：

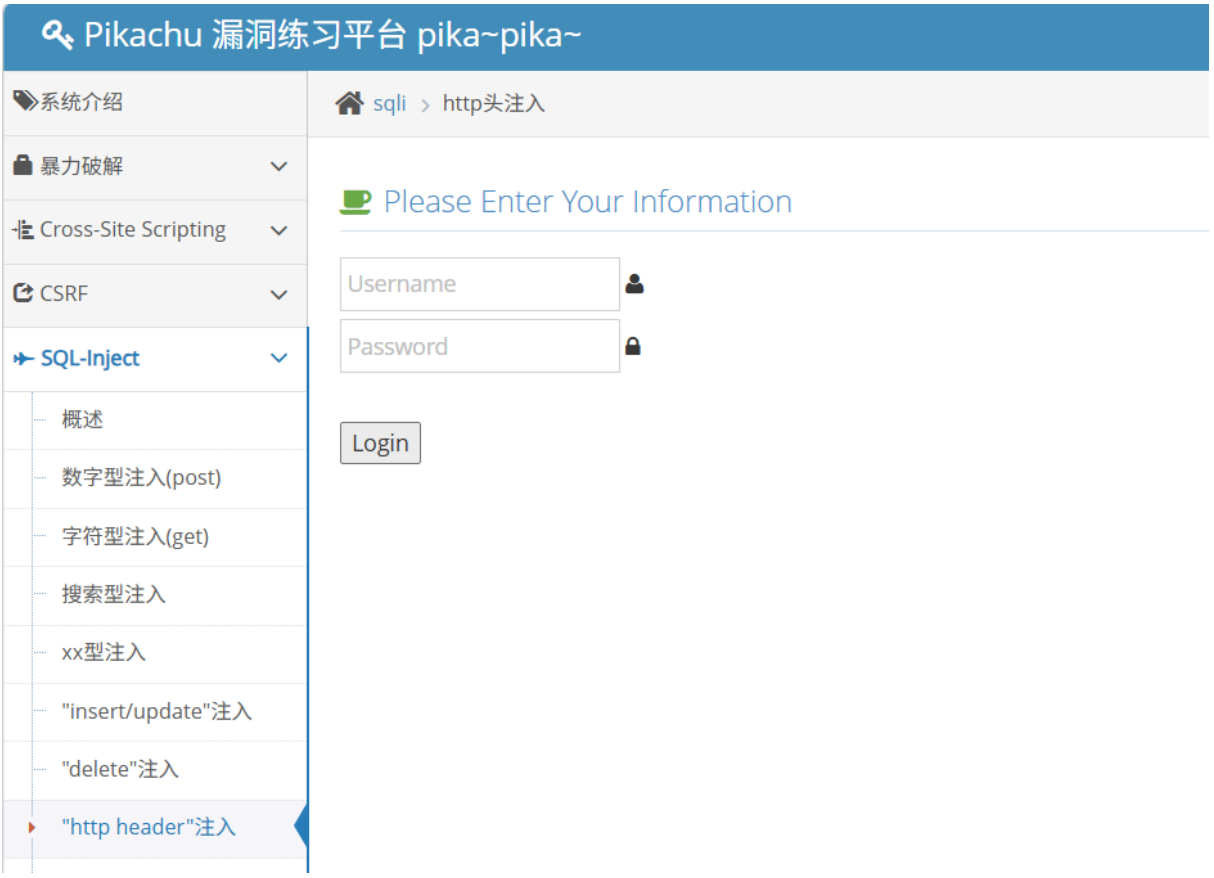
```
delete from message where id=75 or 1=1
```

如此一来，正如我们所预期的，所有留言数据都会满足删除条件而被统统清空！而这就是“delete”注入！

顺带一提，这里开发者注释掉的那行代码，恰恰就是过滤条件——仅仅筛选出纯数字类型的参数。从这里，我们也能看到开发者的小巧思——看来开发者从来都是等着有缘人来读源代码呢！

7、“http header”注入

来到下一题，又是一个登录界面，不过单凭这个我们还不能知道这一题的SQL注入到底是怎么一回事，总之还是先正常走一遍看看：



旁边的提示里面有登录账号信息（admin/123456），咱们直接登录，瞅瞅看隐藏在登录入口后面的究竟是什么面目：

呃，这是……难道是被开盒了？

当然不可能！让我们把这段信息脱敏后发给AI，看看AI怎么说：

> 其实这就是**HTTP头注入**的典型场景——服务器把你的请求头信息（IP、User-Agent、端口）**老老实实地记录到数据库里**，然后**老老实实地显示在页面上**。

这里有两个要点，一个时请求头信息会被发送到后端，另一个是这个信息会被存储到数据库并且返回前端。这段描述恰恰揭示了其与我们之前所做的SQL注入并没有本质区别，只是形式发生了变化——这个请求头信息和输入框或网址栏没有任何关系，而仅仅与我们的网络条件有关。

因此，当我们要进行构造时，既不是在登录框里面进行构造，也不是在网址栏进行构造，而是需要直接修改发往后端的请求——那就必然要借助BurpSuite抓包了！

顺带一提，其实前面所有题目的SQL注入都可以通过BurpSuite抓包后直接修改发往后端的请求来实现，换言之，其实在哪里构造都问题不大，重点是在请求中发送了什么数据给服务器——从这个角度而言，抓包修改其实是最接近本质的方法。

在这里，让我们重新填写登录信息，打开BurpSuite进行抓包，首先会获取到这样一段请求：

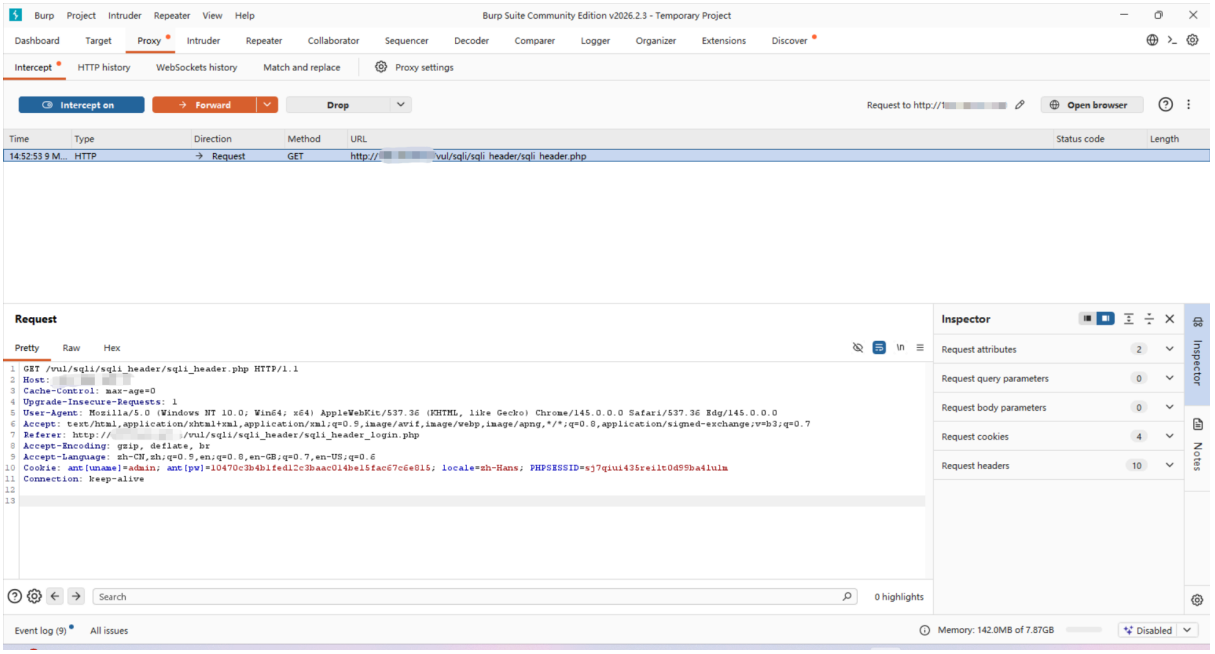
```
POST /vul/sqli/sqli_header/sqli_header_login.php HTTP/1.1
Host:XXX.XXX.XXX.X
Content-Length: 43
Cache-Control: XXX.XXX.XXX.X
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) Apple
WebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/5
37.36 Edg/145.0.0.0
Accept: text/html,application/xhtml+xml,application/xml;q=
0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/
signed-exchange;v=b3;q=0.7
Referer: http://XXX.XXX.XXX.X/vul/sqli/sqli_header/sqli_hea
der_login.php
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;
q=0.6
Cookie: ant[uname]=admin; ant[pw]=10470c3b4b1fed12c3baac014
be15fac67c6e815; locale=zh-Hans; PHPSESSID=sj7qiui435reilt0
d99ba4lulm
Connection: keep-alive
```

```
username=admin&password=123456&submit=Login
```

遗憾的是，这段请求没有意义，因为它不过是我们登录时发出的请求，并没有我们需要的信息。

我们真正需要的，是那个获取我们网络信息并发往后端的请求！如果那个请求不在这里，那就只有一种可能——在登录界面与跳转后的那个页面之间！

当然，跳转速度太快，我们不方便操作（这个时候我就奇怪为什么我们校园网那么好了doge）。其实还有一种方法，我们看到刚才的登录请求中是有一段cookie数据的，这是我们的登录凭证，可以让网页保持登录状态。因此，我们完全可以在登录后，保持抓包功能打开，在页面进行刷新操作！如此一来，如果有请求发生，我们就能顺利抓包了！



成功了！把抓包数据提取出来：

```
GET /vul/sqli/sqli_header/sqli_header.php HTTP/1.1
Host: XXX.XXX.XXX.X
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) Apple
WebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/5
37.36 Edg/145.0.0.0
Accept: text/html,application/xhtml+xml,application/xml;q=
```

```
0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/
signed-exchange;v=b3;q=0.7
Referer: http://XXX.XXX.XXX.X/vul/sqli/sqli_header/sqli_hea
der_login.php
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;
q=0.6
Cookie: ant[uname]=admin; ant[pw]=10470c3b4b1fed12c3baac014
be15fac67c6e815; locale=zh-Hans; PHPSESSID=sj7qiui435reilt0
d99ba4lulm
Connection: keep-alive
```

这里我们再对比一下刚才前端返回的数据，看看能不能发现一点端倪：

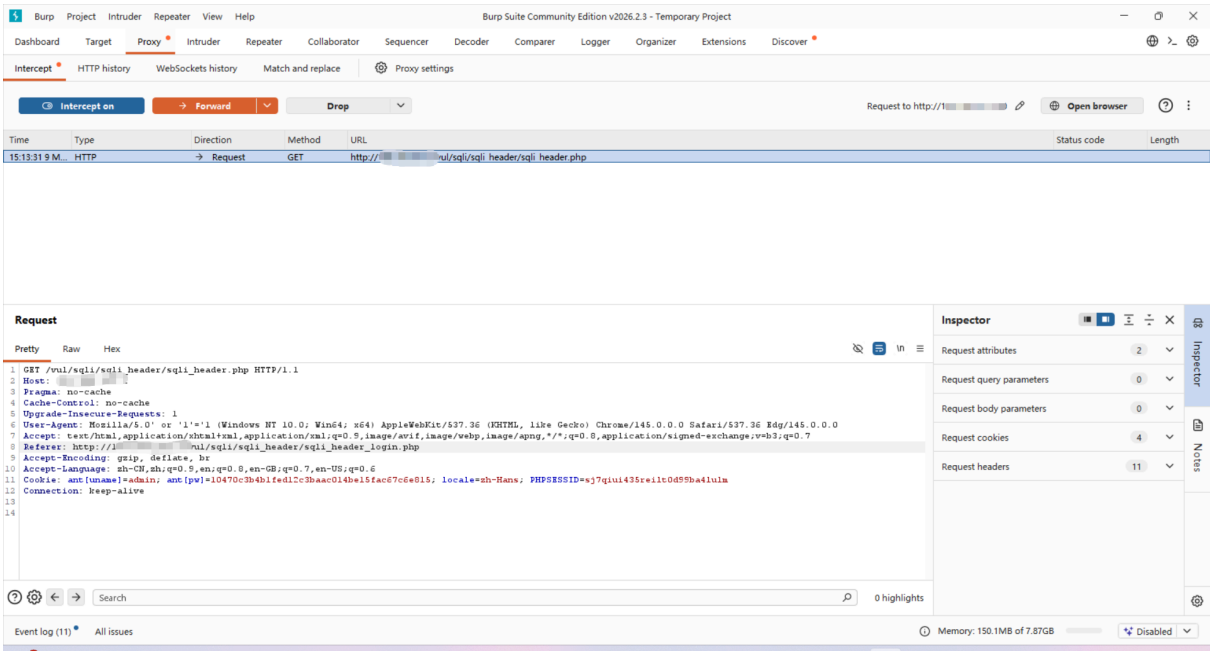
```
你的ip地址:XXX.XXX.XXX.X

你的user agent:Mozilla/5.0 (Windows NT 10.0; Win64; x64) Ap
pleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safar
i/537.36 Edg/145.0.0.0

你的http accept:text/html,application/xhtml+xml,applicatio
n/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,appl
ication/signed-exchange;v=b3;q=0.7

你的端口（本次连接）:tcp38552
```

这里前端返回的ip地址，就是请求头中的Host（不能改，改了就连不上了）；user agent数据，就对应着其中的User-Agent；http accept对应 Accept，而端口信息则对应 TCP 连接的本地端口（不在请求头里，是TCP/IP层的信息，这个我们没法更改）。

之后，后端获取到这些数据之后将其存入数据库，并读取数据库、返回前端让我们看到。因此，这里的重点在于这个存取数据库的过程，后端代码在执行的时候，也恰恰是SQL注入发挥作用的时机！

那么这里就是和先前一样的思路，去修改我们抓包到的这段请求头！由于ip和端口不能更改，所以我们就去试试User-Agent和Accept！

先试试User-Agent！还是按照我们万能构造“闭合前面，注释后面”的思路，把 `Mozilla/5.0` 修改为 `Mozilla/5.0' or '1'='1` ，然后放行这个请求：



回头一看，前端发生了报错：

报错了！这很有价值！

不过，这是一行很奇怪的报错，之前没见过：Truncated incorrect DOUBLE value: 'Mozilla/5.0'。看起来应该是类型转换出错了——具体分析来看，`Mozilla/5.0` 这个值是在转换为数字型（浮点数DOUBLE）时发生了错误，但是为什么这里会把User-Agent转换成数字呢？只有一种可能：在 MySQL 里，当字符串出现在需要数字的语境中时，就会发生隐式类型转换！

换句话说，我们既要执行攻击，又要保证整个表达式的结果不会触发类型转换——也就是说，最终的结果必须是数字，或者至少和端口字段的类型兼容。

好吧，这个就在我们经验之外了。没办法，去找找网上的答案吧！

我又回来了！网上的答案给我们一个思路，答主是这样构造的：

```
firefox' or updatexml(1,concat(0x7e,database()),0) or '
```

这个构造我们之前没见过，而且语法远远比我们之前的构造复杂（仅次于“insert/update”型注入）。不过其构造思路并不复杂，它实际上恰恰验证了我们之前的

结论——“既要执行攻击，又要保证整个表达式的结果不会触发类型转换”。

最引人注目的是中间这个函数 `updatexml()` ，这个函数的功能是报错吐数据——换句话说，我们就是希望服务器报错，然后把必要的数据给出来，此处这个必要的数据就是函数里指定的database：预期的结果就是数据库的名称pikachu（我们之前获取过）！

但是请注意，这里 `updatexml()` 的第三个参数是 **0**，这是一个数字。因此这个函数执行后，不管报不报错，它返回的值就是 **0**（数字）。
而前后的引号构成了闭合，因此前后两个值的类型是字符型。所以，整个表达式就变成了：

```
字符串 or 数字0 or 字符串
```

而MySQL在计算这个表达式时，先看 `字符串 or 数字0` ：遇到数字0，它已经是一个确定的布尔值（false），所以直接返回数字0，不会去计算左边的字符串；然后看 `0 or 字符串` ：同样，遇到数字0，直接返回0；最后整个表达式的结果是 **0**（数字）。

完美！整个过程避开了对字符串 `'firefox'` 的类型转换，同时 `updatexml()` 成功执行，报错吐出了数据。（这个firefox当然也可以换成其他的字符，这个不是重点，但毕竟要尊重原答主嘛！）

我们试试看！

和我们的预期完全一致！攻击成功！

类似User-Agent，读者可以就Accept字段进行类似的构造，原理是一样的，甚至构造方法也是一样的，这里就不多赘述了。

最后，我们还有一个疑惑未能解决，这个触发了类型转换的报错，在后端代码里究竟是如何实现的？我们找出sqli_header.php文件，容易定位到这样一段代码：

```
// 注意看这里——有一行被注释掉的代码
// $remoteipadd=escape($link, $_SERVER['REMOTE_ADDR']);
// $useragent=escape($link, $_SERVER['HTTP_USER_AGENT']);
// $httpaccept=escape($link, $_SERVER['HTTP_ACCEPT']);

// 直接获取前端过来的头信息,没人任何处理,留下安全隐患
$remoteipadd=$_SERVER['REMOTE_ADDR'];
$useragent=$_SERVER['HTTP_USER_AGENT'];
$httpaccept=$_SERVER['HTTP_ACCEPT'];
$remoteport=$_SERVER['REMOTE_PORT'];

// 这里把http的头信息存到数据库里面去了，但是存进去之前没有进行转义
$query="insert httpinfo(userid,ipaddress,useragent,httpacce
pt,remoteport) values('$is_login_id','$remoteipadd','usera
gent','$httpaccept','$remoteport')";
$result=execute($link, $query);
```

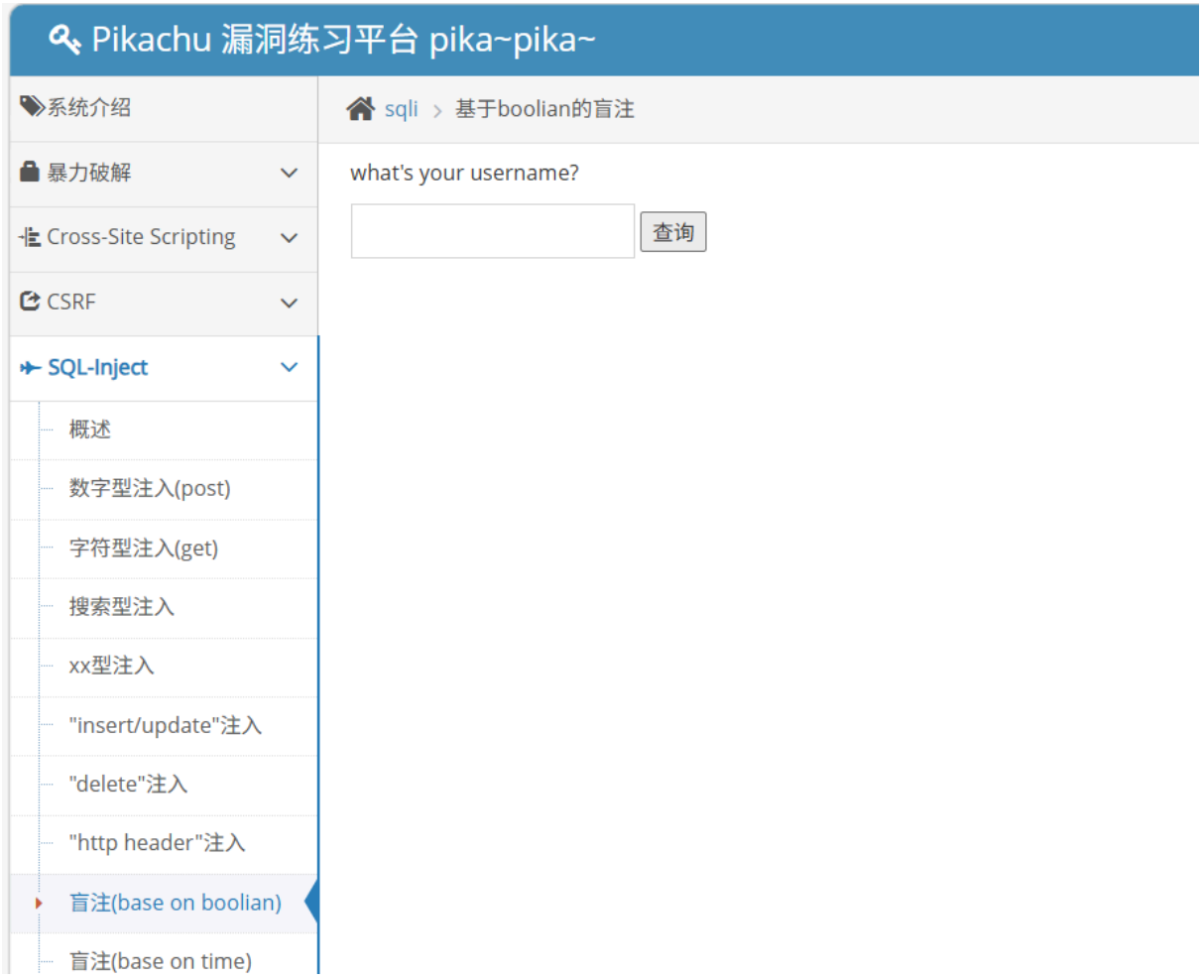
变量看起来复杂，但是逻辑很简单，就是先获取信息然后未经处理直接存入数据库。而这里

`remoteport` 是数字类型！这就是为什么之前的构造 `'Mozilla/5.0' or '1'='1'` 会报 `DOUBLE value` 错误——`'Mozilla/5.0' or '1'='1'` 整个表达式的结果（在MySQL计算过程中，表达式必须被计算，因此要进行类型转换，而单独的一个字符串就无需考虑这一点——因此如果我们不做SQL注入，这就只剩下了 `'Mozilla/5.0'`，这是一个字符串，不会触发转换而发生报错）被试图塞进 `remoteport` 这个数字字段（就是我们在前端看到的端口信息），或者参与了这个数字字段相关的运算，触发了类型转换。而后面我们输入的正确的构造由于本身整体最终会转换为一个数字类型的0，所以不会触发报错转换，因而整个语法就能够顺利执行！

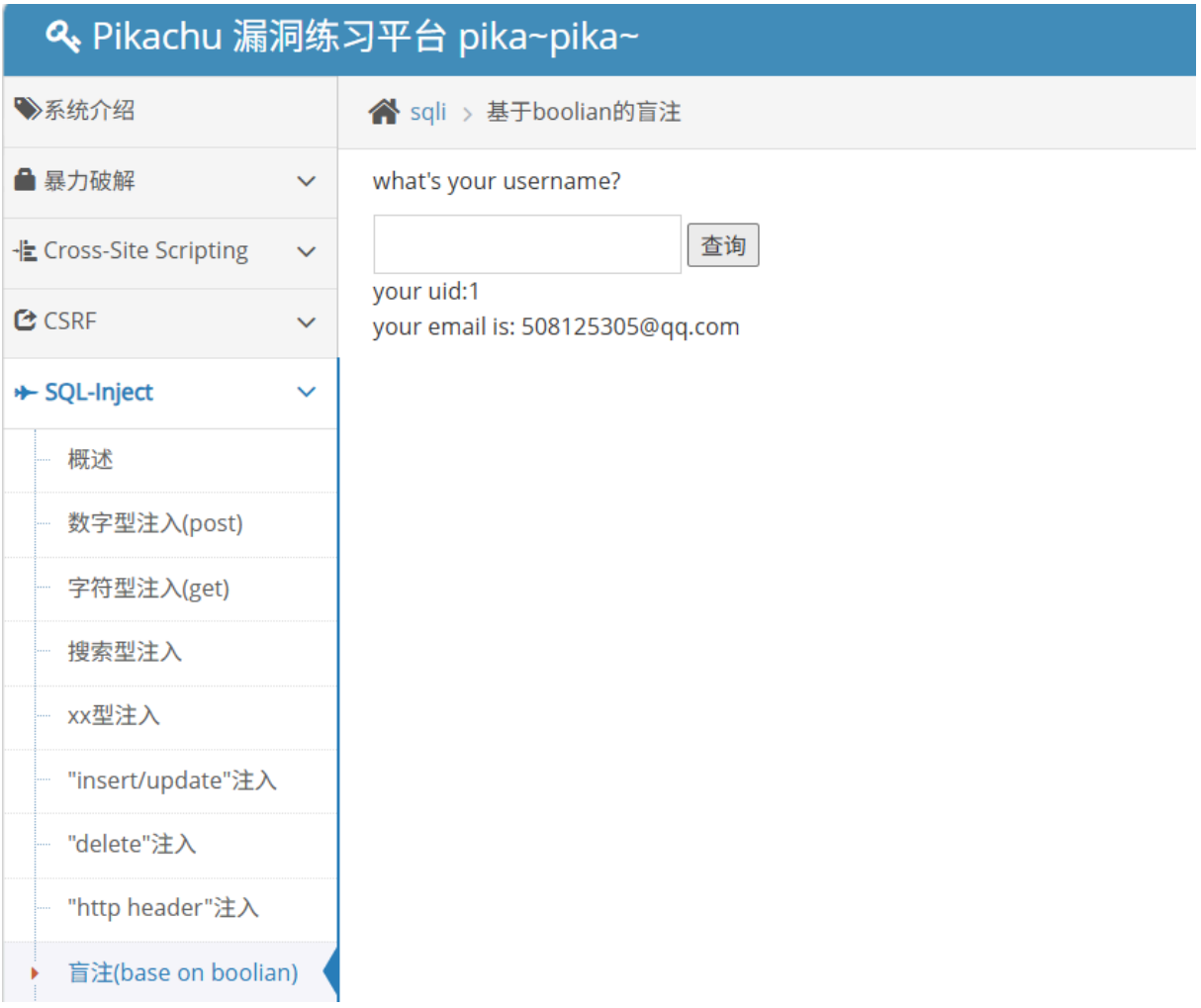
顺带一提，这里被注释掉的代码就是对于参数的转义，如果这些代码没有被注释掉，那么我们的构造肯定会失败——开发者时时刻刻都在用注释告诫我们呢！

8、盲注（base on boolian）

来到下一题，看起来像是一个搜索查询：



还是先试试看咯！这里用的数据库就还是之前的，我们输入vince试试看：



很好，看起来就是一个输入查询的功能。，那么再试试随意输入一个账号名看看，比如输入pikachu：

嗯，看起来就是一个简单的输入验证。有这个账号就显示，没有就提示不存在。似乎和之前的也差不多?

不管怎么说，先试试看输入一个引号，看看能不能闭合:

好吧，仍然失败，这就很难办了。最关键的问题在于，这里应该是存在字符型的注入的，一个引号是会发生闭合的，但是我们验证后并不能得到任何报错数据——看起来是有一个判断逻辑，只要查询失败，就只会返回那段无聊的“不存在”！（至少我们得先这样认为，不然就没法进行下去了）

还有什么办法吗？可以肯定的是，这里存在一个判断逻辑，根据题目，这里用的是布尔类型的判断，只会返回两类结果：是与非。如果能找出这种判断的漏洞，或许还有机会！

那么，该如何利用这种判断逻辑呢？我们已经知道，这种判断逻辑是针对这个输入框传递的参数的，所以如果我们能利用这个输入框构造出一些判断逻辑，或许就可以猜到关于这个网站的一些秘密！

这个判断逻辑是什么样子的？按照我们上述的分析，很有可能是这样一段伪代码：

```
select 某字段 from 某表 where username = '$name'
```

然后查询到数据之后，做了这样一个判断：

```
if (有数据) {
    echo "用户存在";
} else {
    echo "用户不存在";
}
```

但是，这里是与非的判断逻辑是很简单的，只要where后面的条件整体为真，就一定能够返回“用户存在”的判断——因此，假如说我们能构造一个and表达式（A and B），其中A是真实的账号，为真，B是一个待确定的值，那么整个表达式的正确与否仅由B决定。而我们可以修改这个值，根据“存在与否”的结果，判断其真实与否！

这样一来，我们的思路就有了：

首先，通过暴力穷举找出一个存在的账号（当然在这里，我们已经知道了，也没有必要重复这个操作）；

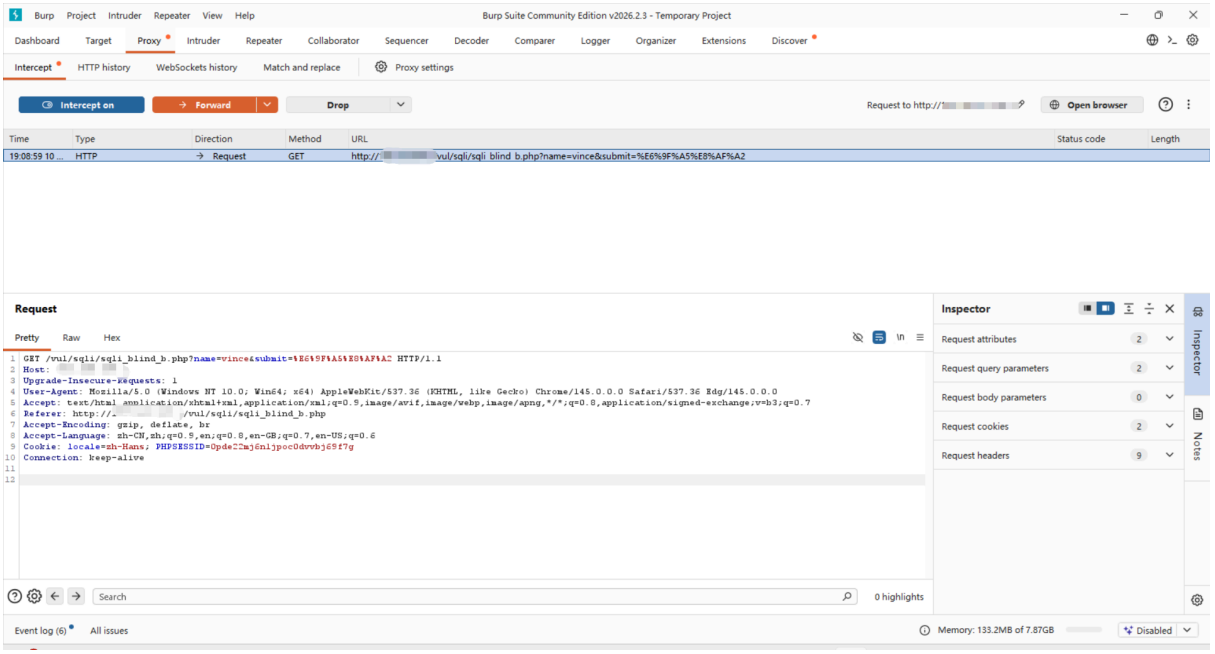
其次，用这个账号作为我们and表达式的A；

然后，在后面构造一个我们想要探寻是否为真的式子，而这个表达式的构造方法，就还是利用引号进行闭合，比如我们要判断数据库名称的长度，可以构造为`vince' and length(database()) > 5 #`；

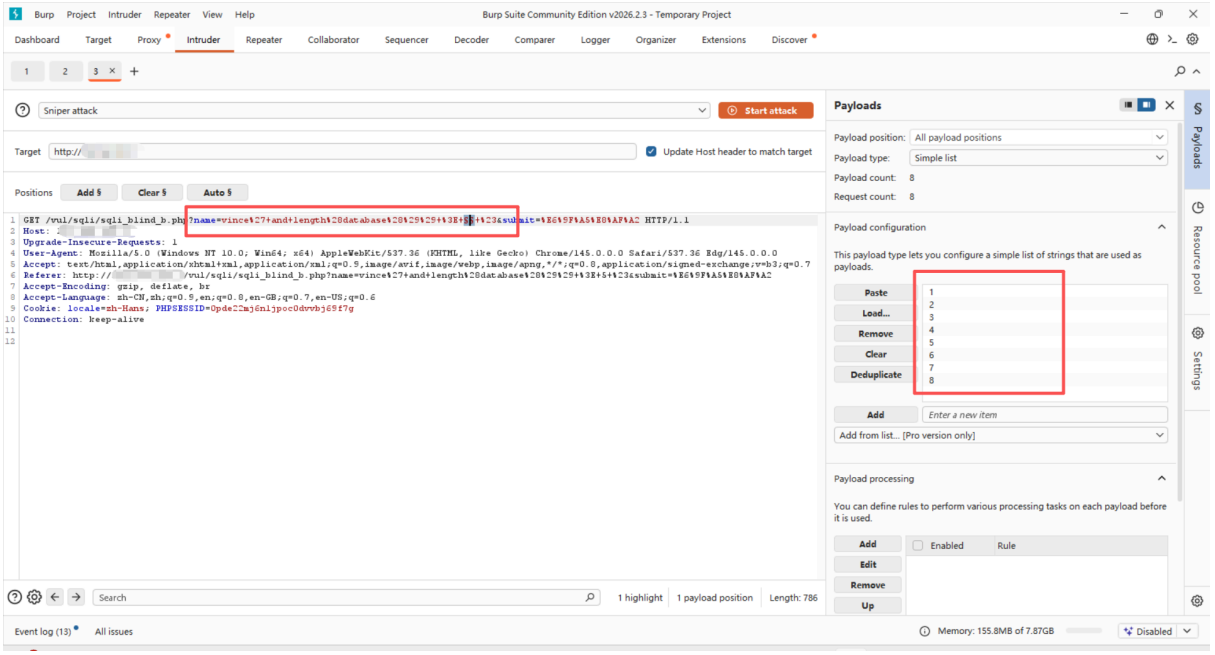
最后，反复测试，直到为真！

不过，既然这里存在反复测试的过程，那么就和我们最初的暴力破解一样，最好就要借助脚本工具了——不过在这里，BurpSuite本身就有这样的工具，方法和暴力破解如出一辙。

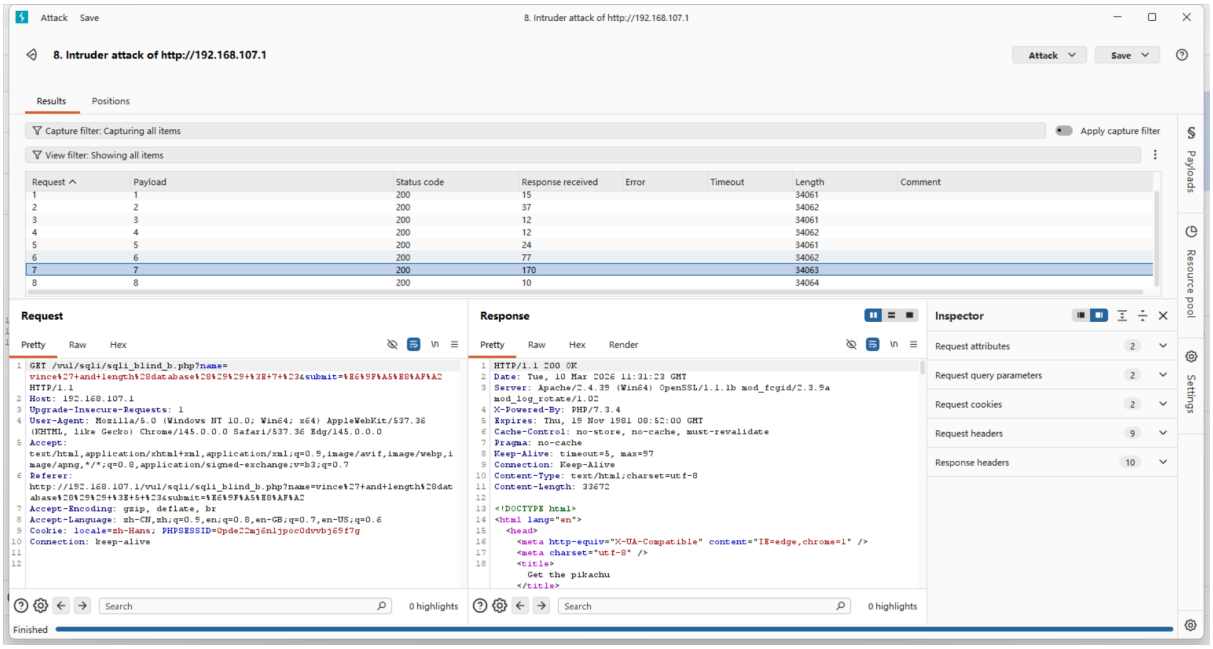
当然第一步还是抓包：



接下来是把抓包的数据发送给Intruder工具，对参数进行格式修改，并在对应位置留下插入符号，在右侧面板直接构造我们接下来要穷举的命令（注意，这里需要先对构造进行url转义，不要直接把字符输入进去，否则会报错！这里没必要照抄我的构造，你只需要确保右侧的数据插入进来形成的表达式格式无误即可）：



然后，进行爆破！



很明显，这里payload为7时，接收到的响应有一些异常，那么大概率这就是转折点！

可以转向前端进行测试，输入 `vince' and length(database()) > 7 #` 看看有没有问题：

显示不存在，那我们输入 `vince' and length(database()) > 6 #` ，看看情况：

用户名存在！这说明这个数据库的名称的长度就是7！从而会在这里发生转折！

按照这样的逻辑继续构造，继续测试，我们可以一点一点摸索出这个数据库名称的线索，最终得到这个完整的名字：pikachu！

如此一来，我们回顾一下本题的标题，什么是“盲注”？就是不管你的构造在后端产生了什么影响，只会给你一个冰冷的答案：是/非，而不会像之前一样告诉我们哪里发生了错误——对于我们而言，就是盲人摸象。而我们也只能根据这种二元的判断，先暴力穷举出一个正确的答案，然后通过and表达式一点一点构造测试，在无数次是与非中找出恰如其分的那个“是”！

最后，看看源码吧！

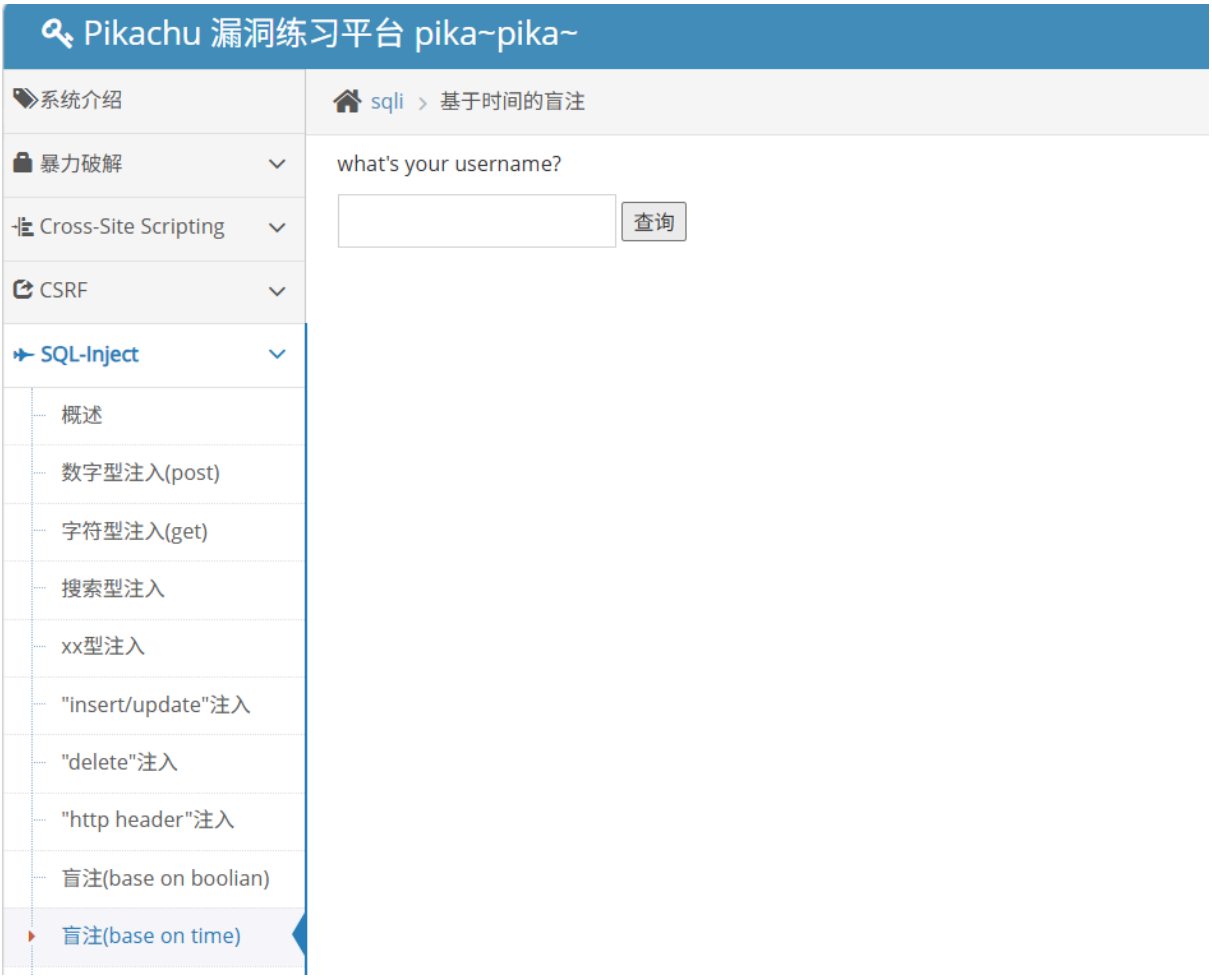
```
$query="select id,email from member where username='$nam
e'";
$result=mysqli_query($link, $query);
if($result && mysqli_num_rows($result)==1){
    while($data=mysqli_fetch_assoc($result)){
        $id=$data['id'];
        $email=$data['email'];
        $html.="<p class='notice'>your uid:{$id} <br />your
email is: {$email}</p>";
```

```
    }
}else{
    $html.="<p class='notice'>您输入的username不存在，请重新输入！</p>";
}
```

和我们猜测的结果基本一致——只要不符合if的条件，不管发生了什么，不存在也好，报错也罢，一律返回else分支！相信读者和我一起看了这么多题目的源代码，这里应该不需要过多解释了，让我们一起奔赴下一道题！

9、盲注（base on time）

下一题看起来是完全一样的，别无他法，还是只能先尝试：

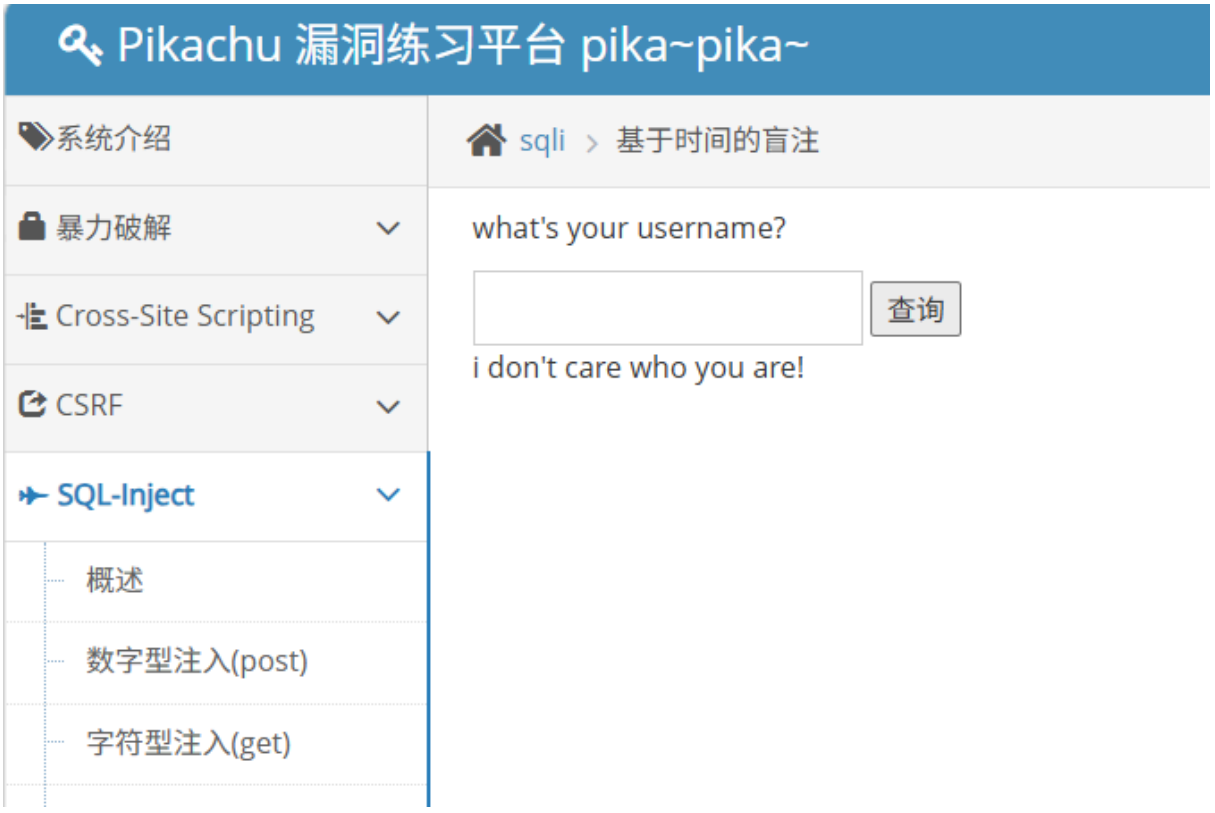


先随便输入一个账号看看，比如输入dream：

“I don’t care who you are!”

霸气侧漏！但是网站真的不在乎吗？

我们试试真的账号vince看看：



好吧，网站是真的不在乎！（这里我打包票，vince绝对是真是的账号，因为这里的数据库都是共享的）

那么，我们就遇到了一个新的困境：上一题虽然前端不会给我们报错信息，但是至少会告诉我们这个账号是否存在——然而到这一题，开发者已经吝啬到一点信息都不肯给我们了！

但是，虽然没有信息，我们已经有了类似题型的经验——同样是“盲注”，那么思路应该是类似的，都是需要直接在输入框中用and表达式构造另外一个条件，进行判断。

不过和上一题不同，这里似乎是没法直接判断了——让我们把思路打开一点，除了可以在and表达式中构造条件，同样也是可以构造命令的啊，只不过是需要把这个命令作为条件的一部分进行构造而已！

而这个命令，一定要能够有助于我们去区分这个账号的真实与否！

在MySQL中，有这样一个函数 `sleep()`，它的作用就是让数据库“休眠”一段时间——也就是说，根据这个响应时间的长度，我们可以判断整个命令是否真实！

那么，我们首先要看看这个账号是否存在，可以构造下面的语句：

```
vince' and if(length(database())>5, sleep(5), 1) #
```

这条语句的意思是，如果数据库名长度大于5，就睡5秒；否则返回1（不睡）。换言之，如果我们等待的时间超过了五秒，那么整条语句就为“真”；而vince也为真的话，则说明后面的整体为真，也就是 `length(database())>5` 为真！（这里sleep(5)和1没有真假的意义，仅仅是为了构造语法而已，这个条件本身的真假取决于第一个值 `length(database())>5`）

当然，采用这个方法需要确保自己的网速比较好，要是网络延迟本身就让这个过程超过了5秒，就完全无法判断了。

那么，试试看！我们观察到，页面等待了五秒多才返回了结果，也就是说——这个数据库名称的长度确实大于5！

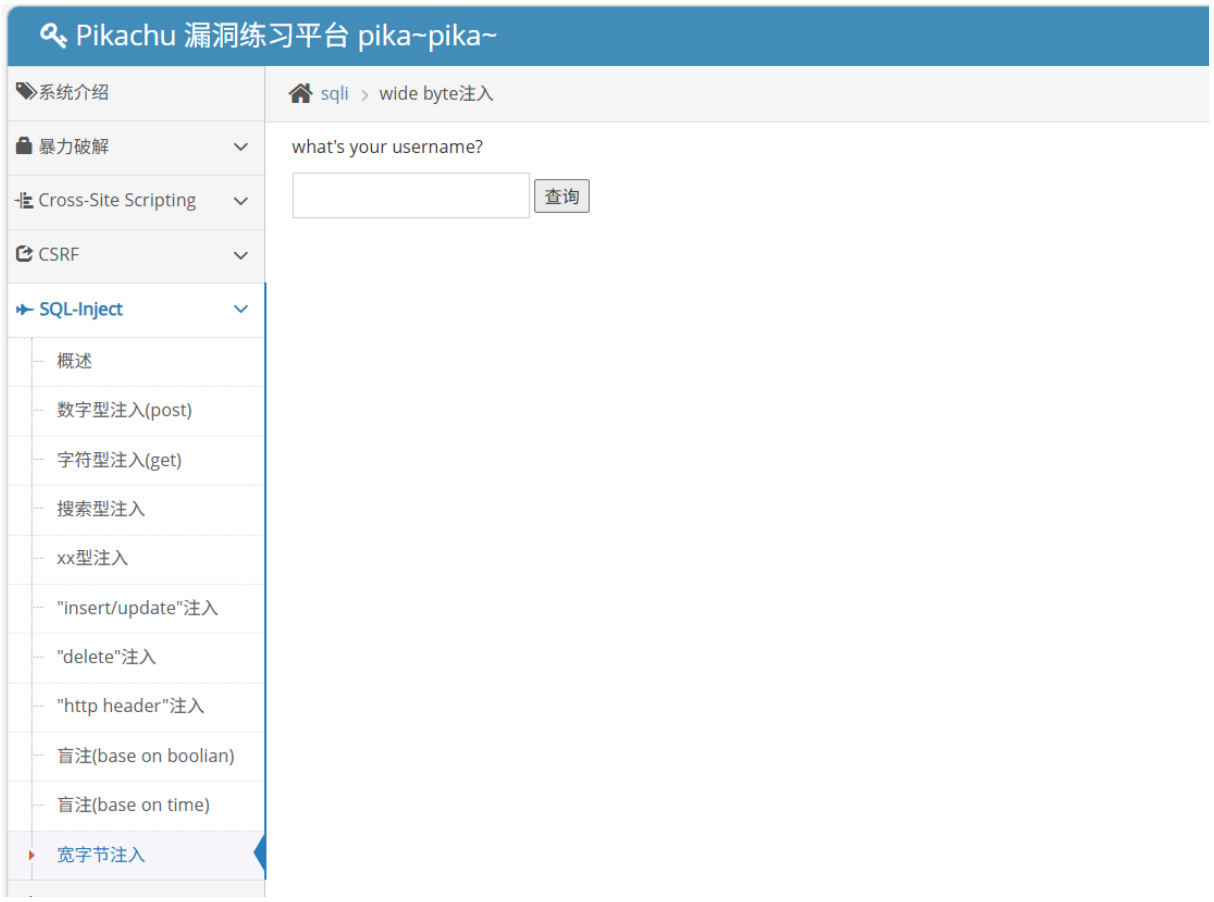
（因为是一个动态的过程，这里没办法截图给大家展示）

然后，就是和上一题完全一样的逻辑，无论是用BurpSuite字典猜解也好，写脚本也罢，或者干脆就直接在前端一个一个试，最终会得到这样一个结果：数据库的名称是pikachu！

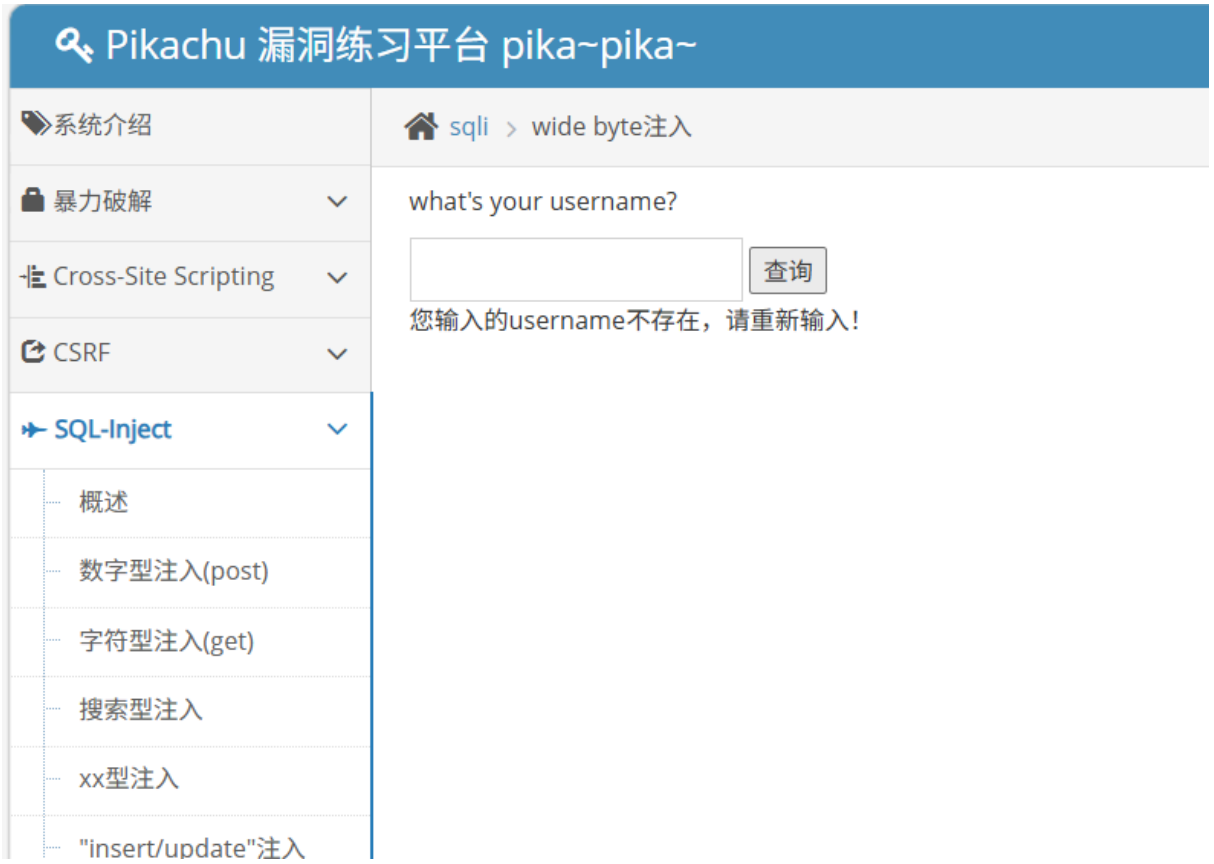
在这一道题中，我们不再是基于网页本身的布尔响应来判断条件是否为真，而是通过自己构造一个“另类”的通过浏览器进程体现出来的”布尔响应“来进行判断。究其本质，还是在是与非的二元圈子里慢慢摸索。

10、宽字节注入

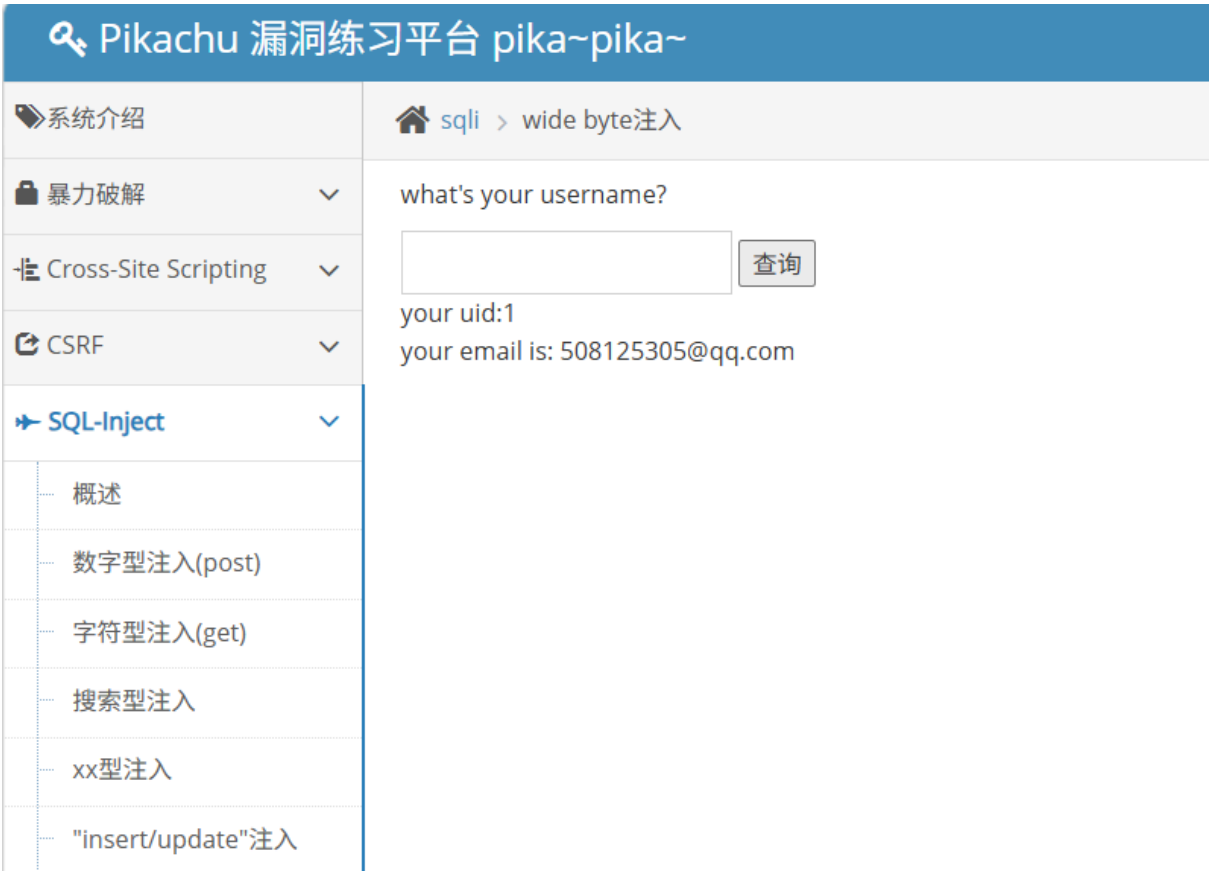
来到下一题，一如既往的朴素页面，但是本题的标题是我们从未见过的类型。但不管怎么说，我们还是先老样子，摸索一下这到底是个啥：



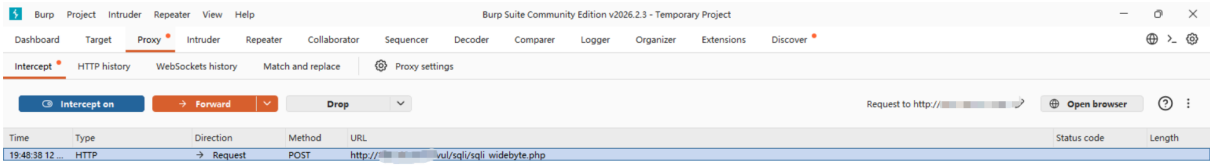
先按部就班，随便输入一个账号，比如zhaolei：



账号不存在，那再试试正确的账号vince：



正确返回了账号信息，不过这道题的机制如何仍然是一团迷雾。那还是先抓包看看吧：



感觉看不出任何线索——这不就是一个很正常的请求嘛！

那没招了，说白了还是见识不够，我们得去补补功课，了解一下什么是宽字节注入了：

> 宽字节注入是一种利用数据库编码特性绕过转义防护的SQL注入技巧。当数据库使用GBK等宽字节编码时，每个字符可能由两个字节组成。开发者常用`addslashes()`等函数在单引号前加反斜杠（`\'`）来防止注入，反斜杠的十六进制是`%5c`。
>
> 攻击者输入`%df'`，经过转义后变成`%df%5c%27`。在GBK编码下，`%df%5c`恰好被解析成一个宽字节字符（如“運”），反斜杠被“吃掉”成为汉字的一部分。剩下的单引号`%27`成功逃逸，可以闭合SQL语句。
>
> 这个漏洞的核心条件是：数据库连接使用GBK编码，同时后端用反斜杠转义单引号。看似安全的转义机制，在编码特性面前反而成了突破口。

看起来，所谓宽字节注入就是为了应对后端对引号的转义而生的——引号可是我们在SQL注入中构造闭合的核心，如果就这么被后端“吃掉”，那么大部分注入手段就完全没用了，而与其想方设法去绕过引号，不如直接把引号保留下来！而这就是宽字节注入，直接从数据库代码层面绕过后端防御机制！

为了方便理解，我们提前把源代码扒下来看看：

```
if(isset($_POST['submit']) && $_POST['name']!=null){

    $name = escape($link,$_POST['name']);
    $query="select id,email from member where username='$name'";//这里的变量是字符型，需要考虑闭合
    //设置mysql客户端来源编码是gbk,这个设置导致出现宽字节注入问题
    $set = "set character_set_client=gbk";
    execute($link,$set);

    //mysqi_query不打印错误描述
    $result=mysqli_query($link, $query);
    if(mysqli_num_rows($result) >= 1){
        while ($data=mysqli_fetch_assoc($result)){
            $id=$data['id'];
            $email=$data['email'];
            $html.="<p class='notice'>your uid:{$id} <br />your email is: {$email}</p>";
        }
    }else{
        $html.="<p class='notice'>您输入的username不存在，请重新输入! </p>";
    }


}
```

这里，后端先把我们输入的内容用escape()函数转义后，直接拼接进SQL语句之中。这里的escape的功能就是在像 ' 这种特殊字符前面加上 / ，把它真正变成一个字符型的引号，而不再具备闭合的功能。如此一来，我们的构造就直接失效了。

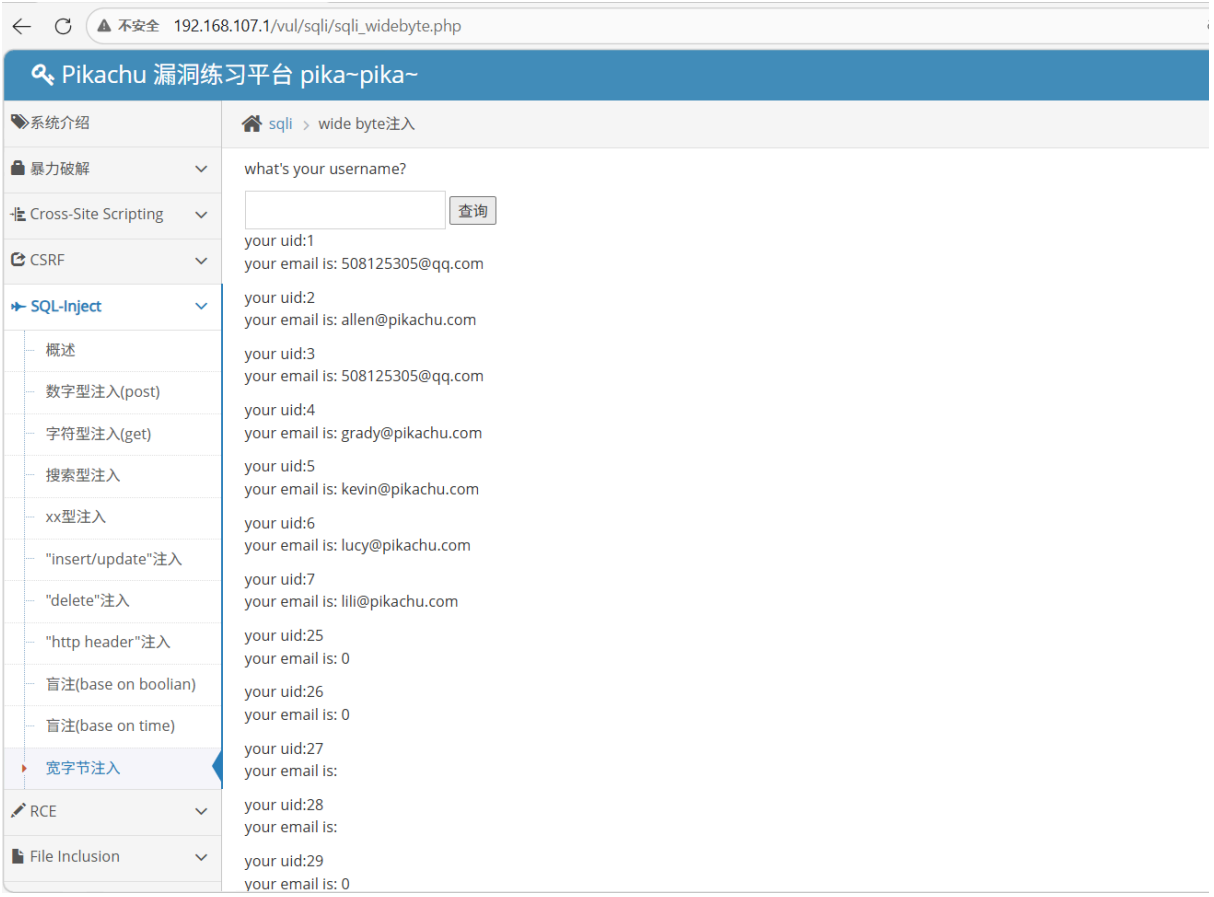
但是，正如这里开发者留下的注释，MySQL客户端来源编码是gbk，这个设置会导致出现宽字节注入问题——因为gbk编码中，反斜杠是可以通过特定的构造直接被吃掉的！

举例而言，前端传入参数 %df' ，转移函数escape看到单引号，在前面加反斜杠，内容就变为 %df%5c%27 。而在数据库执行时（GBK编码），首先会读取 %df%5c ，这是一个完整的汉字“連”，而剩下的 %27 ，就是单引号本身。如此一来，引号就被保留了下来，而前面新增的“連”由于被引号一起闭合掉，所以不会产生任何影响。

从本质上来说，这就是一个直接构造GBK编码的过程！通过直接输入十六进制字符，在GBK编码之中发生了一个新的构造，把后端新加入的反斜杠给无效化了，因而我们输入的引号就能够正常发挥作用！

当然，这里要提醒读者，我们的构造不能直接在网页端进行，因为我们的输入会被浏览器转换成十六进制，而前端输入框之中如果本身就存在十六进制字符就会发生错误。因此，我们需要通过抓包，直接修改传入后端的请求！

还是一样，抓包一个请求，按照万能构造的方法把参数修改为 `vince%df' or 1=1 #` （这里不做赘述啦，本质上就是要在引号前面加一个十六进制的构造），放包！



很好，所有数据全都出来了！这就是我们的预期，理由就和之前一样——所有数据都符合条件，因此全都返回！

至此，SQL注入正式完结！恭喜读者，最难的一章已经过关啦！皮卡丘靶场的通关之路也就不远了！

六、RCE

老规矩，看看开发者怎么说：

> **RCE(remote command/code execute)概述**
>
> RCE漏洞，可以让攻击者直接向后台服务器远程注入操作系统命令或者代码，从而控制后台系统。
>
> **远程系统命令执行**一般出现这种漏洞，是因为应用系统从设计上需要给用户提供指定的远程命令操作的接口，比如我们常见的路由器、防火墙、入侵检测等设备的web管理界面上一般会给用户提供一个ping操作的web界面，用户从web界面输入目标IP，提交后，后台会对该IP地址进行一次ping测试，并返回测试结果。
>
> 而，如果，设计者在完成该功能时，没有做严格的安全控制，则可能会导致攻击者通过该接口提交“意想不到”的命令，从而让后台进行执行，从而控制整个后台服务器现在很多的甲方企业都开始实施自动化运维,大量的系统操作会通过"自动化运维平台"进行操作。
>
> 在这种平台上往往会出现远程系统命令执行的漏洞,不信的话现在就可以找你们运维部的系统测试一下,会有意想不到的"收获"-_-
>
> **远程代码执行**同样的道理,因为需求设计,后台有时候也会把用户的输入作为代码的一部分进行执行,也就造成了远程代码执行漏洞。 不管是使用了代码执行的函数,还是使用了不安全的反序列化等等。因此，如果需要给前端用户提供操作类的API接口，一定需要对接口输入的内容进行严格的判断，比如实施严格的白名单策略会是一个比较好的方法。
>
> 你可以通过“RCE”对应的测试栏目，来进一步的了解该漏洞。

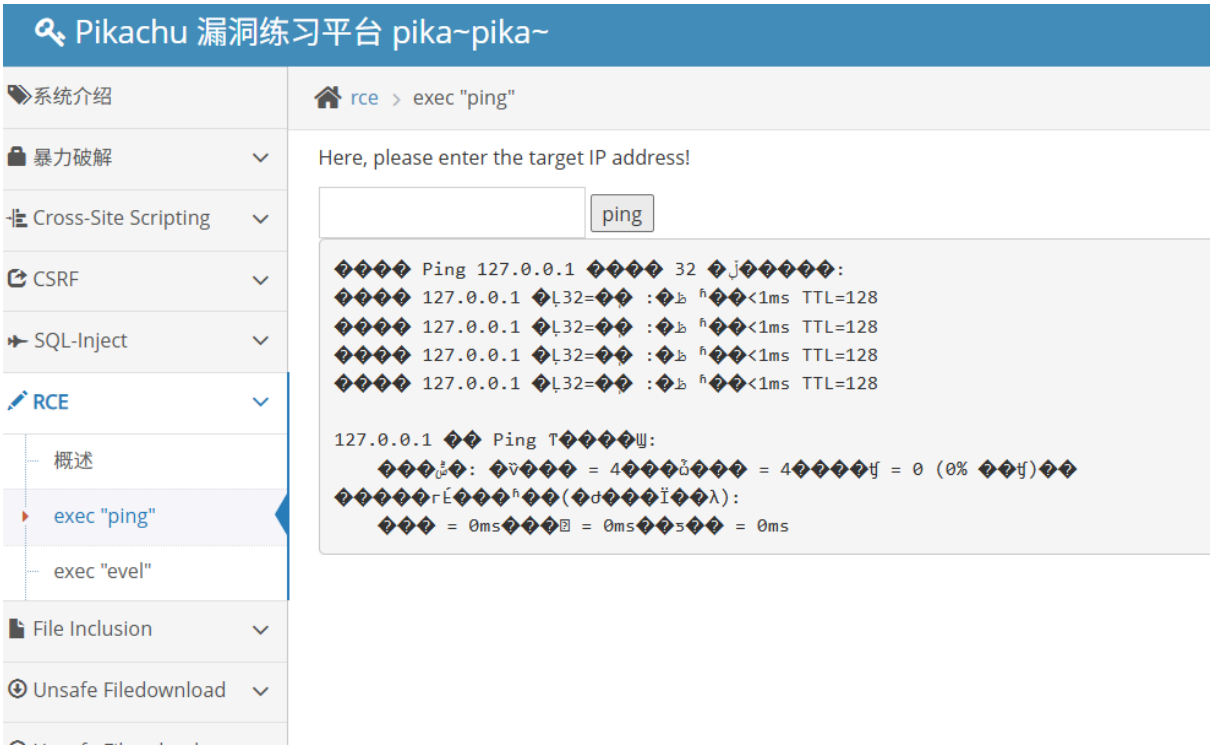
开发者这里写得稍微有点绕，不过我们可以结合先前的SQL注入进行理解——如果说SQL注入的内容会被拼接到后端的SQL语句之中，作为数据库语法而被执行，那么RCE中注入的内容则会被拼接到操作系统之中进行执行。换言之，SQL注入能够操作数据库，而RCE可以直接操作服务器的操作系统！

RCE进一步可以分为远程系统命令执行和远程代码执行，前者是执行系统命令，后者是执行脚本代码，在后面的题目中，我们可以清楚地看到二者的区别。

1、exec "ping”

看看页面，这里和之前的输入框不太一样——读者或许见过这个ping，这是在用来测试网络连通性的命令，既然出现在网页上，那应该是一个命令？

按照要求，我们这里随便输入一个IP地址，不如就试试环回地址 `127.0.0.1`，测试一下与本机的连通性：



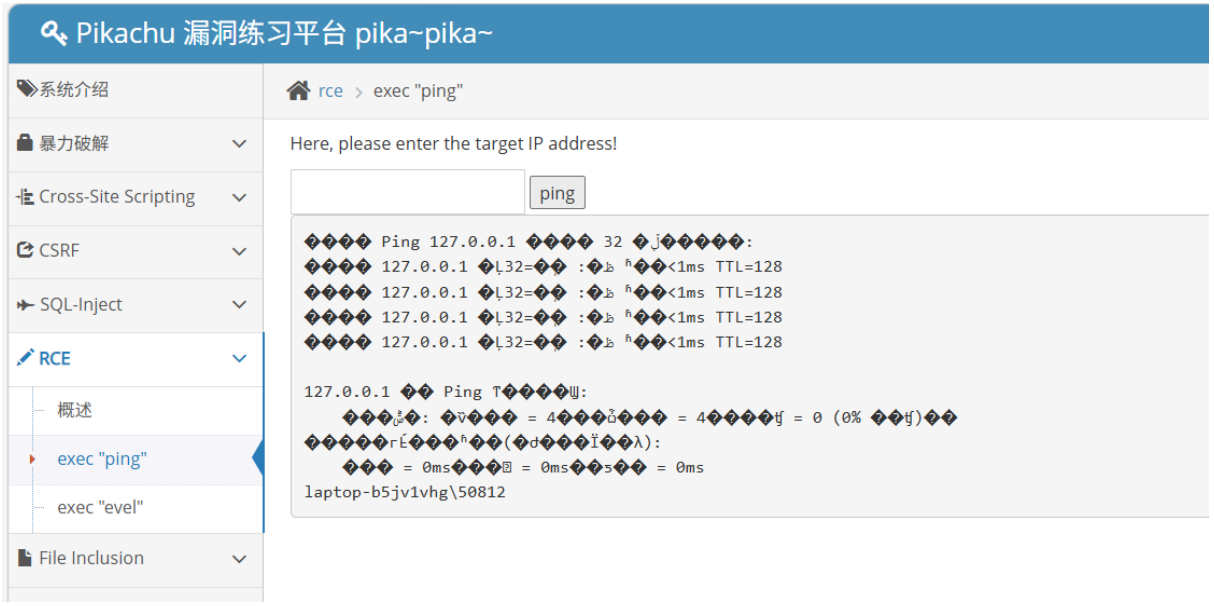
这里显示的全是乱码（如果你是Linux系统，可能显示正常；Windows下乱码是因为CMD的编码和网页编码不一致——但这不影响命令执行，所以不管它），不过问题不大，如果读者熟悉这个命令的话，这里响应结果的意思其实就是往这个IP地址发送了4

个包，看看能不能收到响应。从这里结果的相对位置来看，显然是ping通了（本机和本机肯定能ping通的啦！）。

因为ping命令一般都是在命令行进行操作，因此，我们输入的内容应该是被拼接到了操作系统上加以执行，并把执行结果返回了前端。而这里，我们这里预期的攻击标志就是，让命令行为我们所用，执行其他的命令，并返回前端！

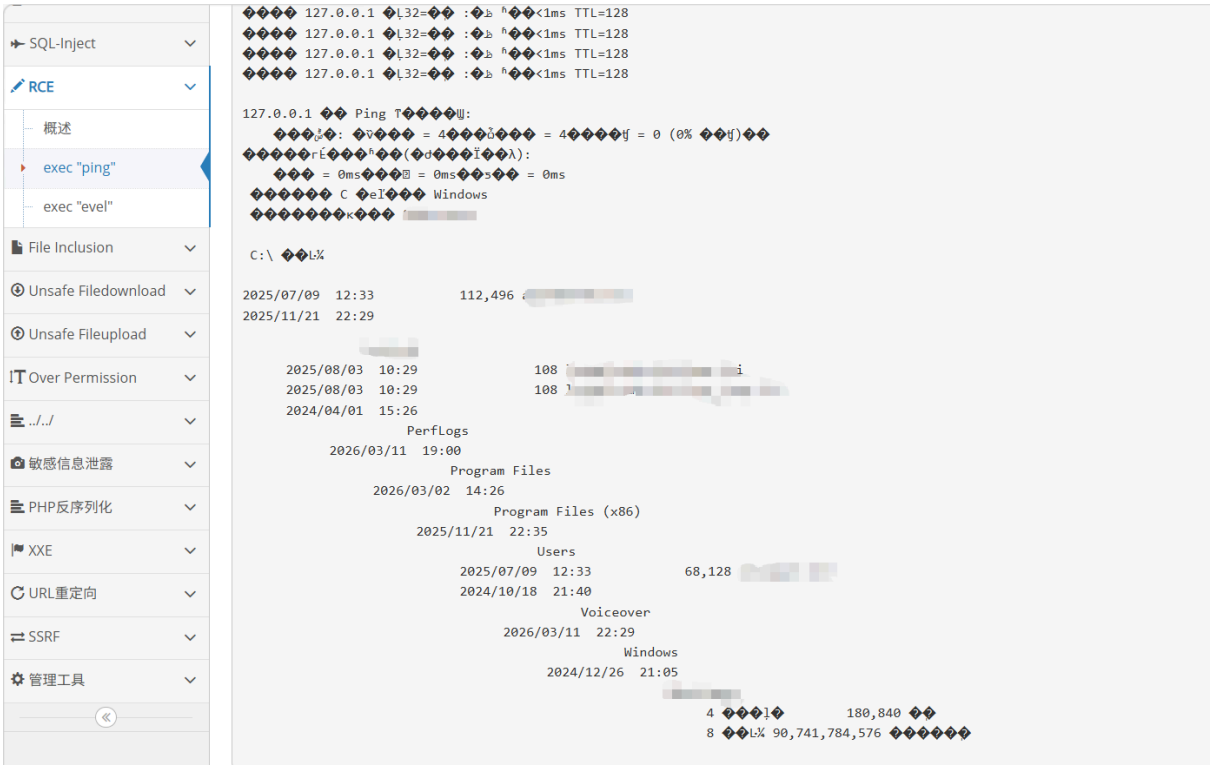
这里的注入方法其实并不复杂，因为在Linux系统中（网站的服务器一般都是Linux系统，感谢开源吧！），可以直接用&&来输入几个连续的命令，让后台依次执行。

比如说，我们这里输入 `127.0.0.1 && whoami` ，看看能不能返回登录服务器的账户：



很好！我们看到底部新增了一行 `laptop-b5jv1vhg\50812` ，这就是服务器的登录账户信息（在Windows里， `whoami` 返回的是 `计算机名\用户名` ，所以这里的 `50812` 就是服务器的用户名——当然，如果你用的是Linux，返回的就是 `root` 或者普通用户名。）！

同理，我们也可以输入一些其他更过分的命令，比如这里可以输入 `127.0.0.1 && dir C:\` ，直接查看服务器的C盘信息！



看起来熟悉不？当然，这里返回的是我们自己电脑的C盘信息——而且这只是开始，能 `dir` 就能 `del`，能 `del` 就能删库跑路。所以RCE的威力，远比你看到的这些输出要大得多（千万不要在这里删掉自己的C盘数据！！！）。

当然，要做到这一点并不复杂，本质上就是一个拼接代码，我们可以看看代码是如何实现的：

```
if(isset($_POST['submit']) && $_POST['ipaddress']!=null){
    $ip=$_POST['ipaddress'];
    // 这里有一行被注释掉的代码
    // $check=explode('.', $ip);可以先拆分，然后校验数字以范围...

    if(stristr(php_uname('s'), 'windows')){
        $result.=shell_exec('ping '.$ip);
    }else {
        $result.=shell_exec('ping -c 4 '.$ip);
    }
}
```

可以看到，这里没有执行任何过滤，直接将前端传回的参数拼接到shell命令中，直接加以执行。比如我们输入 `127.0.0.1 && dir C:\` 之后，系统就会在命令行里执行下面的

命令（因为本地部署的关系，应该是windows系统，这里不进入else分支）：

```
ping 127.0.0.1 && dir C:\
```

如此一来，当系统执行完ping命令之后，就会紧接着执行dir C:\命令，也就是列出C盘目录下所有文件或文件夹，而所有的信息就这样被传回前端了！

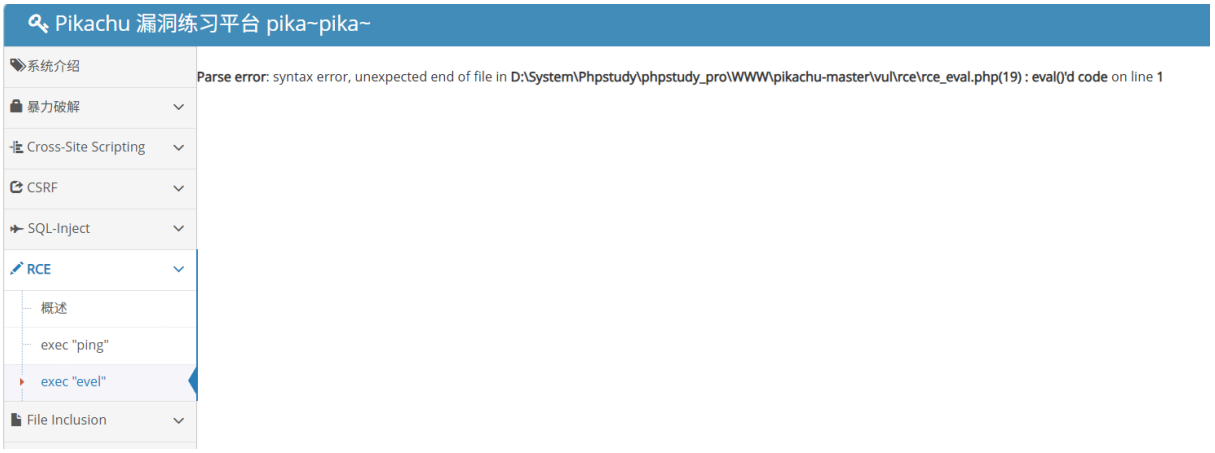
其实，看起来也不是很难，对不对？

2、exec "eval”

我们从一开始的开发者概述中了解到，如果说上一题的注入是在shell中执行命令，那本题就是执行脚本代码。不过，关于这种注入具体是如何实现的，我们仍然一无所知。还是先看看题目吧！

好吧，看起来就是一个普通的文本输入框。但是，什么叫做“提交一个字符串”呢？

也别急着去查概念，不妨随便输入一个，比如“kobe”：



呃，怎么报错了？

别急着重装皮卡丘靶场！不要怀疑自己，这里不是你的问题！

实际上，这里的报错是十分正常的，我们仔细看看这行报错是什么意思：

```
Parse error: syntax error, unexpected end of file in D:\System\Phpstudy\phpstudy_pro\WWW\pikachu-master\vul\rce\rce_eval.php(19) : eval()'d code on line 1
```

这行报错大概是说，在靶场的安装目录下，eval()函数执行的时候出现了问题。那么我们当然需要搞清楚，什么是eval函数？

eval() 是PHP的一个函数，它会把传入的字符串当作PHP代码来执行。举个安全例子：

```
eval('echo "Hello World";');  // 输出 Hello World
```

注意看，如果说我们传入的的参数被拼接到这个函数之中的话，那么应当是在这对单引号之间，因此，我们输入正确的格式——或者说预期的输入应当是类似 `echo "Hello World";` 的完整命令！

那么不妨试试！就输入 `echo "Hello World";` 看看网页有什么响应：

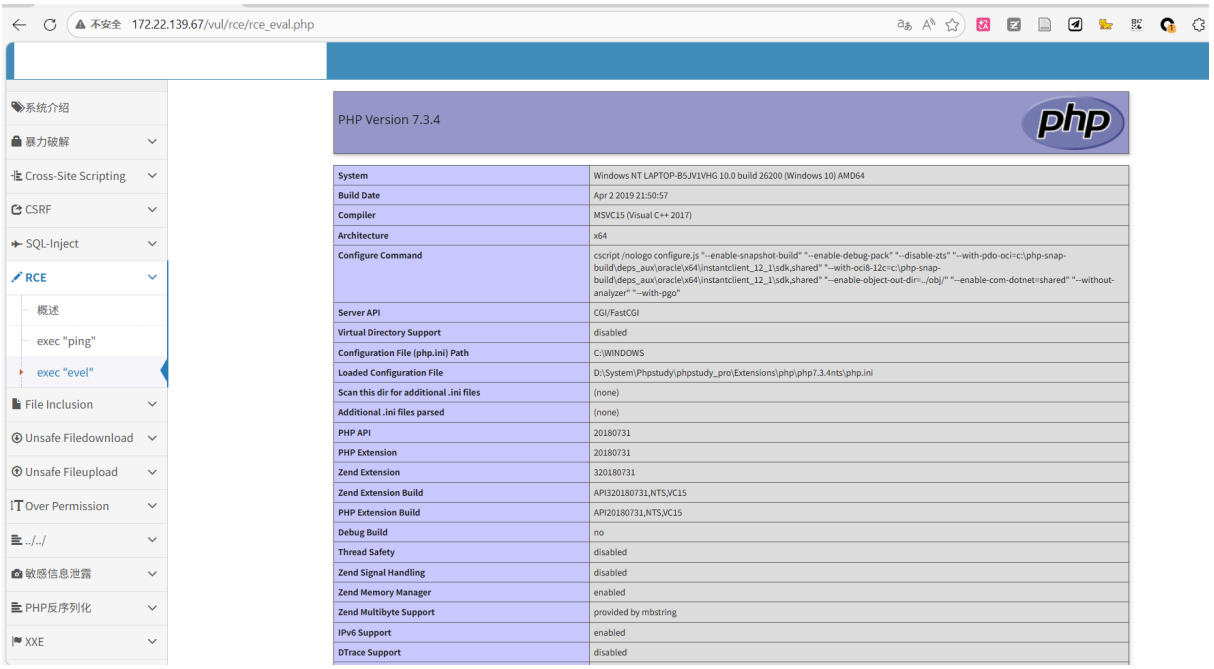
“你喜欢的字符还挺奇怪的！”那确实很奇怪！毕竟这个奇怪的拼接方式居然不只让我们输入字符，还要带上完整的命令——当然，这种网页一般都是用作接口，不过也正因为如此，本题给了我们更大的操作空间！

换言之，我们在eval()函数中无论构造出什么样的字符串，无论安全或危险，符不符合开发者预期，都能够被服务器正常执行！

而这，就是本题“远程代码执行”的关键所在！

那么，既然能够执行任意的代码，我们当然要搞点事情出来，比如说，看一看网站配置？

输入`phpinfo();`，提交：



哇！这是……直接把网站的所有配置信息都打印出来了！（好在这里没有什么敏感信息，我就毫无保留地呈现给读者咯！）

或者，向上一题那样，在shell中执行系统命令？

当然也是可以的，输入 `system('whoami');` ，看看能不能返回服务器的账户信息：（做个补充说明：`system()` 是PHP里执行系统命令的函数，和上一题的 `shell_exec()` 是亲戚——一个返回输出，一个直接打印）

哇哦，网页上方多了一行，`laptop-b5jv1vhg\50812` ——这就和上一题的结果一致，看来这个命令也被正常执行了。读者也可以去尝试一下其他命令，这里就不再演示了。

看起来，通过“远程代码执行”这一漏洞，我们不仅能够通过调 `system` 做到“远程系统命令执行”所能做到的事情，还能做到更多：看配置、读文件（比如 `echo file_get_contents('/etc/passwd');`）、写文件（包括木马文件，比如 `file_put_contents('shell.php', '<?php @eval($_POST[cmd]); ?>');`）……只有你想不到，没有你做不到！

最后，我们还是去看看源代码吧，趁热打铁加深一下记忆：

```
if(isset($_POST['submit']) && $_POST['txt'] != null){
    if(@!eval($_POST['txt'])){
        $html.="<p>你喜欢的字符还挺奇怪的!</p>";
    }
}
```

让我们把关键信息依次提取出来：

```
$_POST['txt']
```

这行代码的意思是直接从POST拿用户输入，赋值给eval的参数——没有任何过滤，没有任何校验。

```
eval($_POST['txt'])
```

这行代码是PHP里最危险的函数之一——把字符串当代码执行。

```
@!eval(...)
```

注意一下前面这个 `@` ，它是PHP的错误抑制符，意思是“不管执行成不成功，都别报错给用户看”。（不过坦率地说，我怀疑这里出了点问题，因为之前我们尝试的时候确实会发生报错啊！可能是因为它无法拦截系统层面的报错吧ヽ(ﾉ ▽ ヽ) ノ）

```
if(@!eval(...)){ ... }
```

这行代码是说，如果eval执行失败，返回 `false` ，再加上前面的 `!` ，也就是 `!false` ，反而为真，就会输出“你喜欢的字符还挺奇怪的！”否则，就会正常返回我们代码执行后的结果。

也就是说，开发者从一开始让我们输入自己喜欢的字符串，就是一个有趣的谎言；反倒是我们后面直接执行了代码之后，才会返回“不报错的结果”，也就是我们代码执行后的结果！（而我们一开始的输入 `echo "Hello World";` ，由于不会返回任何值，所以也被当作“执行失败”处理了）

最后，我们对比总结一下这两种RCE：

“命令执行”是指：你在操作系统层面搞事，能执行命令，但受限于系统命令的语法；而“代码执行”却是：你在PHP层面搞事，能做任何PHP能做的事——包括调用系统命令、读写文件、连接数据库.....

所以，代码执行的危害，通常比命令执行更大！

七、File Inclusion

新的篇章，我们还是先看看开发者怎么说：

> **File Inclusion(文件包含漏洞)概述**
>
> 文件包含，是一个功能。在各种开发语言中都提供了内置的文件包含函数，其可以使开发人员在一个代码文件中直接包含（引入）另外一个代码文件。
>
> 比如 在PHP中，提供了：
>
> include(),include_once()
>
> require(),require_once()
>
> 这些文件包含函数，这些函数在代码设计中被经常使用到。大多数情况下，文件包含函数中包含的代码文件是固定的，因此也不会出现安全问题。
>
> 但是，有些时候，文件包含的代码文件被写成了一个变量，且这个变量可以由前端用户传进来，这种情况下，如果没有做足够的安全考虑，则可能会引发文件包含漏洞。
>
> 攻击者会指定一个“意想不到”的文件让包含函数去执行，从而造成恶意操作。 根据不同的配置环境，文件包含漏洞分为如下两种情况：
>
> **1.本地文件包含漏洞：**仅能够对服务器本地的文件进行包含，由于服务器上的文件并不是攻击者所能够控制的，因此该情况下，攻击着更多的会包含一些 固定的系统配置文件，从而读取系统敏感信息。很多时候本地文件包含漏洞会结合一些特殊的文件上传漏洞，从而形成更大的威力。
>
> **2.远程文件包含漏洞：**能够通过url地址对远程的文件进行包含，这意味着攻击者可以传入任意的代码，这种情况没啥好说的，准备挂彩。
>
> 因此，在web应用系统的功能设计上尽量不要让前端用户直接传变量给包含函数，如果非要这么做，也一定要做严格的白名单策略进行过滤。
>
> 你可以通过“File Inclusion”对应的测试栏目，来进一步的了解该漏洞。

不满读者说，这一章太能引起我的共鸣了。前些日子我出于学习的热情开发了一个网站，一个集成了AI能力的LaTeX线上写作平台，但是网站在公测期间被同学几次三番攻破，在服务器里面写文件。事后我问他是怎么攻进去的，他说，就是利用了这个文件包含漏洞。

嘶~后面我只能加入更多过滤条件来防范此类攻击了——虽然后续又被他用其他方法侵入服务器了。

当然啦，我们都是小打小闹，所谓“攻进服务器”干的事情也不过就是写几个“到此一游”的文件罢了，事实上，后面我还把他加入了网站的致谢名单中去了——要是没有这种源于兴趣的渗透测试（把网站当靶场打），恐怕我的网站早就千疮百孔了吧！

回到正题上来，这个所谓的文件包含漏洞到底是什么呢？其实开发者已经解释得很清楚，不过简单来说就是，如果程序员说“用户想看哪个文件，我就给他看哪个”，而且没检查用户想看的是啥，那用户就能看任何文件——通过构造文件包含函数查看服务器上的文件，而这些文件既可以是开发者想让用户看到的，也可以是网站的源代码、敏感数据、配置文件等等私密文件。

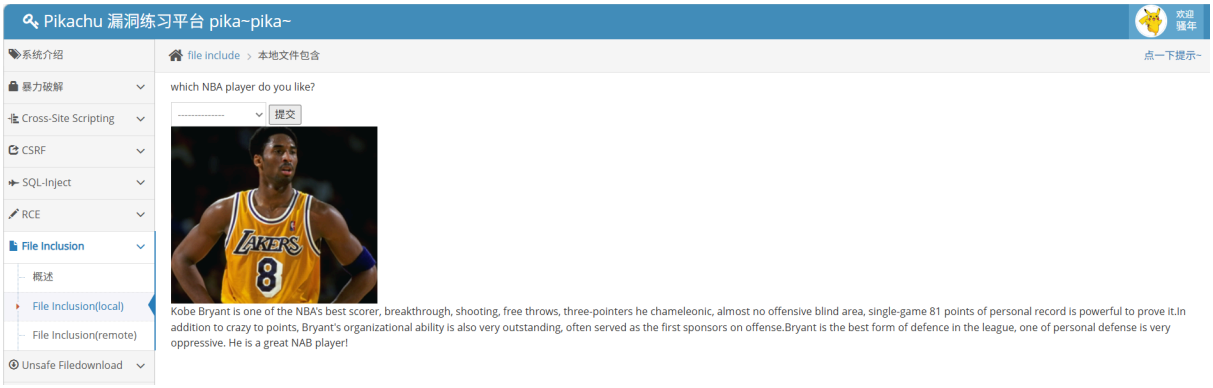
那么，我们就在实操中看看一个文件包含漏洞到底是怎么被利用的吧！

1、File Inclusion(local)

看看题目，又是熟悉的选项栏：

我们在之前SQL注入的第一题“数字型注入”已经了解到，像这种前端的选项栏其实根本无法拦住我们的构造命令，不过我们还是先老老实实走一遍正常流程吧！

在选项框中选择Kobe Bryant，看看牢大怎么说：



翻译一下吧，“科比·布莱恩特是NBA最出色的得分手之一，在突破、投篮、罚球、三分球等方面他都变幻莫测，几乎没有进攻盲区，单场81分的个人纪录就是有力的证明。除了能疯狂得分外，布莱恩特的组织能力也十分出色，经常在进攻中担任第一发起者。布莱恩特是联盟中防守状态最佳的球员之一，个人防守极具压迫性。他是一位伟大的NBA球员！”

伟大的牢大，现在还在为我们的网安学习发力！

其他选项我们就不必测试了，都是类似的。从这个过程中，我们应该能够意识到一个问题，为什么我们前端传入参数之后，能够获得一个图片文件？看来一定是存在一个逻辑，允许把后端的图片传递到前端来，对应一下标题，看来这就是“文件包含”！

也就是说，当我们选择一个选项后，一定同时有一个命令被传递到后端，指定了一个图片文件的传递！

既然如此，我们就需要抓包看看这个前后端通信的过程中发生了什么。我们先在提交选项的时候抓包看看：

```
GET /vul/fileinclude/fi_local.php?filename=file1.php&submit
=%E6%8F%90%E4%BA%A4 HTTP/1.1
Host: XXX.XXX.XXX.X
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) Apple
WebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/5
37.36 Edg/146.0.0.0
Accept: text/html,application/xhtml+xml,application/xml;q=
0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/
signed-exchange;v=b3;q=0.7
Referer: http://XXX.XXX.XXX.X/vul/fileinclude/fi_local.php
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;
q=0.6
Cookie: PHPSESSID=vjvnpno138k2u18f2gl2jhm57p
Connection: keep-alive
```

关注一下第一行传递的参数，线索已经很清晰了：

```
GET /vul/fileinclude/fi_local.php?filename=file1.php&submit
=%E6%8F%90%E4%BA%A4 HTTP/1.1
```

这里，我们选的的kobe，但是实际传递到后端的参数是 `filename=file1.php` ，也就是读取后端的对应文件！这不就是一个典型的文件包含吗！

所以，我们现在要做的就是，修改这个参数，读取到一些“特别”的文件。什么文件比较好呢？

当然，由于皮卡丘靶场是本地部署的，我们再怎么读取也不过是读我们自己的文件罢了。如果你的电脑像我一样是Windows系统，不妨就把filename后面的内容改为 `../../../../Windows/win.ini` ，这就是读取Windows的系统文件内容。（ `../` 是“返回上一级目录”，多写几层，是为了从当前目录一路返回到根目录，然后从根目录出发，指向目标文件）

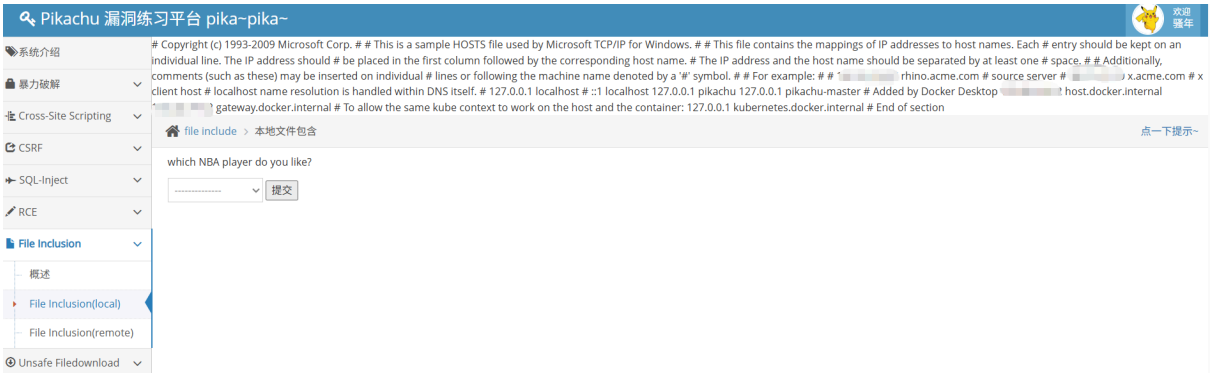
修改后，放行！

欸！网页上面多了一行，这个看来就是 `win.ini` 文件的内容了：

```
; for 16-bit app support [fonts] [extensions] [mci extensio
ns] [files] [Mail] MAPI=1
```

这行代码是Windows系统的一个古老配置文件，记录了16位程序兼容性、字体映射、MAPI邮件设置等信息。在Windows 95/98时代，这个文件很重要；到了Windows 10/11，它基本上就是个“历史遗迹”——但只要能读到它，就说明文件包含成功了！

当然，我们还可以试试其他系统文件，比如 `../../../../Windows/System32/drivers/etc/hosts` ：



这下可不得了，我们刚才读取的是Windows的本地DNS解析文件，作用是把域名映射到IP地址。如果你在一个真实的服务器上读到了这个文件，而且里面有自定义记录，你就能知道：这个服务器连了哪些内网机器；内网机器的IP和域名对应关系；可能还有开发环境的测试域名

这就是信息泄露——本来是给系统自己看的，被你看到了。

话说回来，上次我的网站服务器被同学攻破，就是因为他通过这个漏洞查看到了我的源代码，找出了漏洞所在，并最终获取到了任意写的权限。更令人警醒的是，这个漏洞实施起来一点也不复杂，但是却能直接获取最秘密的数据！不可不防啊！

最后，我们来看看本题对应的源代码吧，核心漏洞在这里：

```
if(isset($_GET['submit']) && $_GET['filename']!=null){
    $filename=$_GET['filename'];
    include "include/$filename";  // 变量传进来直接包含,没做任何安全限制
}
```

此处 `$_GET['filename']` 直接从URL参数拿文件名，赋值给 `$filename` ——没有任何过滤，没有任何校验。

而第三行 `include "include/$filename";` 。正常情况下，你选 `file1.php` ，实际包含的是 `include/file1.php` ，于是就会返回预期结果；但是现在由于没有过滤，你可以控制 `$filename` ，从而可以让 `include` 去包含任何文件！

而这就是文件包含漏洞的代码原理！

当然，开发者同时也用注释示范了应该怎么防范此类漏洞：

```
//      //安全的写法,使用白名单，严格指定包含的文件名
//      if($filename=='file1.php' || $filename=='file2.php' || $filename=='file3.php' || $filename=='file4.php' || $filename=='file5.php'){
```

```
//          include "include/$filename";
//      }
```

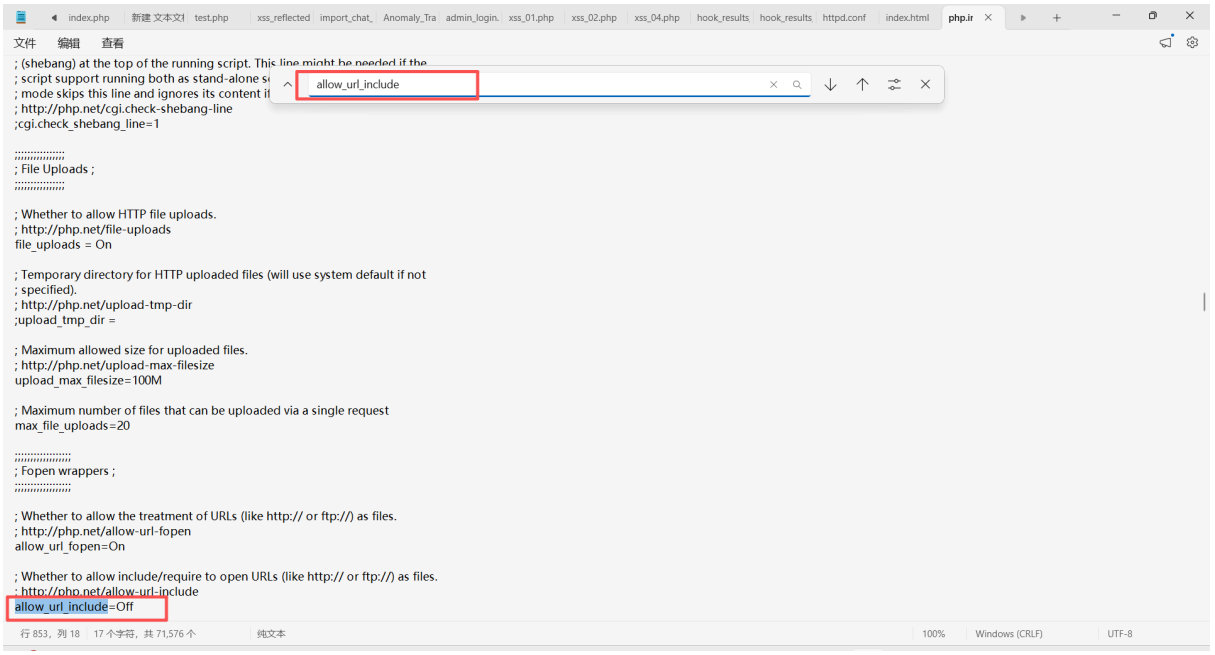
这里的代码是很清晰的——把传入的参数限定在指定的四个文件中，才能正常启动include()函数。这就是针对此类漏洞最常用的防范方式，指定文件白名单！

不过，到目前为止，我们只是读取本地文件（即服务器本地的文件），那么我们能不能读取这个服务器上启用的远程服务对应的文件呢？那就去看看下一题吧！

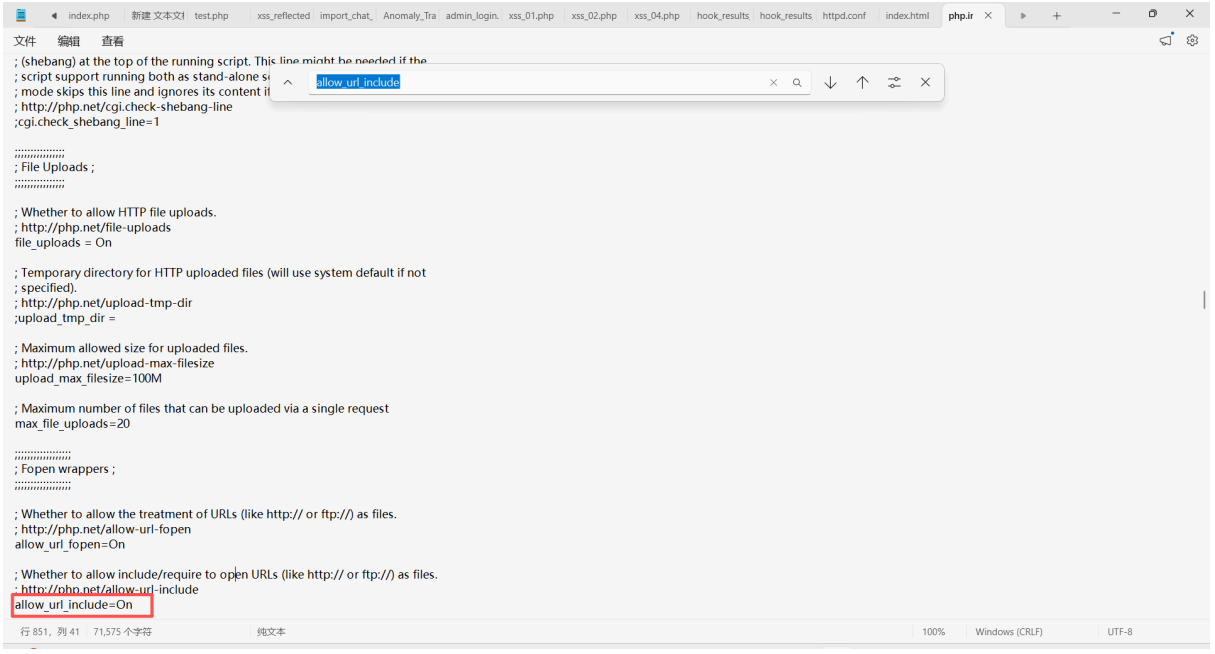
2、File Inclusion(remote)

来到下一题，刚开始就给我们一个提示，说有一个配置没有打开——咱也不要逞强，先开启了再说。

首先找到php.ini文件，看到这样一个界面：嘶~好复杂。不过既然这里是和远程文件包含有关，我们先搜索看看，能不能找到对应的源代码。我们可以直接利用记事本自带的查找功能，输入：`allow_url_include`，回车：



很清楚啦！这里`allow_url_include=Off`明明白白地写在文件里，咱们把它改为On就行！

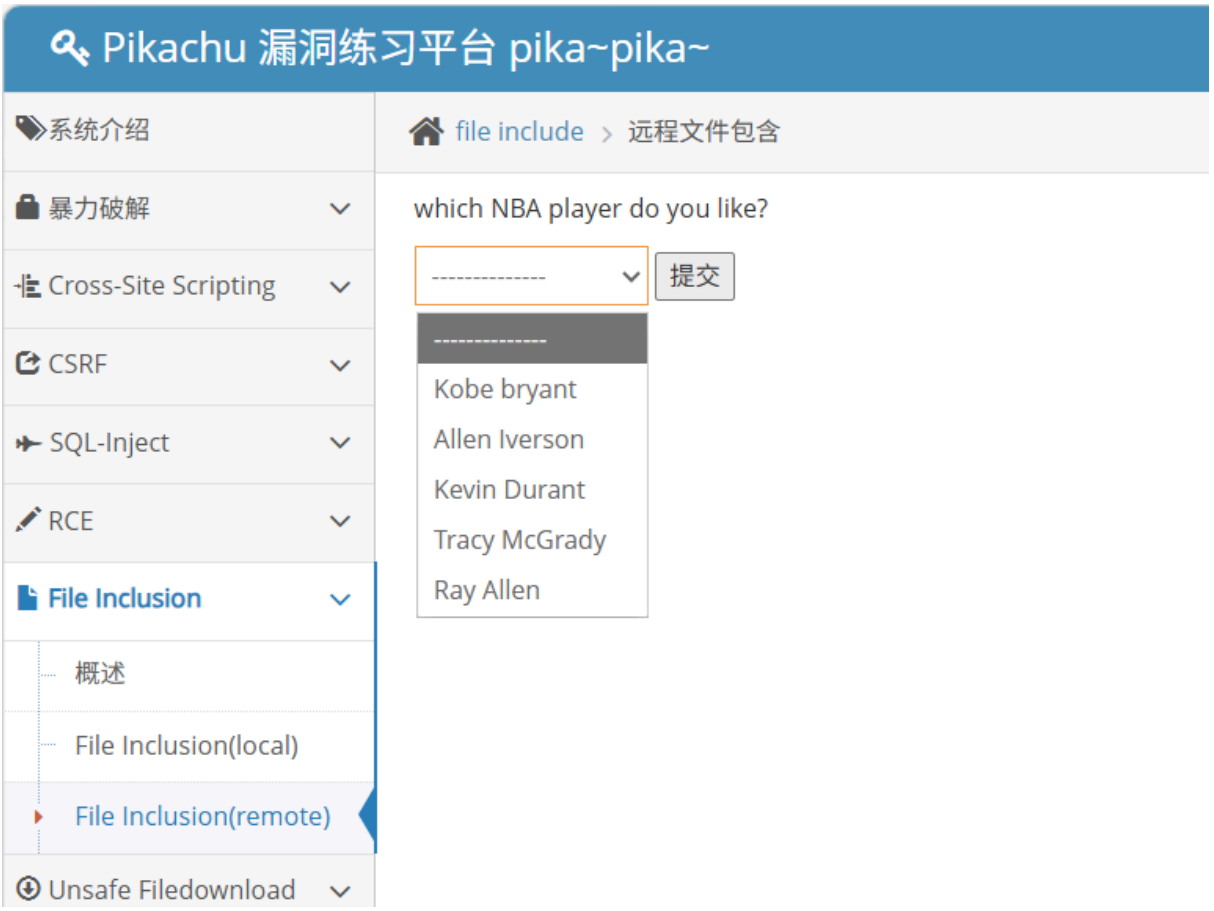


修改完成后保存即可。接下来要重启中间件服务，打开小皮面板，重启一下两个套件就行：

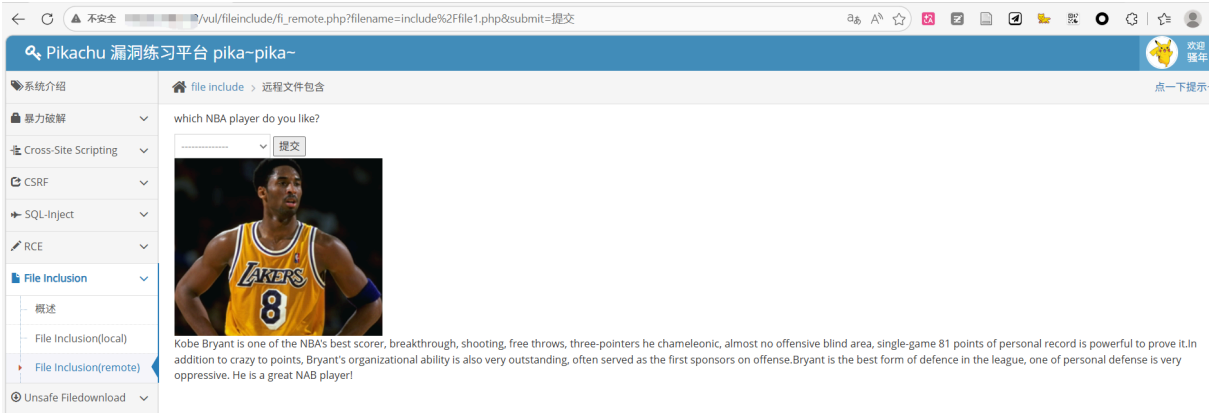
搞定！回头看看皮卡丘靶场的页面吧：

警告消失了！现在咱们就可以开始正式展开攻击了！

首先当然还是正常走一遍，看看这大概是什么一个机制：



有一个下拉列表，这里当然要选一直帮助我们学习的牢大！



跳转到了一个新的页面，有Kobe的图片，以及一段文本介绍——看起来前端是返回了一个新的文件！回想一下上一题，这里大概率也是通过文件包含的方式实现的！

同时，我们注意到这里的网址栏的变化：后缀中多了一点东西：

```
?filename=include%2Ffile1.php&submit=提交
```

这个是我们的老朋友啦，一看就是一个Get类型的参数传递！这里的include清晰地向我们揭示了一个事实——这里同样是通过include命令来实现后端文件的包含的！

因此，我们可以直接修改网址栏来进行攻击——这个方法和我们用BurpSuite抓包修改参数的方式是一样的，而且在这个场景下更加快捷。

回顾一下上一题我们的操作方式，这里很可能就是直接修改include后面的文件名——至少应该先试试看！而这一次我们的目标不再是本地的文件，而是远程的文件！

什么是远程的文件呢？当然，如果你有一个远程文件的链接，那就可以把这个链接用来替代filename后面的参数；但是我们其实还有另外的更加直接的选择：直接输入一个网页链接！

比如在这里，我们把filename改成百度的域名地址：

```
http://XXX.XXX.XXX.XXX/vul/fileinclude/fi_remote.php?filename=http://www.baidu.com&submit=提交
```

访问试试效果吧！

好混乱的页面——看来即便这个filename不是一个传统的“文件”，照样能够被“包含”从而执行。

其实，核心原理就在于：`include()` 不要求“文件”，只要求“资源”——只要 PHP 能通过某个协议（http、https、ftp、php:// 等）获取到内容，它就会首先尝试获取内容，并将其当作PHP代码加以解析，如果解析失败就会原样输出。这里很显然如果访问我们输入的域名地址，会来到百度的首页，而这本质上是一个HTML格式的文件（你可

以在这个域名下ctrl+s保存，会发现保存了一个html文件），当然和PHP不兼容，因此就原样输出在前端，并和已有的页面混淆在一起。

总之，到这里我们已经成功了：通过远程文件包含的漏洞成功让网页返回了百度首页的网页，和已有的页面混在一起成了一个不可名状的产物。

那么，这个漏洞有什么实际价值呢？我们这样污染前端，也就是“恶心”一下我们自己，好像也没啥影响啊？

别忘了，我们刚才只不过是在测试，而我们实际上能做的，远远不止这些——当我们真正上升到攻击层面，我们可以自己编写一个PHP脚本文件，上传到web之中，然后再利用这里远程包含的漏洞，就可以让服务器去解析我们指定的文件，导致文件被执行——你可以去获取服务器的隐私数据，也可以直接控制服务器，而最终的效果比起我们上一节的RCE攻击，也是有过之而不及！

最后，还是老传统，我们一起看看这个漏洞对应的源代码吧！

```
if(isset($_GET['submit']) && $_GET['filename']!=null){
    $filename=$_GET['filename'];
    include "$filename";  // 变量传进来直接包含，没做任何安全限制
}
```

看了这么多代码，其实这段代码还是很平易近人的——首先从url（网址）中获取传递的参数，然后直接拼接到 `include()` 函数中去，同样没有做任何过滤。如此一来，`include()` 函数能做什么，我们就可以做什么！无论是读取本地文件，还是执行远程文件，都不在话下了！

现在，这两类文件包含漏洞我们都见识过了，接下来就去下一章领略一下新漏洞的“风采”吧！

八、Unsafe Filedownload

还是先看看开发者的概述：

> **不安全的文件下载概述**
>
> 文件下载功能在很多web系统上都会出现，一般我们当点击下载链接，便会向后台发送一个下载请求，一般这个请求会包含一个需要下载的文件名称，后台在收到请求后，会开始执行下载代码，将该文件名对应的文件response给浏览器，从而完成

> 下载。 如果后台在收到请求的文件名后,将其直接拼进下载文件的路径中而不对其进行安全判断的话，则可能会引发不安全的文件下载漏洞。
>
> 此时如果 攻击者提交的不是一个程序预期的的文件名，而是一个精心构造的路径(比如../../../etc/passwd),则很有可能会直接将该指定的文件下载下来。 从而导致后台敏感信息(密码文件、源代码等)被下载。
>
> 所以，在设计文件下载功能时，如果下载的目标文件是由前端传进来的，则一定要对传进来的文件进行安全考虑。 切记：所有与前端交互的数据都是不安全的，不能掉以轻心！
>
> 你可以通过“Unsafe file download”对应的测试栏目，来进一步的了解该漏洞。

这个漏洞在说什么?

一句话概括就是：后端直接用了前端传过来的文件名去读文件，没做任何检查。

正常逻辑是：用户点击“下载”，前端告诉后端“我要下载report.pdf”，后端就去 `/downloads/report.pdf` 把这个文件读出来返回。但如果没有校验，攻击者可以把文件名改成 `../../../etc/passwd` 。后端还是老老实实地拼路径： `/downloads/../../../etc/passwd` ，经过路径解析，直接就跑到系统目录里把 `passwd` 文件给下载了。

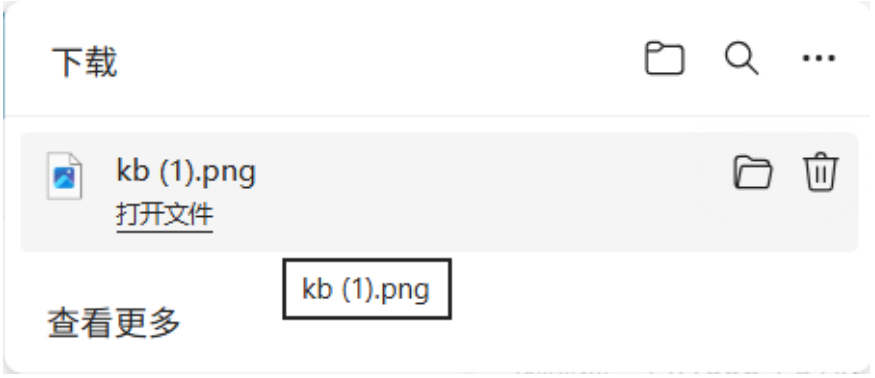
这就是所谓不安全的文件下载。从本质上来说，和我们上一题的文件包含漏洞是相似的，只不过上一题只是用include()来读文件，这里则是直接将文件下载下来。

我们接下来就来体验一下这个漏洞的威力吧!

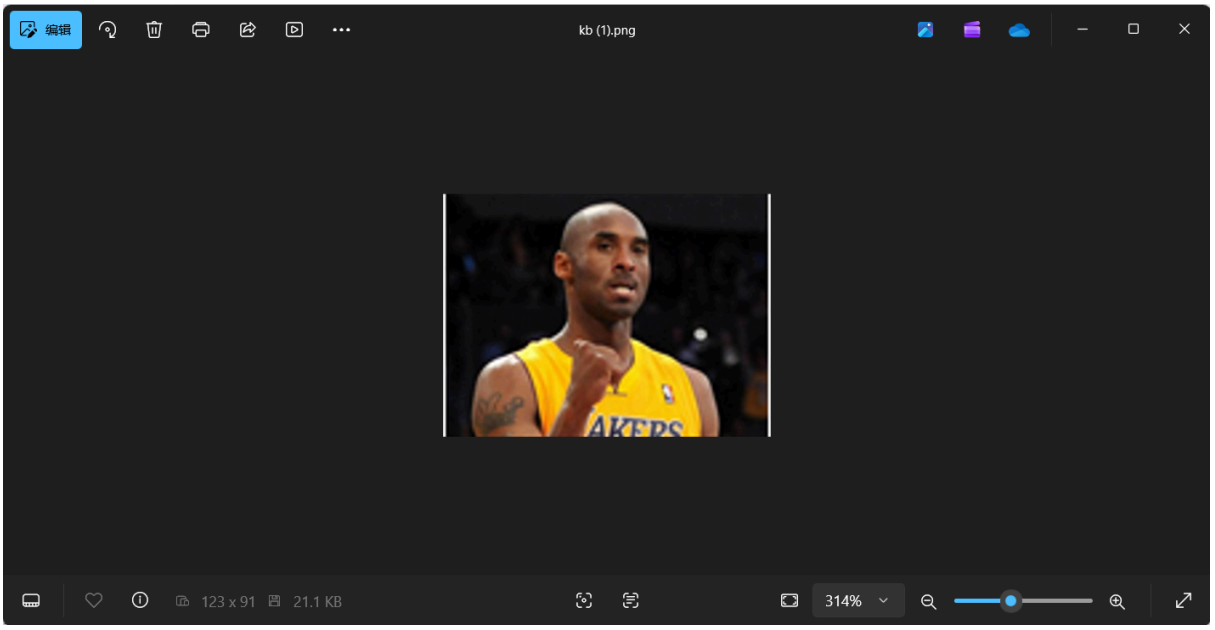
1、不安全的文件下载

这一次的前端页面的元素比前面的题目要丰富多了!

题目提示我们："点击球员名字即可下载头像图片！"那就试试看咯!



打开看看：



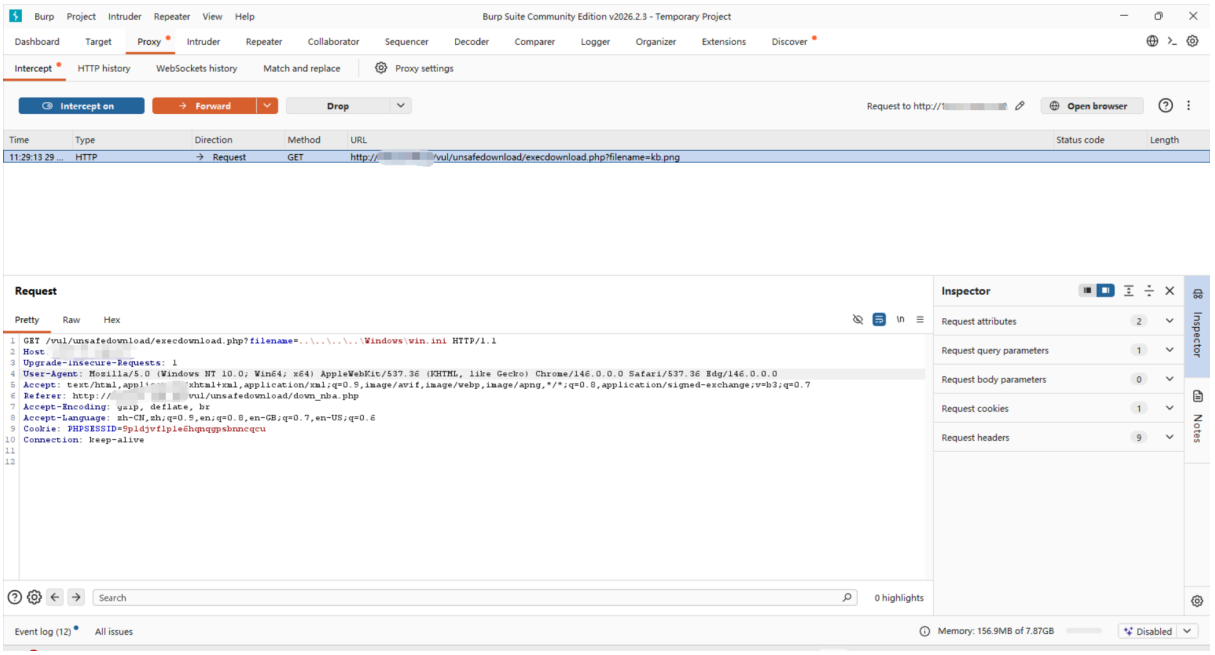
一如既往的牢大!

在这里，类似上一题的思路，我们应该能够猜测到，这里同样是传递了一个参数到后端，后端执行了下载命令。那么，首先就需要抓包来看看这个参数究竟是如何传递的。

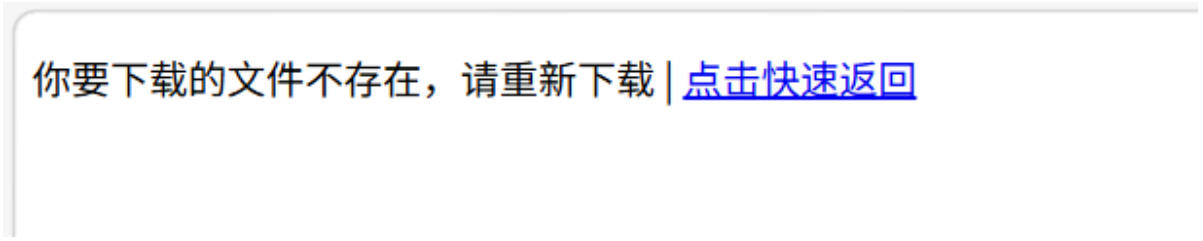
```
GET /vul/unsafedownload/execdownload.php?filename=kb.png HTTP/1.1
Host: XXX.XXX.XXX.XXX
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 Edg/146.0.0.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://XXX.XXX.XXX.XXX/vul/unsafedownload/down_nba.php
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
Cookie: PHPSESSID=9p1djvflp1e6hqnqgpsbnncqcu
Connection: keep-alive
```

这里还是看第一行，是很清晰的——`filename=kb.png` 就是被传递到后端 `execdownload.php` 文件进行处理的参数。因此我们有理由猜测，后端对于下载的处理逻辑就是把这个参数拼接到 `execdownload.php` 文件的对应命令中去并加以执行。

因此，我们可以尝试修改这个参数，看看能不能下载一些其他的东西。由于我这里是Windows系统，可以尝试一下能不能下载系统的配置文件 `C:\Windows\win.ini` 。但是这里要注意，我们这里还处于靶场对应的目录下，因此首先要回到根目录，因此要构造成 `filename=..\..\..\..\Windows\win.ini` ：

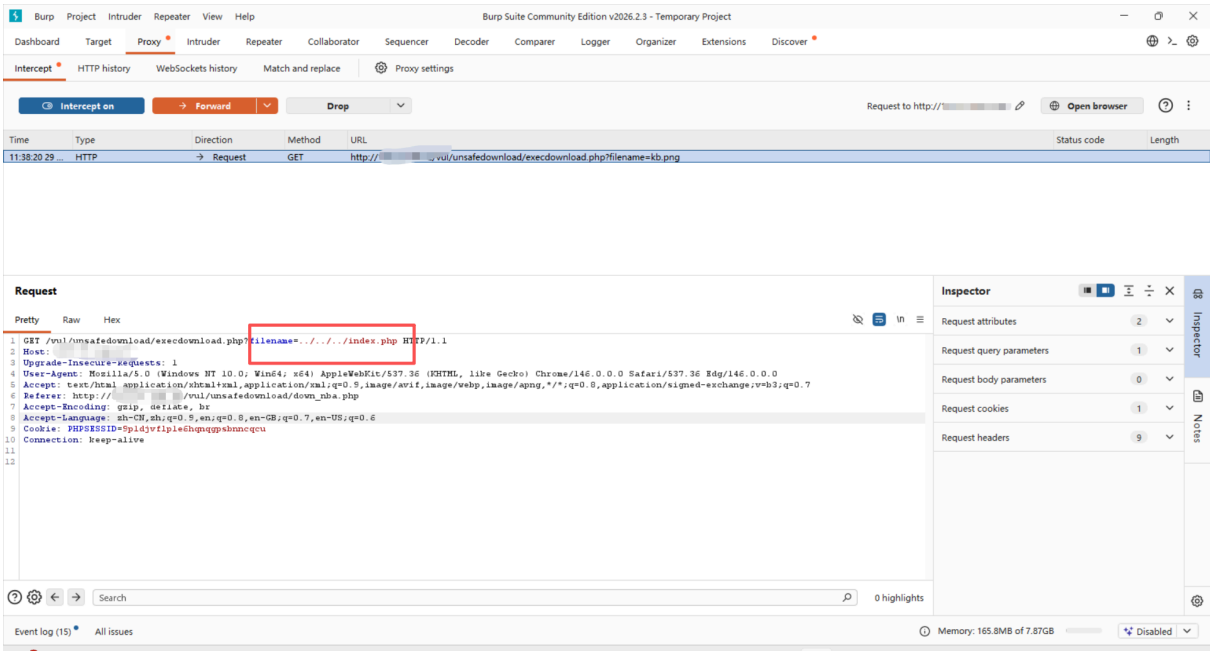


放行这个请求：

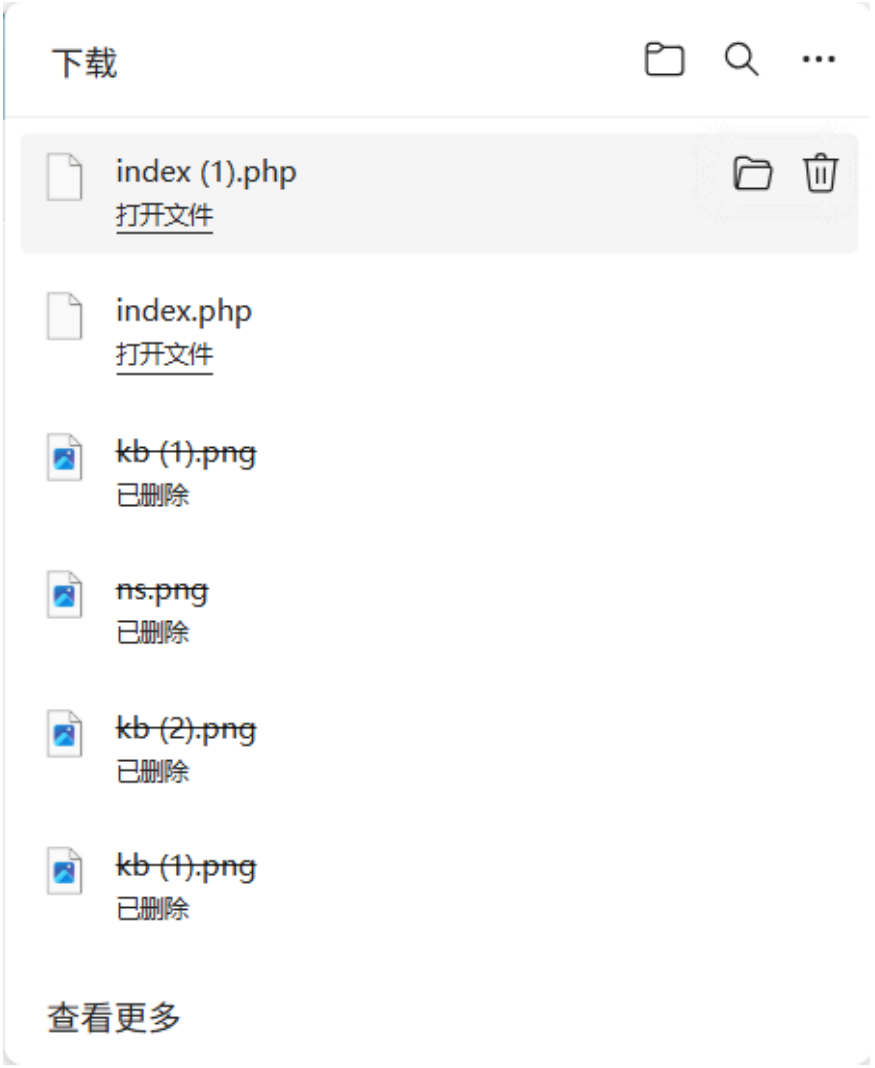


呃，好像没有——不过出现了这个页面就已经给我们吃下了一颗定心丸：我们确实是可以利用这个漏洞下载系统文件的，只是当前这个文件不存在而已，应该是我们路径构造的问题。这个能否成功因系统而异，也许在你的电脑上就能成功下载呢！

不过有一个文件我们肯定是能成功下载的，就是皮卡丘靶场的代码文件本身，它的相对路径都是一致的。这里我们可以构造为 `filename=../../../index.php` ：


Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2026.2.3 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover
Intercept HTTP history WebSockets history Match and replace Proxy settings
Intercept on Forward Drop
Request to http://
Open browser
Time Type Direction Method URL Status code Length
11:38:20 29 ... HTTP Request GET http:// /vul/unsafedownload/execdownload.php?filename=kb.png
Request
Pretty Raw Hex
1 GET /vul/unsafedownload/execdownload.php?filename=../../../index.php HTTP/1.1
2 Host:
3 Upgrade-Insecure-Requests: 1
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 Edg/146.0.0.0
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
6 Referer: http:// /vul/unsafedownload/down_nba.php
7 Accept-Encoding: gzip, deflate, br
8 Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
9 Cookie: PHPSESSID=5pldjvflple6hqnqqpsbnncqcu
10 Connection: keep-alive
11
12
Search
0 highlights
Inspector
Request attributes 2
Request query parameters 1
Request body parameters 0
Request cookies 1
Request headers 9
Inspector
Notes
Event log (15)
All issues
Memory: 165.8MB of 7.87GB
Disabled

放行！


下载
index (1).php
打开文件
index.php
打开文件
kb (1).png
已删除
ns.png
已删除
kb (2).png
已删除
kb (1).png
已删除
查看更多

很好，下载成功啦，我们看看这个文件：

这就是我们皮卡丘靶场的首页代码！如此一来，我们可以如法炮制把所有文件全都下载下来——只要路径构造正确，不仅仅是源代码，也包括系统文件！多么可怕！

最后，咱们一起看看源代码吧，首先看看网页前端对应的`down_nba.php`：（整理了一下）

```
<img src="download/kb.png" />
<a href="execdownload.php?filename=kb.png">科比.布莱恩特</a>
```

嗯，看起来没啥特别的，就只是一个前端展示界面而已，不过我们能从中看到这个传递参数的秘密，以及图片文件的位置是在download目录下。而且从方才我们抓包的信息可以看出来，下载链接指向`execdownload.php?filename=kb.png`，参数应该就是在这里被处理的。因此，我们下一步来看看`execdownload.php`文件的源代码吧！

```
$file_path="download/{$_GET['filename']}";
$file_path=iconv("utf-8","gb2312",$file_path);

if(!file_exists($file_path)){
    skip("你要下载的文件不存在，请重新下载", 'unsafe_down.php');
    return ;
}
```

这就是核心漏洞所在！它直接将 `$_GET['filename']` 拼接到 `download/` 后面，没有任何过滤、校验、白名单限制，直接开始下载！然后用 `file_exists()` 检查文件是否存在，存在则用 `fopen()` 读取并返回给浏览器，不存在则返回 `"你要下载的文件不存在，请重新下载"` ——和我们刚才的探索呼应上了！

因此，我们当然不限于下载指定的那几个作为参数传递的图片文件，而实际上可以通过路径遍历下载整个服务器上的所有文件！

这就是不安全的文件下载漏洞，其实核心也就是把特定的参数拼接到用于执行下载命令的代码中去，和上一章节的文件包含漏洞差不多。思路都是一以贯之——由此可见过滤的重要性！

回过头来，这是网站安全领域的一个最基本的错误：用户输入不可信。

这是安全领域的第一性原则，但也是最容易被忽略的。开发者往往只想着“功能跑起来就行”，默认用户会按设计的流程走——点哪个按钮就下载哪个文件。但攻击者偏偏不按套路出牌。

“能跑就行”？说实话，也就调侃调侃啦——如果你的内心不局限在一些校内的没啥实际应用价值的小项目，而有志于去做做真正的工程，那么就一定要考虑到很多方面的东西啦！毕竟，上线之后被黑客一发 `../../../../etc/passwd` 把家底都搬空了，那时候再说“我当时没想到”可就来不及了。

九、Unsafe Fileupload

下一篇是不安全的文件上传，看看开发者的话：

> **不安全的文件上传漏洞概述**
>
> 文件上传功能在web应用系统很常见，比如很多网站注册的时候需要上传头像、上传附件等等。当用户点击上传按钮后，后台会对上传的文件进行判断 比如是否是指定的类型、后缀名、大小等等，然后将其按照设计的格式进行重命名后存储在指定的目录。 如果说后台对上传的文件没有进行任何的安全判断或者判断条件不够严谨，则攻击着可能会上传一些恶意的文件，比如一句话木马，从而导致后台服务器被webshell。
>
> 所以，在设计文件上传功能时，一定要对传进来的文件进行严格的安全考虑。比如：
>
> --验证文件类型、后缀名、大小;
>
> --验证文件的上传方式;
>
> --对文件进行一定复杂的重命名;

> --不要暴露文件上传后的路径;
>
> --等等...
>
> 你可以通过“Unsafe file upload”对应的测试栏目，来进一步的了解该漏洞。

一句话，如果网站没有对文件上传设置合适的过滤条件，即使我们上传的是木马，文件也一样会被存储到服务器后台，造成严重的安全问题。

因此，我们这里要做的事情就是绕过靶场的文件格式限制，故意去上传一些“非法文件”——当然，这里的意思是说，我们只是去做“它让我们上传图片，我们偏偏要上传txt”这种无伤大雅的操作（可别霍霍自己的计算机了）；而只要我们能够上传成功一个格式不一致的文件，那当然也能上传一个木马文件——这就意味着成功！

现在就来看看题目吧！

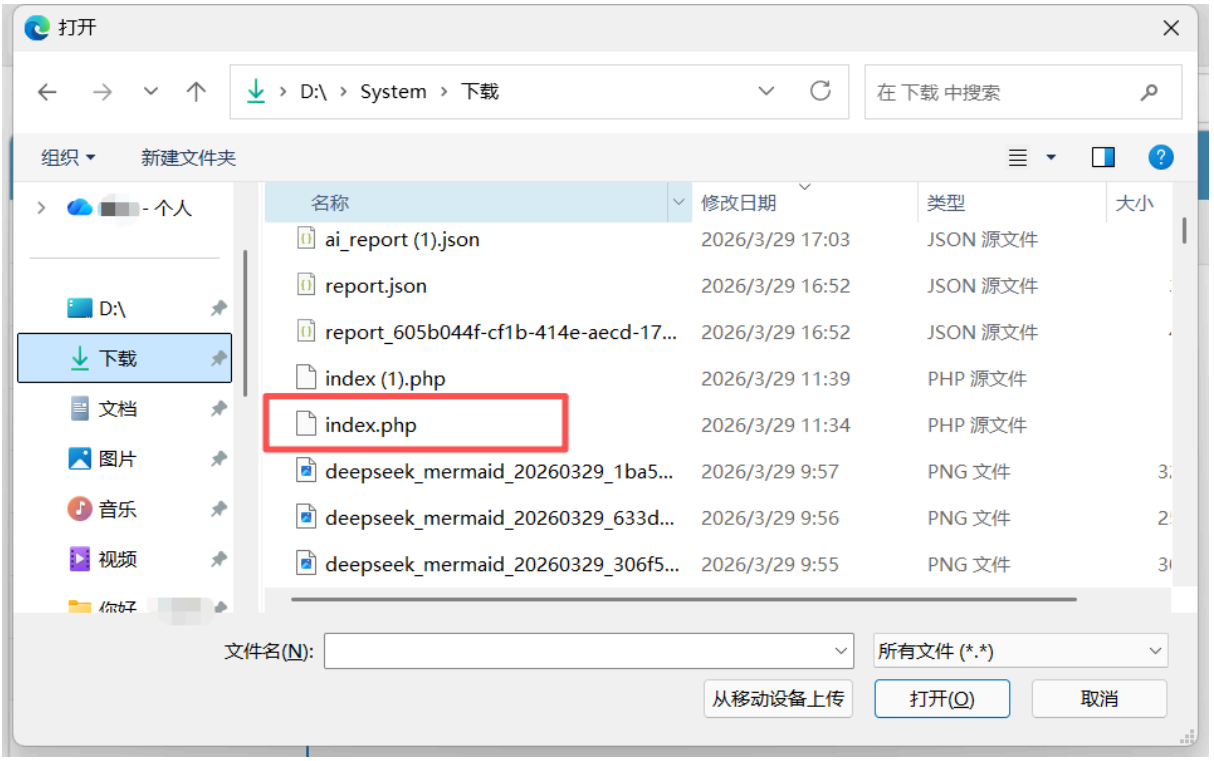
1、client check

第一题是“客户端check”，其实也就是在前端进行格式检查——事到如今，我想读者早已对这种防范机制不屑一顾了吧！诚然如开发者在本题的提示中所说：“一切在前端做的安全措施都是不靠谱的。”

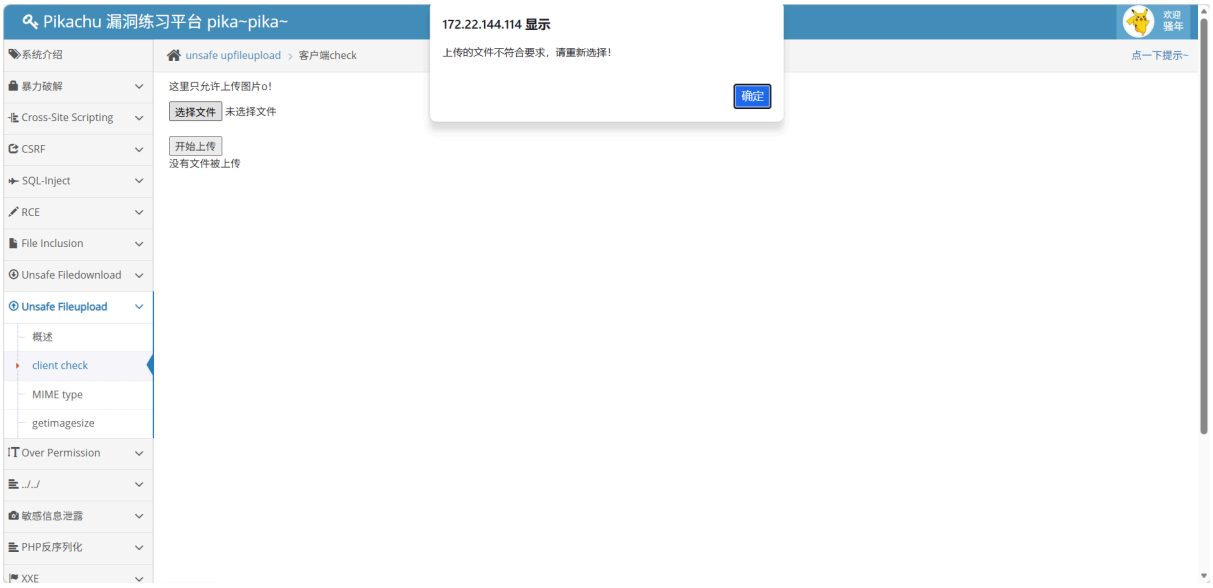
那么怎么去绕过前端的限制呢？最简单有效的方法就是用BurpSuite去抓包，修改请求之后放包——我们之前就是这么做的，实际上当我们抓到请求的时候，浏览器的前端防御机制也就毫无作用了。

看看页面吧，简单的上传功能，让我们只上传图片——我偏不！

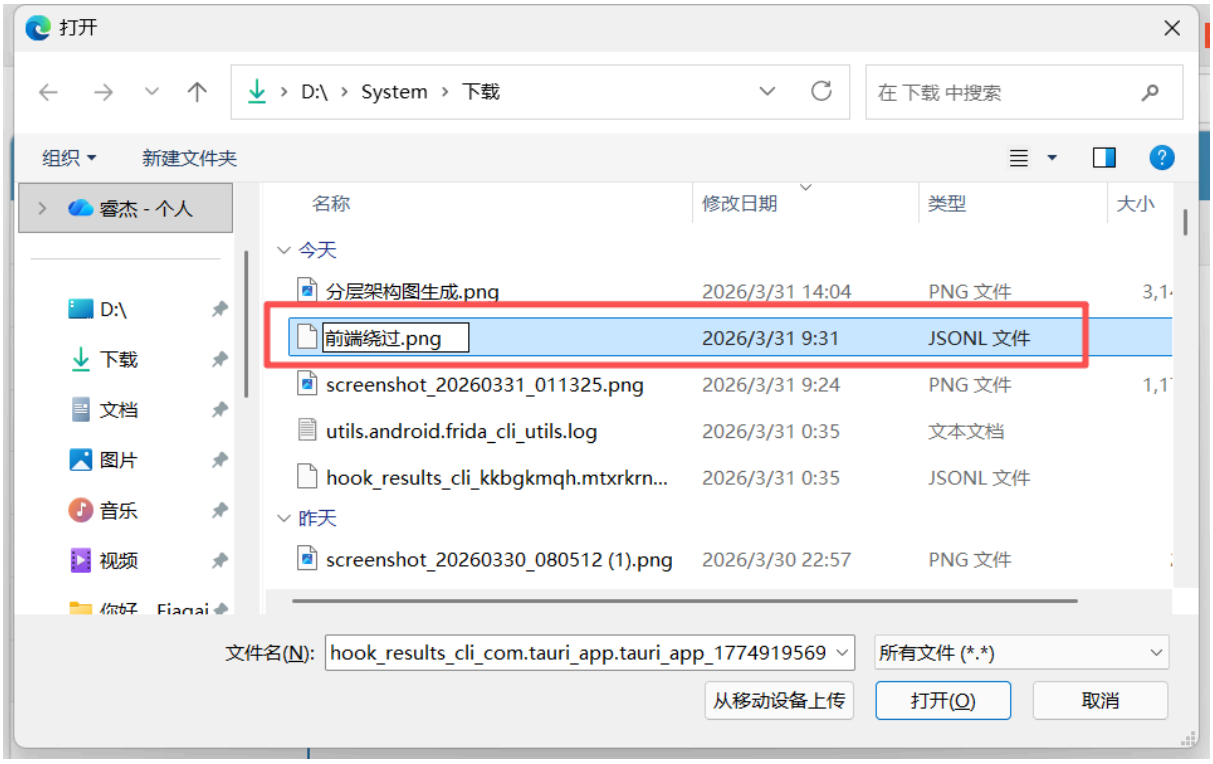
这里随便上传一个文件，先试试看，这里故意先选择php文件：



发现前端直接返回了提示，说我们上传的文件不符合要求——这个熟悉的弹窗，显然是Javascript中的alert弹窗没跑了，而这就是题目中所说的“客户端check”！

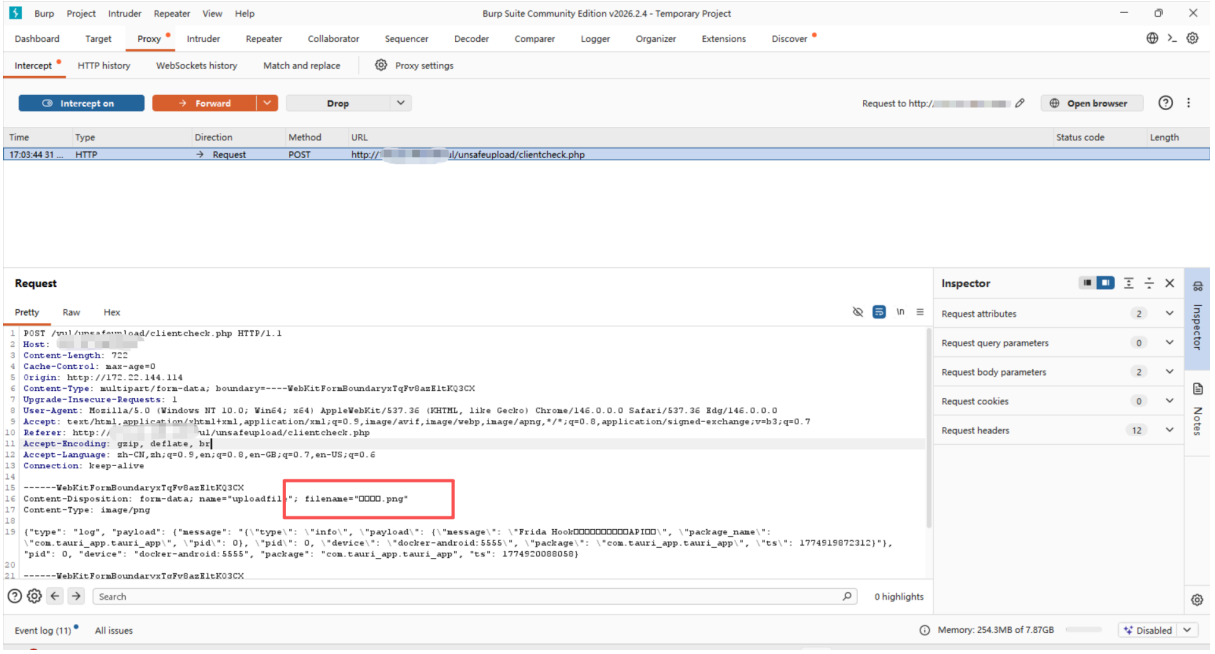


但是，前端到底是怎么进行格式检查的呢？很有可能就是检查文件的后缀名！那么直接修改文件后缀名，后面抓包的时候再改回来不就可以了吗！



这里把一个jsonl文件的后缀修改成png图片格式再打开，这次果然没有弹出任何提示弹窗了：

接下来开启BurpSuite拦截，点击“开始上传”，就能抓到包了：



这次的抓包数据和之前我们见过的都不太一样，下面有一堆编码，这个其实就是我们上传的图片的数据。这里如果这里你上传的是真正的图片，那么就能看到一堆乱码——这是因为图片数据属于二进制格式，本身在文本视图中展示时就会产生乱码。而我这里上传的文件虽然后缀被改成了png，但是实际上仍然是一个jsonl格式的文件，所以这里捕包的数据显示的就是真实的文件编码。

这里我们要做的只是把图中标记的部分后缀名修改回原本的格式名称即可，改完后放包！

于是，我们就成功绕过了前端的限制，上传了一个非图片格式的文件啦！

当然，在这里还有更简单的方法，就和我们一开始在暴力破解系列里验证码绕过那一道题一样，直接在浏览器中关闭JavaScript渲染，即可随意上传文件——毕竟刚才我们测试的时候就已经知道这里用的是JavaScript的拦截命令了！

这里，我们上传的当然只是一个无害的jsonl文件。但是方法是相通的，我们绕过了前端限制，当然也可以上传木马、病毒、破坏脚本，那就是货真价实的网络攻击了！这也再一次提醒我们，前端的防御机制永远是不可靠的，永远阻挡不了真正的网络攻击者！

不过，这一次我们就没有必要看源代码了——毕竟正如题目所说，这里只有一个简单的前端检查，用了一点简单的JavaScript机制，实在没什么意思。我们接下来就一起看看真正有趣的文件上传漏洞吧！

2、MIME type

来到下一题，服务端check，看起来页面和上一题是完全一样的——但是光从题目我们就能看出来，这一次绝对不是简单的前端检查了：

但是真正的问题在于，我们并不知道这个服务端的check到底是什么样的一种检查机制。而如果不清楚这个问题，我们也就不可能绕过后端的检查。

先试试看咯！

这里，就不再像上一题一样随便传一个非图片格式的文件了——为了模拟真实的网安场景，我准备了一个“一句话木马”的脚本文件，编码如下：

```
<?php echo "webshell test success"; ?>
```

所谓“一句话木马”，实际上就是一个精简版的可执行文件。由于皮卡丘靶场是本地部署的，这里当然不可能用一个破坏性的木马脚本，于是我选了这个——它的含义很简单，当文件上传成功后，你通过url访问这个文件路径，浏览器就会显示一行文字：`webshell test success`。

将这个脚本保存为一个php文件，待会测试就用它了！

先不急着打开BurpSuite抓包，咱们先正常上传一次看看：

点击开始上传，发现这次不再有弹窗出现了，前端加载了一会之后直接返回了错误信息，说我们上传的文件格式不对：

“没有弹窗”，说明这次不再是简单的前端检查了，而”加载了一会“似乎说明期间有一个前后端通信的过程？（如果你发现非常流畅，那说明你的电脑性能太好啦！不妨上传一个大文件看看，doge）

到了这里，我们就能大致猜测出来，这里存在一个后端检查的过程！而为了定位问题所在，需要抓包看看！

打开BurpSuite，重新上传一次，抓包数据如下：

```
POST /vul/unsafeupload/servercheck.php HTTP/1.1
Host: XXX.XXX.XXX.XXX
Content-Length: 356
Cache-Control: max-age=0
Origin: http://XXX.XXX.XXX.XXX
Content-Type: multipart/form-data; boundary=----WebKitFormB
oundaryh6IqAxVgQUISfZgc
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) Apple
WebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/5
37.36 Edg/146.0.0.0
Accept: text/html,application/xhtml+xml,application/xml;q=
0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/
signed-exchange;v=b3;q=0.7
Referer: http://XXX.XXX.XXX.XXX/vul/unsafeupload/serverchec
k.php
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;
q=0.6
Cookie: PHPSESSID=b06bcfvl87q2b2jlg71ad847ge
Connection: keep-alive

------WebKitFormBoundaryh6IqAxVgQUISfZgc
Content-Disposition: form-data; name="uploadfile"; filename
="一句话木马.php"
Content-Type: application/octet-stream

<?php echo "webshell test success"; ?>
------WebKitFormBoundaryh6IqAxVgQUISfZgc
Content-Disposition: form-data; name="submit"
```

```
开始上传
------WebKitFormBoundaryh6IqAxVgQUISfZgc--
```

这里数据稍微有点乱，我们提炼一下，其实有用的数据仅仅在下面的几行：

```
-----WebKitFormBoundaryh6IqAxVgQUISfZgc
Content-Disposition: form-data; name="uploadfile"; filename
="一句话木马.php"
Content-Type: application/octet-stream

<?php echo "webshell test success"; ?>
------WebKitFormBoundaryh6IqAxVgQUISfZgc
Content-Disposition: form-data; name="submit"

开始上传
------WebKitFormBoundaryh6IqAxVgQUISfZgc--
```

这是什么意思呢？`------WebKitFormBoundaryh6IqAxVgQUISfZgc` 是表单边界，没有实际含义；`Content-Disposition: form-data; name="uploadfile"; filename="一句话木马.php"` 是这个文件的属性；`<?php echo "webshell test success"; ?>` 是我们上传的文件内容编码；而 `Content-Type: application/octet-stream` 是这里最重要的信息（因为我们没见过）——这是文件的MIME类型。

那么，什么是MIME类型呢？问一下AI：

> MIME 类型（Multipurpose Internet Mail Extensions）是告诉服务器“这个文件是什么格式”的标识。可以理解为文件的“身份证”或“标签”。MIME类型是浏览器根据文件后缀名自动填的，不是服务器检测文件内容得出的。常见例子如下：

文件类型	MIME 类型	说明
`.jpg` 图片	`image/jpeg`	JPEG 图片
`.png` 图片	`image/png`	PNG 图片
`.php` 代码	`application/x-php`	PHP 脚本
`.txt` 文本	`text/plain`	纯文本
`.html` 网页	`text/html`	HTML 页面
任意二进制文件	`application/octet-stream`	未知类型（默认）

原来如此！那么我们当然就要意识到：MIME类型既然代表着文件的"身份证"，那么它很可能就是后端检查的依据！对照一下这张表格和我们实际抓包的字段，确实是复合的——我们上传的PHP脚本就是一个二进制文件！

所以，就像我们上一题做的一样，直接修改这个参数！既然这里后端期望的是图片类型，那么就把这个MIME字段修改为jpg文件对应的 `image/jpeg` ！

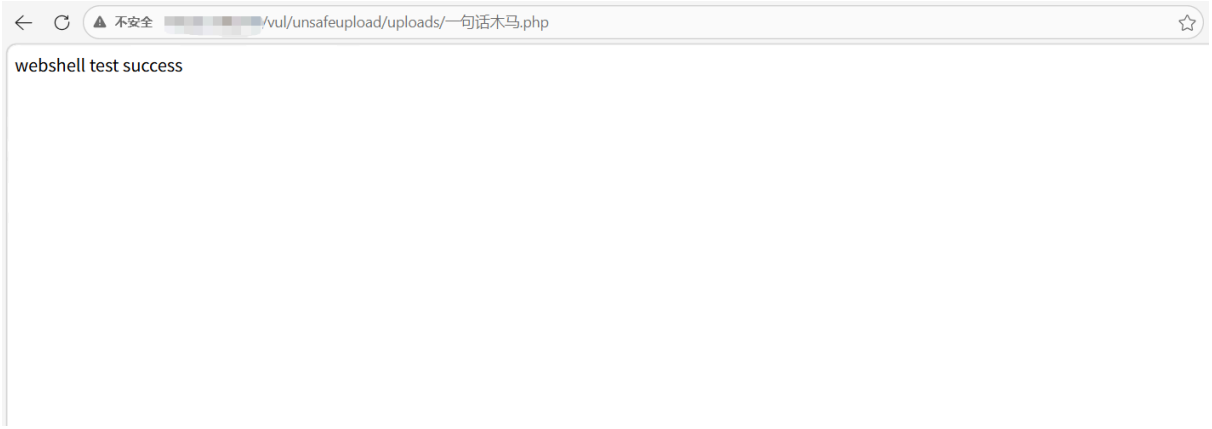
修改后，放包！

果然成功了！看来修改MIME类型的操作的确达到了预期的效果！这个文件保存的路径也很清晰，那么接下来我们就用url去访问它，看看能不能触发这个木马脚本。

在网址栏输入：

```
http://XXX.XXX.XXX.XXX/vul/unsafeupload/uploads/一句话木马.php
```

根据前端刚才返回的存储路径提示，这里url构造就是在本节的目录下加上刚才我们知悉的文件路径，回车访问：



前端返回了 `webshell test success` ，这正是我们"一句话木马"的预期执行效果！到这里，我们也就顺利攻克了这道题！

这道题还是很值得看看源代码的：

```
if(isset($_POST['submit'])){
//      var_dump($_FILES);
    $mime=array('image/jpg','image/jpeg','image/png');//指定MIME类型,这里只是对MIME类型做了判断。
    $save_path='uploads';//指定在当前目录建立一个目录
    $upload=upload_sick('uploadfile',$mime,$save_path);//调用函数
    if($upload['return']){
        $html.="<p class='notice'>文件上传成功</p><p class='notice'>文件保存的路径为：{$upload['new_path']}</p>";
    }else{
        $html.="<p class=notice>{$upload['error']}</p>";
    }
}

.....

        </div>
        <div class="page-content">
            <div id="usu_main">
                <p class="title">这里只允许上传图片，不要乱搞！</p>
                <form class="upload" method="post" enctype
```

```
="multipart/form-data"  action="">
                    <input class="uploadfile" type="file"
name="uploadfile" /><br />
                    <input class="sub" type="submit" name
="submit" value="开始上传" />
                </form>
                <?php
                echo $html;//输出了路径，暴露了
                ?>
            </div>
        </div><!-- /.page-content -->
    </div>
</div><!-- /.main-content -->
```

这里存在两个隐患，分别存在于这两段代码中——第一段代码指定了MIME的几种类型，针对前端传入的参数进行MIME类型的检验，如果符合就存储，不符合就返回错误；第二段代码的关键在于 `echo $html` 输出了上传文件的保存路径，那么用户就可以通过构造url直接访问这个文件路径来去执行它了。

这一切都和我们刚才的操作呼应！

回顾一下本题和上一题，不难发现，如果说前端检查的应对方法是“绕过“，那后端检查的应对措施就是”欺骗“——毕竟前端只是给用户看的，本身代码就是存在于用户自己的浏览器上，是暴露的、可修改的；而后端则是运行在服务器上，用来处理数据的，用户看不到它的代码，也无法直接进行修改，就必然只能通过修改参数等方式骗过后端的处理逻辑了（如果你打穿了网站当我没说）。

当然，这里由于MIME的处理逻辑实质上就是浏览器根据文件后缀名自动生成的，所以这里当然可以沿用上一题的方法，修改文件后缀名，抓包之后把后缀名改回去——但是如果这里我们没有抓包，又何从知晓这里用了MIME呢？

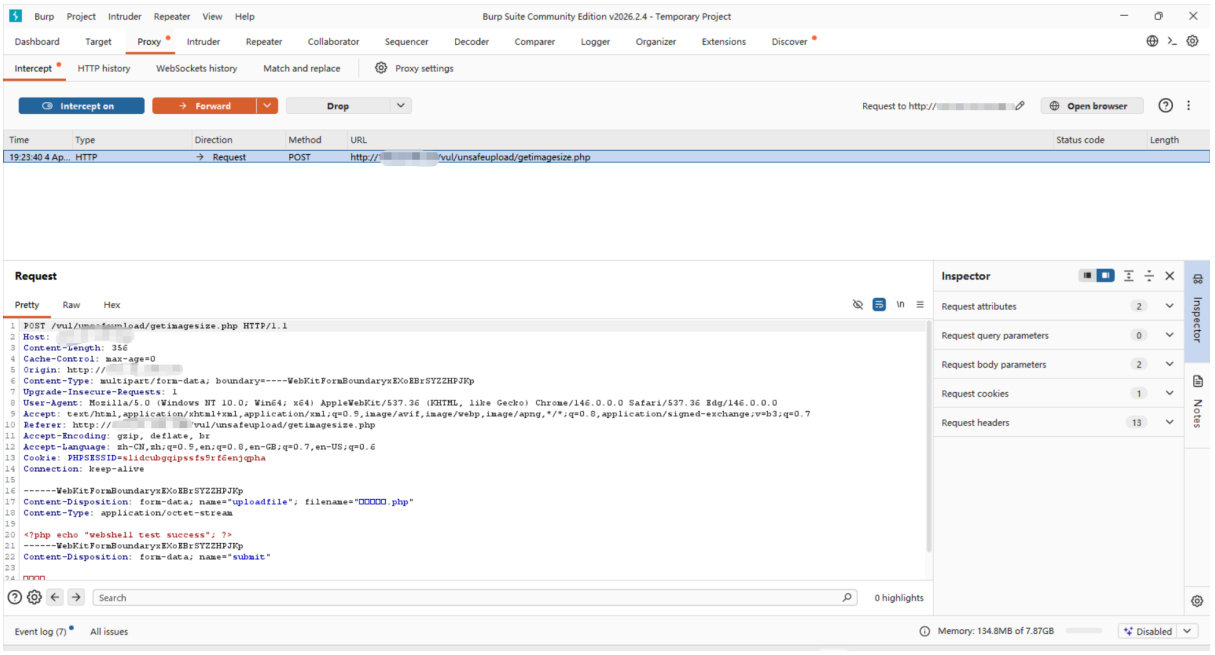
到这里，本题就正式划上了句号。让我们向下一题出发吧！

3、getimagesize

来到这一题，肯定又有什么不同。不过我们暂时还是不得而知。

但是有了前两次测试漏洞的经验，这里肯定还是要先从抓包中寻找线索，还是一样，先上传一个文件看看吧。不如直接上传上次的那个一句话木马，先不用管这里有没有前端检查：

开始上传，准备抓包：



很好，看起来这里没有前端检查（毕竟前端检查并没有任何实际的作用），而是直接发送请求给后端。我们这里就先研究一下这里抓包的数据吧：

```
------WebKitFormBoundaryxEXoEBrSYZZHPJKp
Content-Disposition: form-data; name="uploadfile"; filename
="一句话木马.php"
Content-Type: application/octet-stream

<?php echo "webshell test success"; ?>
------WebKitFormBoundaryxEXoEBrSYZZHPJKp
Content-Disposition: form-data; name="submit"

开始上传
------WebKitFormBoundaryxEXoEBrSYZZHPJKp--
```

呃，看起来好像非常正常，从请求里面我们看不到任何检查标记，就是只有文件名、文件内容、文件信息这些数据。那么，后端又是如何检查文件类型的呢？

前端传入了什么请求，后端只能检查什么！所以这里的问题实际上就是：后端通过检查这个请求的什么内容，能够分辨文件类型？

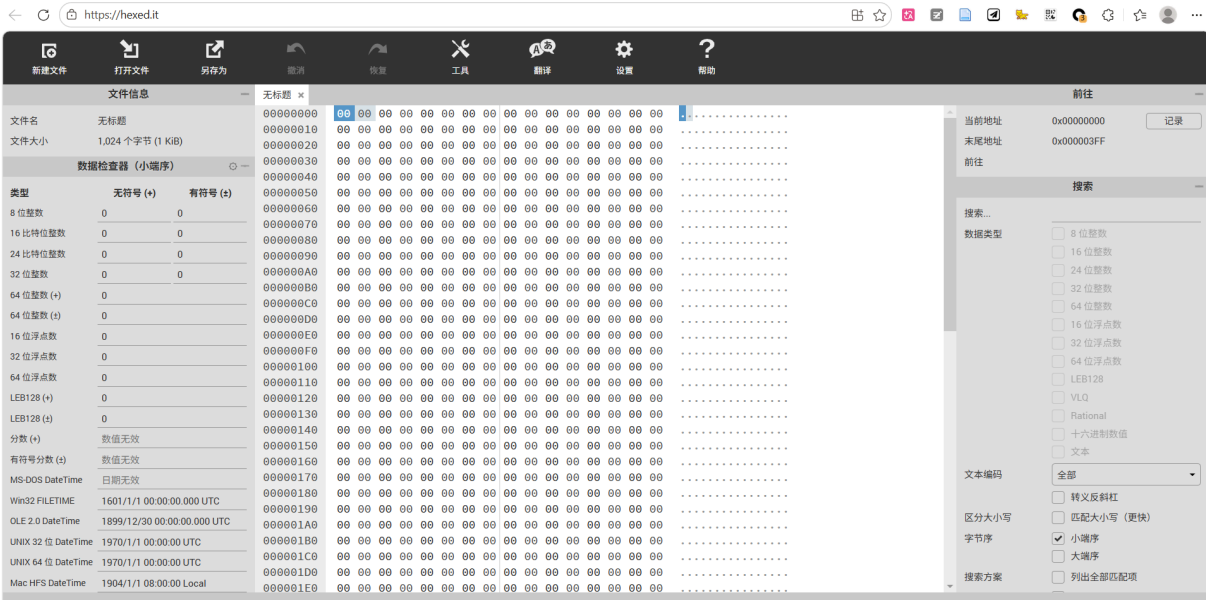
光是到这里，其实答案并不固定，我们可以有三个答案：检查文件名、检查文件内容、以及都检查——实际上，而只有这两个对应的数据才与文件类型相关！

那么，就先试试修改文件名吧，看把php改成png，看看能不能绕过后端检测：

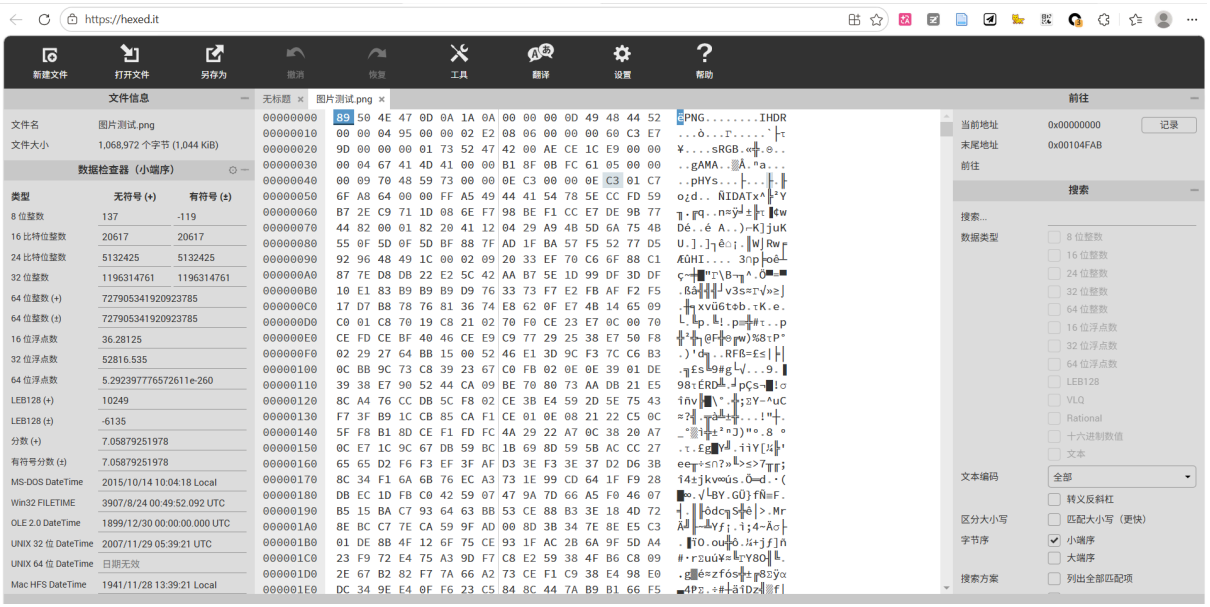
呃，开发者显然想到了这一层。那么答案只剩下一个了：后端检查的还包括文件内容！

但是这里我们要有一个常识——如果后端要去读完整的图片内容，那是绝对不可能的；实际上，后端所检查的一定是一些共性的东西，而且这个共性的标签一定不会太长！

那么，为了鉴别图片，就需要先知道图片究竟对应着什么样的数据。这里我们随便找一张图片，通过网站HexEd.it : 免费的基于JavaScript的十六进制编辑器，就可以看到图片的数据了：



上传我们的图片：



看到了一堆奇奇怪怪的字符和一堆乱码——不要见怪，图片本身就是通过16进制编码的，所以这里后端如果要检查，那么实际上检查的就是十六进制数据。

这里有一个很明显的线索，右侧有一个PNG标签，而且它是和左边的十六进制字符对应着的！把这串十六进制编码列出来：

```
89 50 4E 47 0D 0A 1A 0A
```

这就是PNG文件的固定文件头！

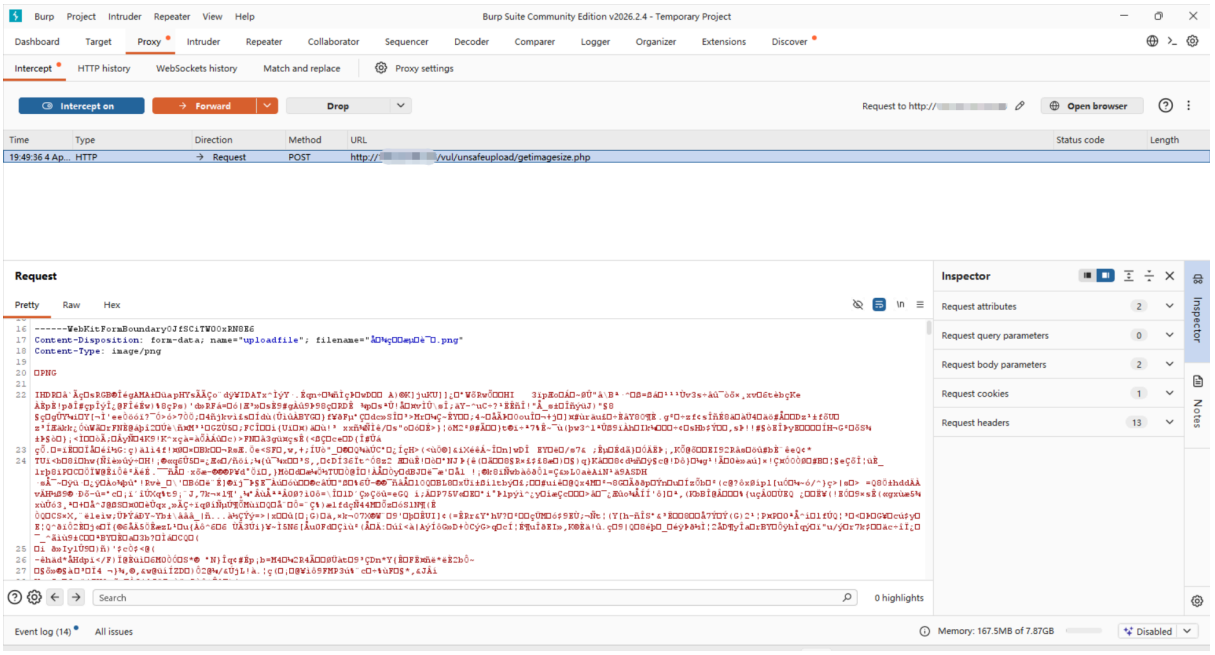
那么，这是否就说明，在文件头有这样一个特定的编码，用来标注文件的类型？答案是肯定的。

因此，我们也理所应当地想到，后端所检查的标记，一定是这个文件头的数据！也就是所谓“读文件头“的检查方法！

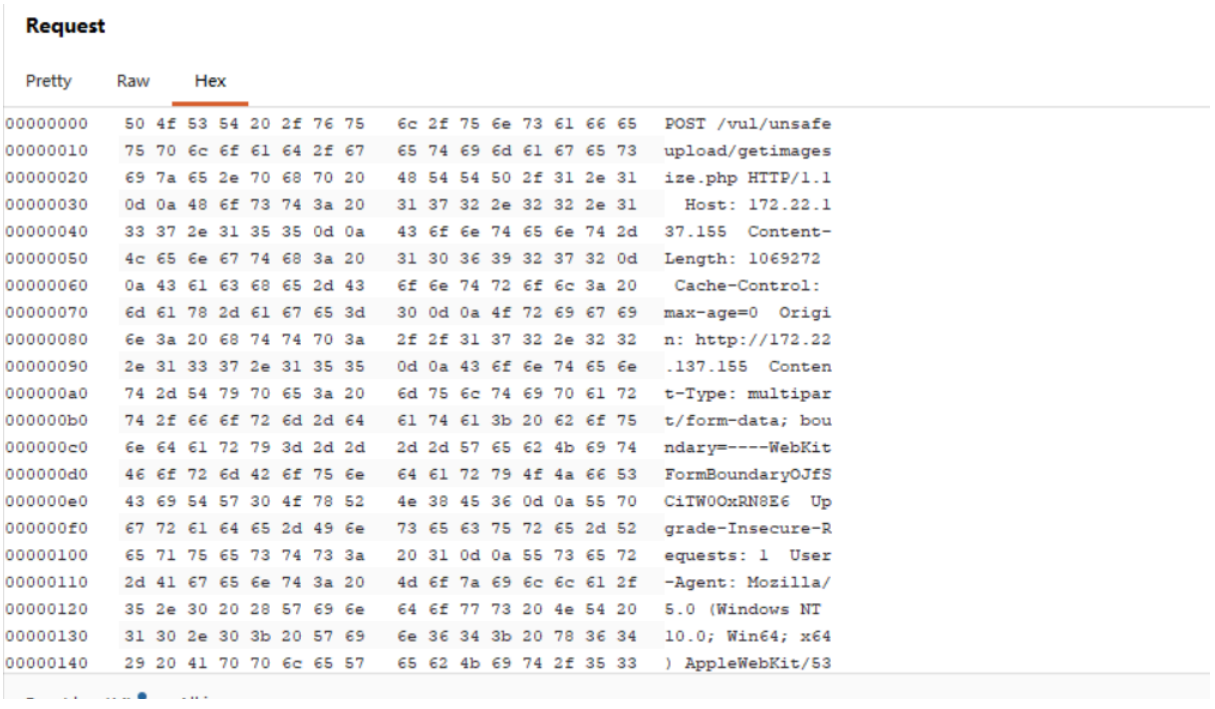
那么，我们应该通过什么方式绕过后端的检查逻辑呢——那就是留出文件头，在文件尾部构造木马！

我们可以在本地直接修改文件（改后缀为txt，用记事本打开后修改即可），也有更简单的方法，那就是上传一个正常的图片，然后在抓包的请求中修改。

那就试试看咯！



抓包的请求中很明显，这个文件内容的开头有一个PNG标签，就像我们刚才通过网站看到的那样。这里我们可以切换到Hex标签页，就能看到十六进制编码和解码之后的对应关系了：



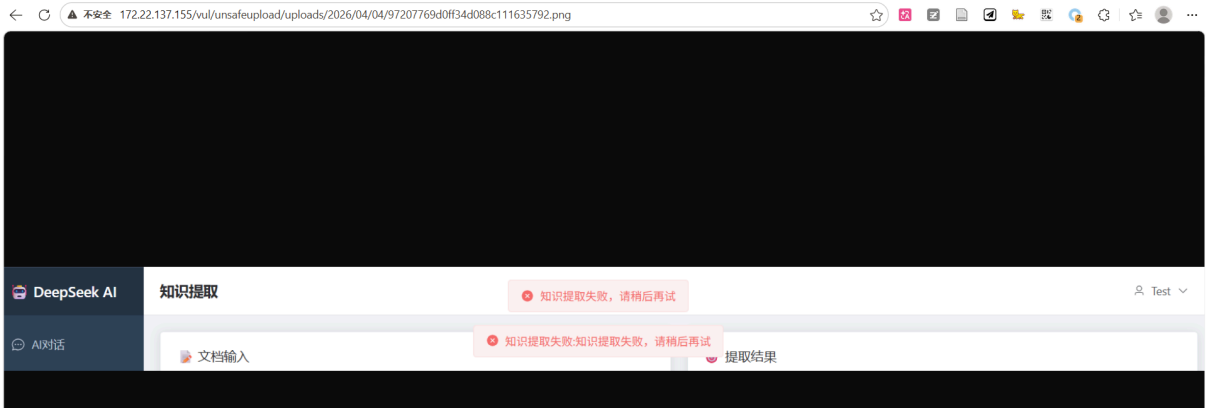
这里直接来到文件尾部，在末尾加上我们构造的一句话木马 `<?php echo "webshell test success"; ?>` ：

放包！这次应该能通过啦！

呃，还是失败了啊，原来还有文件大小检查！

不过方法是很明确了。这次我们按照同样的方法，上传一个体积很小的图片并改包：

很好，这次就上传成功了！那么还是先按照我们上一题的方法，访问路径 `/vul/unsafeupload/uploads/2026/04/04/97207769d0ff34d088c111635792.png` 来触发我们上传的木马——由于这次木马是放在图片中的，因此就叫“图片马”。



咦？怎么显示的是我们上传的图片？

别忘了，刚才我们上传的文件名后缀还是png（后端有对这个后缀的检查），因此它被当作图片存储，访问时直接输出图片数据（理论上来说，如果图片大小足够小，或者格式特殊，像GIF这种，那么我们刚才的构造也是可行的，但是那种方法太没有确定性了）！因此，这里后缀还是png的话，通过访问url的方式是没办法执行木马的！

那么该怎么做？我们需要配合文件包含漏洞才能执行它！

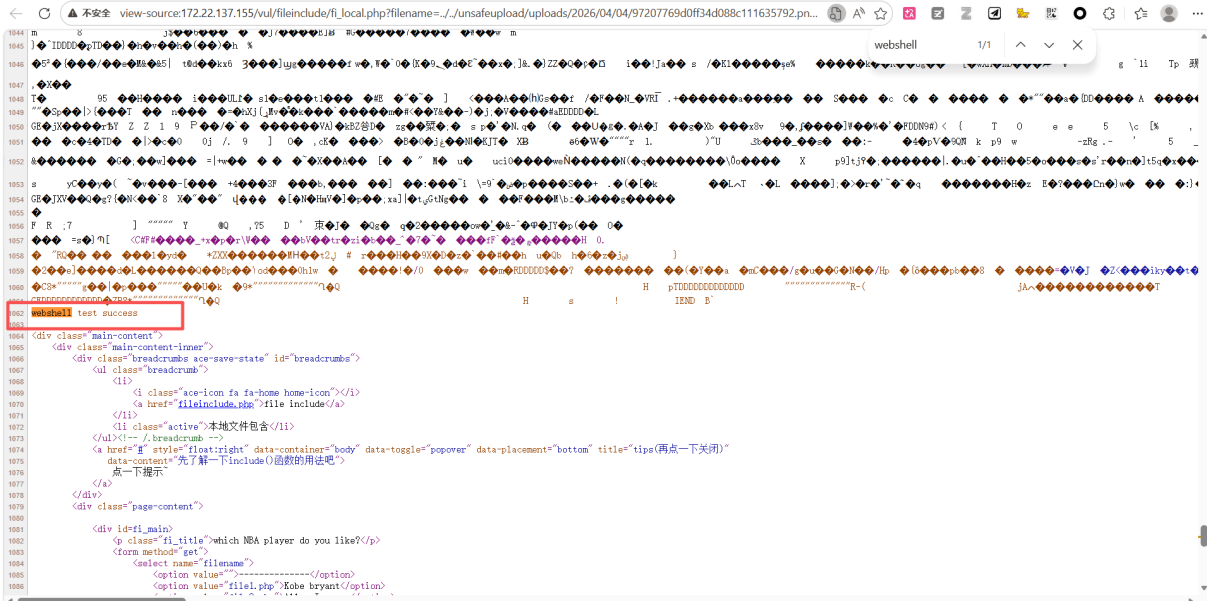
哪里有文件包含漏洞呢？我们前面刚做过，那么就按照第七节的第一题的思路，在网址栏构造这样的路径（我们当时是通过改包的方式攻击的，但是这里其实是一个GET

类型的传参方式，所以在网址栏构造是一样的），就能够把我们在这里上传的文件“包含”起来从而执行！

```
http://XXX.XXX.XXX.XXX/vul/fileinclude/fi_local.php?filename=../../unsafeupload/uploads/2026/04/04/97207769d0ff34d088c111635792.png&submit=%E6%8F%90%E4%BA%A4
```

前端显示了乱码，这其实就说明我们上传的图片码被顺利执行了！（文件尾部看不到我们构造的一句话木马，就是被执行的标志）

这里，由于我们的一句话木马相当无害，而且输出被一并包含在乱码中，所以读者看不出来这里有什么效果是很正常的。要看到我们构造的木马，可以查看页面源代码：



看到了！通过搜索功能，我们找到了我们之前埋在图片中的木马！它已经被顺利执行，返回了 `webshell test success` 的结果。

如果读者想要看一点刺激的反应，可以构造一些其他的木马，比如`<?php phpinfo(); ?>`可以访问php信息页面，`<?php @eval($_POST['cmd']); ?>`获取shell等等，这里就不再演示了。

最后，咱们还是看看源代码：

```
$type = array('jpg','jpeg','png');  // 只允许这些后缀
$mime = array('image/jpg','image/jpeg','image/png');  // 只允许这些MIME
$upload = upload('uploadfile','512000',$type,$mime,$save_path);
……
getimagesize($_FILES[$key]['tmp_name'])
```

嗯，看起来这里除了检查文件后缀名和MIME，还有一个`getimagesize()`函数，这个函数就起着读文件头的功能——当你传入一个临时文件路径，它会首先读取文件开头的若干字节（通常是几十到几百字节），然后尝试解析为图片格式（JPEG/PNG/GIF/BMP等），最后根据文件头中的魔术数字（Magic Number）判断类型。

那么，真相大白！

到这里，我们就正式通关了皮卡丘靶场中关于文件上传漏洞的部分。当然在真实的场景中，网站开发者还会利用更多的防御机制，但是万变不离其宗，黑客们所做的也不过就是通过特殊的构造欺骗后端检查罢了——理解了原理，一切都是浮云。

而作为网站开发者，则需要明白，“任何来自客户端的数据都是不可信的。”就像我们从小接受的教育一样，防人之心不可无——在纷繁复杂的公网世界，尤为如此。

十、Over Permission

还是老样子，概述开篇：

> 如果使用A用户的权限去操作B用户的数据，A的权限小于B的权限，如果能够成功操作，则称之为越权操作。 越权漏洞形成的原因是后台使用了 不合理的权限校验规则导致的。
>
> 一般越权漏洞容易出现在权限页面（需要登录的页面）增、删、改、查的的地方，当用户对权限页面内的信息进行这些操作时，后台需要对当前用户的权限进行校

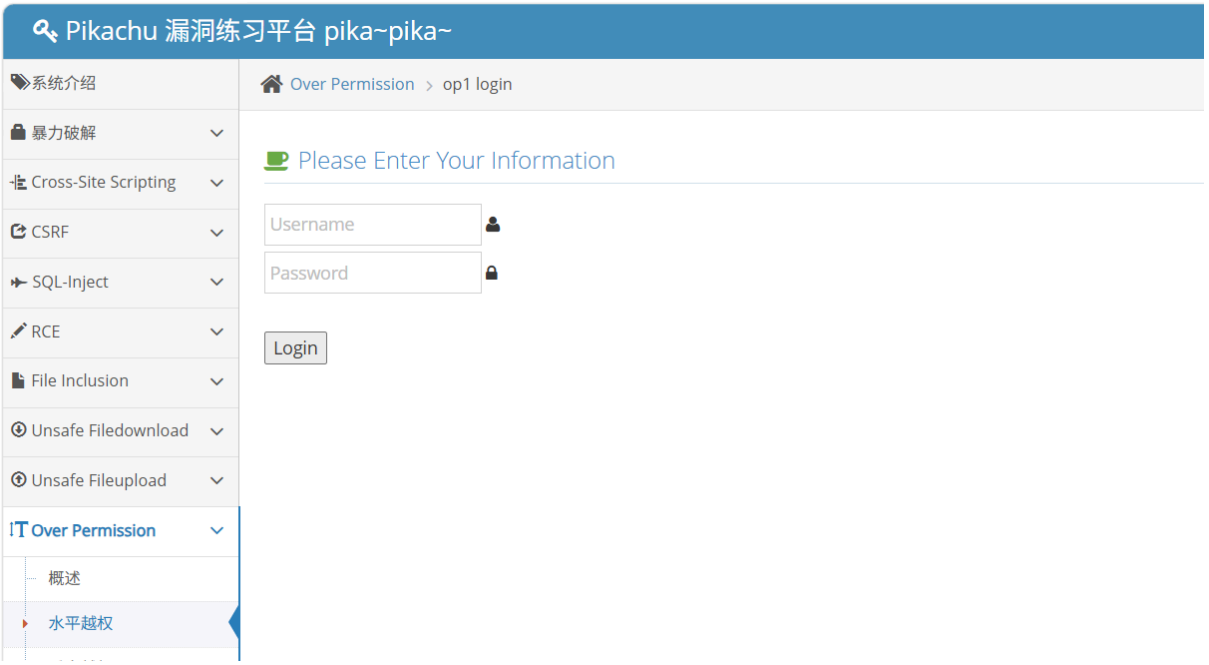
> 验，看其是否具备操作的权限，从而给出响应，而如果校验的规则过于简单则容易出现越权漏洞。
>
> 因此，在在权限管理中应该遵守：
>
> 1.使用最小权限原则对用户进行赋权;
>
> 2.使用合理（严格）的权限校验规则;
>
> 3.使用后台登录态作为条件进行权限判断,别动不动就瞎用前端传进来的条件;
>
> 你可以通过“Over permission”对应的测试栏目，来进一步的了解该漏洞。

简单来说，所谓“越权漏洞”，就是用户通过某种方法获取了不属于自己的权限，而越权又分为两种，水平越权和垂直越权——水平越权就是获取了和自己同一层次的其他用户权限，比如A、B都注册了一个普通用户账号，拥有各自账号的管理权限，但是A却能够通过水平越权攻击获取B的账号权限；而垂直越权则是获取了更高层次的账号权限，比如普通用户通过垂直越权攻击可以获取管理员权限。

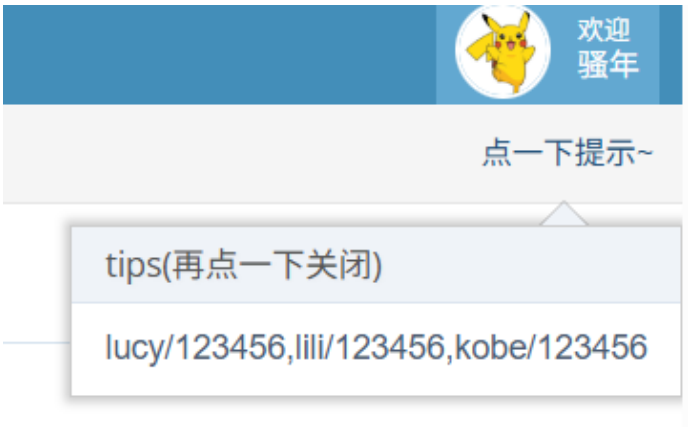
这样单纯介绍还是太抽象了，虽然意思不难理解，但总是少了一点什么——那就是实践成功的喜悦。事不宜迟，让我们开始爆破靶场吧！我们的目标就是以自己的账号作跳板，去获取其他账号的权限（其实也就是操作其他账号的控制面板）！

1、水平越权

又是熟悉的登录界面：



这里当然不可能是让我们猜密码，看看提示，应该有登录账号的：

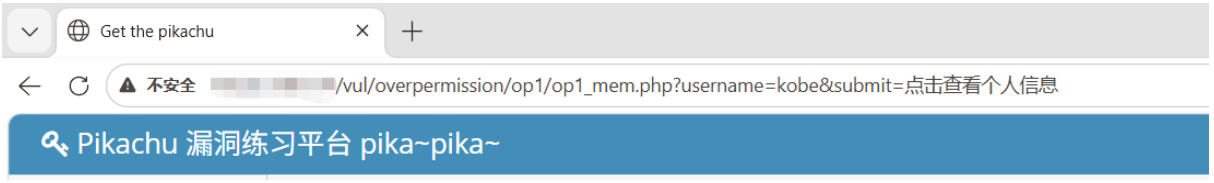


欧克！账号到手，先登录kobe的帐号吧！（快说，谢谢牢大！）

很好，登录之后的页面很简单，只有一个查询自己个人信息的功能。而此处的水平越权要实现的目标，就是要去“点击查看别人的个人信息”！

到这里，似乎没有什么头绪，毕竟看起来这个页面上并没有能够操作的空间——等下，不要忘了。url同样是我们操作的对象，而我们之前利用的各种Get类型的参数传递漏洞，本质上就是构造这个网址栏的url！

所以，关注一下这里的url吧！



咦？看起来好像就只是在原来的网址后面拼接了一个参数查询的后缀`username=kobe&submit=点击查看个人信息`，那么，如果我们去修改这个后缀，会发生什么呢？

这里值得关注的只有一个参数，`username=kobe`，也就是查询我们自己的账号用户名。那么，我们就很自然地想到，可以把这里的参数修改为其他用户名！

当然，如果在真实场景下，我们是不知道其他账号的用户名是什么的，所以只能去利用我们最开始用到的暴力破解的方法，构造一个用户名字典拼接到这里的username参数，反复地去访问对应的url，看看能不能返回结果。不过这里，我们是已经知道其他账号的用户名的————感谢开发者，这里把username修改为lili试试看咯！

成功了！我们原本处于kobe的个人信息页面，现在构造url访问之后，竟然直接来到了lili用户的控制页面，同时也能点击查看她的个人信息。注意，我们这里并没有去猜解lili用户的用户名和密码，而是通过自己的账号直接来到了她的控制页面，直接获取了她的用户权限——正常情况下，这个权限只能通过账号认证的方式才能取得，而我们这里做的，就是通过一个越权漏洞绕过了这套认证机制！

这就是水平越权——由于同一层次的用户的代码结构大同小异，往往只是一个小小的参数变化控制着各个账号的权限，所以一旦你发现了一些参数传递的端倪，就可以尝试一下参数的修改咯！

现在，我们还是按照光荣传统分析一下源代码咯！

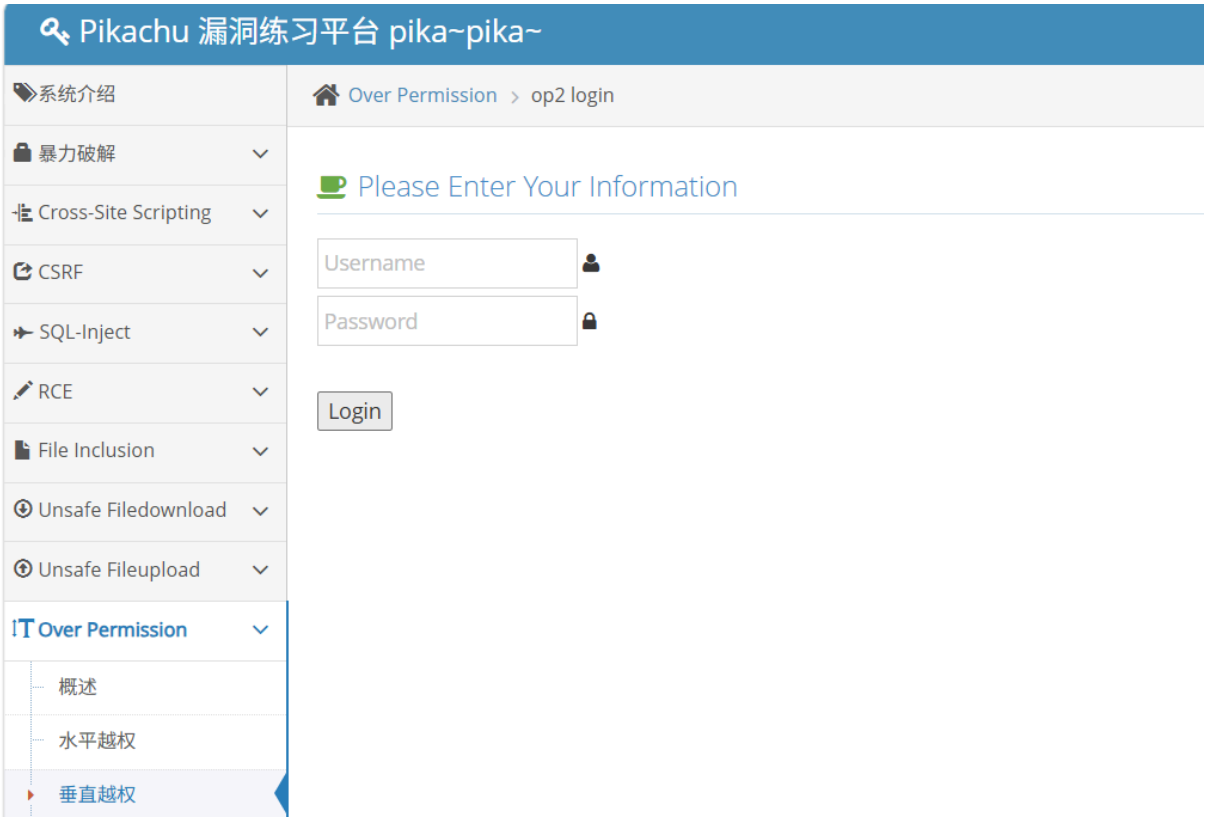
```
$link=connect();
// 判断是否登录，没有登录不能访问
if(!check_op_login($link)){
    header("location:op1_login.php");
}
$html='';
if(isset($_GET['submit']) && $_GET['username']!=null){
    //没有使用session来校验,而是使用的传进来的值，权限校验出现问题，
这里应该跟登录态关系进行绑定
    $username=escape($link, $_GET['username']);
    $query="select * from member where username='$usernam
e'";
    $result=execute($link, $query);
```

这里的代码逻辑还是很清晰的：先判断是否登录，如果处于登录状态（不检查登录状态的实际账号），那么就通过所有的查询——也就是说，你可以在自己的登录账号下通过特定构造去传递查询其他账号的参数，从而获取其他用户的权限！这就是水平越权漏洞的实质：只问“你在”，不问“你是”。

那么，现在去看看更有趣的垂直越权吧！

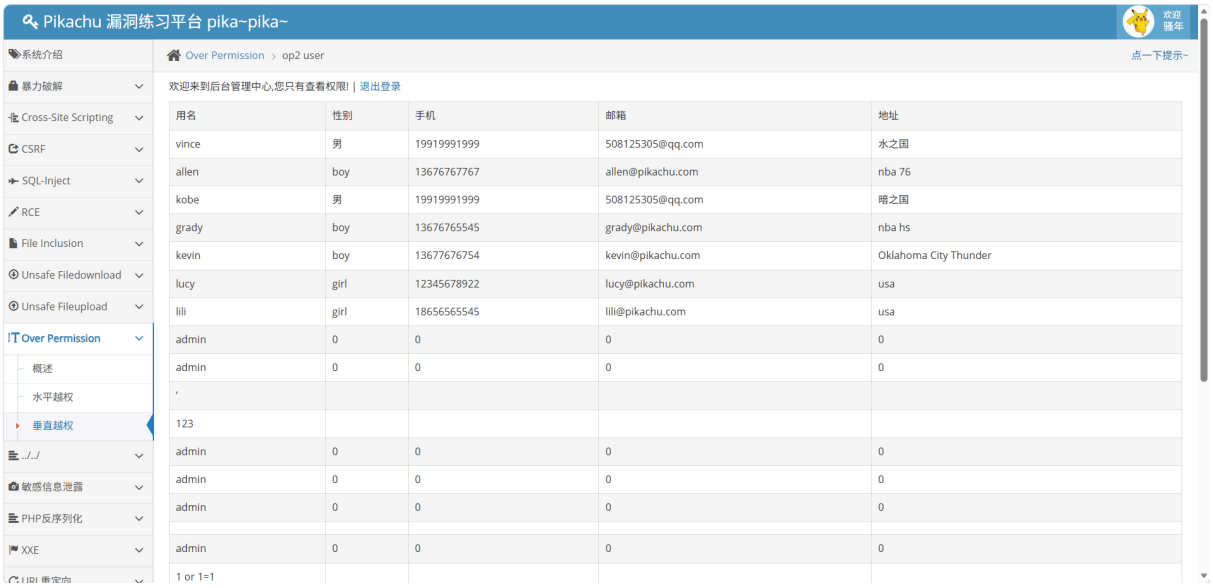
2、垂直越权

如果说上一题是同级用户之前的权限获取，各个用户的权限是一样的，那这里就是低权限账号获取高权限账号的权限——这样说比较拗口，实际上就是普通账号获取了管理员账号的权限，获取了更高的权限。难道这里有什么区别吗？不妨一起来看看怎么操作吧！

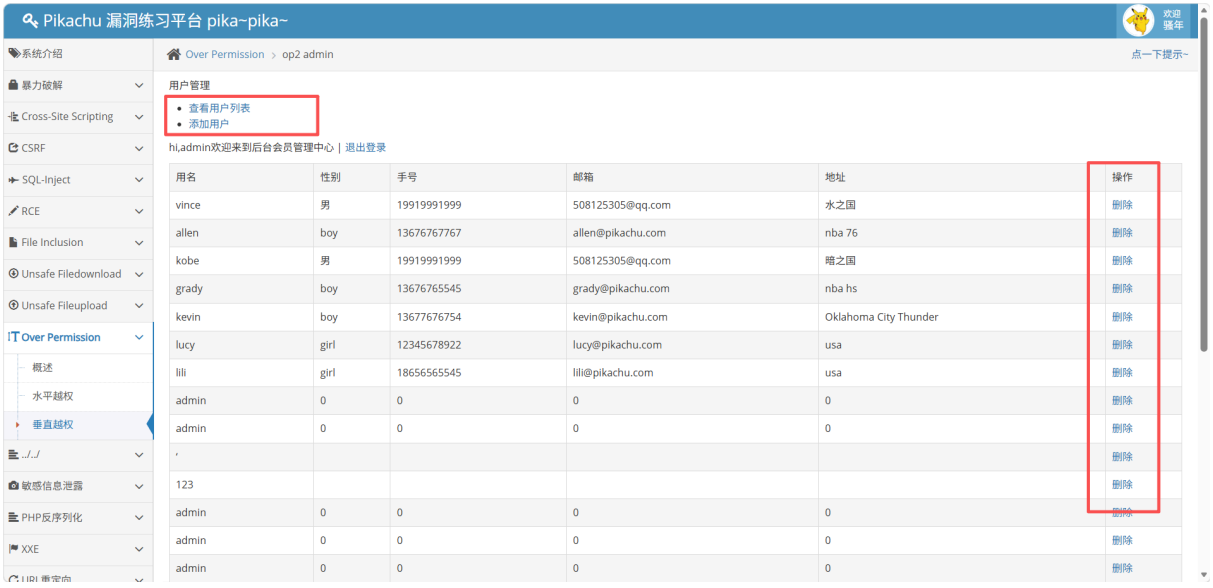


还是一样的登录页面，看看提示有没有提供账号咯！

有的，而且一个是普通账号，一个是boss账号，这两者有何区别呢？咱们先不着急挖漏洞，先尝试着看看。首先登录普通账号：



这里看起来是一个所有用户的账号数据面板，而且写的很清楚，这个普通账号只有查看权限，无法对这些账号数据进行编辑。那么，boss账号多出来的权限是什么呢？



这次进入了admin账号的控制台，发现多了很多“按钮”——我们不仅可以查看用户列表，还可以添加用户、删除用户。看来，这就是boss多出来的权限了！

因此，我们要做的事情就很清楚了，我们要先登录普通账号的后台页面，然后利用垂直越权漏洞跳转到admin账号的控制台，也就是获取了该账号的更高权限！

怎么做呢？有了上一题的经验，我们理所应当地想到，这里应该同样可能是构造url，不管怎么说，先观察一下url的区别吧！

首先是admin管理员账号控制台页面的url：

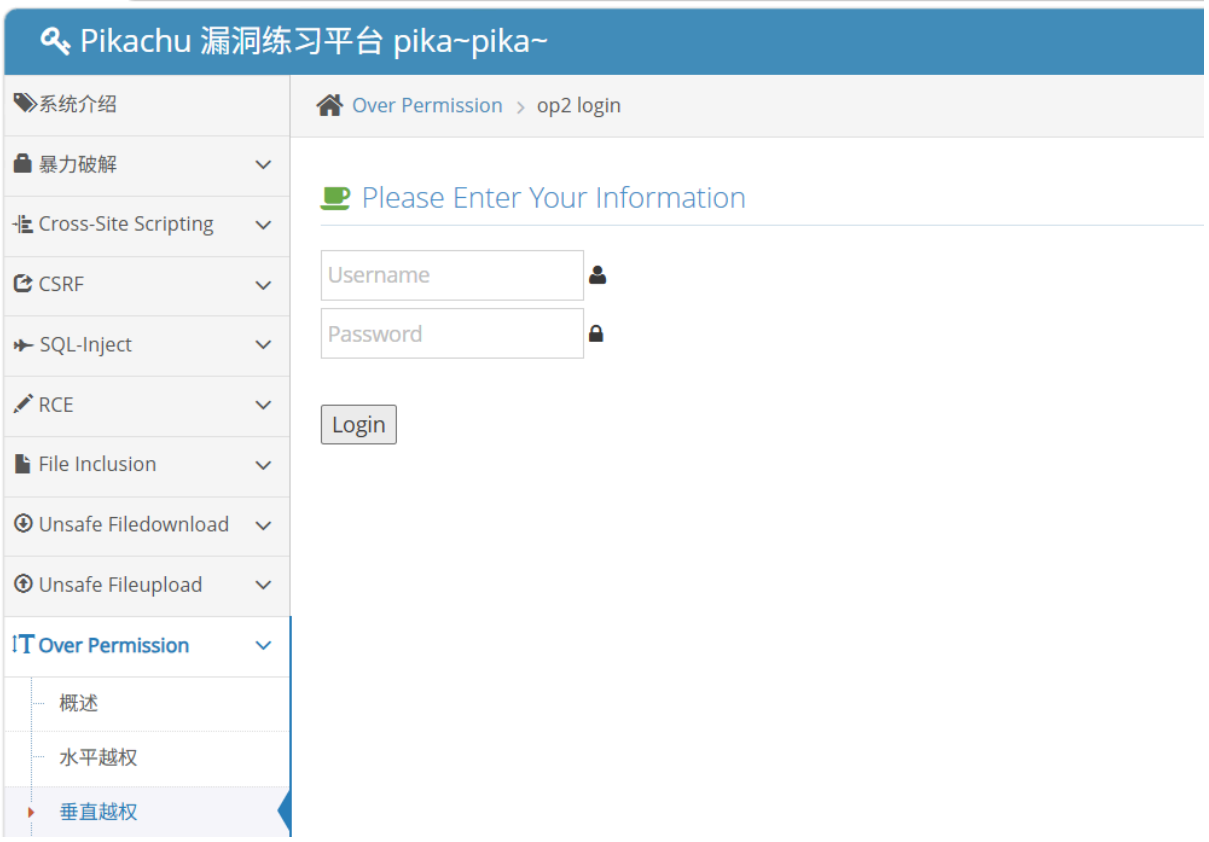
```
http://XXX.XXX.XXX.XXX/vul/overpermission/op2/op2_admin.php
```

接下来是pikachu普通账号控制台页面的url：

```
http://XXX.XXX.XXX.XXX/vul/overpermission/op2/op2_user.php
```

区别很明显，还是在于后缀的拼接，指向了不同的目录文件——一个是`op2_admin.php`，一个是`op2_user.php`。那么，我们就改改看咯！

首先登录pikachu普通账号，进入控制台之后，将网址栏处的`op2_user.php`改成`op2_admin.php`，回车：



咦？怎么跳转回到了登录页面？看来这里一定存在着某种检查机制，大概是发现我们本身是普通账号，所以自动重定向到了登录页面吧，这里显然和上一题不太一样。

那么，只能去想想其他办法了。如果这里在控制台页面添加了一个用户检查的话，那么不妨试试另辟蹊径，有没有一些关于admin账号的其他的url，同样是可以访问的呢？

别忘了，刚才我们登录admin账号时，我们是有权限添加和删除用户的！这里的传参方式也是Get类型的话，就可以从这里入手，看看能不能产生垂直越权攻击！

也就是说，我们第一步就是登录admin账号，看看这些操作对应的url是什么。首先试试添加用户：

发现跳转到了一个用户信息填写的页面，而且更重要的是，这里url变了：

```
http://XXX.XXX.XXX.XXX/vul/overpermission/op2/op2_admin_edit.php
```

那么，我们是否可以尝试一下，在pikachu普通账号的控制台页面去访问这个url呢？

这里暂且按下，我们再试试删除账号的功能。点击之后，界面发生了刷新，url也发生了变化，末尾拼接了一个查询逻辑：

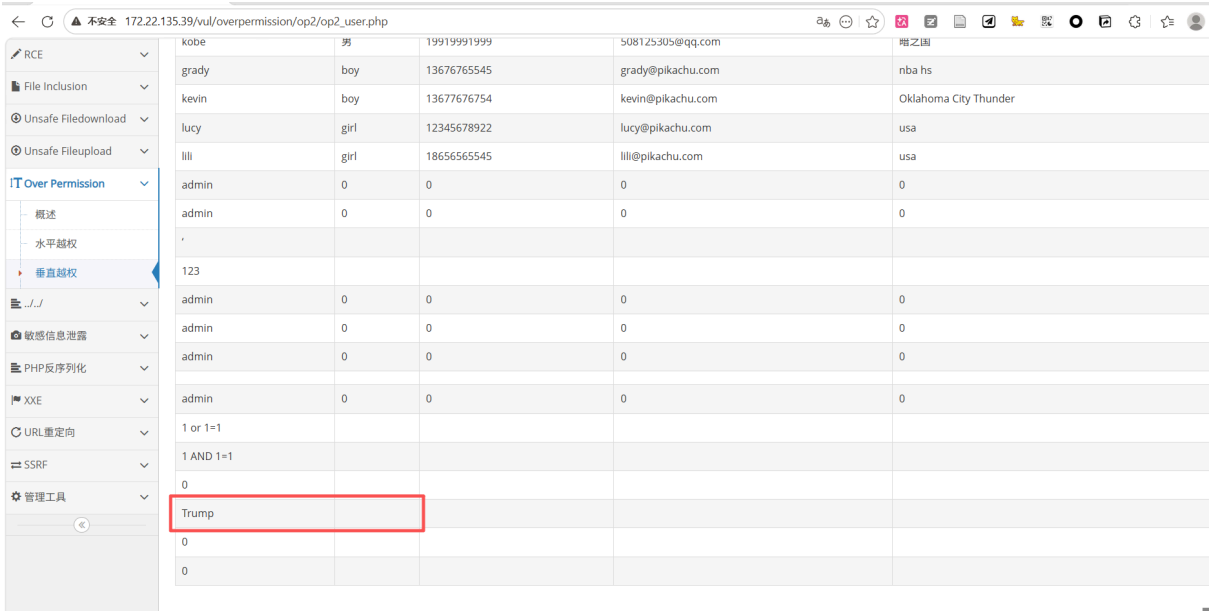
```
http://XXX.XXX.XXX.XXX/vul/overpermission/op2/op2_admin.php?id=39
```

看来，这里是把id这个参数传递到后端的一个MySQL语句中，实现了删除操作。这里同样是Get类型的传参方式，当然也是可以加以利用的。

现在，我们多了两种url的构造可能，那么就回到pikachu普通账号的控制台，试试看这两种构造吧！

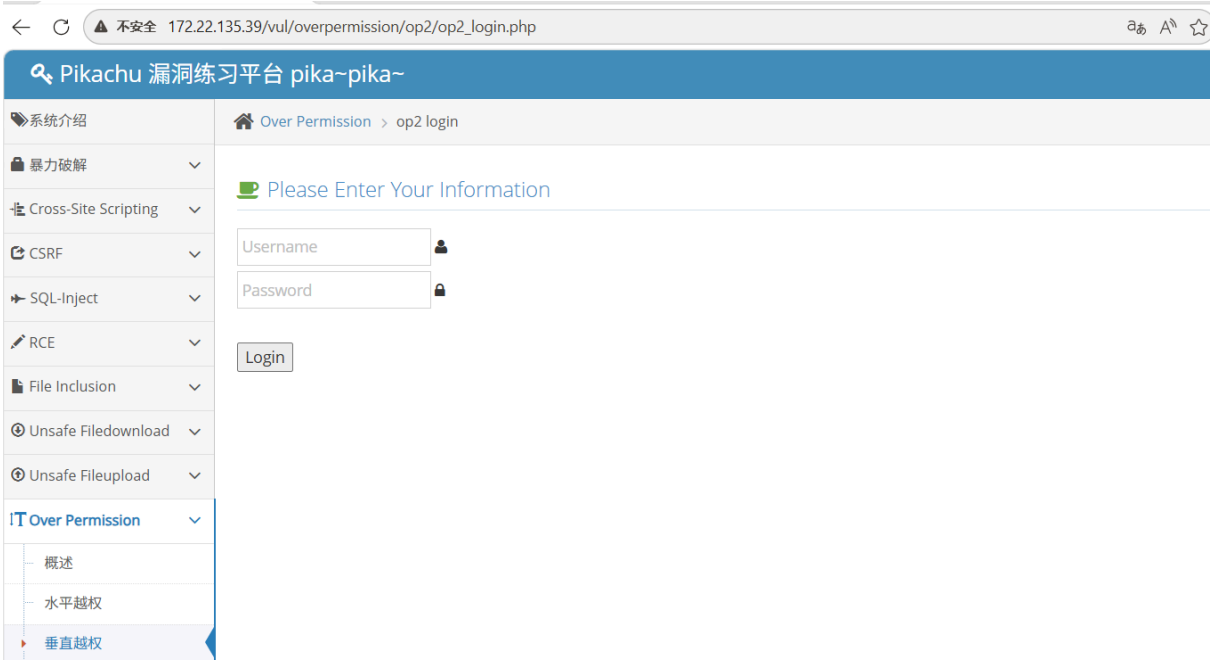
首先，试试看能不能添加账号，直接把刚才用户信息填写页面的url复制粘贴过来：

页面成功发生了跳转！十有八九成了！不过还需要检查一下我们是不是真的拿到了创建账号的权限，输入用户信息看看。我这里创建了一个Trump的帐号，创建完成后再次访问控制台，看看账号列表有没有变化：



看到了，Trump出现了。看来我们刚才的确成功获取了admin账号的创建账号的权限！

现在，我们试试看能不能获取删除账号的权限，在网址栏输入另一串url，id我们这里不是很清楚，先随便输入一个看看：



嘶~直接回到了登录页面，和我们最初的构造结果一样，看来这次确实是失败了。读者也可以多试试，这里确实是无法删除任何账号的。

但是问题不大，我们毕竟已经拿到了boss创建账号的权限了，这已经是很大的胜利了，不是吗？这就叫做垂直越权，还是通过构造url，从而获取了更高的、本不属于自己的一些权限。

最后，还是看看源代码咯！这里的关键是 `op2_admin.php` 和 `op2_admin_edit.php` 源文件——虽然我们的操作是在普通账号的控制台页面中进行的，但是实际上依赖于我们构造的url所对应的目录文件的一些检查机制：

```
$link=connect();
// 判断是否登录，没有登录不能访问
//如果没登录，或者level不等于1，都就干掉
if(!check_op2_login($link) || $_SESSION['op2']['level']!=1)
{
    header("location:op2_login.php");
    exit();
}

//删除
if(isset($_GET['id'])){
    $id=escape($link, $_GET['id']);//转义
    $query="delete from member where id={$id}";
```

```
    execute($link, $query);
}


if(isset($_GET['logout']) && $_GET['logout'] == 1){
    session_unset();
    session_destroy();
    setcookie(session_name(),'',time()-3600,'/');
    header("location:op2_login.php");

}
```

显然，这里的检查机制比上一题多了一道，也就是开发者留下的注释所说的“level不等于1”——这就是权限检查标记。如果level不等于1，那么即便处于登录状态，也不会对操作进行授权，反而直接重定向到登录页面。这道检查机制不仅存在于进入控制台中，也存在于删除操作中——因此，这也就解释了我们最初试图进入admin账号的控制台，以及后来试图获得删除权限时，为什么会直接回到登录页面了。

再看看 `op2_admin_edit.php` 的源代码：

```
$link=connect();
// 判断是否登录，没有登录不能访问
//这里只是验证了登录状态，并没有验证级别，所以存在越权问题。
if(!check_op2_login($link)){
    header("location:op2_login.php");
    exit();
}
if(isset($_POST['submit'])){
    if($_POST['username']!=null && $_POST['password']!=nul
l){//用户名密码必填
        $getdata=escape($link, $_POST);//转义
        $query="insert into member(username,pw,sex,phonenu
m,email,address) values('{$getdata['username']}',md5('{$get
data['password']}'),'{$getdata['sex']}','{$getdata['phonenu
m']}','{$getdata['email']}','{$getdata['address']}')";
        $result=execute($link, $query);
        if(mysqli_affected_rows($link)==1){//判断是否插入
            header("location:op2_admin.php");
        }else {
```

```
                $html.="<p>修改失败,请检查下数据库是不是还是活着的</p
>";

            }
        }
}
```

这里的判断标准很清楚，此处没有再加上那一条"level不等于1"的检查规则，而是只有是否登录的检查——因此，只要我们处于登录状态，就可以直接构造url来获得创建账号的权限！

也就是说，这道题存在的漏洞在于，后端没在每个功能点重复检查"权限等级"！因此，我们就利用了某些功能点上的检查环节的缺失，成功获取了对应的权限！

所以，概括一下，低权限用户通过直接访问高权限功能页面的URL，绕过了前端的权限控制按钮，如果后端没有在每个功能点重复校验权限等级，就会产生垂直越权。

回顾一下这两道"越权"的题目，水平越权偷同级数据，垂直越权拿高权限功能。越权漏洞，说到底是后端的一次"偷懒"——以为前端藏了按钮用户就点不到，以为检查了登录状态就万事大吉。结果呢？换一个URL，别人的数据、老板的权限，就这样轻轻松松到手了。

安全没有"想当然"，每一个参数都值得被校验，每一个接口都值得被守护。

十一、目录遍历

老规矩，看概述：

> **目录遍历漏洞概述**
>
> 在web功能设计中,很多时候我们会要将需要访问的文件定义成变量，从而让前端的功能变的更加灵活。 当用户发起一个前端的请求时，便会将请求的这个文件的值(比如文件名称)传递到后台，后台再执行其对应的文件。 在这个过程中，如果后台没有对前端传进来的值进行严格的安全考虑，则攻击者可能会通过"../"这样的手段让后台打开或者执行一些其他的文件。 从而导致后台服务器上其他目录的文件结果被遍历出来，形成目录遍历漏洞。
>
> 看到这里,你可能会觉得目录遍历漏洞和不安全的文件下载，甚至文件包含漏洞有差不多的意思，是的，目录遍历漏洞形成的最主要的原因跟这两者一样，都是在功能设计中将要操作的文件使用变量的 方式传递给了后台，而又没有进行严格的安全考

> 虑而造成的，只是出现的位置所展现的现象不一样，因此，这里还是单独拿出来定义一下。
>
> 需要区分一下的是,如果你通过不带参数的url（比如：http://xxxx/doc）列出了doc文件夹里面所有的文件，这种情况，我们成为敏感信息泄露。 而并不归为目录遍历漏洞。（关于敏感信息泄露你你可以在"i can see you ABC"中了解更多）
>
> 你可以通过“../../”对应的测试栏目，来进一步的了解该漏洞。

简单来说，目录遍历，就是用 `../../` 这样的“后退一步”符号，跳出Web程序规定的文件夹，去读服务器上其他位置的文件。

比如，正常情况你应该只能看 `/var/www/images/1.jpg`，但你输入 `../../etc/passwd`，结果服务器真的把密码文件给你了——这就是目录遍历漏洞。

这确实和我们之前做过的文件包含、文件下载漏洞很像，都是跳到其他目录去。不过它跟文件包含、不安全文件下载是什么关系？

皮卡丘靶场的开发者说得好：成因相同，场景不同。可以参考一下下面的表格：

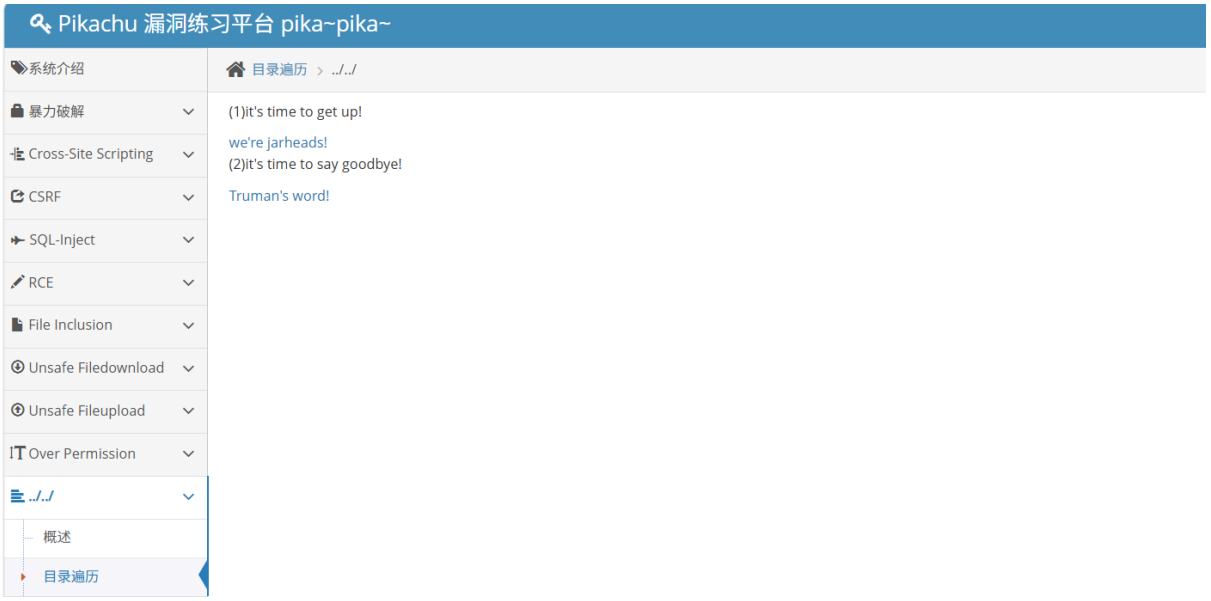
漏洞类型	核心操作	后果
文件包含	`include($_GET['file'])`	执行代码
不安全下载	`readfile($_GET['file'])`	下载文件
目录遍历	`readfile($_GET['file'])` 或 `file_get_contents()`	读取文件内容并显示

三者的核心毛病都是：直接把用户输入拼到文件路径里，没过滤 `../`。而目录遍历看起来更“温柔”一点，只是读取一下目录下的文件罢了。

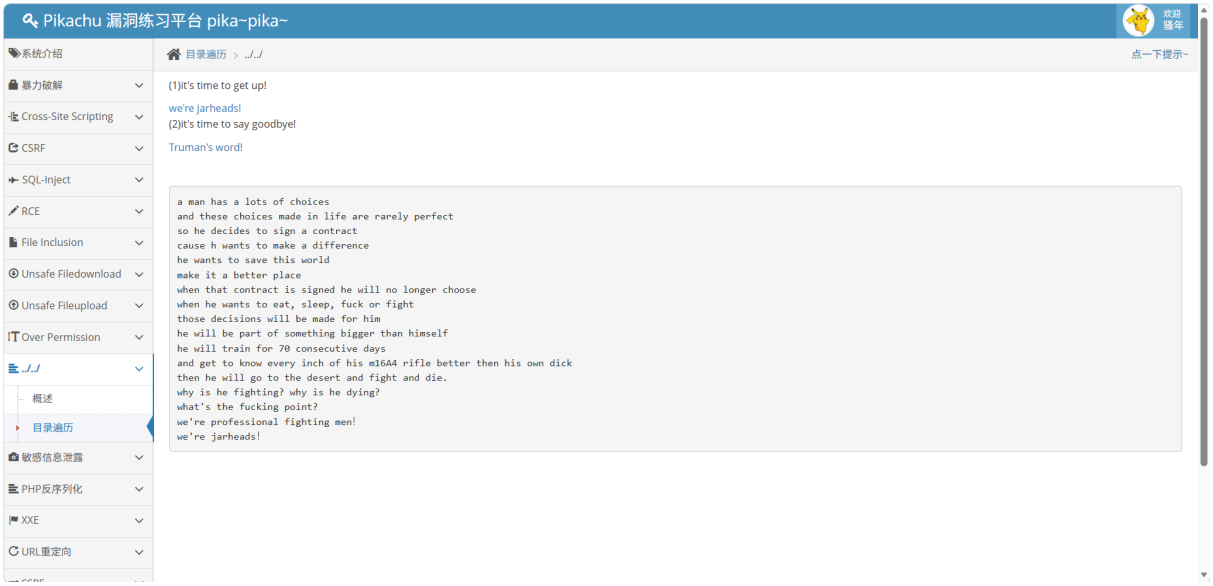
现在，就去做做题吧！

1、../../

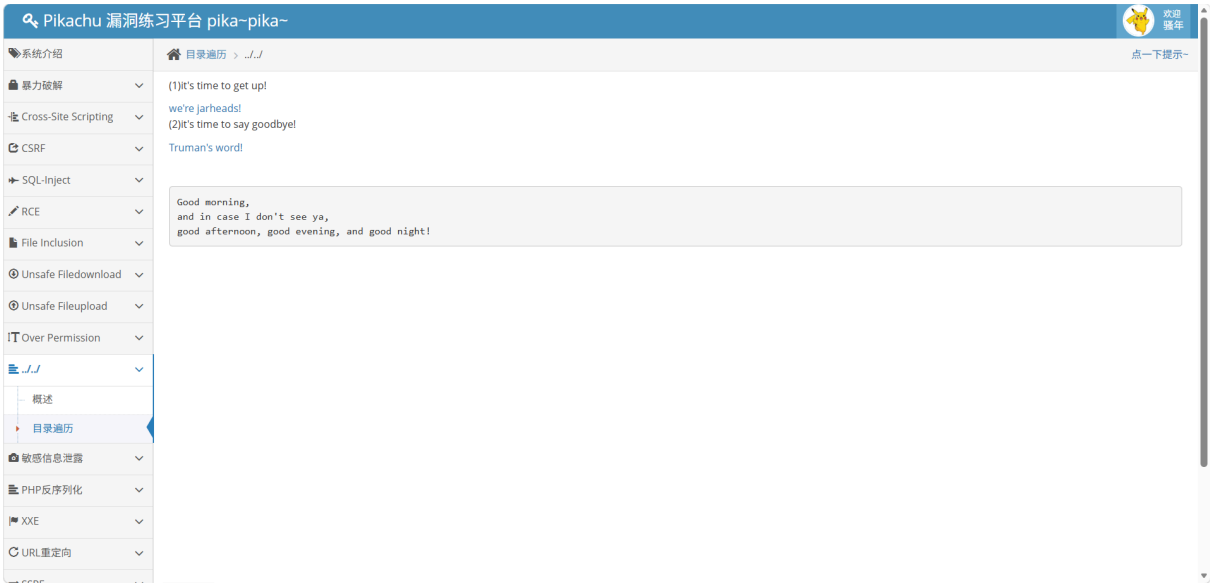
看看题目，很特别：



这个蓝色的东西我们早就见过了，这是可以点的“<a>标签”。先点点看咯！



嗯，看起来是导入了一篇文章，再点另一个：



这两篇文章其实有背景的哦！这里不急着爆破，不妨先陶冶一下情操~

第一篇文字来自电影《锅盖头》（Jarhead），翻译如下：

> 一个男人面前有很多选择，
>
> 而人生中做出的这些选择，很少能尽善尽美。
>
> 于是他决定签下一份合同，
>
> 因为他想有所作为，
>
> 他想拯救这个世界，
>
> 让它变得更好。
>
> 一旦签下那份合同，他就不再拥有选择的权利——
>
> 他想吃、想睡、想做爱、想战斗，
>
> 这些决定都将由别人替他做出。
>
> 他将成为某个比自己更宏大的事物的一部分。
>
> 他将连续训练70天，
>
> 对那把M16A4步枪的每一个角落，了解得比对自己的命根子还要透彻。
>
> 然后他会奔赴沙漠，战斗，然后死去。
>
> 他为何而战？他为何而死？
>
> 这他妈到底有什么意义？
>
> “我们是职业军人！”
>
> “我们是海军陆战队员！”

第二篇文字大有来头，正是著名电影《楚门的世界》的经典台词：

> 早安。假如再也见不到你，
>
> 那就祝你午安、晚安、都好！

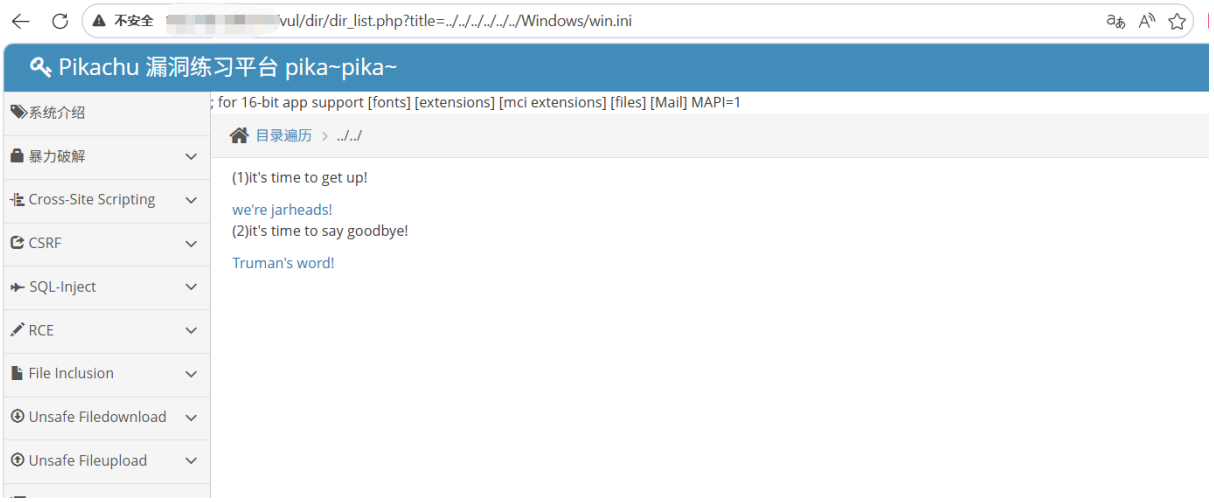
欧克！回到网安领域来吧！我们至少可以确认一件事——这两篇文章一定是经过参数传递之后从后端传回前端的。所以这个参数就至关重要了，有趣的是，这里仍然是一个简单的Get类型的传参方法，我们可以不用去劳烦BurpSuite，直接在网址栏就能观察到端倪：

```
http://XXX.XXX.XXX.XXX/vul/dir/dir_list.php?title=truman.php
```

这种样式的url我们见的很多了，后缀被拼接了一个查询代码 `title=` ，而就在这里我们大有可为。既然参数是 `title` ，后端大概率是去某个目录下读取 `title` 指定的文件，然后返回内容。因此我们直接修改这个参数的内容试试看：

```
http://XXX.XXX.XXX.XXX/vul/dir/dir_list.php?title=../../../../../../Windows/win.ini
```

这就i是经典的 `../` 大法，利用 `../` 跳出当前目录，连续回退到根目录之后，再根据默认路径去读取一些服务器的配置文件。这里回退多少步因人而已，总之多退几步不会有问题。构造完成后，回车访问：



出现了！这就是我们读取的 `win.ini` 文件配置信息！

呃，所以，搞定了？

对，搞定了。虽然有点突兀，虽然看起来和之前的文件包含漏洞完全一样，但我们确实成功了。但是有一点本质区别，文件包含漏洞是代码执行，参数传递到后端之后是通过`include()`函数执行后返回前端的，而目录遍历漏洞就是简单的读取文件内容并显示。

关于这一点，我们可以看看源代码：

```
if(isset($_GET['title'])){
    $filename=$_GET['title'];
    //这里直接把传进来的内容进行了require(),造成问题
    require "soup/$filename";
}
```

嘶~何意味?

老实说，亲爱的读者，这和我预想的完全不一样。这个require()是一个函数啊，本质上就是对传到后端的参数进行了代码执行，和我们之前做的文件包含漏洞是一样的啊!

对此，我只能说，大抵是开发者搞错了。事到如今，也只能和这串源代码大眼瞪小眼了。

那么，真正的目录遍历漏洞是什么样子的呢？理论上应该是下面这种代码：

```
$filename = $_GET['title'];
echo file_get_contents("soup/" . $filename);

或者
$filename = $_GET['title'];
readfile("soup/" . $filename);
```

注意看，这才是经典的目录遍历漏洞，不去执行代码，而是用`file_get_contents()` / `readfile()` / `fopen()`直接读取文件。而皮卡丘靶场这道题用`require()`演示“目录遍历”，其实不太严谨，倒有点“目录遍历+文件包含”的综合。但考虑到这是给初学者练习路径穿越的手法，也说得过去。

毕竟，有多少人像我们一样愿意去读源代码呢？但是从源代码中，我们能学到更多东西。也许将来有人会说“我通关了皮卡丘靶场”，这最多不过是一个夸耀，而只有懂了并且学以致用才是真的啊!

十二、敏感信息泄露

还是先看看开发者要说啥：

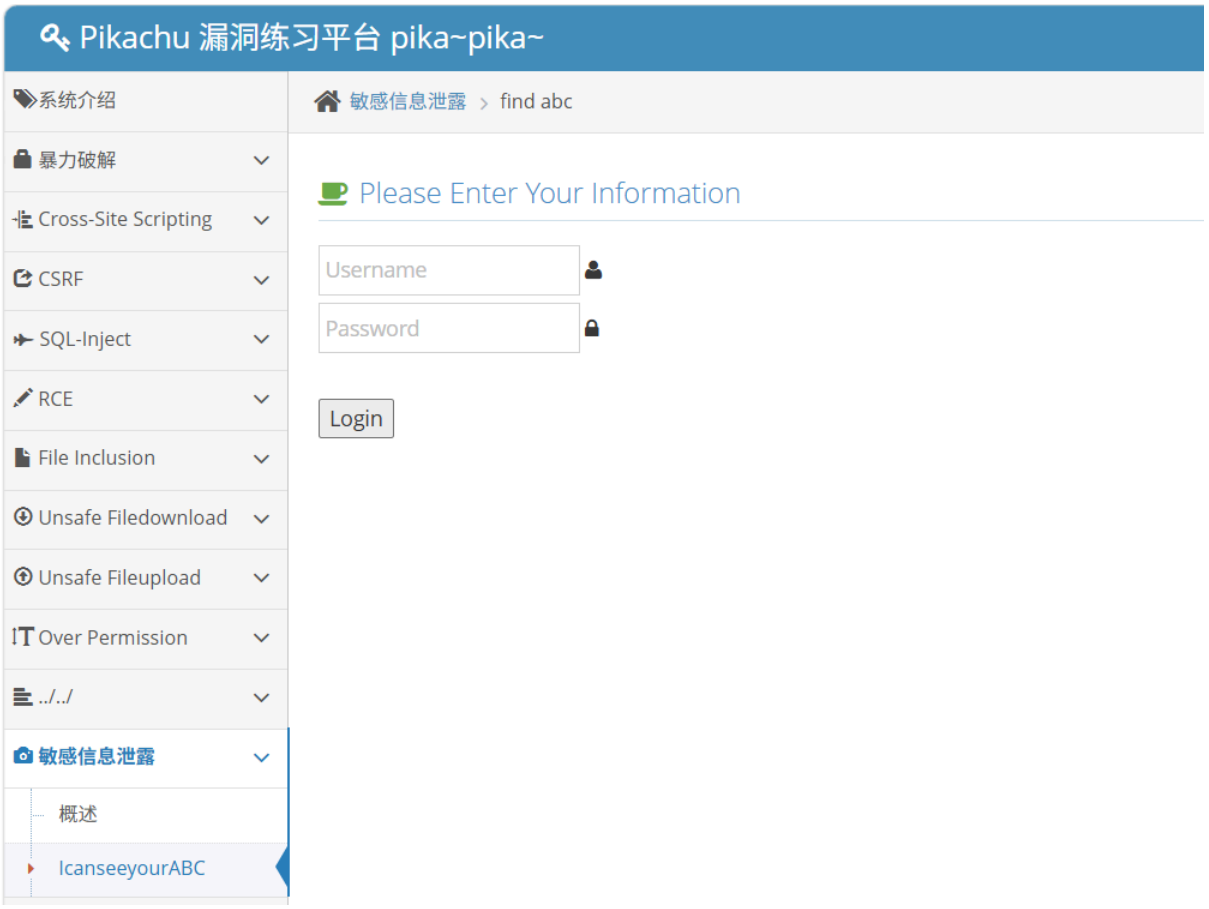
> **敏感信息泄露概述**
>
> 由于后台人员的疏忽或者不当的设计，导致不应该被前端用户看到的数据被轻易的访问到。 比如：
>
> ---通过访问url下的目录，可以直接列出目录下的文件列表;
>
> ---输入错误的url参数后报错信息里面包含操作系统、中间件、开发语言的版本或其他信息;
>
> ---前端的源码（html,css,js）里面包含了敏感信息，比如后台登录地址、内网接口信息、甚至账号密码等;
>
> 类似以上这些情况，我们成为敏感信息泄露。敏感信息泄露虽然一直被评为危害比较低的漏洞，但这些敏感信息往往给攻击着实施进一步的攻击提供很大的帮助,甚至“离谱”的敏感信息泄露也会直接造成严重的损失。 因此,在web应用的开发上，除了要进行安全的代码编写，也需要注意对敏感信息的合理处理。
>
> 你可以通过“i can see your abc”对应的测试栏目，来进一步的了解该漏洞。

简单来说，所谓“敏感信息泄露”漏洞，就是由于开发者的粗心，把一些敏感的信息直接暴露在前端代码中，虽然可能直接通过网页看不到，但是前端的代码其实是透明的——我们随时可以通过浏览器的“查看页面源代码”功能看到这些信息。

道理是很简单的，我们直接看题目吧！

1、I can see your abc

来到这一题，是一个登录页面：



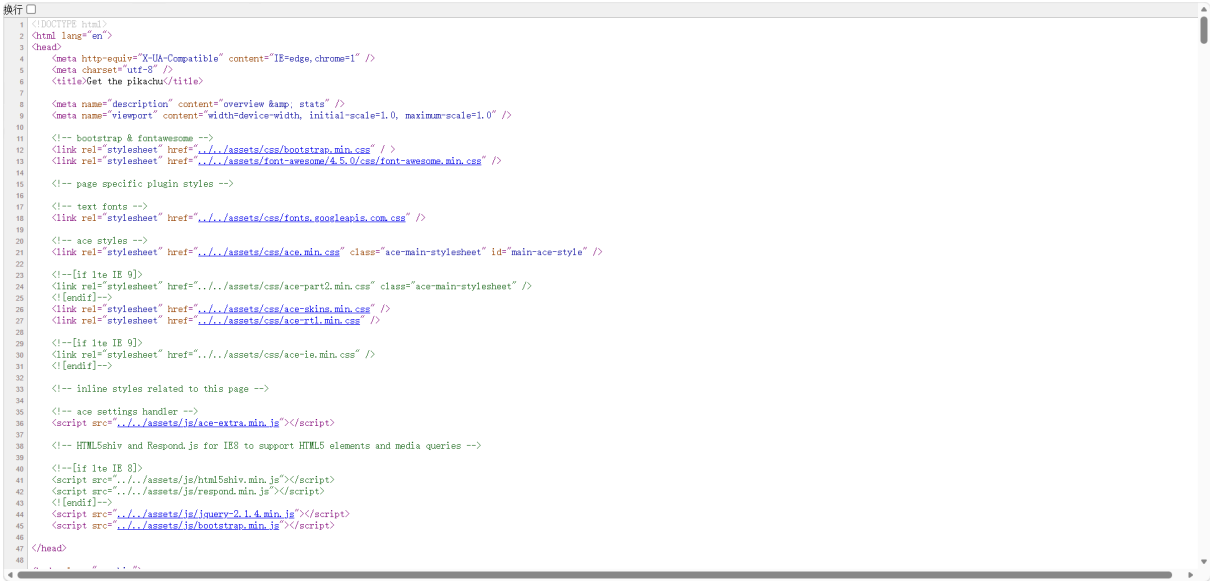
不过这一次，提示中没有给我们任何账号信息了——相反，开发者提示我们："找找看，很多地方都漏点了..."

这就意味着，我们需要找到一些关于账号的蛛丝马迹——这不同于我们一开始做的暴力破解，前端没有任何提示，纯靠用户名和密码的多种组合试错；也不同于后来的注入漏洞、越权漏洞等，登录只是为了区分各个账号，但登录账号本身不是要点。

这个蛛丝马迹从哪里来？结合本节的概述，只有从前端代码中寻找！

在网页中右键，就可以看到一个“查看页面源代码”的功能：

点击后，我们就来到了一个新的标签页，这里显示的就是当前页面的源代码：



那么，开始检查有没有什么敏感信息咯！

由于我们需要的是有关账号的信息，所以敏感信息如果有的话，大概率是在登录功能相关的代码附近，我们简单用“password“查找定位一下，就能找到了：

等下，这是什么？测试账号？就这样明明白白地写在了前端代码中！

这就是我们需要的"敏感信息"，拿回去试验一下，看看能不能登录进去吧！

成功了！看来开发者是为了方便，顺手把测试账号写到前端代码中，却在网站上线忘记清理这个硬编码的账号信息，同时测试账号也没有清理，这就让攻击者有机可乘了！

此外，不得不说，开发者的留言也颇有一种浪漫风范呢！

> 那一天我二十一岁，在我一生的黄金时代
>
> 我有好多奢望。我想爱，想吃，还想在一瞬间变成天上半明半暗的云

> 后来我才知道，生活就是个缓慢受锤的过程，人一天天老下去，奢望也一天天消失，最后变得像挨了锤的牛一样
>
> 可是我过二十一岁生日时没有预见到这一点。我觉得自己会永远生猛下去，什么也锤不了我
>
> ----王小波《黄金时代》

这就是"敏感信息泄露"，还是我们之前强调的一个简单事实——一切前端代码对于用户而言都是透明的，一切关于前端的隐藏和防御机制都是自欺欺人。

你以为是秘密的东西，其实浏览器早就替你说出来了。开发者图一时方便留下的"后门"，往往是攻击者最先敲门的地方。

十三、PHP反序列化

这是一个全新的名词，看看开发者是怎么介绍的吧！

> 在理解这个漏洞前,你需要先搞清楚php中serialize()，unserialize()这两个函数。
>
> **序列化serialize()**
>
> 序列化说通俗点就是把一个对象变成可以传输的字符串,比如下面是一个对象:
>
> ```
> class S{
>     public $test="pikachu";
> }
> $s=new S(); //创建一个对象
> serialize($s); //把这个对象进行序列化
> 序列化后得到的结果是这个样子的:O:1:"S":1:{s:4:"test";s:7:"pikachu";}
>     O:代表object
>     1:代表对象名字长度为一个字符
>     S:对象的名称
>     1:代表对象里面有一个变量
>     s:数据类型
>     4:变量名称的长度
>     test:变量名称
>     s:数据类型
>     7:变量值的长度
>     pikachu:变量值
> ```
>
> **反序列化unserialize()**
>
> 就是把被序列化的字符串还原为对象,然后在接下来的代码中继续使用。

```
$u=unserialize("O:1:"S":1:{s:4:"test";s:7:"pikachu";}");
echo $u->test; //得到的结果为pikachu
```

序列化和反序列化本身没有问题,但是如果反序列化的内容是用户可以控制的,且后台不正当的使用了PHP中的魔法函数,就会导致安全问题

```
常见的几个魔法函数:
__construct()当一个对象创建时被调用

__destruct()当一个对象销毁时被调用

__toString()当一个对象被当作一个字符串使用

__sleep() 在对象在被序列化之前运行

__wakeup将在序列化之后立即被调用

漏洞举例:

class S{
    var $test = "pikachu";
    function __destruct(){
        echo $this->test;
    }
}
$s = $_GET['test'];
@$unser = unserialize($a);

payload:O:1:"S":1:{s:4:"test";s:29:"<script>alert('xss')</script>";}
```

这里关于序列化和反序列化的解释是很清楚的，但是对于我们这样的初学者来说，理解起来还是有点抽象。其实，序列化就是这样一种东西——程序里的对象（比如一个 `$user` 对象，有姓名、年龄属性）是存在于内存中的，于是 PHP 提供了 `serialize()`，把对象“打包”成一个字符串。

那么，为什么非要把对象转换成字符串不可呢？

简单来说，序列化之所以要把对象“打包成字符串”，是因为对象活在内存里，无法直接保存到文件或通过网络传输。只有转换成字符串这种线性、可存储的格式，才能跨脚本、跨请求传递。之所以叫“序列化”，就是把这个立体的、复杂的对象结构，“按顺序拍平”成一个连续的字符序列。因此，这个函数 `serialize()` 就像一个“液压机”一样，将数据罐头压瘪，但却不丧失其本质。反过来，反序列化就是再把这段序列重新拼装成原来的对象。

这个过程，就和压缩和解压缩一样，只不过是同一串数据的不同表现形式罢了。只不过，压缩（如 zip）的目标是减小体积，压缩后人类完全看不懂；而序列化（serialize）的目标是保留结构与类型（字符串、数字、类、属性等），体积可能反而变大，但能无损还原成原来的对象。

但是问题在于，反序列化函数会无条件还原对象，并且还原过程中会自动触发魔术方法（如 `__destruct`、`__wakeup`），所以攻击者只要构造一个包含恶意属性值的序列化字符串，就能在对象被还原时“借刀杀人”。这就好比别人给你发来了一个压缩包，你刚一解压缩，你的系统就中招了。

因此，PHP反序列化漏洞的核心就是：`unserialize()` 不是一个纯“数据解析”函数，而是一个有“代码执行副作用”的函数。如果用户可以构造特殊的数据，经过后端的反序列化后就会生成一个可执行的恶意代码，从而欺骗系统执行以达到攻击目的。

接下来，我们就一起看看怎么利用这个反序列化漏洞来进行攻击吧！

1、PHP反序列化漏洞

就像我们之前做的RCE一样，这里同样给了我们一个API接口，专门用于接收执行序列化数据。

换言之，我们就需要在这里构造一个序列化数据，提交后，后端通过反序列化函数将其解析为一个对象——而在这个解析过程中，我们构造的代码就会被执行，这就是攻

击的核心所在！

那么，要构造什么序列化数据才能满足攻击的条件呢？当然是需要符合序列化数据的格式——我们就不必专门去学这一点了，不妨就让AI代劳吧！

```
O:1:"S":1:{s:4:"test";s:29:"<script>alert('XSS')</script>";}
```

经过反序列化之后，这行数据就会变成：

```
object(S)#1 (1) {
  ["test"]=>
  string(29) "<script>alert('XSS')</script>"
}
```

这是AI给了我们一个构造，不过这个序列化数据是什么意思呢？让AI解释一下：

> 这个序列化字符串的意思是：创建一个类名为 `S` 的对象，它有一个属性叫 `test`，而这个属性的值是一段29个字符的XSS代码 `<script>alert('XSS')</script>`。当后端用 `unserialize()` 解析这个字符串时，就会原样重建出这个对象，等程序运行结束、对象被销毁时，`__destruct()` 魔术方法会自动执行，把 `test` 属性里的内容输出到页面上，从而触发弹窗。

也就是说，提交这串数据后，预期的效果就是系统执行 `<script>alert('XSS')</script>`，网页跳出一个弹窗，显示“XSS”。

看看是不是这样吧！

页面成功出现了弹窗！攻击成功！

这就是针对反序列化漏洞的攻击，我们这里只是让网页跳出一个弹窗，同样的，我们当然也可以获取服务器权限、数据、破坏网站等等——本质都是一样的，就和我们之

前做的RCE一样，我们可以利用这个测试接口执行任意代码。区别在于RCE的构造是显性的，可以被后端直接执行，而这里是把恶意代码构造在序列化数据中，而反序列化函数一经解析，这个恶意代码就会作为对象被执行，从而达到攻击目的。

这里，我们还是发挥传统艺能看看源代码：

```
class S{
    var $test = "pikachu";
    function __construct(){
        echo $this->test;
    }
}
#定义了一个类S，__construct() 是魔术方法，对象被创建时自动调用,它会直接输出 $test 属性的值
……
if(isset($_POST['o'])){
    $s = $_POST['o'];
    $unser = unserialize($s);  // 反序列化用户输入
    $html .= "<p>{$unser->test}</p>";
}
```

此处，反序列化的代码大致分为两部分——第一部分是用于定义类和方法，第二部分是反序列化函数 `unserialize()` 的执行。因此，我们提交序列化数据后，系统进行反序列化时会发生：

1. PHP 创建一个 `s` 类的对象
2. `__construct()` 自动执行 → 输出 `<script>alert('xss')</script>`（弹窗）
3. 然后代码继续执行 `{$unser->test}`，又把这段脚本输出一次

于是，我们提交的序列化字符串被反序列化后，自动创建了 `s` 对象，创建过程中 `__construct()` 被触发，把注入的 XSS 代码输出到了页面上。这就是反序列化漏洞利用的全过程。

回顾一下整个过程，其实序列化和反序列化本身都没有什么问题，我们这道题留下来的API接口也只是为了测试——因此，避免这个漏洞的关键在于权限控制，关键在于只让测试人员有权限使用这个API接口，而绝不能将其暴露在其他用户的账号下！（或者干脆就不要反序列化啦！）

这就是反序列化漏洞——不过是一个新名词而已，其实做下来感觉也不是很难，对吧！

十四、XXE

老规矩，看概述：

> XXE -"xml external entity injection"
>
> 既"xml外部实体注入漏洞"。
>
> 概括一下就是"攻击者通过向服务器注入指定的xml实体内容,从而让服务器按照指定的配置进行执行,导致问题"
>
> 也就是说服务端接收和解析了来自用户端的xml数据,而又没有做严格的安全控制,从而导致xml外部实体注入。
>
> 具体的关于xml实体的介绍,网络上有很多,自己动手先查一下。
>
> 现在很多语言里面对应的解析xml的函数默认是禁止解析外部实体内容的,从而也就直接避免了这个漏洞。
>
> 以PHP为例,在PHP里面解析xml用的是libxml,其在≥2.9.0的版本中,默认是禁止解析xml外部实体内容的。
>
> 本章提供的案例中,为了模拟漏洞,通过手动指定LIBXML_NOENT选项开启了xml外部实体解析。

这次的介绍相当简洁哈，对于不了解XML的人而言，实在看得是一头雾水。其实，所谓XEE漏洞，就是在服务器对传入的xml实体的解析过程中，系统执行了参数中包含的危险代码。而xml实体，其实就是一种类似html的一种“用尖括号写标签”的文本格式，用来存数据或传数据。

我们一起看一个xml的例子：

```
<user>
    <name>张三</name>
    <age>25</age>
    <city>北京</city>
</user>
```

这就是一个典型的xml实体，它的意思就是“有一个用户，名字叫张三，年龄25岁，城市在北京”。因此，它本质上不过就是一个带结构的文本罢了；而这个结构，恰恰就是和后端相匹配对应的，从而能够让服务器去解析和存储这个参数。

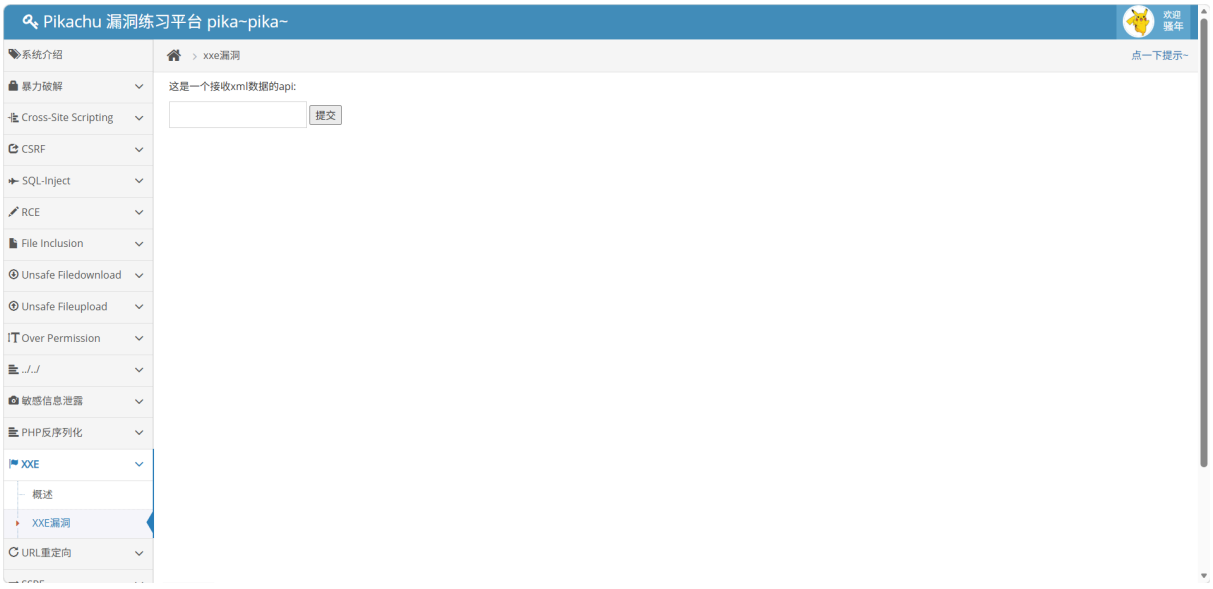
如果光看这一点，其实它就和现在流行的json格式很像，都是用特定的格式来传递数据。不过json格式由于其轻量性的特点被广为使用，而xml现在多存在于一些老系统之中。

我们本节的XXL漏洞，就是这种文本格式xml的漏洞——正常情况下，系统会去读取xml中的数据，而一旦这个xml中带有恶意的暗号，那么就会按照暗号的指引去执行一些命令、读取一些敏感文件。这就是XXL漏洞的本质。

至于什么是“恶意暗号”？这就要在具体的题目中去理解啦！

1、XXE漏洞

又是熟悉的API接口：



看来这题和上一题很类似，都是需要利用这个API接口，构造出一个包含恶意代码的数据。那么，具体应该如何构造呢？首先，需要符合xml的语法格式；其次，需要使用“恶意暗号”去实现一些攻击操作，比如读取系统配置文件等等（咱们还是温和一些为好）。

接下来，就让AI来帮我们构造一个吧！

```
<?xml version="1.0"?>
<!DOCTYPE ANY [
```

```
    <!ENTITY xxe SYSTEM "file:///c:/windows/system32/drivers/etc/hosts"> ]>
<a>&xxe;</a>
```

这就是一个简单的包含恶意暗号的xml语法。先别急着提交，我们先来理解一下这个构造：

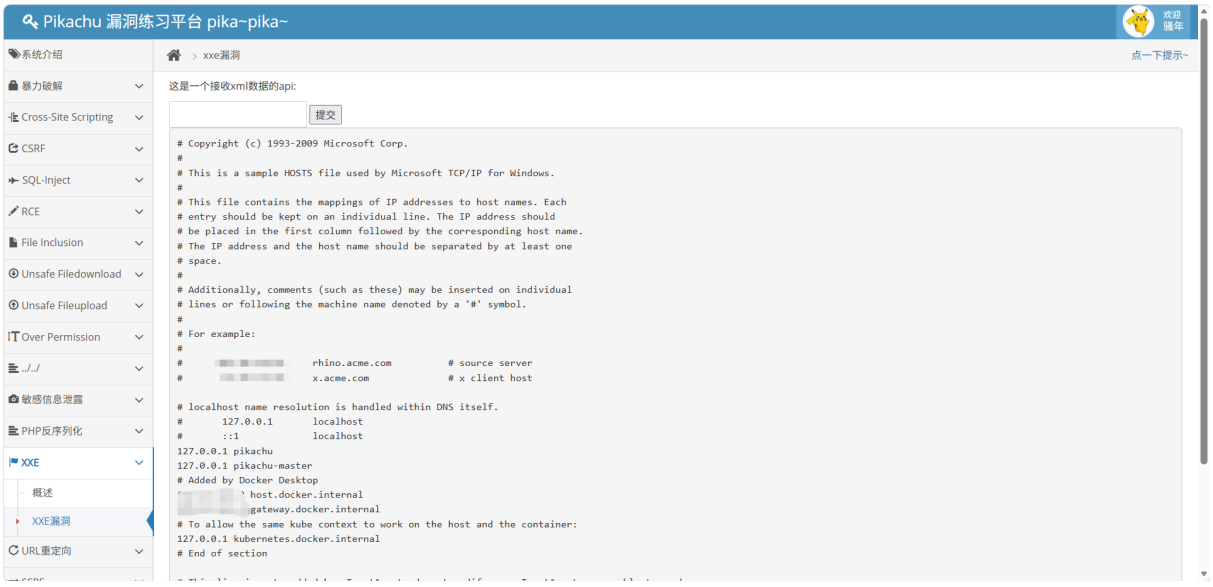
第一行的 `<?xml version="1.0"?>` 是一个声明，声明这是一个XML文档，版本1.0。这是XML的标准开头，告诉解析器“你要处理的是XML格式的数据”。而第二行的 `<!DOCTYPE ANY [` 则是对文档类型的定义，`DOCTYPE` 是文档类型声明的关键字，`ANY` 是这个文档类型（DID）的名字（可以随便起，叫 `root`、`foo`、`test` 都行），最后 `[` 则表示开始定义具体的实体。

第三行是重头戏，这是在定义外部实体，即整个XXE的核心！`ENTITY` 是定义实体的标志，`xxe` 是实体的名字（同样可以随便起），`SYSTEM` 则表示这是一个外部资源（来自系统文件或网络），最后 `"file:///c:/windows/system32/drivers/etc/hosts"` 则注明了外部资源的路径（这个文件是Windows系统的本地DNS映射表，作用是把域名指向特定的IP地址。）。换言之，这是在定义一个暗号：“当我说 `&xxe;` 的时候，你就去把 `C:\windows\system32\drivers\etc\hosts` 这个文件的内容拿来”。

第四行就是根元素了，也就是触发系统解析的地方。`<a>&xxe;</a>` 中引用了&xxe的实体，而这个实体就是我们刚才在上一行定义的暗号。

也就是说，当我们提交这个xml数据后，预期系统就会读取 `C:\windows\system32\drivers\etc\hosts` 文件，并返回前端——如此一来，我们就成功利用XXE漏洞读取了系统敏感信息。实际上，读取任意文件就是XXE漏洞最稳定的利用方式。

试试看咯！



网页成功返回了文件内容！这就成功了！

最后，咱们还是一起看看源代码吧：

```
if(isset($_POST['submit']) and $_POST['xml'] != null){
    $xml = $_POST['xml'];
    $data = @simplexml_load_string($xml,'SimpleXMLElement',
LIBXML_NOENT);
    if($data){
        $html.="<pre>{$data}</pre>";
    }
}
```

这里不需要看得很仔细，因为这其实是xml自身的一个旧版本的漏洞（现在已经被修复了，但是这里开发者强行打开了这个危险开关，复现了这个漏洞），而不是代码层面的漏洞。这个漏洞是因为代码里解析XML时，故意打开了一个危险开关（`LIBXML_NOENT`），允许解析外部实体，又没有检查用户输入，所以攻击者可以在XML里写“去把某个文件的内容拿来”，服务器就会乖乖照做，并把文件内容返回给攻击者。

到了这里，我们就会发现，其实XXE漏洞的本质并不复杂——就是网站解析XML数据时，没有禁用“外部实体”这个功能，攻击者可以在XML里塞一句“帮我把某个文件的内容拿来”，解析器就会乖乖照做，并把文件内容吐出来。

这次我们能成功读取 `hosts` 文件，关键原因在于代码里手动开启了 `LIBXML_NOENT` 这个“危险开关”，又没对用户输入做任何过滤。说到底，这不是什么高深的黑客技术，而是开发者对底层配置的不了解，给攻击者留了一扇不该开的门。

好啦！接下来乘胜追击吧！

十五、URL重定向

看看概述咯：

> 不安全的url跳转
>
> 不安全的url跳转问题可能发生在一切执行了url地址跳转的地方。

> 如果后端采用了前端传进来的(可能是用户传参,或者之前预埋在前端页面的url地址)参数作为了跳转的目的地,而又没有做判断的话
>
> 就可能发生"跳错对象"的问题。
>
> url跳转比较直接的危害是:
>
> →钓鱼,既攻击者使用漏洞方的域名(比如一个比较出名的公司域名往往会让用户放心的点击)做掩盖,而最终跳转的确实钓鱼网站
>
> 这个漏洞比较简单,come on,来测一把!

简单来说，这就是一个关于url的传参的漏洞——当前端传入的参数是一个url链接时，如果后端没有充分的检查，那么通过修改这个参数，就可以实现不安全的url跳转——设想一下，别人给你发来一个链接，你一看域名，是个特别正规的网站，就放心点了进去，但是这个链接的url后缀却是指向了一个钓鱼站点，那么你就会直接跳转到这个钓鱼网站了！

举个例子来说，假如说淘宝网存在这个漏洞的话，攻击者就可以构造这样一个链接：

```
https://www.taobao.com/redirect?url=https://fake-taobao.com
```

注意看，光看前面的域名的话，真的会毫无防备——淘宝网嘛！中国最知名的网站之一，有什么安全隐患呢？结果忽略了这个链接后面拼接的url，它指向了一个 `https://fake-taobao.com` 的钓鱼网站。而只要你一点击链接，就会跳转到这个钓鱼站点了！真是猝不及防！

所以，用一句话来总结，URL 重定向漏洞是指应用程序在接受用户控制的参数作为跳转目标时，未做充分校验，导致攻击者可构造一个指向可信域名的恶意链接，诱骗用户点击后跳转至钓鱼网站或恶意站点。

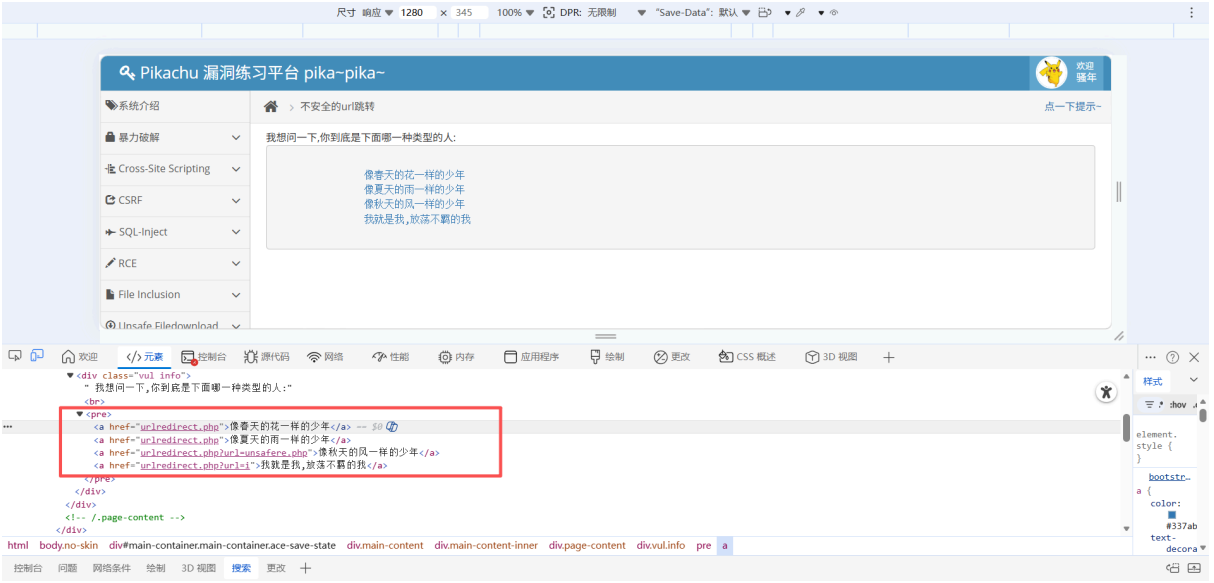
而我们之后要做的事情，就是构造这样一个链接。

那么，就来看看题目吧！

1、不安全的URL跳转

看题！

这一题又给了我们很多<a>标签，不出意外的话，有一个<a>标签一定存在一个特殊的url的跳转。这里当然可以一个一个点击看看，但是还有一个取巧的方法，我们直接打开浏览器的开发者工具（好久没用过了~），选中元素，就可以看到这四个<a>标签分别对应的链接是啥：



很清晰了！这四个<a>标签分别对应的四个链接中，最后两个多出来了url的后缀，也就是说，这里存在url的跳转！读者可以点击看看，点第三个<a>标签就会跳转到 `unsafere.php` 页面，点第四个就会触发 `url=i` 的特殊效果——开发者的深情寄语："好的，希望你能坚持做你自己！"

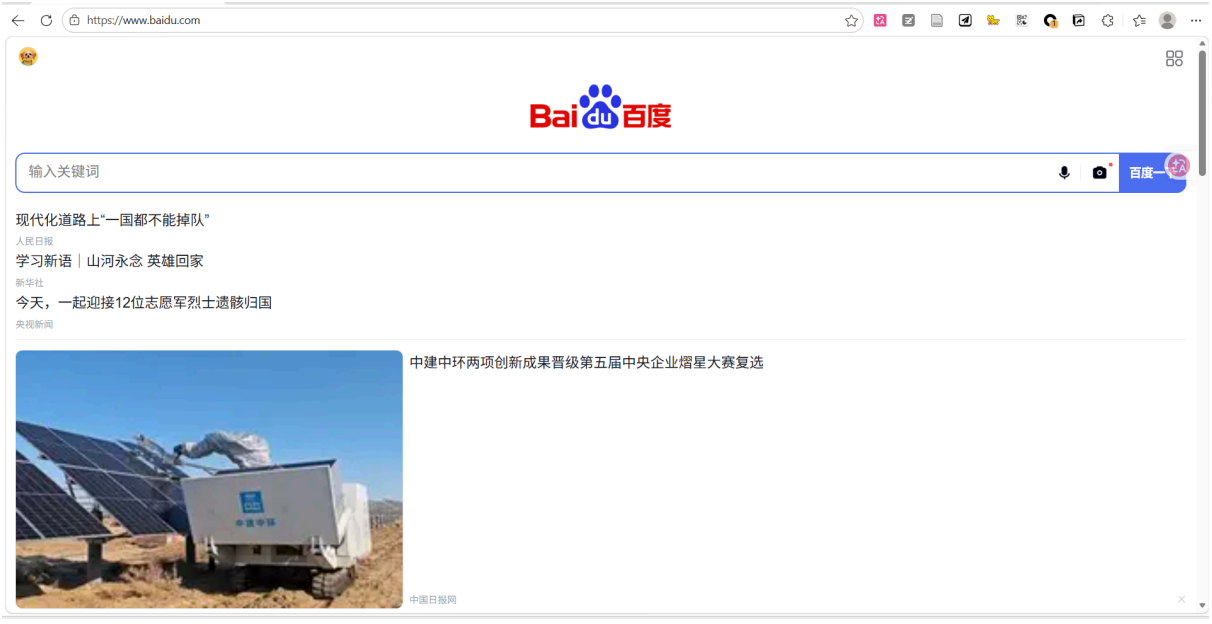
现在要做的事情很清晰了，就是在当前页面的浏览器的网址栏加上这样一个url跳转的后缀格式： `?url=` 。具体指向的链接你可以自定义，比如，我们先跳转到亲爱的度娘看看？

```
?url=http://www.baidu.com
```

于是就形成了这样一个链接：

```
http://XXX.XXX.XXX.XXX:XX/vul/urlredirect/urlredirect.php?url=http://www.baidu.com
```

回车！看看能否访问：



果然跳转成功了！那么我们的解题就完成啦！而如果这是一个真实的攻击场景，那么接下来攻击者就可以将上述的这个特殊链接发布出去，这就是"不安全的URL跳转"的最终攻击形态。

最后，还是一起看看源代码吧！

```php
if(isset($_GET['url']) && $_GET['url'] != null){
    $url = $_GET['url'];
    // 只有一条特殊规则：如果等于 'i' 就输出一句话
    if($url == 'i'){
        $html.="<p>好的,希望你能坚持做你自己!</p>";
    }else {
        header("location:{$url}");    // ← 漏洞在这里
    }
}
```

问题是很明确的，这里没有任何针对url传参的安全检验机制，就只有一条特殊规则“`如果等于 'i' 就输出一句话`”。所以，无论我们构造出什么样的url，都会正常返回跳转结果——如此一来，我们就可以利用这个官网链接实现一些指向特殊网站的构造了！

这就是“不安全的url跳转”漏洞，其实核心原理是很简单的——不安全的URL跳转漏洞，本质是开发者“信任了用户传入的URL”却没有做任何校验——信任一旦放错了地方，原本安全的域名就会成为攻击者的钓鱼跳板。

用户可以控制跳转目的地，而代码照单全收——这就是URL重定向漏洞的全部秘密。

十六、SSRF

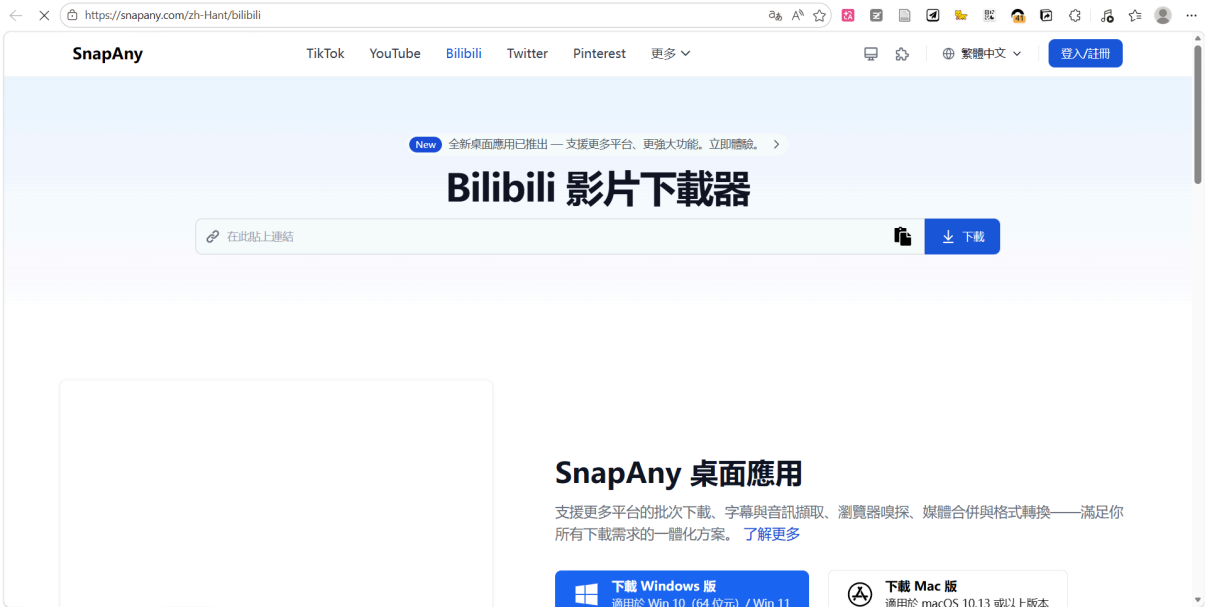
最后一章节啦！这也是最后一篇概述了，且看且珍惜咯！

> **SSRF(Server-Side Request Forgery:服务器端请求伪造)**
>
> 其形成的原因大都是由于服务端**提供了从其他服务器应用获取数据的功能**,但又没有对目标地址做严格过滤与限制,导致攻击者可以传入任意的地址来让后端服务器对其发起请求,并返回对该目标地址请求的数据
>
> 数据流:攻击者--——→服务器-——→目标地址
>
> 根据后台使用的函数的不同,对应的影响和利用方法又有不一样
>
> ```
> PHP中下面函数的使用不当会导致SSRF:
> file_get_contents()
> fsockopen()
> curl_exec()
> ```

> 如果一定要通过后台服务器远程去对用户指定("或者预埋在前端的请求")的地址进行资源请求,**则请做好目标地址的过滤**。
>
> 你可以根据"SSRF"里面的项目来搞懂问题的原因。

简单来说，所谓SSRF漏洞，就是攻击者利用服务端提供的请求外部资源的功能，让服务器向内网或外网的任意地址发起请求，从而绕过访问控制、探测内网、攻击内部系统、读取敏感文件等。在这里，服务端就好像变成了攻击者的"肉鸡代理"(毕竟没有真的控制网站服务器，而只是利用漏洞)，借用它的网络身份和权限去发起请求。

那么，在哪些情境下可能会出现这种漏洞呢？此漏洞一般都发生在网站提供一个链接输入选项，用户提交链接后，服务器前往指定链接下载资源的情形下——这种情况在一些代理下载网站中十分常见，下面的网站就是一个示例，不过有没有这个漏洞我就不清楚了（大抵是没有吧）：



总之，正如靶场开发者所说，如果需要提供此类服务，就一定要做好目标地址的过滤，把获取资源的对象局限在目标地址下，而不要让攻击者有可乘之机。

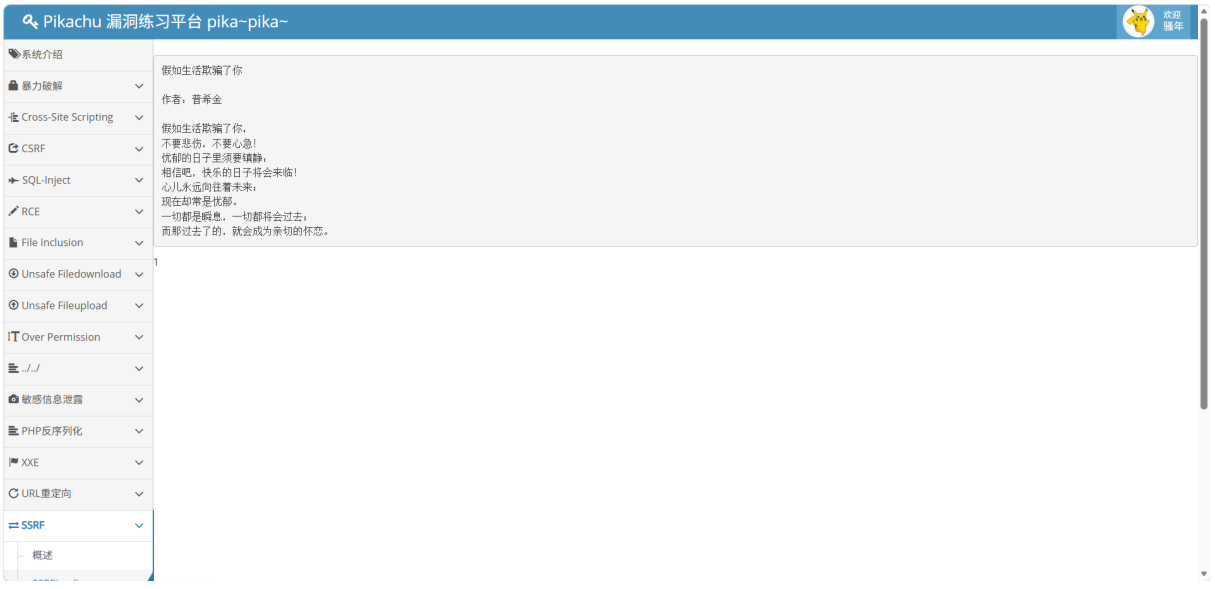
现在，我们就一起来看看靶场吧！

1、SSRF(curl)

"累了吧，来读一首诗吧"——靶场一开始就扔给我们一个<a>标签：

那么，就点击看看咯！

是一首诗啊，顿时勾起了我中学时的回忆：



> 假如生活欺骗了你
>
> 作者：普希金
>
> 假如生活欺骗了你，
> 不要悲伤，不要心急！
> 忧郁的日子里须要镇静：

> 相信吧，快乐的日子将会来临！
> 心儿永远向往着未来；
> 现在却常是忧郁。
> 一切都是瞬息，一切都将会过去；
> 而那过去了的，就会成为亲切的怀恋。

不过，这里要提醒一下读者，如果你这里皮卡丘靶场用的是默认端口80以外的端口，那么这里跳转链接会失败——因为这里代码就指定了跳转链接哦，请看网址栏的url：

```
http://127.0.0.1/vul/ssrf/ssrf_curl.php?url=http://127.0.0.1/vul/ssrf/ssrf_info/info1.php
```

此处点击<a>标签之后，会给网址尾部拼接上一个后缀：`?url=http://127.0.0.1/vul/ssrf/ssrf_info/info1.php`，这个后缀指向的是本机的环回回路，IP地址默认的是127.0.0.1，默认端口是硬编码的80。也就是说，如果你为皮卡丘靶场配置的端口做了修改，那么此处就会指向一个不存在的端口，当然会报错了。不过解决方法也很简单，只需要在IP地址后面加上端口号即可，比如我的配置端口号是83，那么就把网址改为：

```
http://127.0.0.1:83/vul/ssrf/ssrf_curl.php?url=http://127.0.0.1:83/vul/ssrf/ssrf_info/info1.php
```

从这里，我们其实也能看出，这里给网址尾部拼接的同样是一个url，只不过指向了本机的一个默认地址，相当于从本机的服务器上的`info1.php`文件获取了一个数据——《假如生活欺骗了你》。

所以，这正印证了一个事实——这里提供了从其他服务器应用获取数据的功能！那么很可能就会存在SSRF漏洞！那么我们试试看，能不能获取百度服务器的首页数据。修改一下url的参数（这里我用的是localhost，其实是127.0.0.1环回回路的另一种写法，功能是完全一样的）：

```
http://localhost:83/vul/ssrf/ssrf_curl.php?url=http://www.baidu.com
```

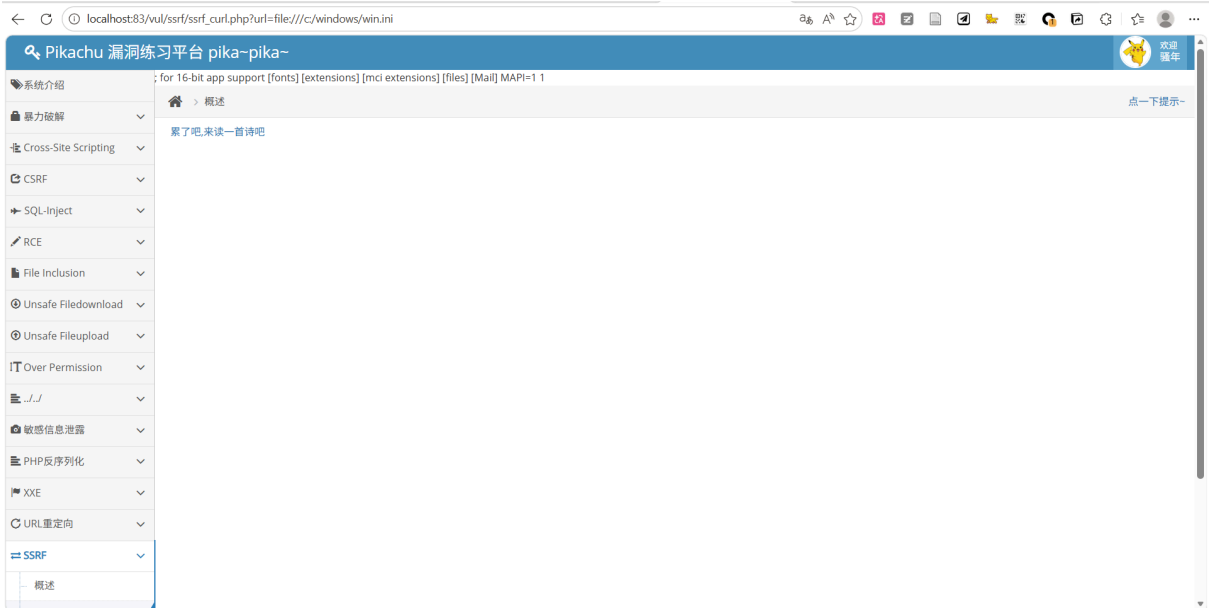
回车之后，发现页面中出现了百度首页，靶场和百度首页的数据全混在一起了。而且这上面的百度元素其实都是可以交互的——毕竟这不过就是一些前端代码而已，直接拿过来的话，相关的链接参数都是正常工作的啦。

你还可以试试看获取其他的服务，不过这里就不再演示了。其实，SSRF漏洞的本质就是很简单的，如果没有对请求服务的地址加以检查，那么攻击者就可以利用此服务任意地获取其他服务器应用的数据，甚至对其他服务器展开攻击。

不过，SSRF漏洞真正的威力在于探测内网，攻击内网服务——换言之，对于本机部署的皮卡丘靶场而言，那就是探测服务器上的文件结构和隐私信息。而且，这里从题目来看，用的是 `curl_exec()` 函数方法（这个方法支持很多协议，包括文件读取的协议file），那么我们就可以使用file协议进行构造，从而直接读取主机上的其他文件了：

```
http://localhost:83/vul/ssrf/ssrf_curl.php?url=file:///c:/windows/win.ini
```

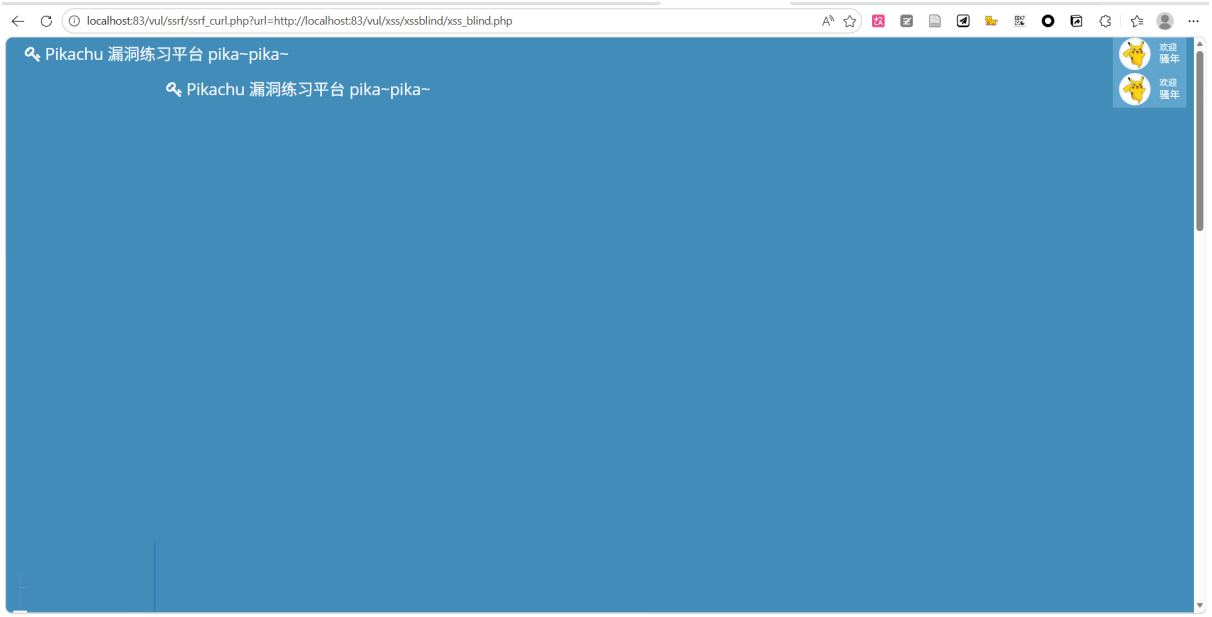
输入这串网址，就能在前端看到这个配置文件的内容了：

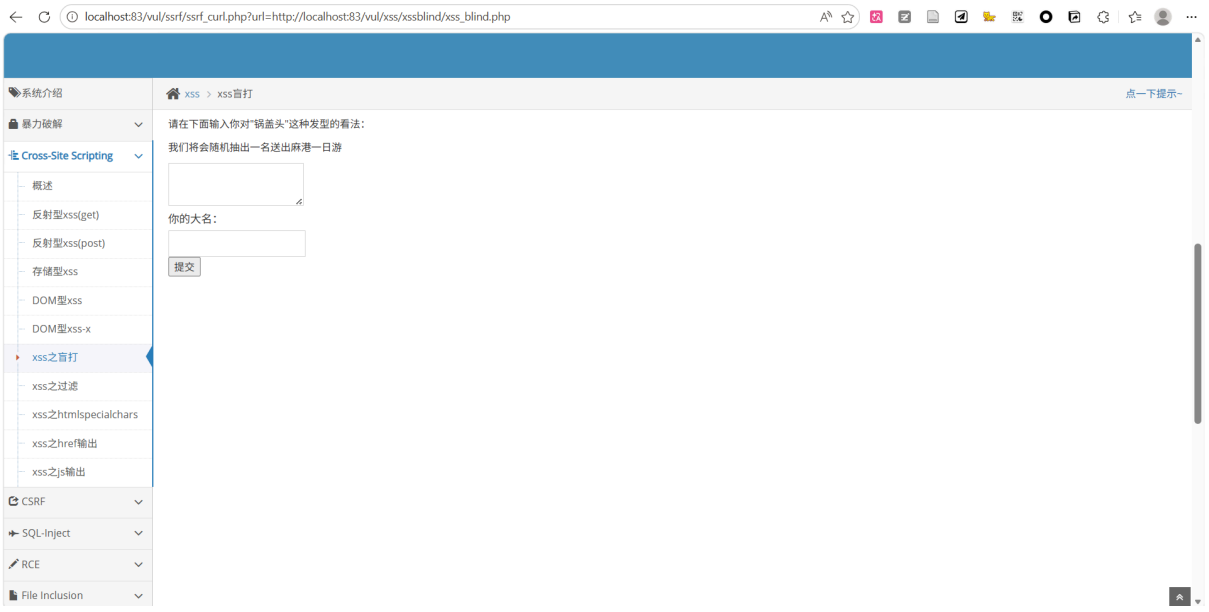


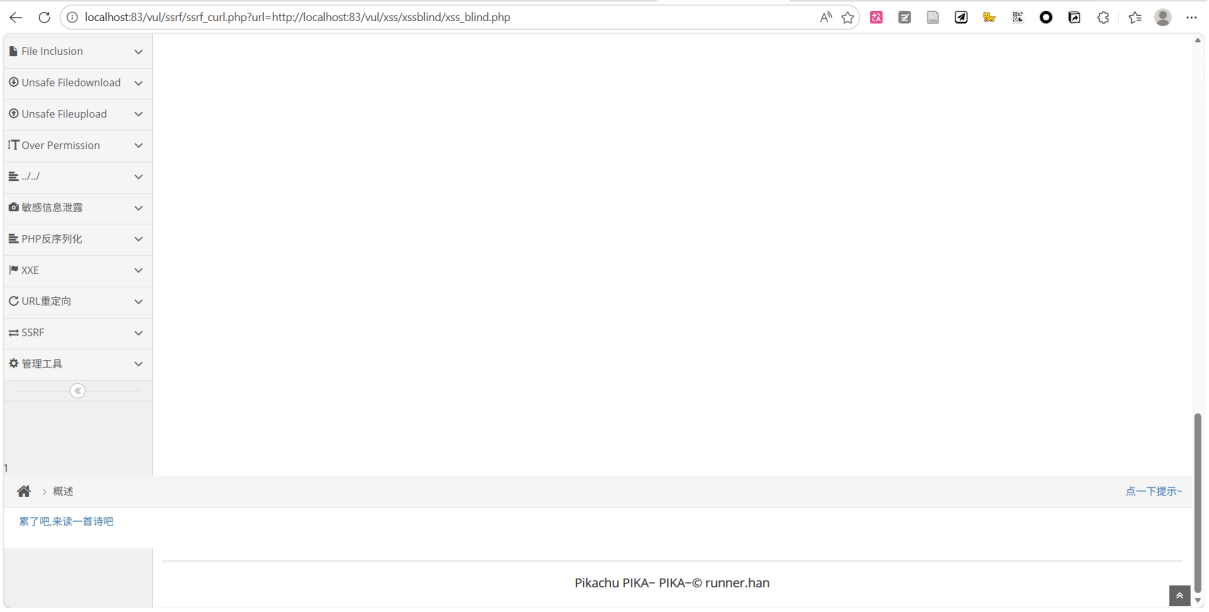
而且，就算不用这个这个读文件的方法，我们也可以就按照题目给的思路走，切换到其他路径，读取其他挂在83端口下的服务器文件，比如，这里我把url后缀修改为xxs节中“盲打“的那道题对应的url路径：

```
http://localhost:83/vul/ssrf/ssrf_curl.php?url=http://localhost:83/vul/xss/xssblind/xss_blind.php
```

回车访问后，发现那一题和这题的网页也混在一起了：







呐~就是这样咯！而在真实的网站环境下，攻击者完全可以利用这个漏洞，把网站相关的所有文件全都读取一遍，然后再利用刚才那个file协议的方法把服务器上的其他文件也全都搞到手——可见危害之大了！

最后，还是一起看看源代码吧：

```
if(isset($_GET['url']) && $_GET['url'] != null){
    $URL = $_GET['url'];                    // ① 直接取用户输入，无任何过滤
    $CH = curl_init($URL);                  // ② 直接用curl发起请求
    curl_setopt($CH, CURLOPT_HEADER, FALSE);
```

```
        curl_setopt($CH, CURLOPT_SSL_VERIFYPEER, FALSE);
        $RES = curl_exec($CH);                    // ③ 执行请求
        curl_close($CH);
        echo $RES;                                // ④ 直接把结果
    返回给前端
    }
```

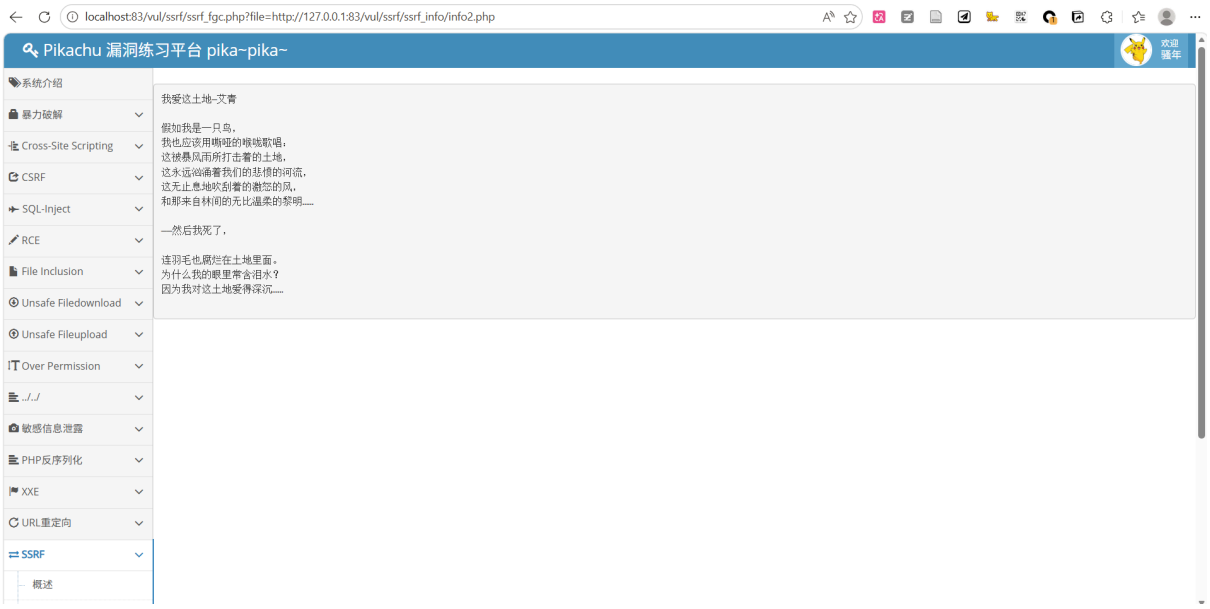
这里代码的问题很清晰，正如我们一开始所介绍的那样，提供了curl的请求服务，却没有对请求服务的对象加以检查和过滤——这就是SSRF漏洞的本质所在啊！

最后，快马加鞭把最后一题也给解决了吧！

2、SSRF（file_get_content）

最后一题，和上一题看起来如出一辙：

看看能不能速通吧！点击<a>标签看看会跳转到哪里：



还是一首诗欸！再次梦回高中：

> 我爱这土地--艾青
>
> 假如我是一只鸟，
> 我也应该用嘶哑的喉咙歌唱：
> 这被暴风雨所打击着的土地，
> 这永远汹涌着我们的悲愤的河流，
> 这无止息地吹刮着的激怒的风，
> 和那来自林间的无比温柔的黎明……
>
> —然后我死了，
>
> 连羽毛也腐烂在土地里面。
> 为什么我的眼里常含泪水？
> 因为我对这土地爱得深沉……

咳咳，这里我们更应该关注的当然还是网址栏，看来这次后缀拼接的东西不太一样：

```
http://localhost:83/vul/ssrf/ssrf_fgc.php?file=http://127.0.0.1:83/vul/ssrf/ssrf_info/info2.php
```

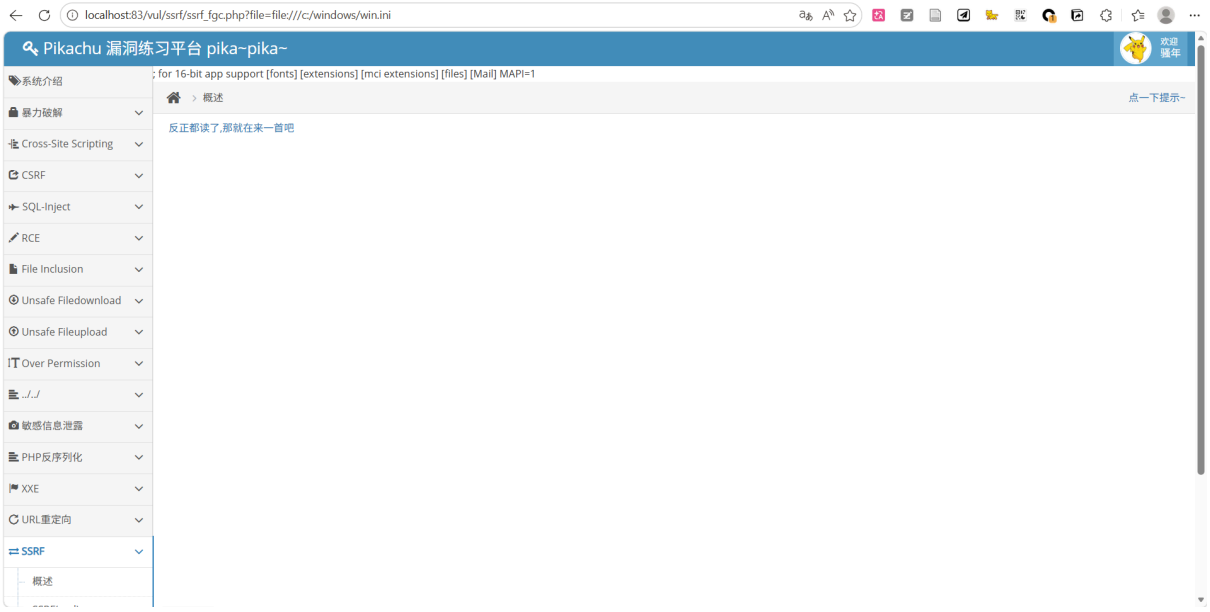
确实！上一题我们的网址后缀是 `url=` ，但是这里是 `file=` ，刚好和题目相呼应，看来这就是区别所在！这里参数名的不同对应的是后端函数的不同，从题目来看的话，这里的函数当然就是 `file_get_contents()` 了。

在这里，我们可以把两道题进行一个对比：

对比项	上一关（ssrf_curl.php）	本关（ssrf_fgc.php）
参数名	?url=	?file=
后端函数	curl_exec()	file_get_contents()
支持协议	多协议（dict、gopher、file、http等）	较少（主要 http/https/file）

那么，其实构造的方法也是类似的，比如我们可以仿照上一题的方式，同样利用这里支持的file协议去读取主机文件：

```
http://localhost:83/vul/ssrf/ssrf_fgc.php?file=file:///c:/windows/win.ini
```



真是似曾相识啊！那么，我们这样也就成功了。

当然，这里和上一题一样，凡是 `file_get_contents()` 函数支持的协议，我们都可以进行对应的构造，去获取外网的数据，去读取本地的文件等等。这里就不再做更多的演示了。

最后，咱们还是一起来看看源代码，其实这里用脚趾头都能想到，除了用的函数不一样之外，其他地方和上一题应该是如出一辙：

```
if(isset($_GET['file']) && $_GET['file'] !=null){
    $filename = $_GET['file'];            // ① 直接取用户输入，无任何过滤
    $str = file_get_contents($filename); // ② 直接读取用户指
```

```
定的内容
    echo $str;                          // ③ 直接输出
}
```

没错啦，除了用到的函数不同，其他的完全一样——直接读取输入，不做过滤，不做检查，直接输出。其实用的函数不同对于漏洞的存在与否没有影响，只是对漏洞的利用方式有关罢了。

所以，综合这两道题，SSRF漏洞的核心不在于用了什么函数，而在于“用户输入是否被允许直接决定服务端发起的请求”。函数决定的是攻击面大小，漏洞存在与否取决于有无过滤。

尾声

到这里，全部皮卡丘靶场的学习笔记就全部结束了，感谢各位读者的陪伴！

从零开始“爆破皮卡丘靶场”，这个最基础的靶场，一边做，一边学，一边写笔记，竟然已经过去两个多月了。回想起来，真是感慨万分——当时我第一次接触靶场，面对那些陌生的名词、奇怪的语法（PHP是世界上最奇怪的语言！），实在是感到无从下手；而且尤其是对于刚开始的那些题目，基本上就是照着攻略抄，不会的地方就只有反复和AI探讨。那可真是一段很痛苦的时光，而那些我在笔记里留下来的幽默感，全是我在自我调侃呢！

但是慢慢的，我也逐渐摸透了这些题目的套路，竟然也能自己独立地去探索和尝试，甚至养成了翻看源代码的习惯（还带着读者一起去看源代码，还请恕罪哈），做起这些题目来也越来越得心应手了，甚至于后来加入了pikachu的QQ群，竟然也能带给大家一点启发，那可真是我的荣幸。

不过坦白地说，我从来不认为自己有什么厉害之处，如果硬要说有的话，那可能就是学习新知识的劲头吧。虽然说，这份笔记有诸多错漏，和网上那些大佬的笔记相比专业性也有很大欠缺，但是真实是它的底色——从零开始的网安学习之路，从零开始的我，从零开始的知识与文字.....而如果你能从这份笔记中体会到这份真实，那么这也是对我最好的回音。

现在，终点也是新的起点——我现在就好像是小智从真新镇领到了第一只宝可梦，正满怀希望地踏上未知的宝可梦大师之旅呢！

去吧！皮卡丘！